

Univbersity of Catania
Degree course in Data Science

Beans Analysis

Prepared by:

Andrea Isaia

Supervisor
Antonio Punzo

2026

Contents

1 Introduction **3**

2 Description of the dataset **3**

3 Library used during the analysis **4**

4 Univariate analysis **5**

4.1 Area . . . 5
4.1.1 Log-Normal . . . 8
4.1.2 Gamma . . . 9
4.1.3 Weibull . . . 10
4.1.4 Exponential . . . 11
4.1.5 Inverse Gaussian . . . 12
4.1.6 Comparison . . . 13
4.2 Perimeter . . . 14
4.2.1 Log-Normal . . . 16
4.2.2 Gamma . . . 17
4.2.3 Weibull . . . 18
4.2.4 Exponential . . . 19
4.2.5 Inverse Gaussian . . . 20
4.2.6 Comparison . . . 21
4.3 MajorAxisLength . . . 22
4.3.1 Log-Normal . . . 24
4.3.2 Gamma . . . 25
4.3.3 Weibull . . . 26
4.3.4 Exponential . . . 27
4.3.5 Inverse Gaussian . . . 28
4.3.6 Comparison . . . 29
4.4 MinorAxisLength . . . 30
4.4.1 Log-Normal . . . 32
4.4.2 Gamma . . . 33
4.4.3 Weibull . . . 34
4.4.4 Exponential . . . 35
4.4.5 Inverse Gaussian . . . 36
4.4.6 Comparison . . . 37
4.5 AspectRation . . . 38
4.5.1 Log-Normal . . . 40
4.5.2 Gamma . . . 41
4.5.3 Weibull . . . 42
4.5.4 Exponential . . . 43
4.5.5 Inverse Gaussian . . . 44
4.5.6 Mixture model of Inverse Gaussian . . . 45
4.5.7 Comparison . . . 47

4.6 Eccentricity . . . 48
4.6.1 Beta . . . 50
4.6.2 Simplex . . . 51
4.6.3 Comparison . . . 52
4.7 ConvexArea . . . 52
4.7.1 Log-Normal . . . 56
4.7.2 Gamma . . . 57
4.7.3 Weibull . . . 58
4.7.4 Exponential . . . 59
4.7.5 Inverse Gaussian . . . 60
4.7.6 Comparison . . . 61
4.8 EquivDiameter . . . 62
4.8.1 Log-Normal . . . 64
4.8.2 Gamma . . . 65
4.8.3 Weibull . . . 66
4.8.4 Exponential . . . 67
4.8.5 Inverse Gaussian . . . 68
4.8.6 Comparison . . . 69
4.9 Extent . . . 70
4.9.1 Beta . . . 72
4.9.2 Simplex . . . 73
4.9.3 Comparison . . . 74
4.10 Solidity . . . 74
4.10.1 Beta . . . 77
4.10.2 Simplex . . . 78
4.10.3 Comparison . . . 79
4.11 Roundness . . . 79
4.11.1 Beta . . . 82
4.11.2 Simplex . . . 83
4.11.3 Comparison . . . 84
4.12 Compactness . . . 84
4.12.1 Beta . . . 87
4.12.2 Simplex . . . 88
4.12.3 Comparison . . . 89
4.13 Class . . . 89

5 PCA **91**
5.1 Correlation . . . 91
5.2 PCA . . . 93
5.2.1 Cumulative proportion of variance explained . . . 96
5.2.2 Kaiser's rule . . . 96
5.2.3 Scree plot . . . 96
5.3 PCs plot . . . 97

6 Multivariate analysis : Cluster analysis **98**

6.1 Clustering tendency . . . 98
6.1.1 Hopkins statistic . . . 102
6.1.2 VAT Algorithm . . . 102
6.2 Hierarchial clustering . . . 104
6.2.1 Euclidean distance . . . 105
6.2.2 Manhattan distance . . . 134
6.3 Partitioning clustering . . . 151
6.3.1 K-medoids . . . 163
6.4 Stability measure . . . 171

7 Fuzzy Clustering (Fuzzy K-means) 178
7.1 Algorithms . . . 178
7.2 Clusters and centroids . . . 179
7.3 Plots . . . 182

8 Model based Clustering 189

9 Conclusion 193

1 Introduction

The dataset is called Bean dataset. This dataset is useful to analyze the different types of beans depending on their geometrical characteristics. The dataset can be downloaded on this site: https://www.kaggle.com/datasets/sansuthi/dry-bean-dataset. Our purpose is to try to create different clusters in which each bean should belongs. The single cluster is described by specific property that we will see during the analysis.

There is a reduction on the original dataset: We only take at random 672 measures of the original 13612 and only 13 features of the original 17.

2 Description of the dataset

We first analyze the dataset using different functions in R.

```
Beans <- read.csv("C:/Users/Andrea/Desktop/Bean analysis/Data/Dry_Beans.csv",
                  header = TRUE)
str(Beans)
```

```
## 'data.frame':    672 obs. of  13 variables:
##  $ Area           : int  90393 34655 32026 39452 237270 48230 57126 52808 49483 36029
##  $ Perimeter      : num  1166 704 662 736 1922 ...
##  $ MajorAxisLength: num  458 266 244 261 738 ...
##  $ MinorAxisLength: num  253 167 168 193 413 ...
##  $ AspectRation   : num  1.81 1.59 1.46 1.36 1.79 ...
##  $ Eccentricity   : num  0.833 0.777 0.727 0.676 0.829 ...
```

```
##  $ ConvexArea     : int  91924 35158 32428 39950 240671 48983 58284 54100 50289 36371
##  $ EquivDiameter  : num  339 210 202 224 550 ...
##  $ Extent         : num  0.756 0.707 0.719 0.736 0.784 ...
##  $ Solidity       : num  0.983 0.986 0.988 0.988 0.986 ...
##  $ roundness      : num  0.836 0.88 0.919 0.916 0.807 ...
##  $ Compactness    : num  0.74 0.791 0.827 0.858 0.745 ...
##  $ Class          : chr  "CALI" "DERMASON" "DERMASON" "SEKER" ...
```

There are 13 features. In particular:

- 10 are represented by real numbers:
  - **Perimeter**: Bean circumference is defined as the length of its border.
  - **MajorAxisLength**: The distance between the ends of the longest line that can be drawn from a bean.
  - **MinorAxisLength**: The longest line that can be drawn from the bean while standing perpendicular to the main axis.
  - **AspectRation**: Defines the relationship between MajorAxisLength and MinorAxisLength
  - **Eccentricity**: Eccentricity of the ellipse having the same moments as the region.
  - **Equivalent diameter**: The diameter of a circle having the same Perimeter as a bean seed area.
  - **Extent** : The ratio between Area and the Bounding Box(the minimal rectangle that contains it) in pixel measurement.
  - **Solidity** : Also known as convexity. The ratio of the pixels in the area of the beans and convex area.
  - **Roundness** : Calculated with the following formula: (4piArea)/(Perimeter^2)
  - **Compactness** : Measures the roundness of an object: Equivalentdiameter/MajorAxisLength
- 2 are represented by integer numbers:
  - **Area** : The area of a bean zone and the number of pixels within its boundaries.
  - **ConvexArea**: Number of pixels in the smallest convex polygon that can contain the area of a bean seed.
- 1 is represented by strings:
  - **Class** (Seker, Barbunya, Bombay, Cali, Dermosan, Horoz and Sira). This represent the different family of the beans. This column in particular will be excluded during the analysis and used as a comparison at the end with the found cluster.

3 Library used during the analysis

```
library("moments") # Useful to evaluate kurtosis and skewness
library("gamlss")
library("gamlss.mx")
```

```r
library("ggcorrplot") # Useful for doing correlation
library("factoextra") # Useful for doing cluster analysis
library("clustertend")
library("hclust1d")
library("gridExtra") # Useful to have a grid of different factorextra method
library("NbClust") # Useful to evaluate the number of clsuters
library("cluster") # Useful for pam algorithm inside fviz_nbclust
library("fpc") # Useful for the clustering validation
library("clValid") # Useful for the internal validation and clustering
                   # stability measueres
library("fclust") # Useful for the k-means algorithm
library("mclust") # Useful for the mixture model
```

4 Univariate analysis

To do the univariate analysis first of all we attach the data, so we can deal with the different features of the data.

```r
attach(Beans)
```

4.1 Area

```r
summary(Area)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   21666   35604   43124   51441   56196  237270
```

```r
V<- var(Area)
cat("Variance : ",V,"\n")
```

```
## Variance :  822736790
```

```r
Sd <- sd(Area)
cat("Standard deviation : ",Sd,"\n")
```

```
## Standard deviation :  28683.39
```

```r
Area.table <- table(Area)
mode <- names(Area.table)[which.max(Area.table)]
cat("Mode : ",mode,"\n")
```

```
## Mode :  27457
```

```r
K <- kurtosis(Area)
cat("Kurtosis : ",K,"\n")
```

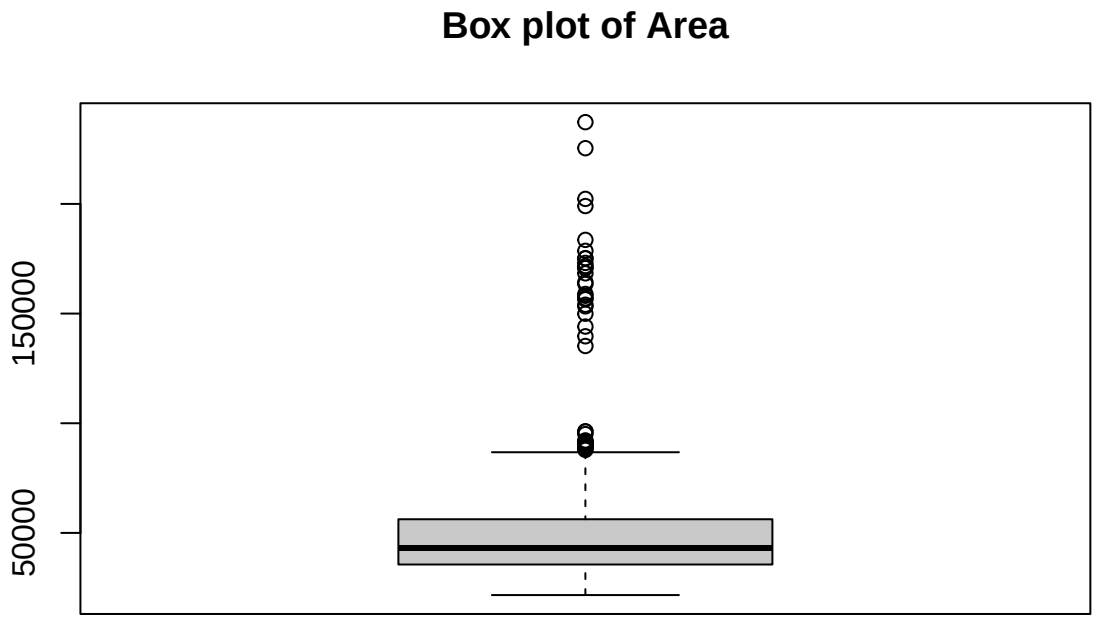
```
## Kurtosis :  14.55907
```

```r
S <- skewness(Area)
cat("Skewness : ",S,"\n")
```

```
## Skewness :  3.054848
```

The data are right-skewed, as observed from the skewness index. The high kurtosis indicates that the distribution is leptokurtic, meaning there is a great concentration of values around the mean, but also a high frequency of outliers that are far away from it.

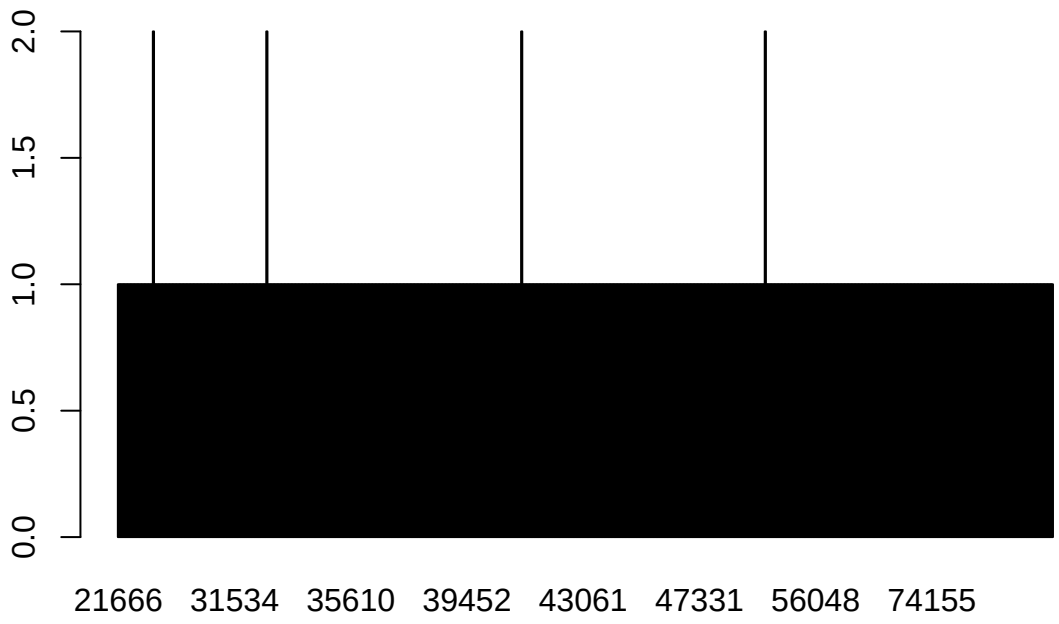
```r
box <- boxplot(Area,main="Box plot of Area")
```




```r
out <- length(box$out)/length(Area)
cat("Percentage of outliers : ",out*100,"%")
```

```
## Percentage of outliers :  6.10119 %
```

```r
barplot(Area.table)
```



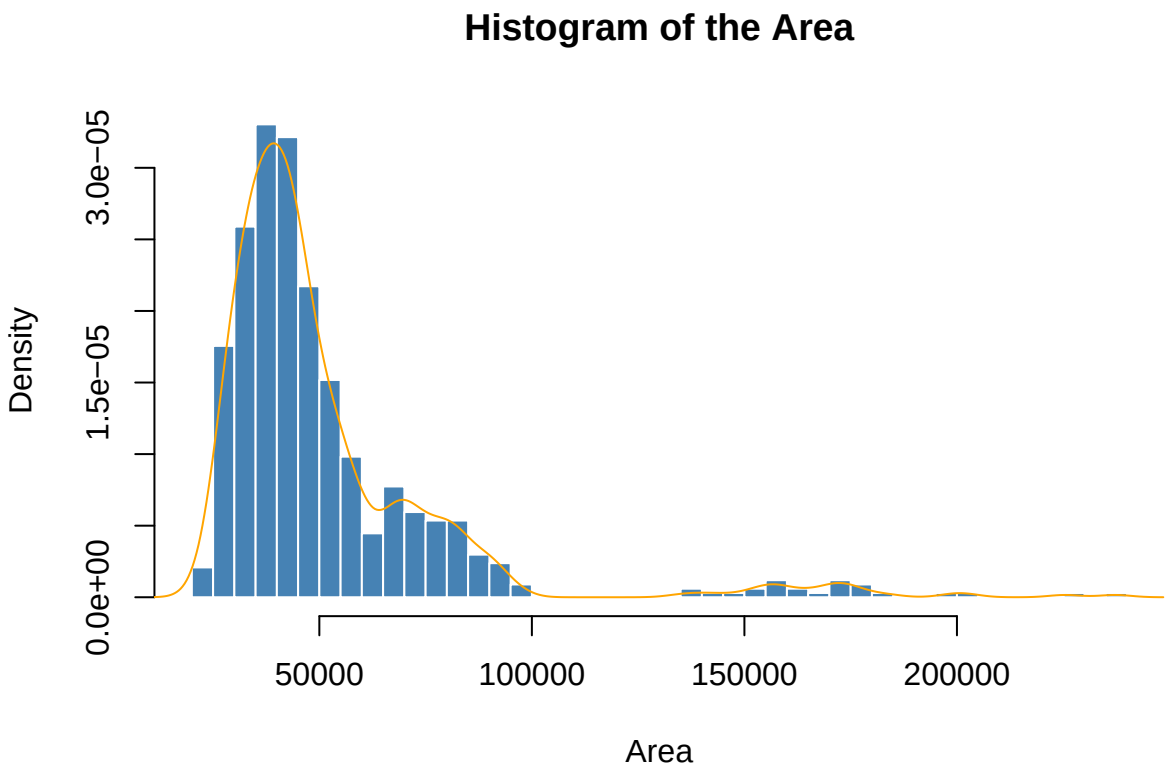
We can see from the bar plot that the data work as a continuous variable, not like an integer where values repeat multiple times. So in this case we treat all like a continuous variable.

For the histogram we can use the Freedman-Diaconis method. The equation is:

$$h = 2 \cdot \frac{IQR(x)}{\sqrt[3]{n}}$$

Where IQR represent the difference between 1°Quantile and the 3° Quantile and n represent the number of data.

```
hist(Area,
     breaks = "FD",
     main = "Histogram of the Area",
     col = "steelblue",
     border = "white",
     freq = FALSE)
lines(density(Area),col = "orange")
```



Now we can try to fit different models to the data. First of all we observe that the support of the data is $[0, +Inf[$. So the distribution from which we could choose are:

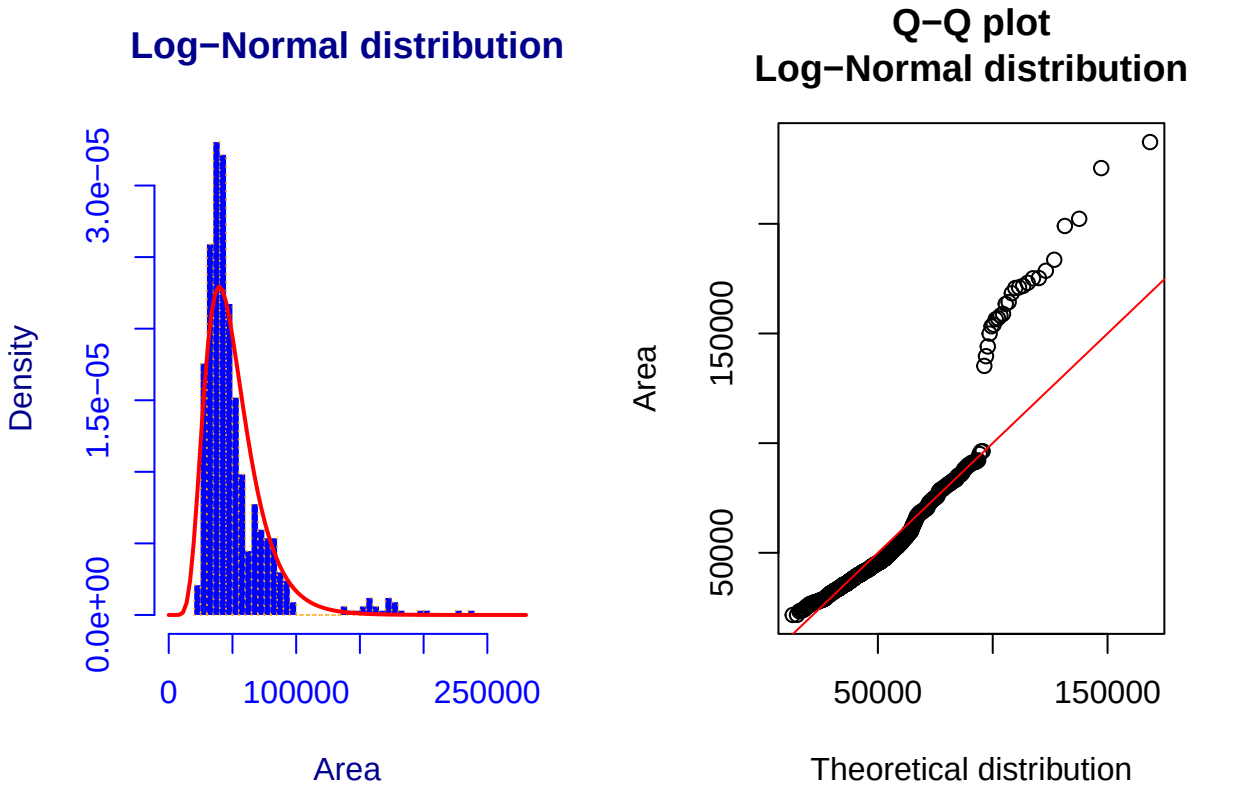
- **Log-Normal**
- **Gamma**
- **Weibull**
- **Exponential**
- **Inverse Gaussian**

4.1.1 Log-Normal

```
par(mfrow = c(1,2))
Log.N1<- histDist(Area,family = LOGNO,nbins = "fd",
                  main = "Log-Normal distribution",
                  col.hist="blue",border.hist="orange",line.col = "red"
                  ,ylab = "Density")

theoA <- ppoints(length(Area))
mu <- Log.N1$mu.coefficients
sigma <- exp(Log.N1$sigma.coefficients)
Log.N1.Theoretical <- qLOGNO(theoA ,mu= mu, sigma = sigma)
```

```
qqplot(x = Log.N1.Theoretical,y = Area,
       xlab = "Theoretical distribution",
       main="Q-Q plot\nLog-Normal distribution")
abline(a = 0, b=1,col="red")
```



But why:

```
sigma <- exp(Log.N1$sigma.coefficients)
cat("Function applied to sigma",Log.N1$sigma.link)
```

```
## Function applied to sigma log
```

We can see that the link value is log. This means that the value obtained from the histDist function gives as the $log(\sigma)$ not the value it self. So we do the exponential.

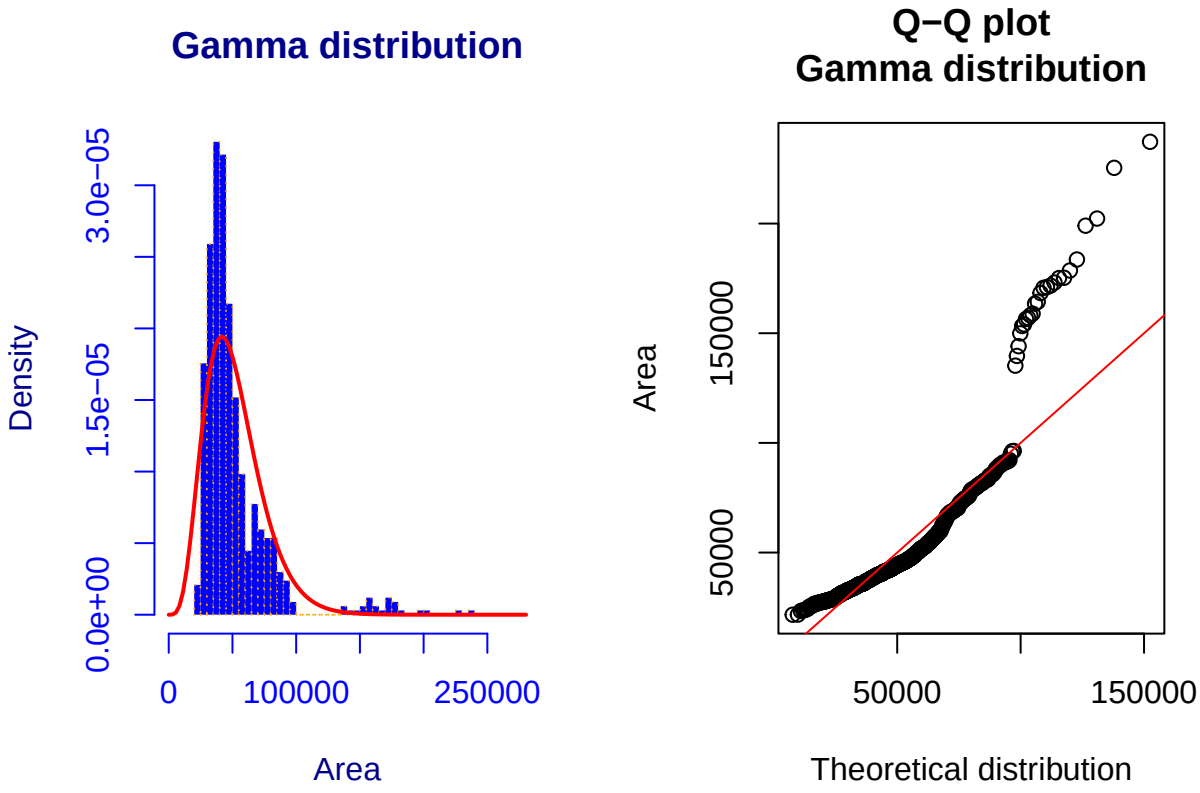
Another important aspect is the usage of ppoints: this function generates a vector of probabilities in $[0, 1]$, but given by the formula $\frac{i-0.5}{n}$ where i is the element and n the number of elements. This with the qLOG gives us the quantile for that probabilities. Then we do the Q-Q plot.

4.1.2 Gamma

```
par(mfrow = c(1,2))
Gamma.N1 <- histDist(Area,family = GA ,nbins = "fd",
                     main = "Gamma distribution",
                     col.hist="blue",border.hist="orange",line.col = "red"
                     ,ylab = "Density")


mu <- exp(Gamma.N1$mu.coefficients)
sigma <- exp(Gamma.N1$sigma.coefficients)
Gamma.N1.Theoretical <- qGA(theoA,mu= mu, sigma = sigma)

qqplot(x = Gamma.N1.Theoretical,y = Area,
       xlab = "Theoretical distribution",
       main="Q-Q plot\nGamma distribution")
abline(a = 0, b=1,col="red")
```



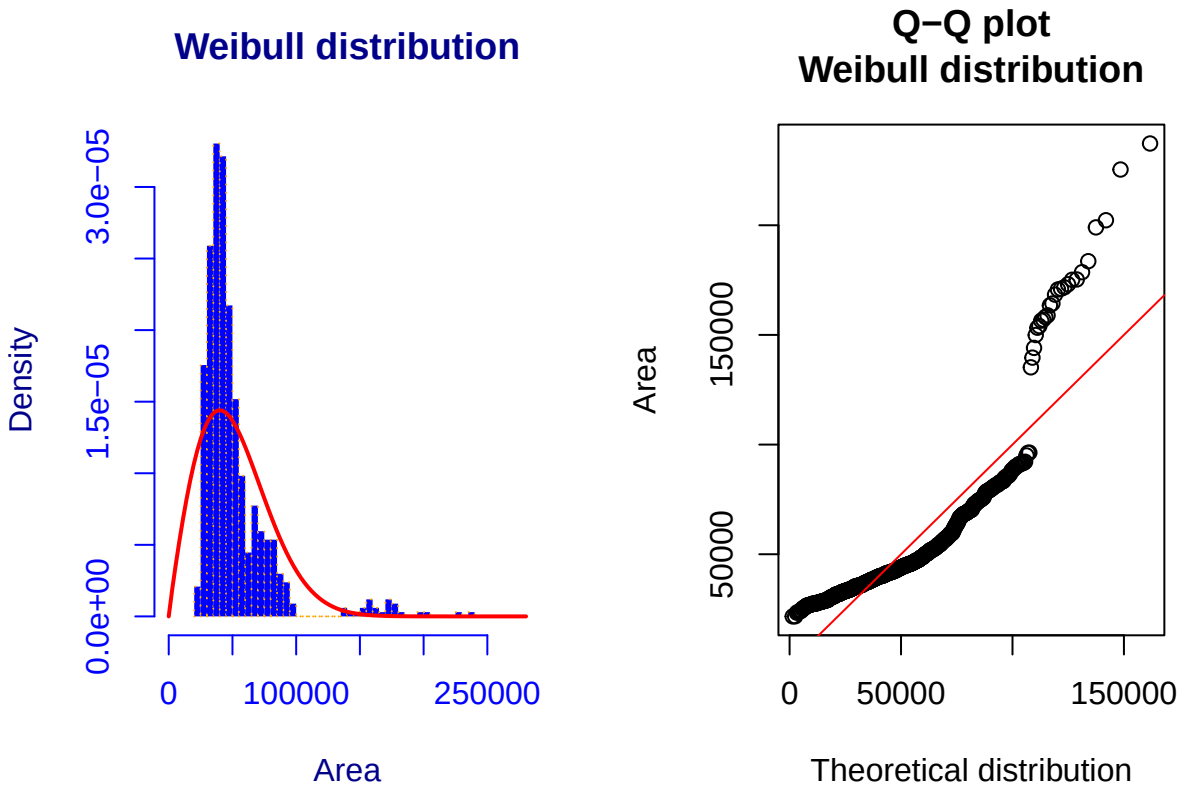
4.1.3 Weibull

```
par(mfrow = c(1,2))
Weibull.N1 <- histDist(Area,family = WEI ,nbins = "fd",
                     main = "Weibull distribution",
```

```
                  col.hist="blue",border.hist="orange",line.col = "red"
                  ,ylab = "Density")

mu <- exp(Weibull.N1$mu.coefficients)
sigma <- exp(Weibull.N1$sigma.coefficients)
Weibull.N1.Theoretical <- qWEI(theoA,mu= mu, sigma = sigma)

qqplot(x = Weibull.N1.Theoretical,y = Area,
       xlab = "Theoretical distribution",
       main="Q-Q plot\nWeibull distribution")
abline(a = 0, b=1,col="red")
```



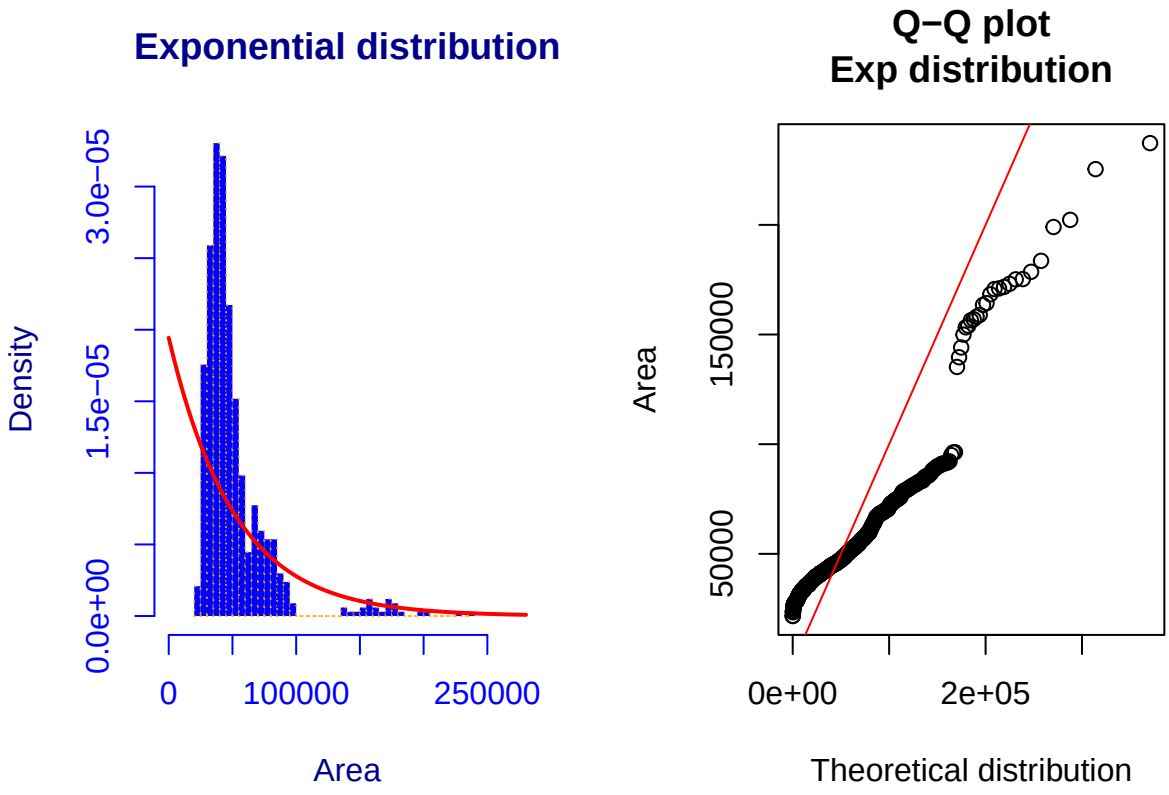
4.1.4 Exponential

```
par(mfrow = c(1,2))
Exp.N1 <- histDist(Area,family = EXP ,nbins = "fd",
                  main = "Exponential distribution",
                  col.hist="blue",border.hist="orange",line.col = "red"
                  ,ylab = "Density")

mu <- exp(Exp.N1$mu.coefficients)
```

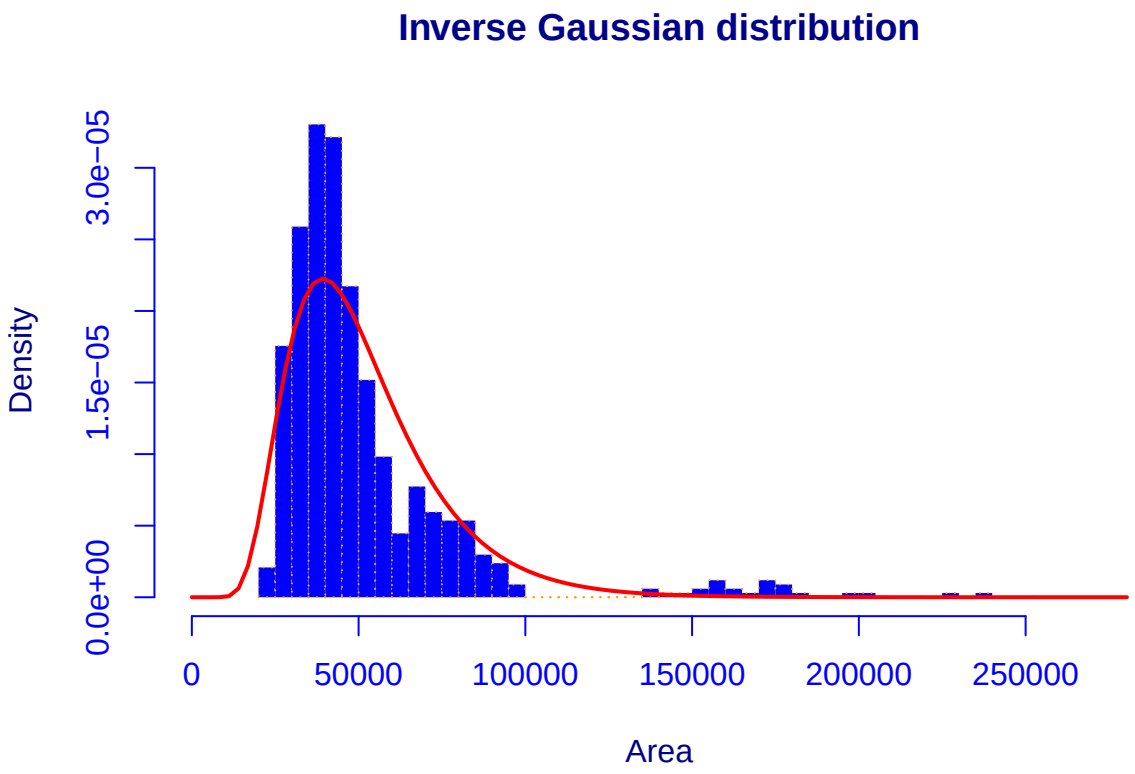
```
Exp.N1.Theoretical <- qEXP(theoA,mu= mu)

qqplot(x = Exp.N1.Theoretical,y = Area,
       xlab = "Theoretical distribution",
       main="Q-Q plot\nExp distribution")
abline(a = 0, b=1,col="red")
```



4.1.5 Inverse Gaussian

```
Ig.N1 <- histDist(Area,family = IG ,nbins = "fd",
                  main = "Inverse Gaussian distribution",
                  col.hist="blue",border.hist="orange",line.col = "red"
                  ,ylab = "Density")
```



In this case we can't use the Q-Q plot, because the variance is given by $\sigma^2\mu^3$, so R cannot generates easily value on that range.

4.1.6 Comparison

```
Area.comp <- data.frame(AIC=c(Log.N1$aic,Gamma.N1$aic,Weibull.N1$aic,
                              Exp.N1$aic,Ig.N1$aic),
                        BIC = c(Log.N1$sbc,Gamma.N1$sbc,Weibull.N1$sbc,
                                Exp.N1$sbc,Ig.N1$sbc),
                        row.names=c("Log-Normal","Gamma","Weibull",
                                    "Exponential","Inverse Gaussian"))
Area.comp <- Area.comp[order(Area.comp$AIC, decreasing = FALSE), ]
Area.comp
```

```
##                         AIC      BIC
## Log-Normal          15142.44 15151.46
## Inverse Gaussian    15151.53 15160.55
## Gamma               15282.53 15291.55
## Weibull             15488.96 15497.98
## Exponential         15925.97 15930.48
```

From the AIC and BIC criteria the better model that fits the data in terms of parsimony and accuracy is the Log-Normal distribution.

4.2 Perimeter

```
summary(Perimeter)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   531.3   692.4   781.9   841.6   935.6  1921.7
```

```
V<- var(Perimeter)
cat("Variance : ",V,"\n")
```

```
## Variance :  44364.6
```

```
Sd <- sd(Perimeter)
cat("Standard deviation : ",Sd,"\n")
```

```
## Standard deviation :  210.6291
```

```
Perimeter.table <- table(Perimeter)
mode <- names(Perimeter.table)[which.max(Perimeter.table)]
cat("Mode : ",mode,"\n")
```

```
## Mode :  796.996
```

```
K <- kurtosis(Perimeter)
cat("Kurtosis : ",K,"\n")
```

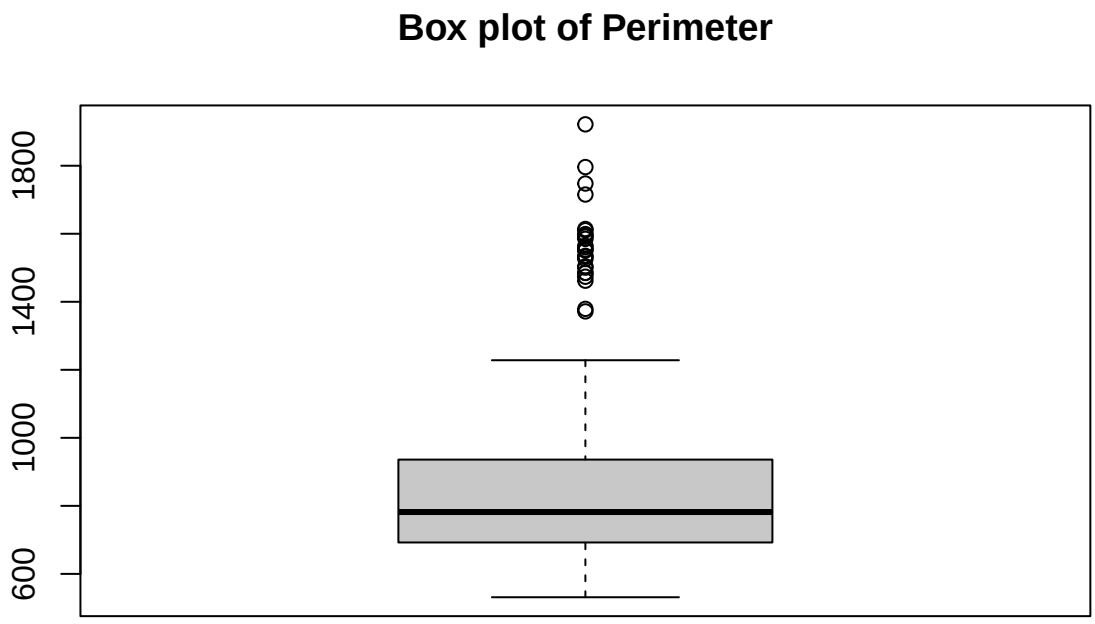
```
## Kurtosis :  6.991247
```

```
S <- skewness(Perimeter)
cat("Skewness : ",S,"\n")
```

```
## Skewness :  1.752217
```

The data are right-skewed, as observed from the skewness index. The high kurtosis indicates that the distribution is leptokurtic, meaning there is a great concentration of values around the mean, but also a high frequency of outliers that are far away from it.

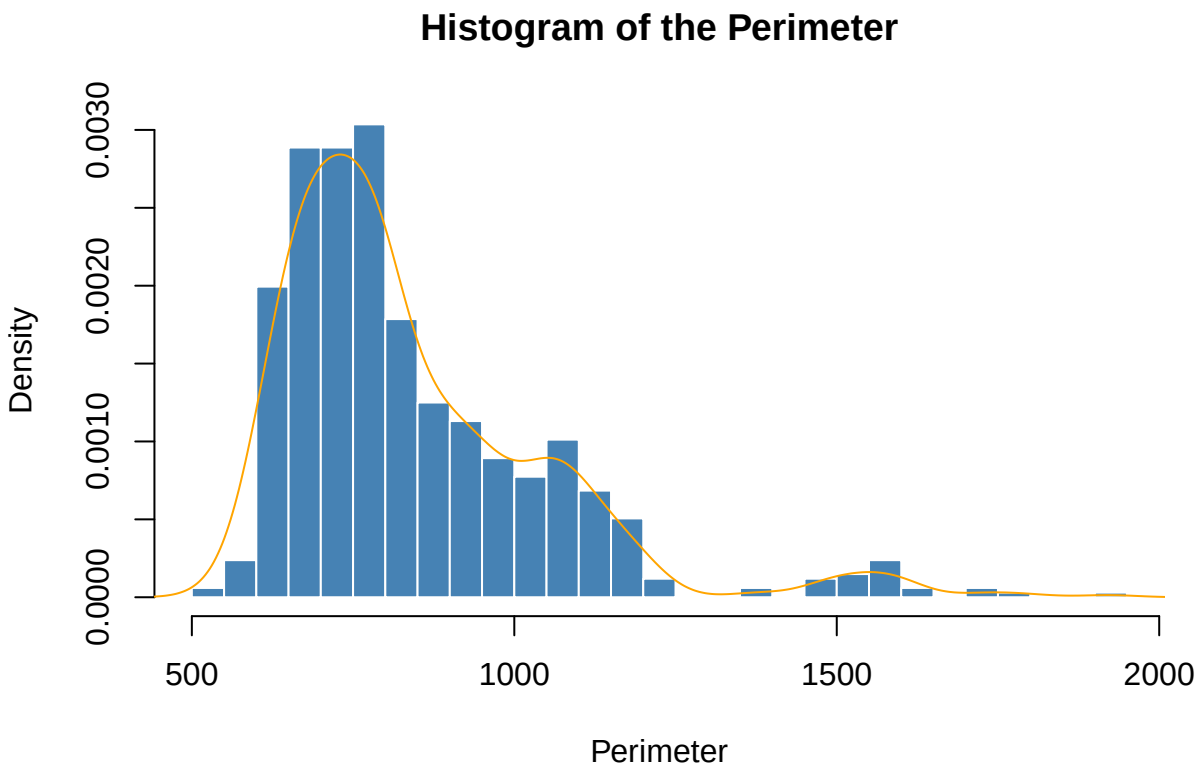
```
box <- boxplot(Perimeter,main="Box plot of Perimeter")
```




```
out <- length(box$out)/length(Perimeter)
cat("Percentage of outliers : ",out*100,"%")
```

```
## Percentage of outliers :  3.720238 %
```

```
hist(Perimeter,
     breaks = "FD",
     main = "Histogram of the Perimeter",
     col = "steelblue",
     border = "white",
     freq = FALSE)
lines(density(Perimeter),col = "orange")
```



Now we can try to fit different models to the data. First of all we observe that the support of the data is $[0, +Inf[$. So the distribution from which we could choose are:

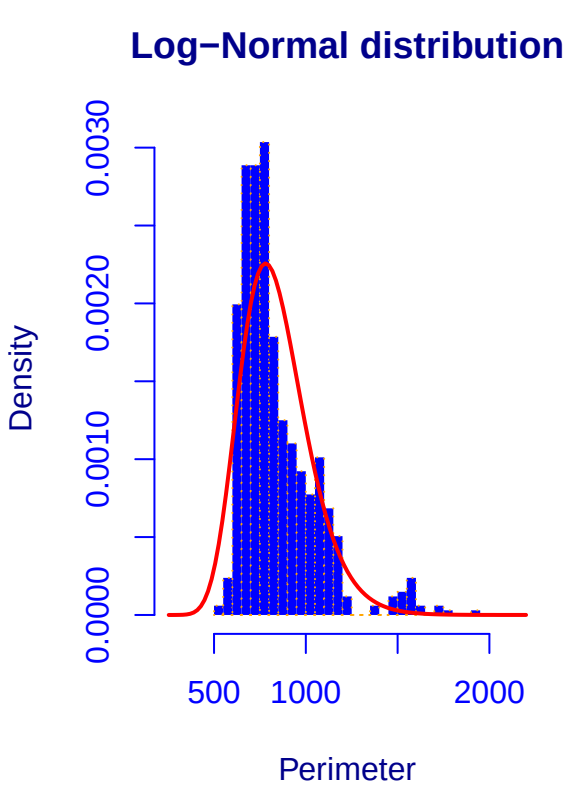
- **Log-Normal**
- **Gamma**
- **Weibull**
- **Exponential**
- **Inverse Gaussian**

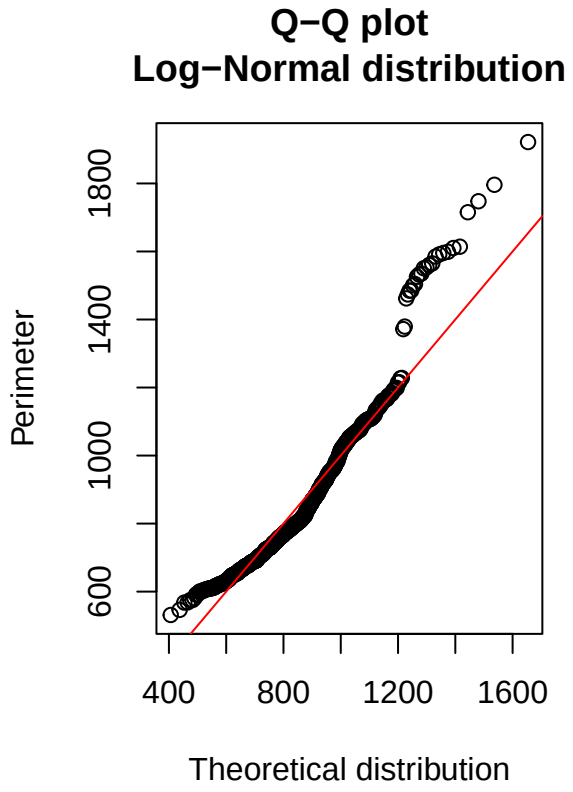
4.2.1 Log-Normal

```
par(mfrow = c(1,2))
Log.N2 <- histDist(Perimeter,family = LOGNO,nbins = "fd",
                   main = "Log-Normal distribution",
                   col.hist="blue",border.hist="orange",line.col = "red"
                   ,ylab = "Density")

theoP <- ppoints(length(Perimeter))
mu <- Log.N2$mu.coefficients
sigma <- exp(Log.N2$sigma.coefficients)
Log.N2.Theoretical <- qLOGNO(theoP ,mu= mu, sigma = sigma)
```

```
qqplot(x = Log.N2.Theoretical,y = Perimeter,
       xlab = "Theoretical distribution",
       main="Q-Q plot\nLog-Normal distribution")
abline(a = 0, b=1,col="red")
```



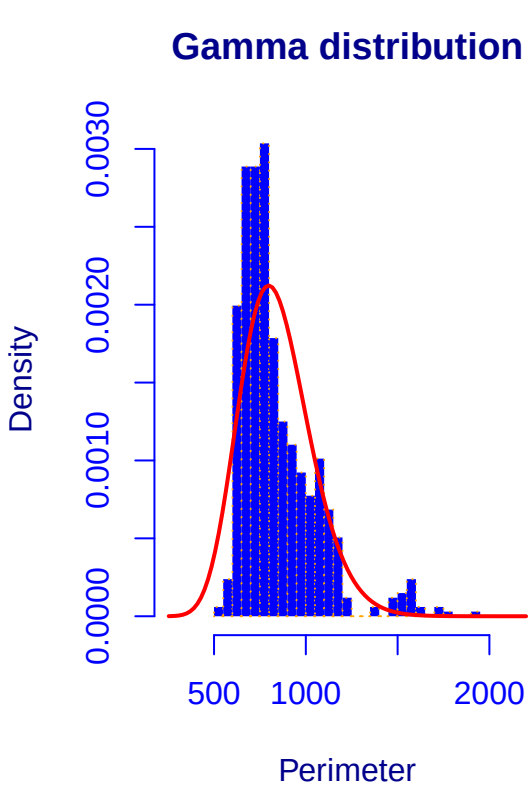


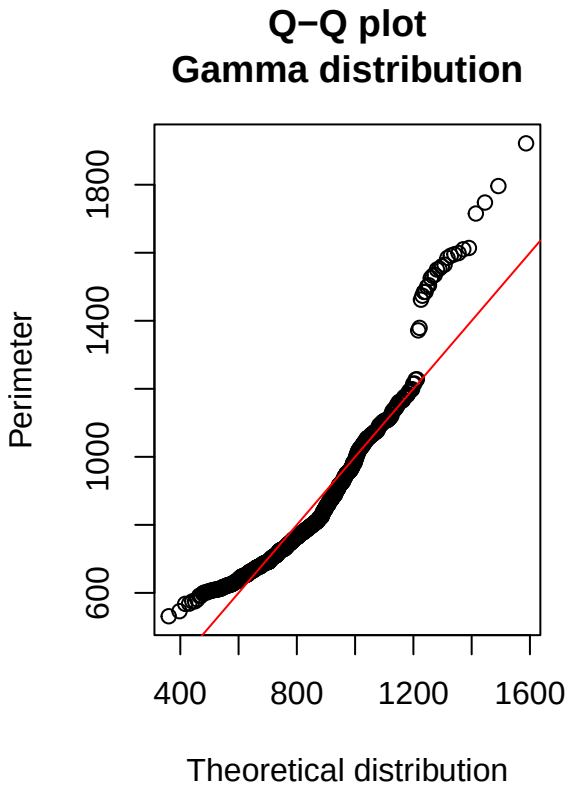
4.2.2 Gamma

```
par(mfrow = c(1,2))
Gamma.N2 <- histDist(Perimeter,family = GA ,nbins = "fd",
                     main = "Gamma distribution",
                     col.hist="blue",border.hist="orange",line.col = "red"
                     ,ylab = "Density")

mu <- exp(Gamma.N2$mu.coefficients)
sigma <- exp(Gamma.N2$sigma.coefficients)
Gamma.N2.Theoretical <- qGA(theoP,mu= mu, sigma = sigma)

qqplot(x = Gamma.N2.Theoretical,y = Perimeter,
       xlab = "Theoretical distribution",
       main="Q-Q plot\nGamma distribution")
abline(a = 0, b=1,col="red")
```



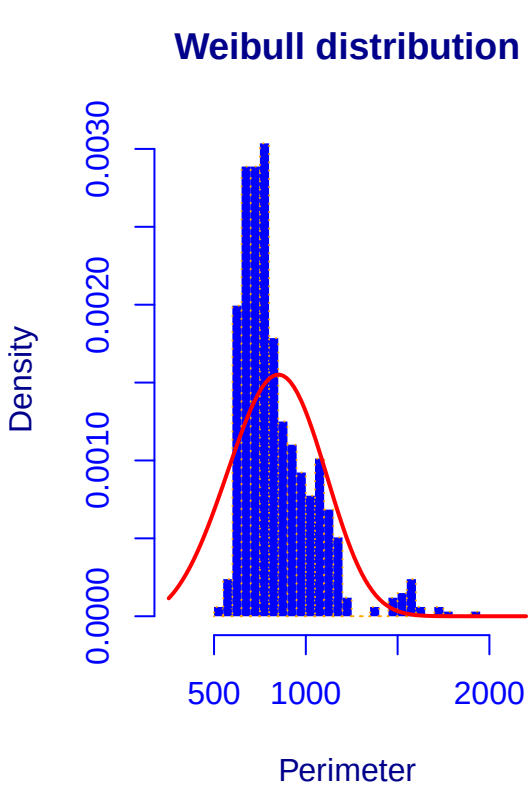


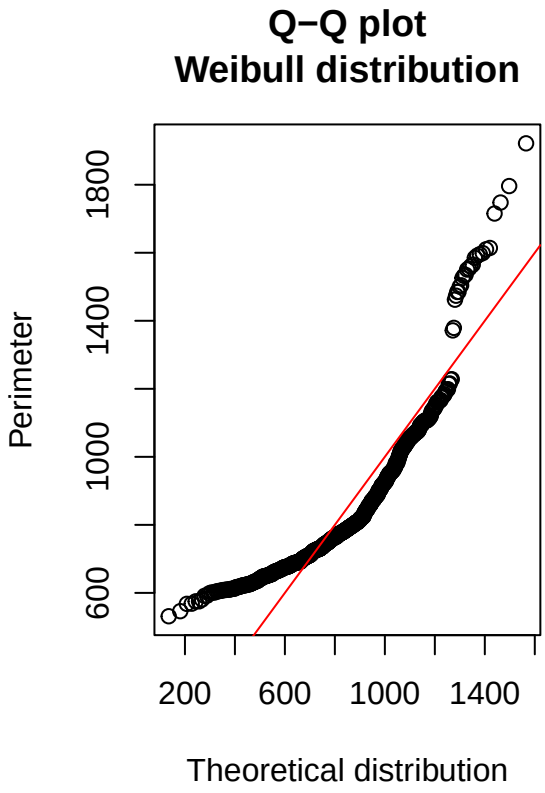
4.2.3 Weibull

```
par(mfrow = c(1,2))
Weibull.N2 <- histDist(Perimeter,family = WEI ,nbins = "fd",
                  main = "Weibull distribution",
                  col.hist="blue",border.hist="orange",line.col = "red"
                  ,ylab = "Density")

mu <- exp(Weibull.N2$mu.coefficients)
sigma <- exp(Weibull.N2$sigma.coefficients)
Weibull.N2.Theoretical <- qWEI(theoP,mu= mu, sigma = sigma)

qqplot(x = Weibull.N2.Theoretical,y = Perimeter,
       xlab = "Theoretical distribution",
       main="Q-Q plot\nWeibull distribution")
abline(a = 0, b=1,col="red")
```



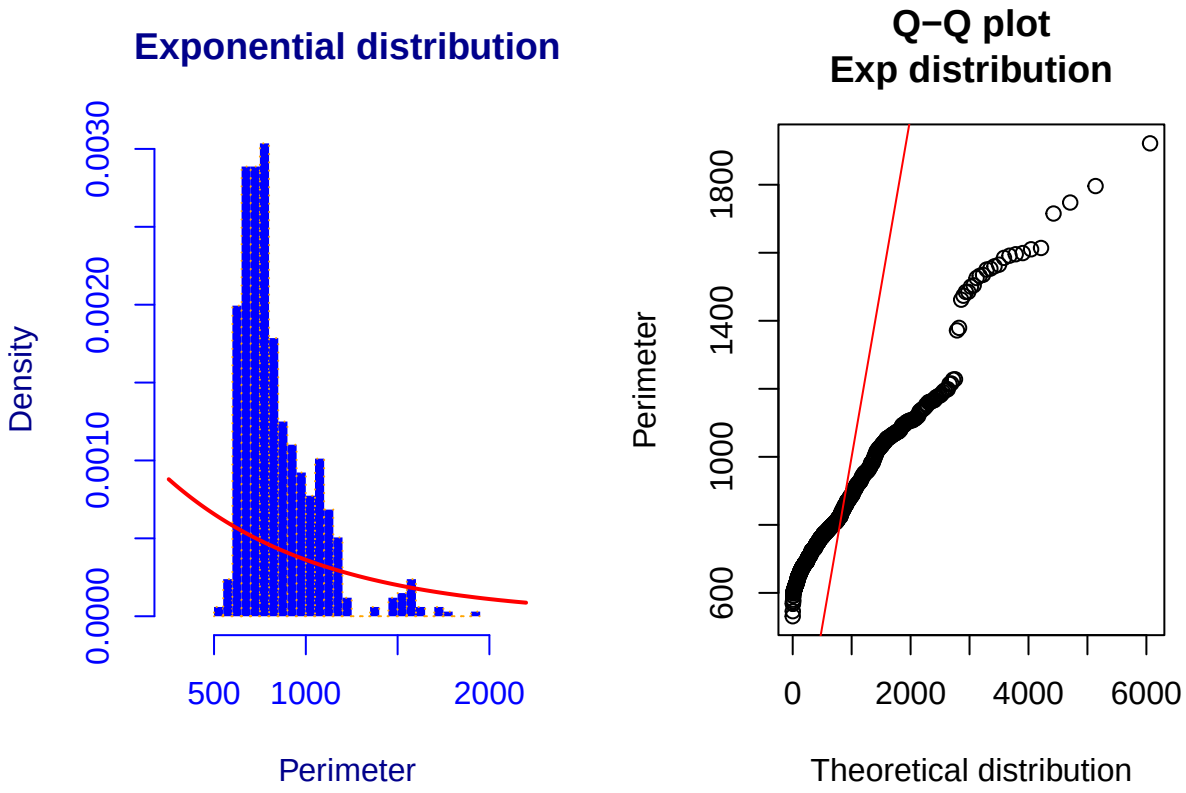


4.2.4 Exponential

```
par(mfrow = c(1,2))
Exp.N2 <- histDist(Perimeter,family = EXP ,nbins = "fd",
                   main = "Exponential distribution",
                   col.hist="blue",border.hist="orange",line.col = "red"
                   ,ylab = "Density")

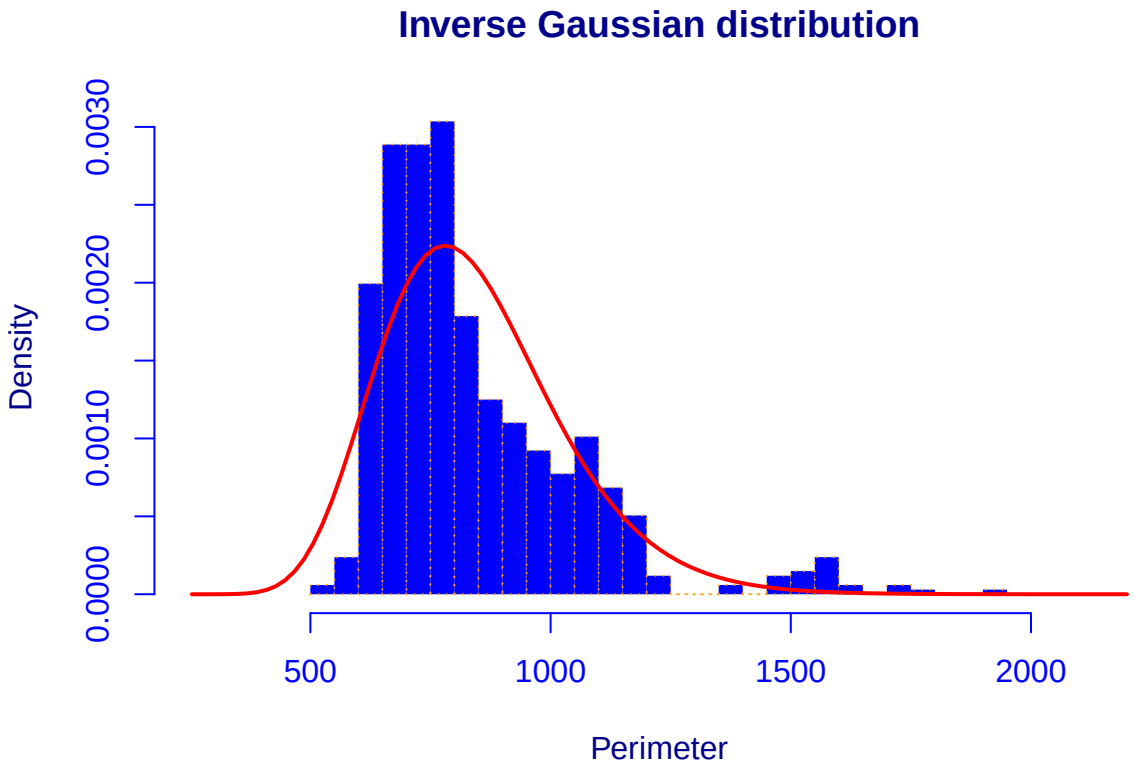
mu <- exp(Exp.N2$mu.coefficients)
Exp.N2.Theoretical <- qEXP(theoP,mu= mu)

qqplot(x = Exp.N2.Theoretical,y = Perimeter,
       xlab = "Theoretical distribution",
       main="Q-Q plot\nExp distribution")
abline(a = 0, b=1,col="red")
```



4.2.5 Inverse Gaussian

```
Ig.N2 <- histDist(Perimeter,family = IG ,nbins = "fd",
                  main = "Inverse Gaussian distribution",
                  col.hist="blue",border.hist="orange",line.col = "red"
                  ,ylab = "Density")
```



In this case we can't use the Q-Q plot, because the variance is given by $\sigma^2\mu^3$, so R cannot generates easily value on that range.

4.2.6 Comparison

```
Perimeter.comp <- data.frame(AIC=c(Log.N2$aic,Gamma.N2$aic,Weibull.N2$aic,
                                   Exp.N2$aic,Ig.N2$aic),
                     BIC = c(Log.N2$sbc,Gamma.N2$sbc,Weibull.N2$sbc,
                             Exp.N2$sbc,Ig.N2$sbc),
                     row.names=c("Log-Normal","Gamma","Weibull",
                                 "Exponential","Inverse Gaussian"))
Perimeter.comp <- Perimeter.comp[order(Perimeter.comp$AIC, decreasing = FALSE), ]
Perimeter.comp
```

```
##                         AIC       BIC
## Log-Normal          8898.723  8907.743
## Inverse Gaussian    8900.211  8909.232
## Gamma               8954.229  8963.250
## Weibull             9191.999  9201.019
## Exponential        10398.286 10402.797
```

From the AIC and BIC criteria the better model that fits the data in terms of parsimony and accuracy is the Log-Normal distribution.

4.3 MajorAxisLength

```
summary(MajorAxisLength)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   187.2   251.9   291.8   315.0   366.6   738.1
```

```
V<- var(MajorAxisLength)
cat("Variance : ",V,"\n")
```

```
## Variance :  7068.067
```

```
Sd <- sd(MajorAxisLength)
cat("Standard deviation : ",Sd,"\n")
```

```
## Standard deviation :  84.07179
```

```
MajorAxisLength.table <- table(MajorAxisLength)
mode <- names(MajorAxisLength.table)[which.max(MajorAxisLength.table)]
cat("Mode : ",mode,"\n")
```

```
## Mode :  187.1686348
```

```
K <- kurtosis(MajorAxisLength)
cat("Kurtosis : ",K,"\n")
```

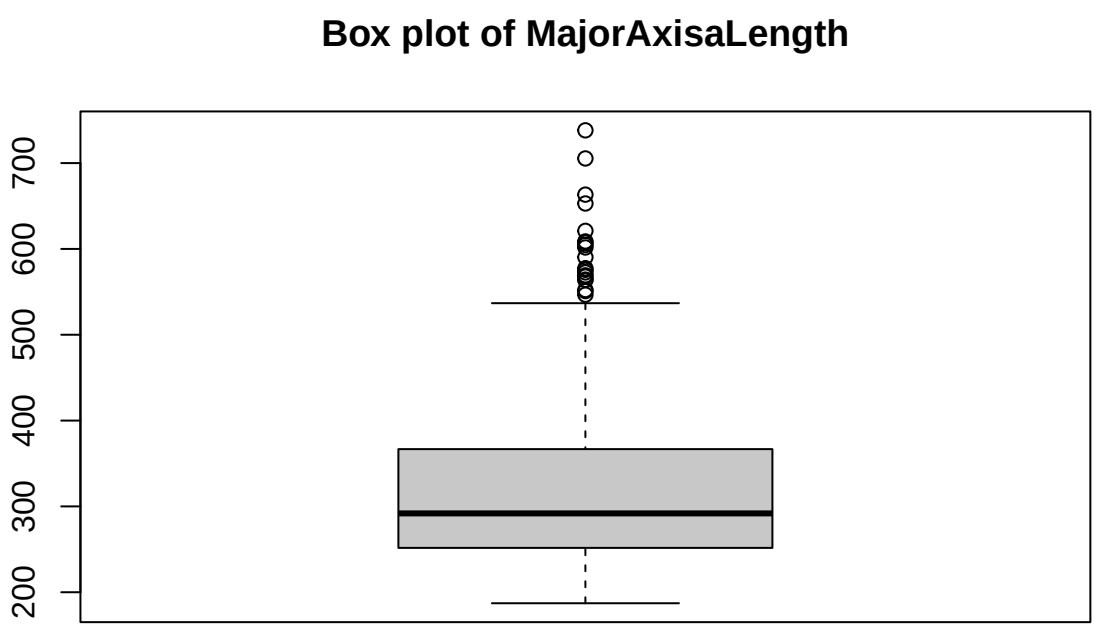
```
## Kurtosis :  5.89532
```

```
S <- skewness(MajorAxisLength)
cat("Skewness : ",S,"\n")
```

```
## Skewness :  1.457575
```

The data are right-skewed, as observed from the skewness index. The high kurtosis indicates that the distribution is leptokurtic, meaning there is a great concentration of values around the mean, but also a high frequency of outliers that are far away from it.

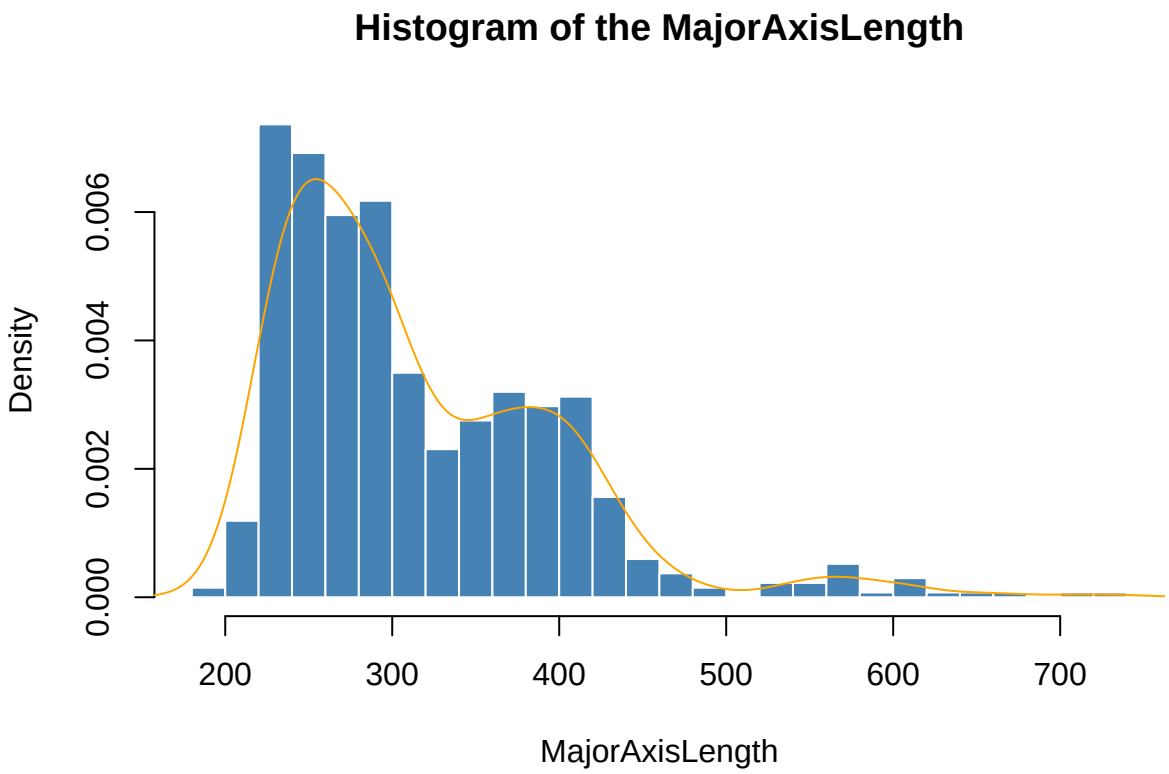
```
box <- boxplot(MajorAxisLength,main="Box plot of MajorAxisaLength")
```




```
out <- length(box$out)/length(MajorAxisLength)
cat("Percentage of outliers : ",out*100,"%")
```

```
## Percentage of outliers :  2.97619 %
```

```
hist(MajorAxisLength,
     breaks = "FD",
     main = "Histogram of the MajorAxisLength",
     col = "steelblue",
     border = "white",
     freq = FALSE)
lines(density(MajorAxisLength),col = "orange")
```



Now we can try to fit different models to the data. First of all we observe that the support of the data is $[0, +Inf[$. So the distribution from which we could choose are:

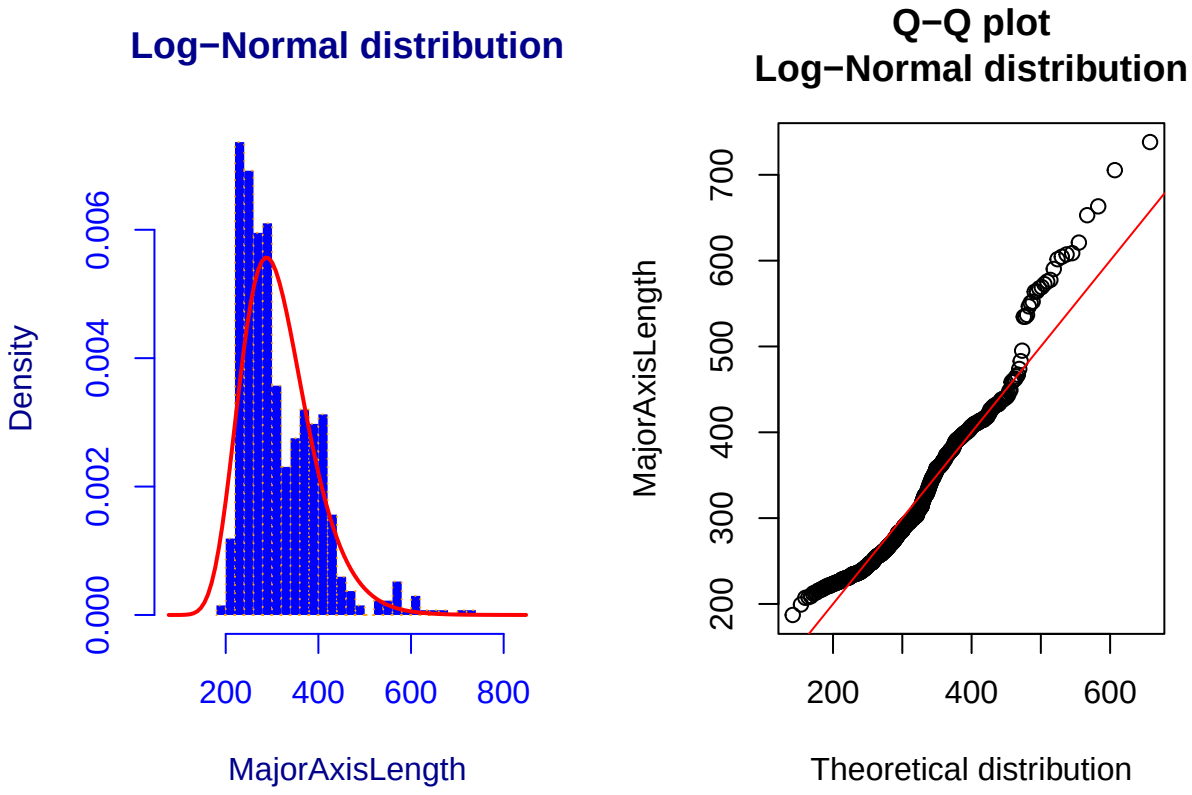
- **Log-Normal**
- **Gamma**
- **Weibull**
- **Exponential**
- **Inverse Gaussian**

4.3.1 Log-Normal

```
par(mfrow = c(1,2))
Log.N3 <- histDist(MajorAxisLength,family = LOGNO,nbins = "fd",
                   main = "Log-Normal distribution",
                   col.hist="blue",border.hist="orange",line.col = "red"
                   ,ylab = "Density")

theoM <- ppoints(length(MajorAxisLength))
mu <- Log.N3$mu.coefficients
sigma <- exp(Log.N3$sigma.coefficients)
Log.N3.Theoretical <- qLOGNO(theoM,mu= mu, sigma = sigma)
```

```
qqplot(x = Log.N3.Theoretical,y = MajorAxisLength,
       xlab = "Theoretical distribution",
       main="Q-Q plot\nLog-Normal distribution")
abline(a = 0, b=1,col="red")
```

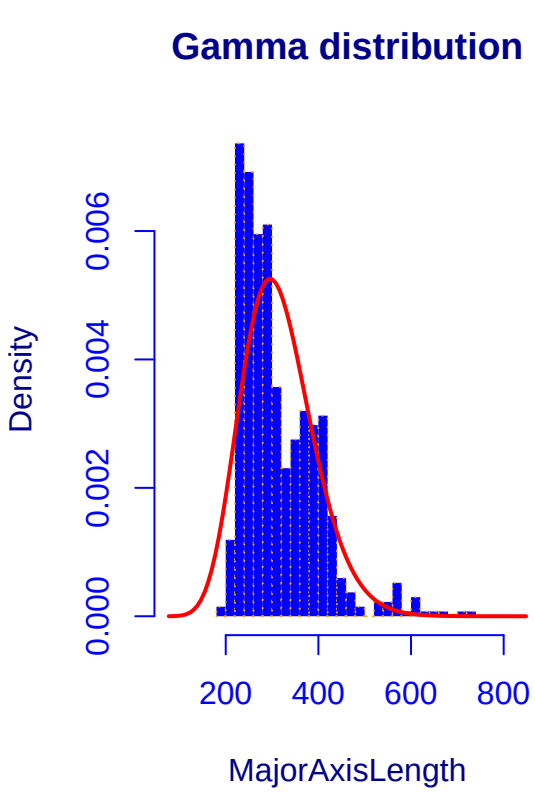


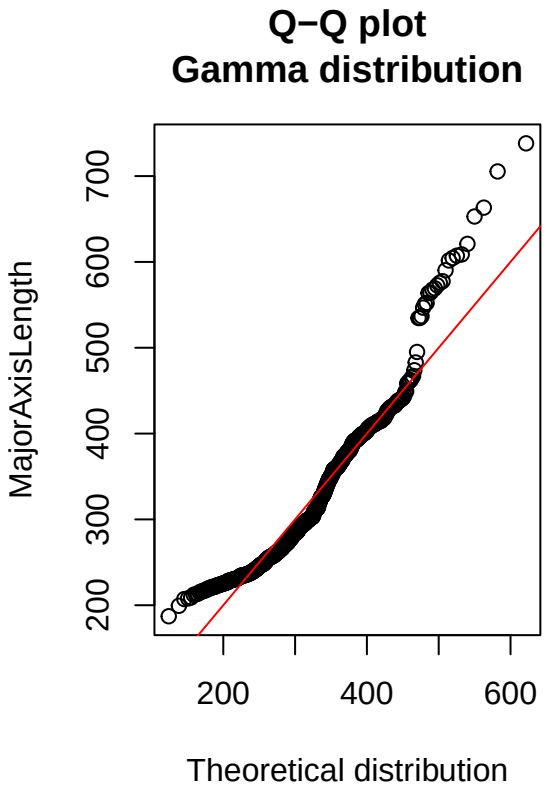
4.3.2 Gamma

```
par(mfrow = c(1,2))
Gamma.N3 <- histDist(MajorAxisLength,family = GA ,nbins = "fd",
                     main = "Gamma distribution",
                     col.hist="blue",border.hist="orange",line.col = "red"
                     ,ylab = "Density")

mu <- exp(Gamma.N3$mu.coefficients)
sigma <- exp(Gamma.N3$sigma.coefficients)
Gamma.N3.Theoretical <- qGA(theoM,mu= mu, sigma = sigma)

qqplot(x = Gamma.N3.Theoretical,y = MajorAxisLength,
       xlab = "Theoretical distribution",
       main="Q-Q plot\nGamma distribution")
abline(a = 0, b=1,col="red")
```



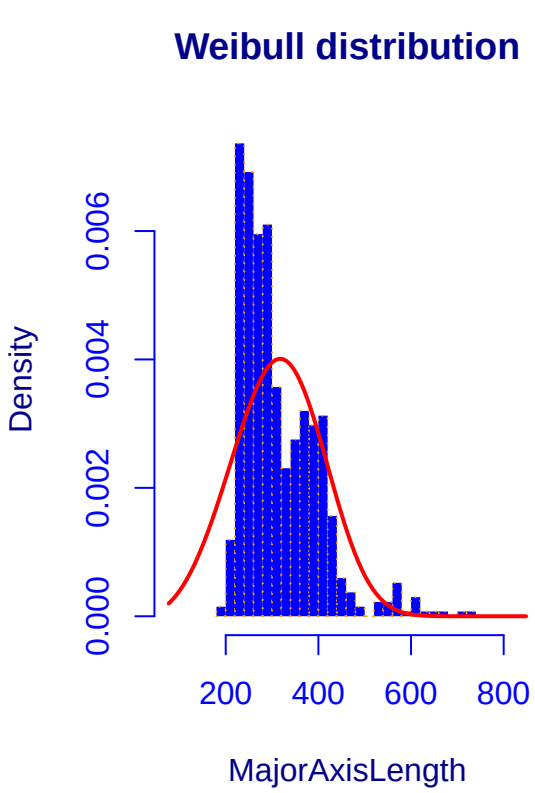


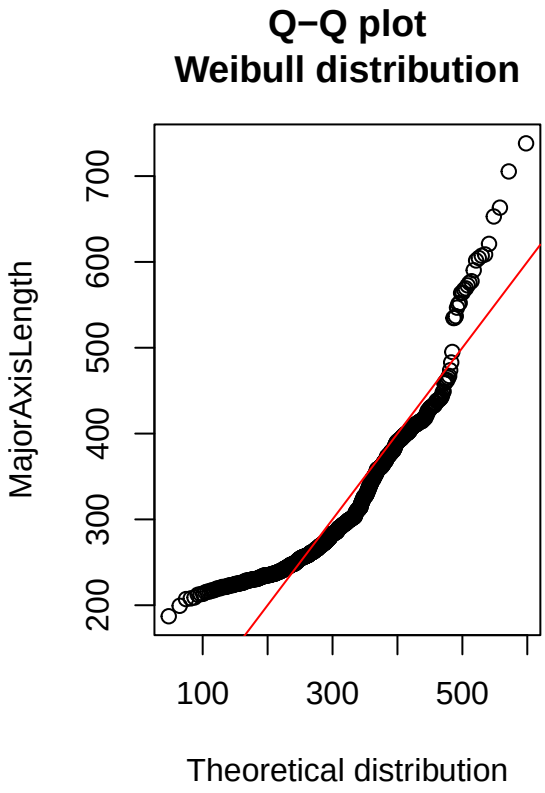
4.3.3 Weibull

```
par(mfrow = c(1,2))
Weibull.N3 <- histDist(MajorAxisLength,family = WEI ,nbins = "fd",
                  main = "Weibull distribution",
                  col.hist="blue",border.hist="orange",line.col = "red"
                  ,ylab = "Density")

mu <- exp(Weibull.N3$mu.coefficients)
sigma <- exp(Weibull.N3$sigma.coefficients)
Weibull.N3.Theoretical <- qWEI(theoM,mu= mu, sigma = sigma)

qqplot(x = Weibull.N3.Theoretical,y = MajorAxisLength,
       xlab = "Theoretical distribution",
       main="Q-Q plot\nWeibull distribution")
abline(a = 0, b=1,col="red")
```



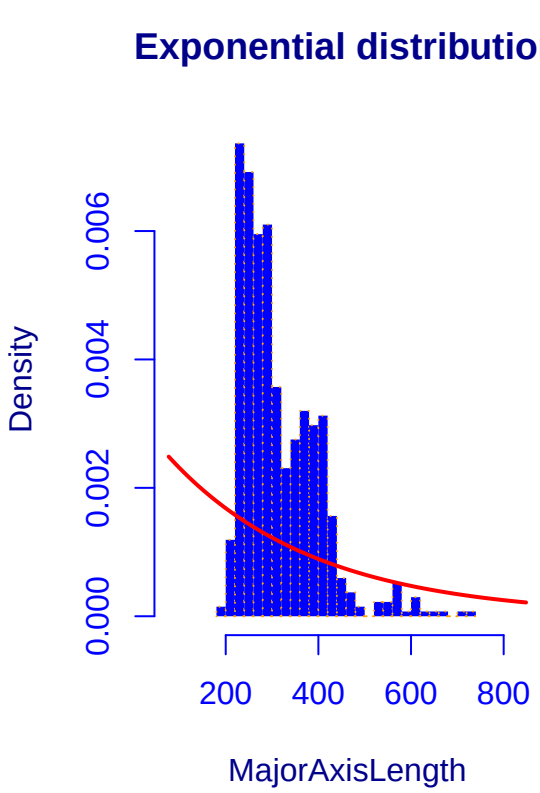


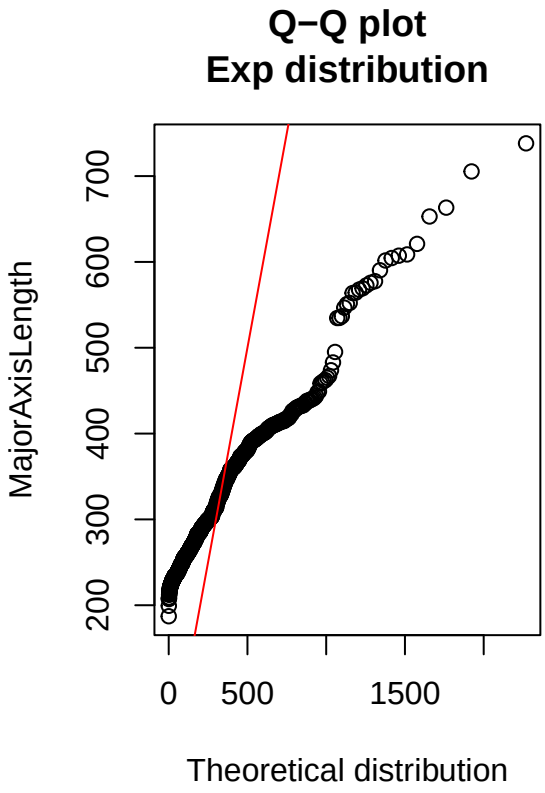
4.3.4 Exponential

```
par(mfrow = c(1,2))
Exp.N3 <- histDist(MajorAxisLength,family = EXP ,nbins = "fd",
                   main = "Exponential distribution",
                   col.hist="blue",border.hist="orange",line.col = "red"
                   ,ylab = "Density")

mu <- exp(Exp.N3$mu.coefficients)
Exp.N3.Theoretical <- qEXP(theoM,mu= mu)

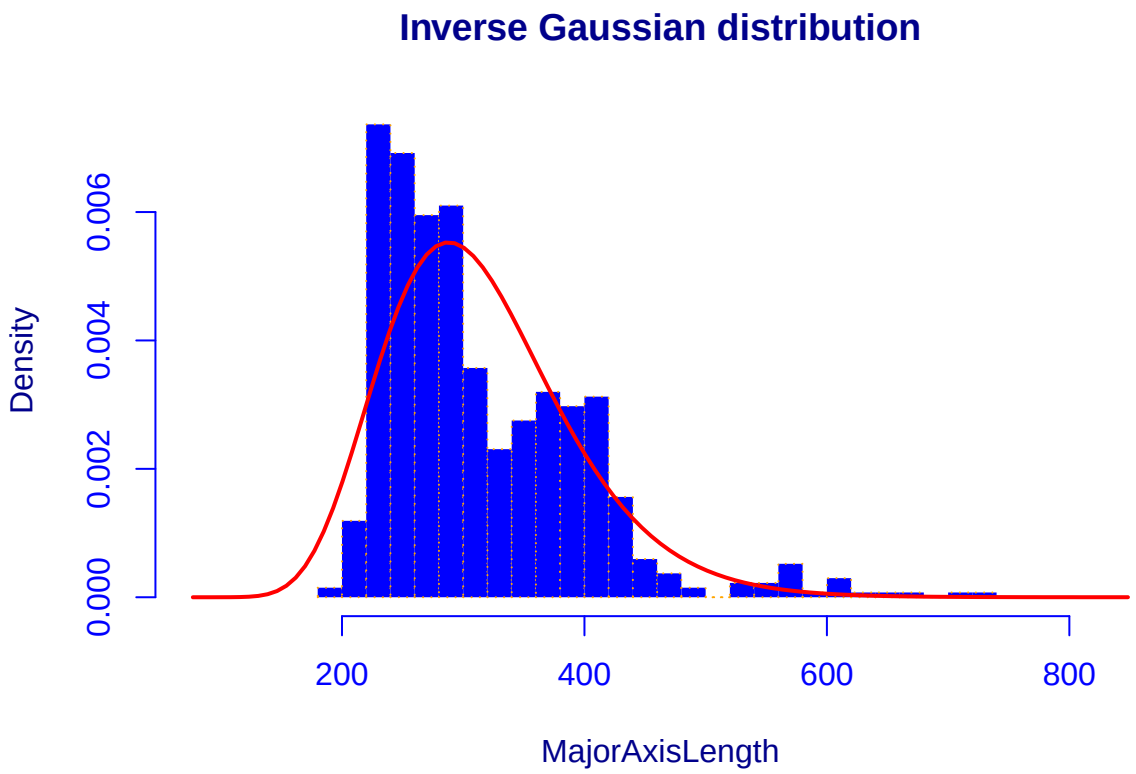
qqplot(x = Exp.N3.Theoretical,y = MajorAxisLength,
       xlab = "Theoretical distribution",
       main="Q-Q plot\nExp distribution")
abline(a = 0, b=1,col="red")
```





4.3.5 Inverse Gaussian

```
Ig.N3 <- histDist(MajorAxisLength,family = IG ,nbins = "fd",
                  main = "Inverse Gaussian distribution",
                  col.hist="blue",border.hist="orange",line.col = "red"
                  ,ylab = "Density")
```



In this case we can't use the Q-Q plot, because the variance is given by $\sigma^2\mu^3$, so R cannot generates easily value on that range.

4.3.6 Comparison

```
MajorAxisLength.comp <- data.frame(AIC=c(Log.N3$aic,Gamma.N3$aic,Weibull.N3$aic,
                                         Exp.N3$aic,Ig.N3$aic),
                       BIC = c(Log.N3$sbc,Gamma.N3$sbc,Weibull.N3$sbc,
                               Exp.N3$sbc,Ig.N3$sbc),
                       row.names=c("Log-Normal","Gamma","Weibull",
                                   "Exponential","Inverse Gaussian"))
MajorAxisLength.comp <- MajorAxisLength.comp[order(MajorAxisLength.comp$AIC,
                                                   decreasing = FALSE), ]
MajorAxisLength.comp
```

```
##                         AIC      BIC
## Inverse Gaussian 7690.400 7699.420
## Log-Normal       7690.908 7699.928
## Gamma            7736.719 7745.740
## Weibull          7929.750 7938.770
## Exponential      9077.449 9081.959
```

From the AIC and BIC criteria the better model that fits the data in terms of parsimony

and accuracy is the Inverse Gaussian distribution.

4.4 MinorAxisLength

```
summary(MinorAxisLength)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   137.0   173.5   189.3   199.4   209.2   435.8
```

```
V<- var(MinorAxisLength)
cat("Variance : ",V,"\n")
```

```
## Variance :  1977.452
```

```
Sd <- sd(MinorAxisLength)
cat("Standard deviation : ",Sd,"\n")
```

```
## Standard deviation :  44.46855
```

```
MinorAxisLength.table <- table(MinorAxisLength)
mode <- names(MinorAxisLength.table)[which.max(MinorAxisLength.table)]
cat("Mode : ",mode,"\n")
```

```
## Mode :  136.9516069
```

```
K <- kurtosis(MinorAxisLength)
cat("Kurtosis : ",K,"\n")
```

```
## Kurtosis :  10.23167
```

```
S <- skewness(MinorAxisLength)
cat("Skewness : ",S,"\n")
```

```
## Skewness :  2.390908
```

The data are right-skewed, as observed from the skewness index. The high kurtosis indicates that the distribution is leptokurtic, meaning there is a great concentration of values around the mean, but also a high frequency of outliers that are far away from it.

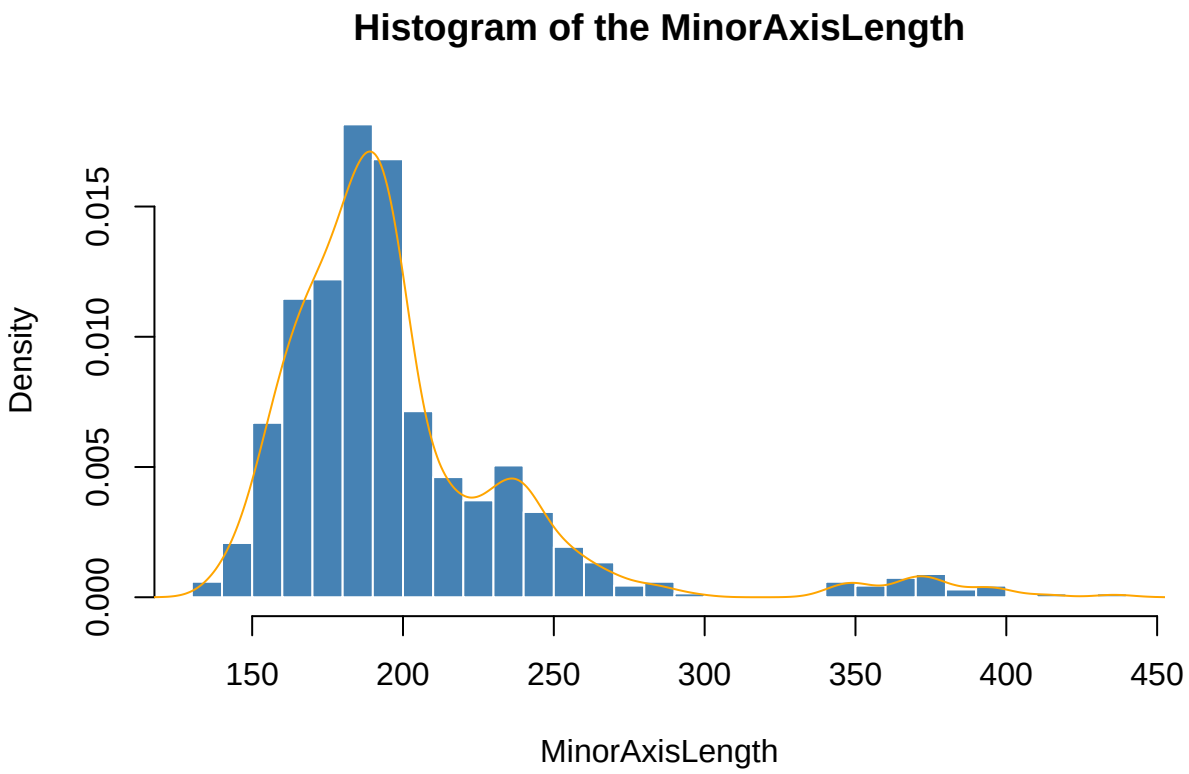
```
box <- boxplot(MinorAxisLength,main="Box plot of MinorAxisLength")
```




```
out <- length(box$out)/length(MinorAxisLength)
cat("Percentage of outliers : ",out*100,"%")
```

```
## Percentage of outliers :  5.803571 %
```

```
hist(MinorAxisLength,
     breaks = "FD",
     main = "Histogram of the MinorAxisLength",
     col = "steelblue",
     border = "white",
     freq = FALSE)
lines(density(MinorAxisLength),col = "orange")
```



Now we can try to fit different models to the data. First of all we observe that the support of the data is $[0, +Inf[$. So the distribution from which we could choose are:

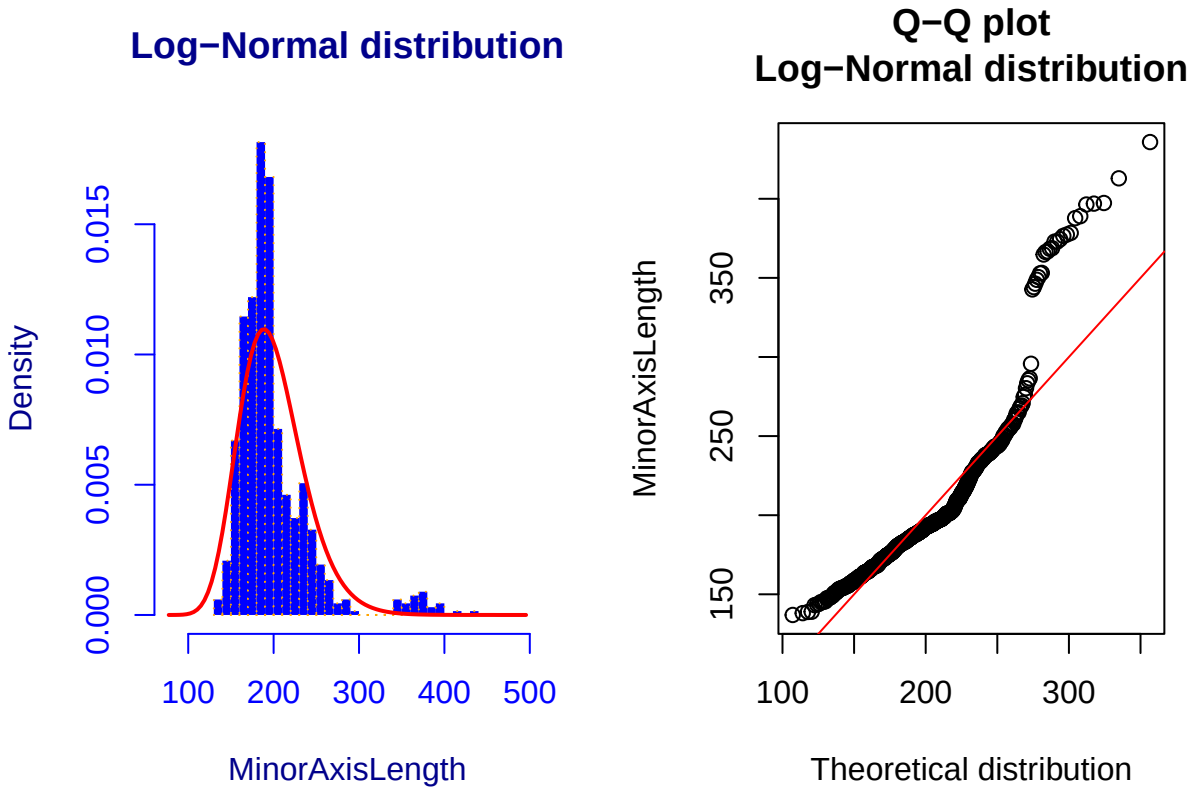
- **Log-Normal**
- **Gamma**
- **Weibull**
- **Exponential**
- **Inverse Gaussian**

4.4.1 Log-Normal

```
par(mfrow = c(1,2))
Log.N4 <- histDist(MinorAxisLength,family = LOGNO,nbins = "fd",
                   main = "Log-Normal distribution",
                   col.hist="blue",border.hist="orange",line.col = "red"
                   ,ylab = "Density")

theom <- ppoints(length(MinorAxisLength))
mu <- Log.N4$mu.coefficients
sigma <- exp(Log.N4$sigma.coefficients)
Log.N4.Theoretical <- qLOGNO(theom,mu= mu, sigma = sigma)
```

```
qqplot(x = Log.N4.Theoretical,y = MinorAxisLength,
       xlab = "Theoretical distribution",
       main="Q-Q plot\nLog-Normal distribution")
abline(a = 0, b=1,col="red")
```

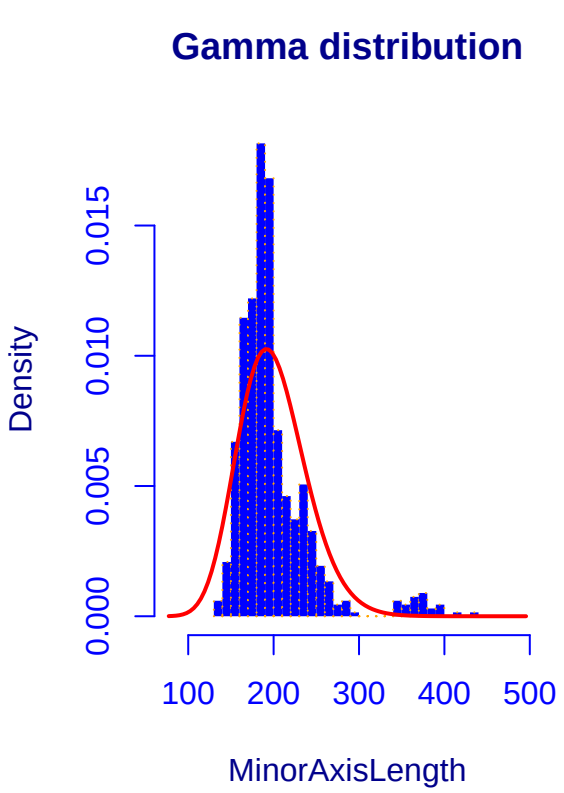


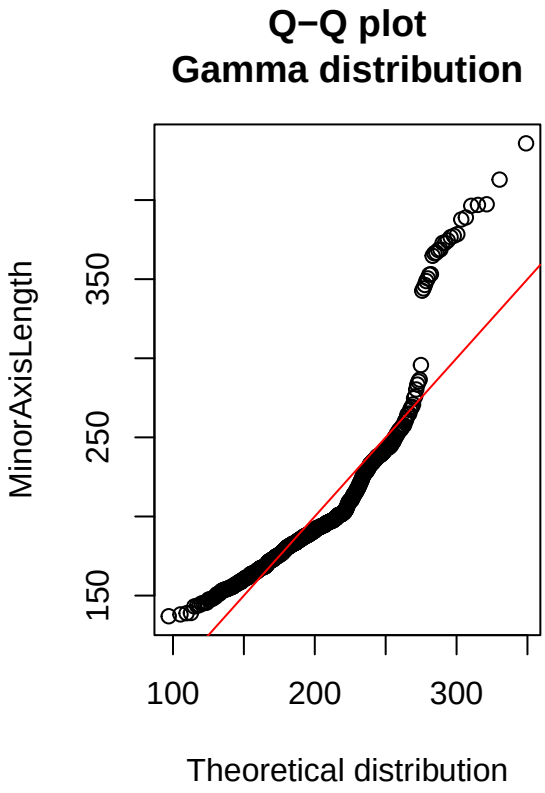
4.4.2 Gamma

```
par(mfrow = c(1,2))
Gamma.N4 <- histDist(MinorAxisLength,family = GA ,nbins = "fd",
                     main = "Gamma distribution",
                     col.hist="blue",border.hist="orange",line.col = "red"
                     ,ylab = "Density")

mu <- exp(Gamma.N4$mu.coefficients)
sigma <- exp(Gamma.N4$sigma.coefficients)
Gamma.N4.Theoretical <- qGA(theom,mu= mu, sigma = sigma)

qqplot(x = Gamma.N4.Theoretical,y = MinorAxisLength,
       xlab = "Theoretical distribution",
       main="Q-Q plot\nGamma distribution")
abline(a = 0, b=1,col="red")
```



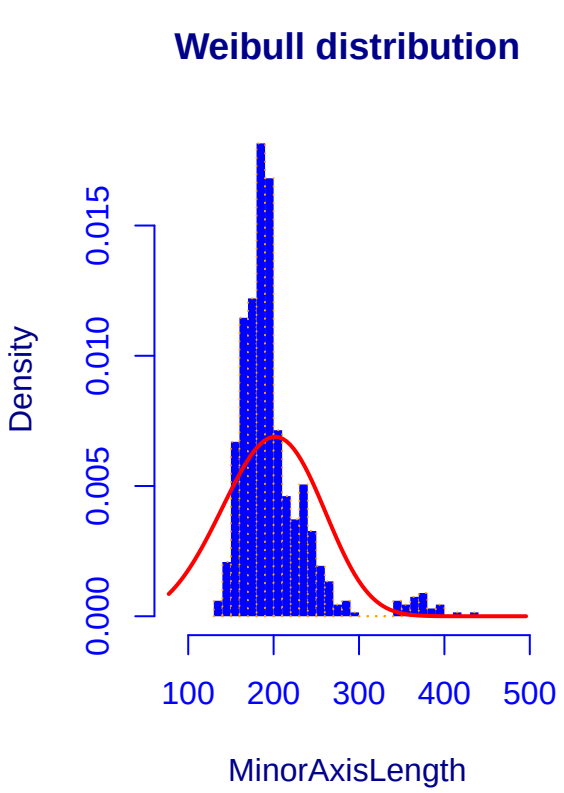


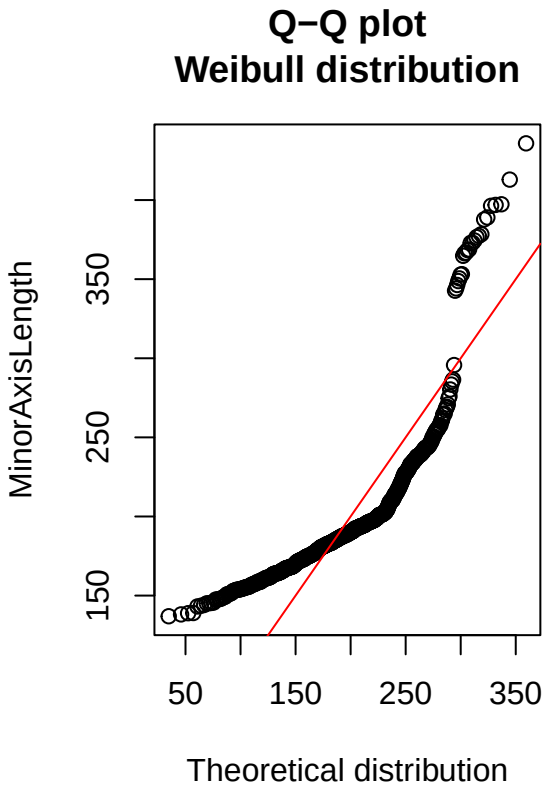
4.4.3 Weibull

```
par(mfrow = c(1,2))
Weibull.N4 <- histDist(MinorAxisLength,family = WEI ,nbins = "fd",
                  main = "Weibull distribution",
                  col.hist="blue",border.hist="orange",line.col = "red"
                  ,ylab = "Density")

mu <- exp(Weibull.N4$mu.coefficients)
sigma <- exp(Weibull.N4$sigma.coefficients)
Weibull.N4.Theoretical <- qWEI(theom,mu= mu, sigma = sigma)

qqplot(x = Weibull.N4.Theoretical,y = MinorAxisLength,
       xlab = "Theoretical distribution",
       main="Q-Q plot\nWeibull distribution")
abline(a = 0, b=1,col="red")
```



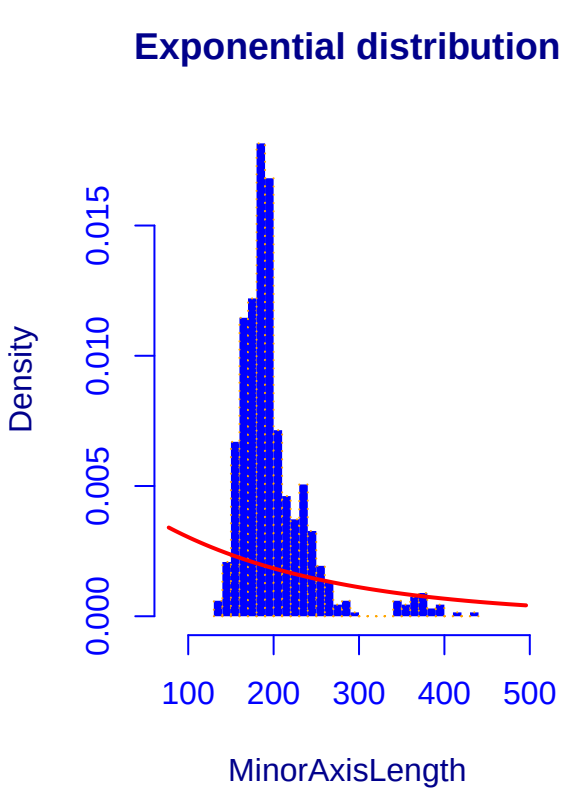


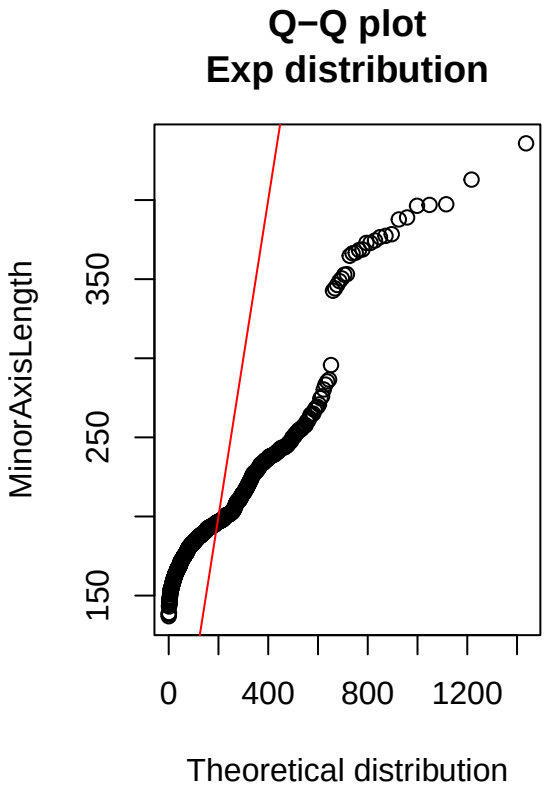
4.4.4 Exponential

```
par(mfrow = c(1,2))
Exp.N4 <- histDist(MinorAxisLength,family = EXP ,nbins = "fd",
                   main = "Exponential distribution",
                   col.hist="blue",border.hist="orange",line.col = "red"
                   ,ylab = "Density")

mu <- exp(Exp.N4$mu.coefficients)
Exp.N4.Theoretical <- qEXP(theom,mu= mu)

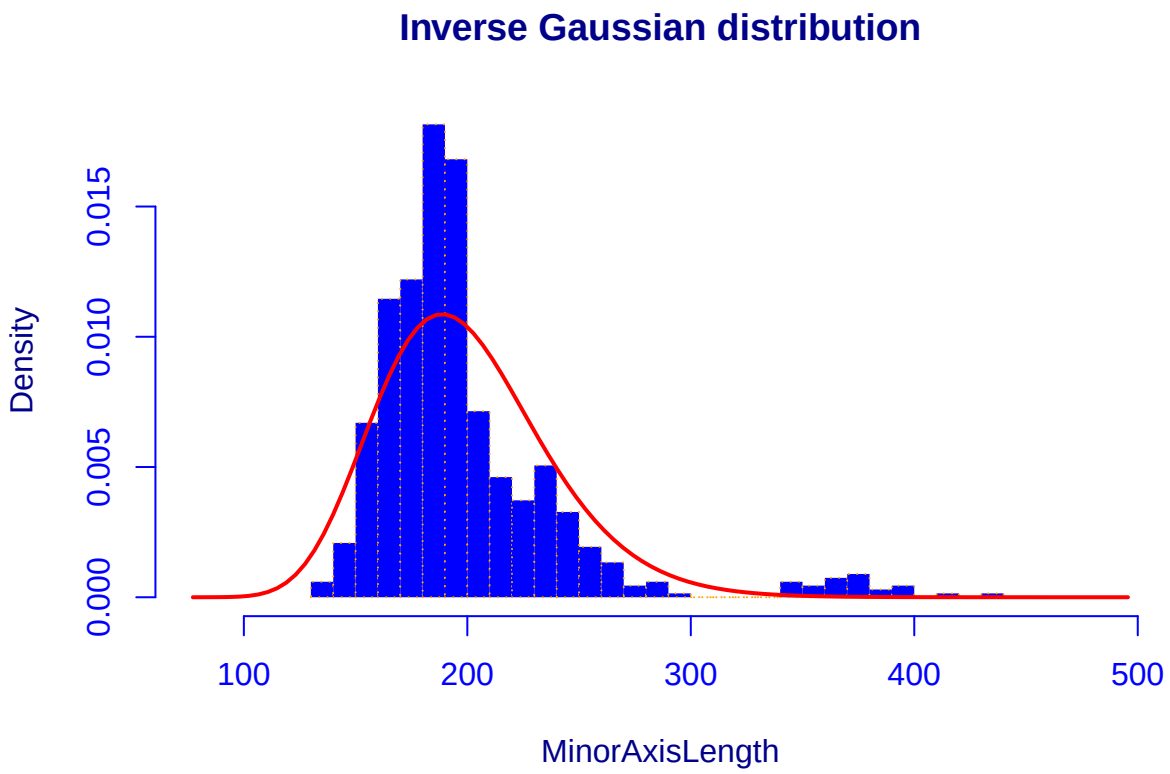
qqplot(x = Exp.N4.Theoretical,y = MinorAxisLength,
       xlab = "Theoretical distribution",
       main="Q-Q plot\nExp distribution")
abline(a = 0, b=1,col="red")
```





4.4.5 Inverse Gaussian

```
Ig.N4 <- histDist(MinorAxisLength,family = IG ,nbins = "fd",
                  main = "Inverse Gaussian distribution",
                  col.hist="blue",border.hist="orange",line.col = "red"
                  ,ylab = "Density")
```



In this case we can't use the Q-Q plot, because the variance is given by $\sigma^2\mu^3$, so R cannot generates easily value on that range.

4.4.6 Comparison

```
MinorAxisLength.comp <- data.frame(AIC=c(Log.N4$aic,Gamma.N4$aic,Weibull.N4$aic,
                                         Exp.N4$aic,Ig.N4$aic),
                       BIC = c(Log.N4$sbc,Gamma.N4$sbc,Weibull.N4$sbc,
                               Exp.N4$sbc,Ig.N4$sbc),
                       row.names=c("Log-Normal","Gamma","Weibull",
                                   "Exponential","Inverse Gaussian"))
MinorAxisLength.comp <- MinorAxisLength.comp[order(MinorAxisLength.comp$AIC,
                                                   decreasing = FALSE), ]
MinorAxisLength.comp
```

```
##                         AIC      BIC
## Log-Normal          6764.067 6773.087
## Inverse Gaussian    6769.292 6778.313
## Gamma               6835.065 6844.086
## Weibull             7160.714 7169.734
## Exponential         8462.858 8467.369
```

From the AIC and BIC criteria the better model that fits the data in terms of parsimony

and accuracy is the Log-Normal distribution.

4.5 AspectRation

```
summary(AspectRation)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   1.084   1.434   1.543   1.581   1.704   2.289
```

```
V<- var(AspectRation)
cat("Variance : ",V,"\n")
```

```
## Variance :  0.06008017
```

```
Sd <- sd(AspectRation)
cat("Standard deviation : ",Sd,"\n")
```

```
## Standard deviation :  0.2451126
```

```
AspectRation.table <- table(AspectRation)
mode <- names(AspectRation.table)[which.max(AspectRation.table)]
cat("Mode : ",mode,"\n")
```

```
## Mode :  1.084289995
```

```
K <- kurtosis(AspectRation)
cat("Kurtosis : ",K,"\n")
```

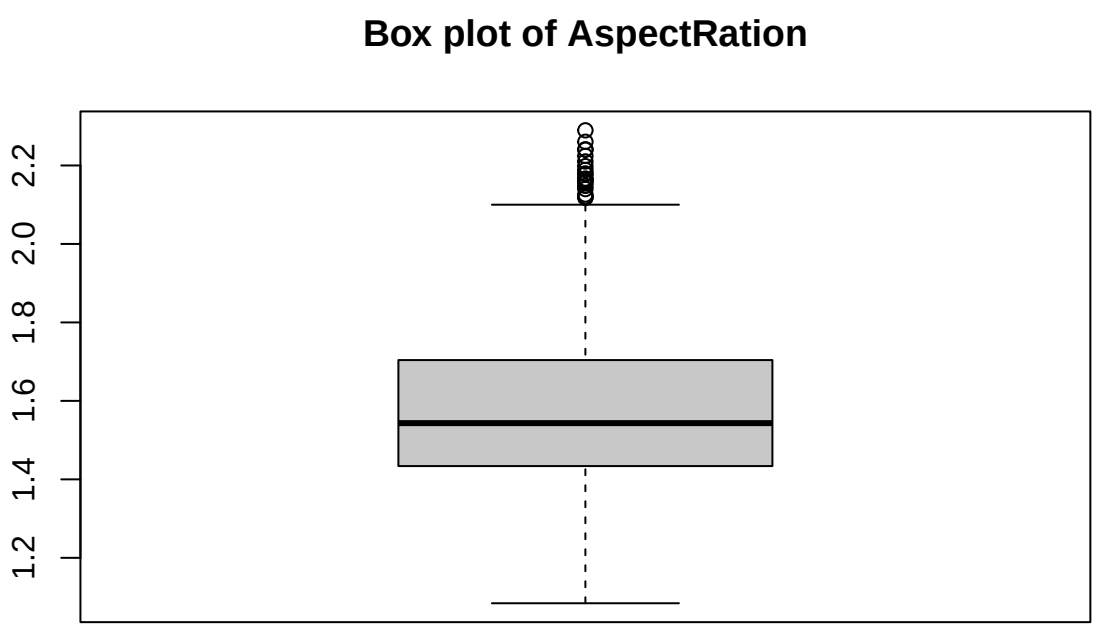
```
## Kurtosis :  2.997186
```

```
S <- skewness(AspectRation)
cat("Skewness : ",S,"\n")
```

```
## Skewness :  0.5042253
```

The data are right-skewed, as observed from the skewness index. The high kurtosis indicates that the distribution is leptokurtic, meaning there is a great concentration of values around the mean, but also a high frequency of outliers that are far away from it.

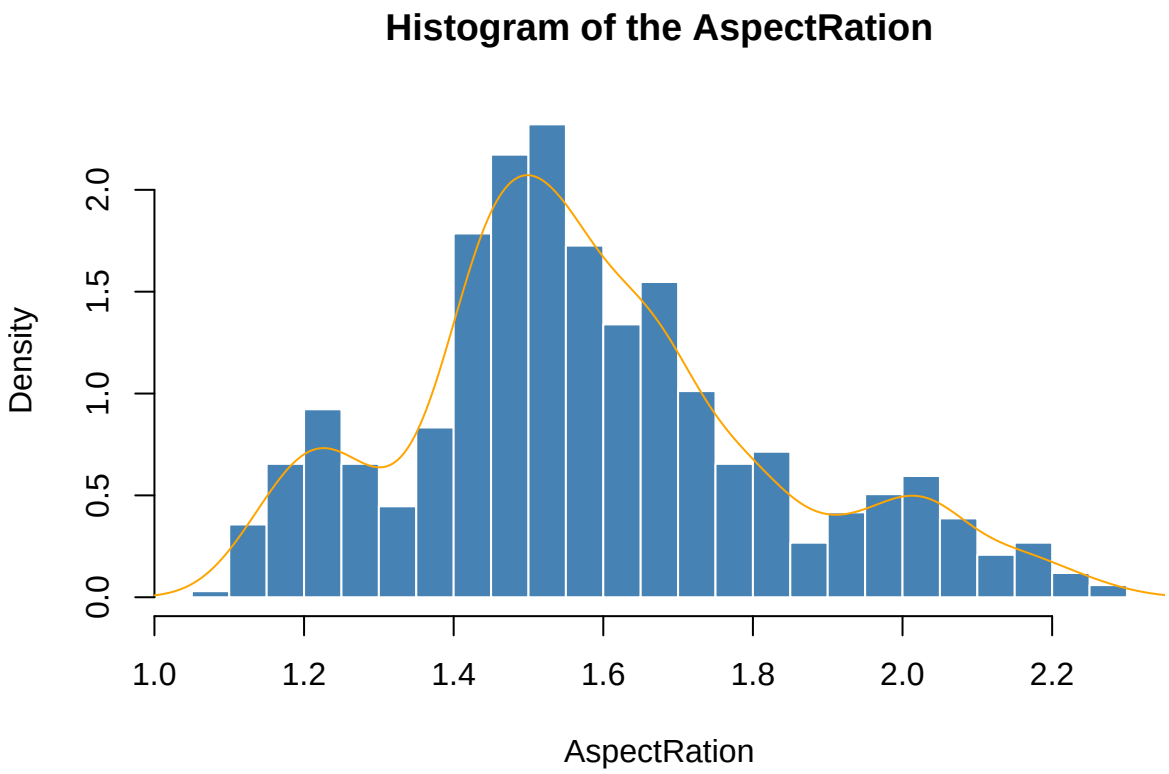
```
box <- boxplot(AspectRation,main="Box plot of AspectRation")
```




```
out <- length(box$out)/length(AspectRation)
cat("Percentage of outliers : ",out*100,"%")
```

```
## Percentage of outliers :  3.125 %
```

```
hist(AspectRation,
     breaks = "FD",
     main = "Histogram of the AspectRation",
     col = "steelblue",
     border = "white",
     freq = FALSE)
lines(density(AspectRation),col = "orange")
```



Now we can try to fit different models to the data. First of all we observe that the support of the data is $[0, +Inf[$. So the distribution from which we could choose are:

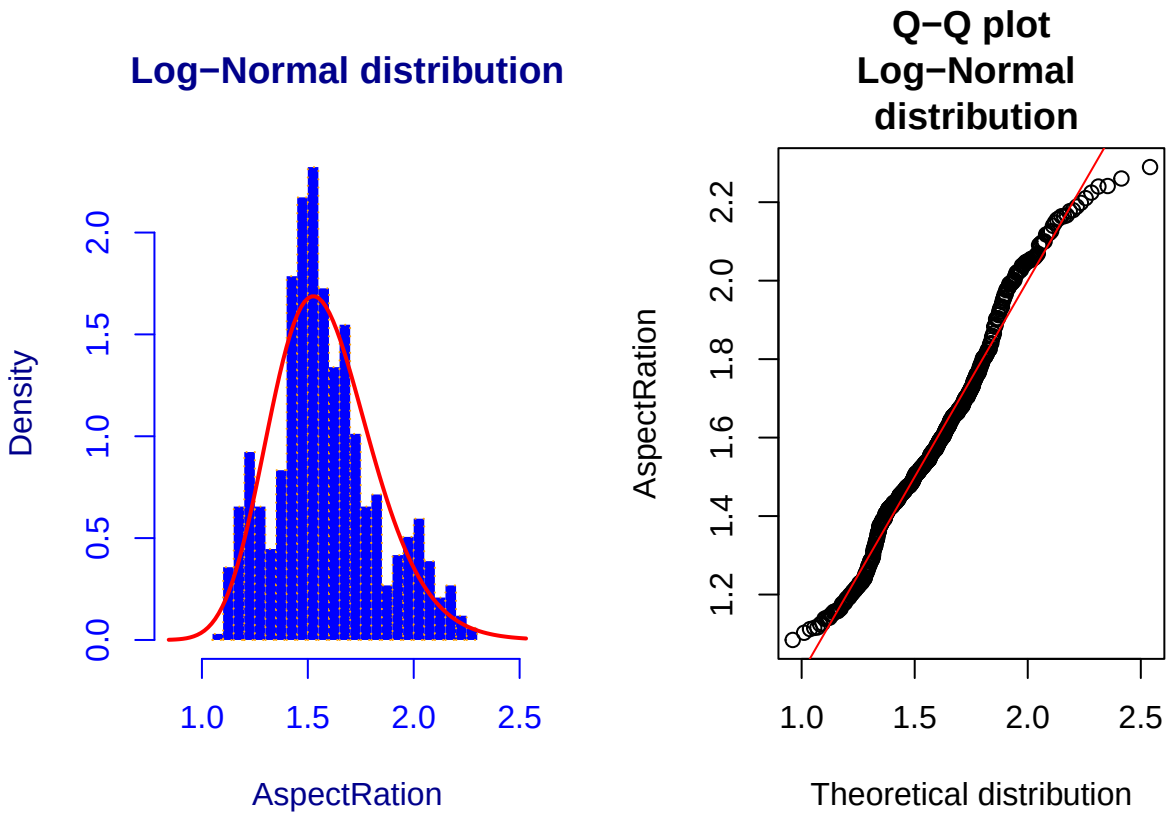
- **Log-Normal**
- **Gamma**
- **Weibull**
- **Exponential**
- **Inverse Gaussian**

4.5.1 Log-Normal

```
par(mfrow = c(1,2))
Log.N5 <- histDist(AspectRation,family = LOGNO,nbins = "fd",
                   main = "Log-Normal distribution",
                   col.hist="blue",border.hist="orange",line.col = "red"
                   ,ylab = "Density")

theoAR <- ppoints(length(AspectRation))
mu <- Log.N5$mu.coefficients
sigma <- exp(Log.N5$sigma.coefficients)
Log.N5.Theoretical <- qLOGNO(theoAR,mu= mu, sigma = sigma)
```

```
qqplot(x = Log.N5.Theoretical,y = AspectRation,
       xlab = "Theoretical distribution",
       main="Q-Q plot\nLog-Normal \n distribution")
abline(a = 0, b=1,col="red")
```

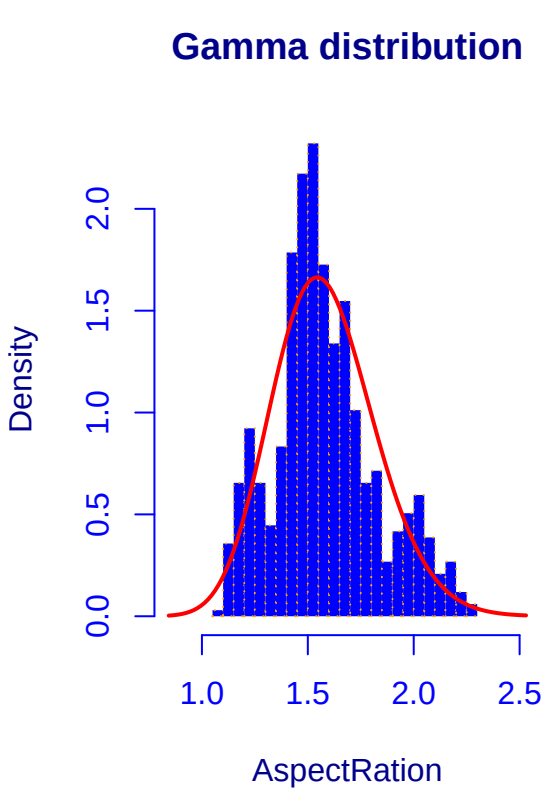


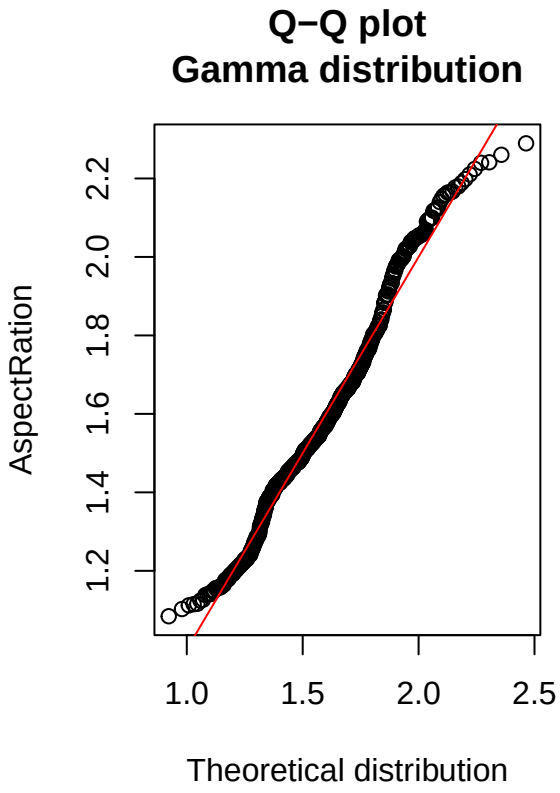
4.5.2 Gamma

```
par(mfrow = c(1,2))
Gamma.N5 <- histDist(AspectRation,family = GA ,nbins = "fd",
                     main = "Gamma distribution",
                     col.hist="blue",border.hist="orange",line.col = "red"
                     ,ylab = "Density")

mu <- exp(Gamma.N5$mu.coefficients)
sigma <- exp(Gamma.N5$sigma.coefficients)
Gamma.N5.Theoretical <- qGA(theoAR,mu= mu, sigma = sigma)

qqplot(x = Gamma.N5.Theoretical,y = AspectRation,
       xlab = "Theoretical distribution",
       main="Q-Q plot\nGamma distribution")
abline(a = 0, b=1,col="red")
```



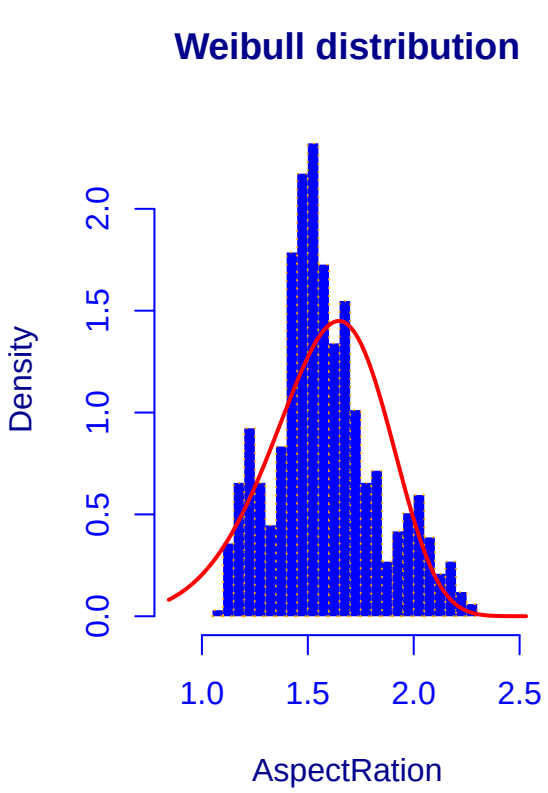


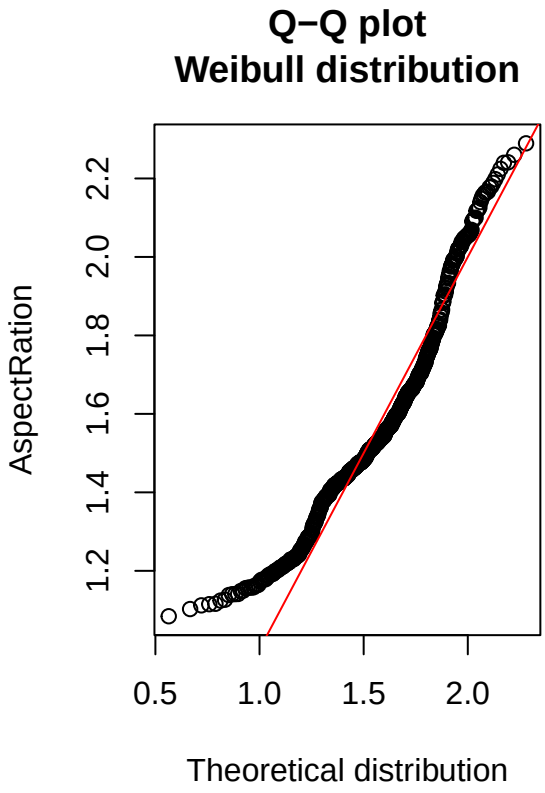
4.5.3 Weibull

```
par(mfrow = c(1,2))
Weibull.N5 <- histDist(AspectRation,family = WEI ,nbins = "fd",
                  main = "Weibull distribution",
                  col.hist="blue",border.hist="orange",line.col = "red"
                  ,ylab = "Density")

mu <- exp(Weibull.N5$mu.coefficients)
sigma <- exp(Weibull.N5$sigma.coefficients)
Weibull.N5.Theoretical <- qWEI(theoAR,mu= mu, sigma = sigma)

qqplot(x = Weibull.N5.Theoretical,y = AspectRation,
       xlab = "Theoretical distribution",
       main="Q-Q plot\nWeibull distribution")
abline(a = 0, b=1,col="red")
```



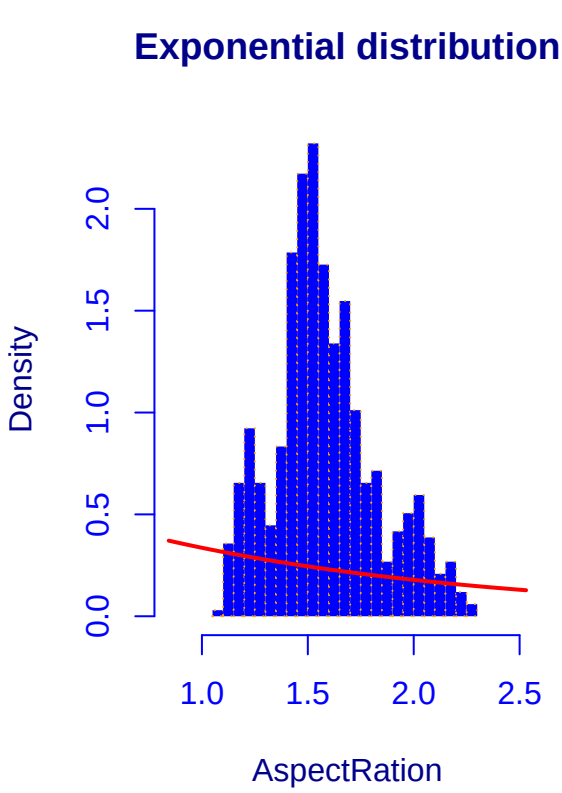


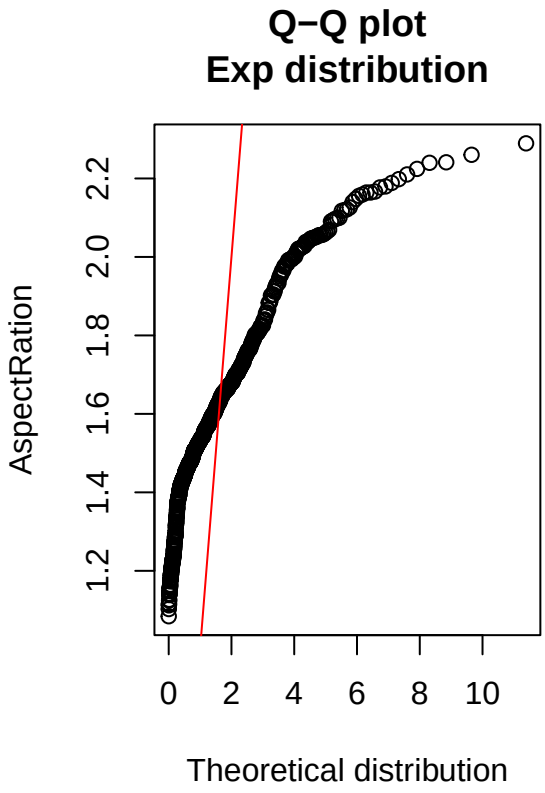
4.5.4 Exponential

```
par(mfrow = c(1,2))
Exp.N5 <- histDist(AspectRation,family = EXP ,nbins = "fd",
                   main = "Exponential distribution",
                   col.hist="blue",border.hist="orange",line.col = "red"
                   ,ylab = "Density")

mu <- exp(Exp.N5$mu.coefficients)
Exp.N5.Theoretical <- qEXP(theoAR,mu= mu)

qqplot(x = Exp.N5.Theoretical,y = AspectRation,
       xlab = "Theoretical distribution",
       main="Q-Q plot\nExp distribution")
abline(a = 0, b=1,col="red")
```



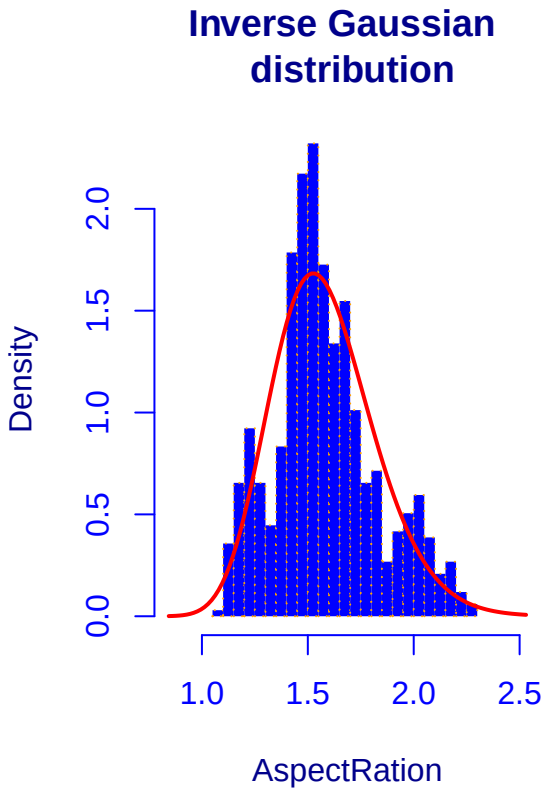


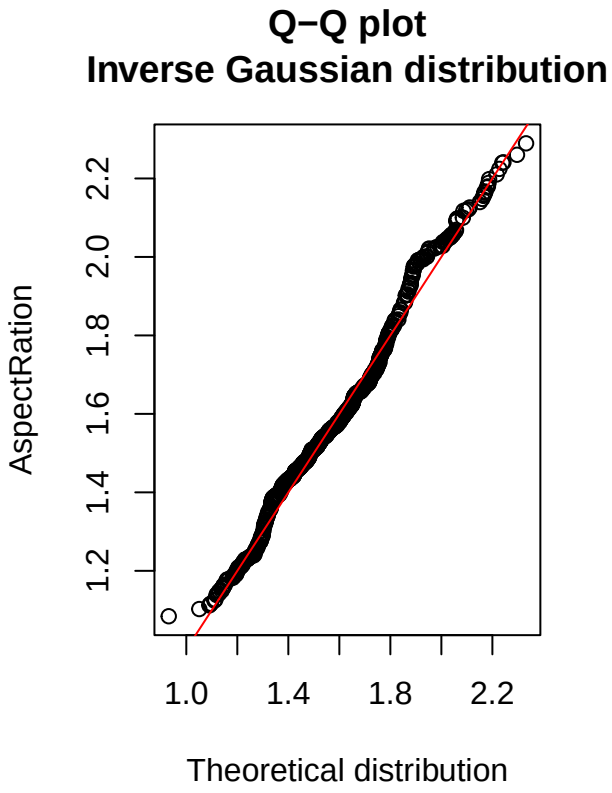
4.5.5 Inverse Gaussian

```
par(mfrow = c(1,2))
Ig.N5 <- histDist(AspectRation,family = IG ,nbins = "fd",
                  main = "Inverse Gaussian \n distribution",
                  col.hist="blue",border.hist="orange",line.col = "red"
                  ,ylab = "Density")

mu <- exp(Ig.N5$mu.coefficients)
sigma <- exp(Ig.N5$sigma.coefficients)
Ig.N5.Theoretical <- rIG(theoAR,mu= mu, sigma = sigma)

qqplot(x = Ig.N5.Theoretical,y = AspectRation,
       xlab = "Theoretical distribution",
       main="Q-Q plot\nInverse Gaussian distribution")
abline(a = 0, b=1,col="red")
```





In this case we can't use the Q-Q plot, because the variance is given by $\sigma^2\mu^3$, so R cannot generates easily value on that range.

4.5.6 Mixture model of Inverse Gaussian

In this case we can try a mixture model of Inverse Gaussian, we can try three different inverse Gaussian because from the histogram we see three peaks.

```
set.seed(123) # We fix the seed because we always have different result
              # due to the fact that the function gamlssMX start in
              # a different point every time
Mx.Ig <- gamlssMX(AspectRation ~ 1,
                  family = IG,
                  K = 3,
                  data = Beans)
```

```
# We extract from the model the parameters that we need

weights <- Mx.Ig$prob

mu_vals <- exp(c(Mx.Ig$models[[1]]$mu.coefficients,
                 Mx.Ig$models[[2]]$mu.coefficients,
                 Mx.Ig$models[[3]]$mu.coefficients))
```

```r
sigma_vals <- exp(c(Mx.Ig$models[[1]]$sigma.coefficients,
                    Mx.Ig$models[[2]]$sigma.coefficients,
                    Mx.Ig$models[[3]]$sigma.coefficients))

# Then we create the histogram

hist(AspectRation,
     breaks = "FD",
     main = "Mixture model of Inverse Gaussian",
     col = "steelblue",
     border = "white",
     freq = FALSE)

# we create a vector to represent the distribution

x_seq <- seq(min(AspectRation), max(AspectRation), length.out = 1000)

colors <- c("red", "blue", "green")

# In this cycle we create the respectively y value and plot
# the each distribution

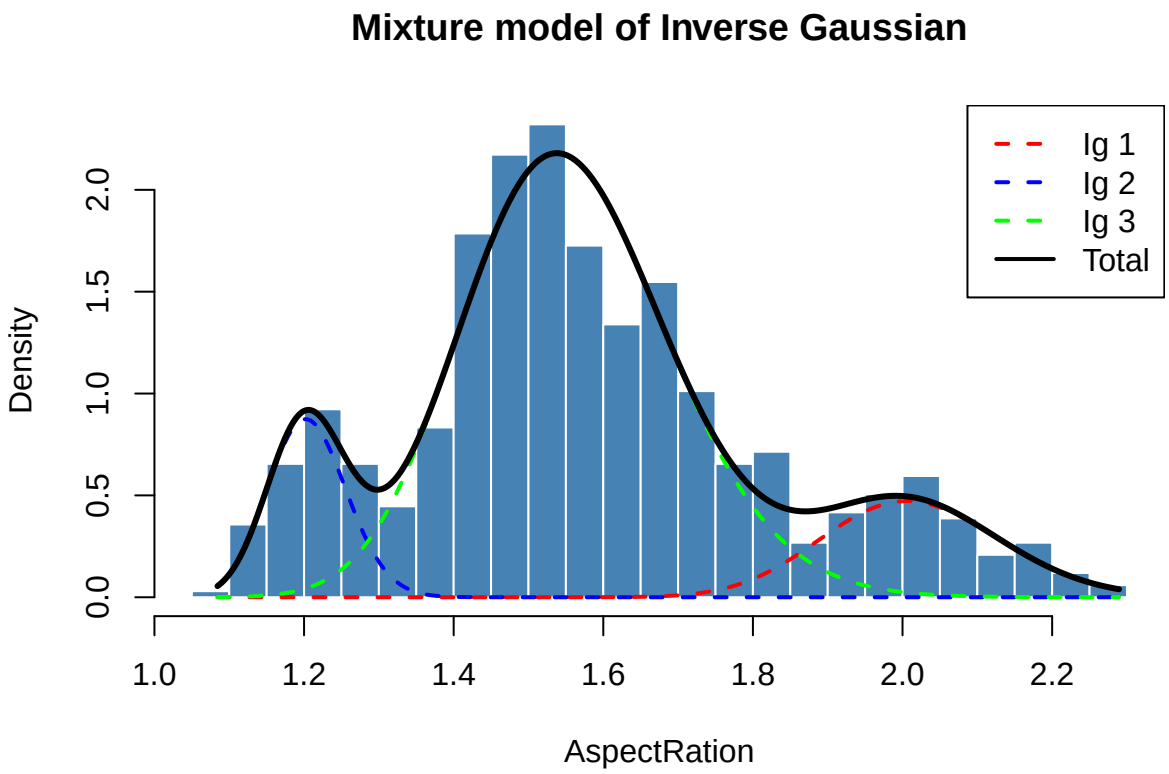
for (i in 1:3) {
  y_comp <- weights[i] * dIG(x_seq, mu = mu_vals[i],
                             sigma = sigma_vals[i])
  lines(x_seq, y_comp, col = colors[i],
        lwd = 2, lty = 2)
}

# Creates the total distribution

y_total <- (weights[1] * dIG(x_seq, mu = mu_vals[1], sigma = sigma_vals[1])) +
           (weights[2] * dIG(x_seq, mu = mu_vals[2], sigma = sigma_vals[2])) +
           (weights[3] * dIG(x_seq, mu = mu_vals[3], sigma = sigma_vals[3]))

lines(x_seq, y_total, col = "black", lwd = 3)

legend("topright", legend = c("Ig 1", "Ig 2", "Ig 3", "Total"),
       col = c(colors, "black"), lwd = 2, lty = c(2,2,2,1))
```



4.5.7 Comparison

```
AspectRation.comp <- data.frame(AIC=c(Log.N5$aic,Gamma.N5$aic,Weibull.N5$aic,
                                      Exp.N5$aic,Ig.N5$aic,
                                    Mx.Ig$aic),
                        BIC = c(Log.N5$sbc,Gamma.N5$sbc,Weibull.N5$sbc,
                                Exp.N5$sbc,Ig.N5$sbc,
                                Mx.Ig$sbc),
                        row.names=c("Log-Normal","Gamma","Weibull",
                                    "Exponential","Inverse Gaussian",
                                   "Mixed Inverse Gaussian"))
AspectRation.comp <- AspectRation.comp[order(AspectRation.comp$AIC,
                                             decreasing = FALSE), ]
AspectRation.comp
```

```
##                              AIC         BIC
## Mixed Inverse Gaussian  -84.981256  -48.899189
## Inverse Gaussian        -11.946265   -2.925748
## Log-Normal              -11.627819   -2.607302
## Gamma                    -5.791441    3.229076
## Weibull                 100.927926  109.948443
```

```
## Exponential             1961.488358 1965.998617
```

From the AIC and BIC criteria the better model that fits the data in terms of parsimony and accuracy is the Mixture Model of Inverse Gaussian distribution.

4.6 Eccentricity

```
summary(Eccentricity)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.3866  0.7167  0.7617  0.7497  0.8097  0.8996
```

```
V<- var(Eccentricity)
cat("Variance : ",V,"\n")
```

```
## Variance :  0.008802923
```

```
Sd <- sd(Eccentricity)
cat("Standard deviation : ",Sd,"\n")
```

```
## Standard deviation :  0.09382389
```

```
Eccentricity.table <- table(Eccentricity)
mode <- names(Eccentricity.table)[which.max(Eccentricity.table)]
cat("Mode : ",mode,"\n")
```

```
## Mode :  0.386564204
```

```
K <- kurtosis(Eccentricity)
cat("Kurtosis : ",K,"\n")
```

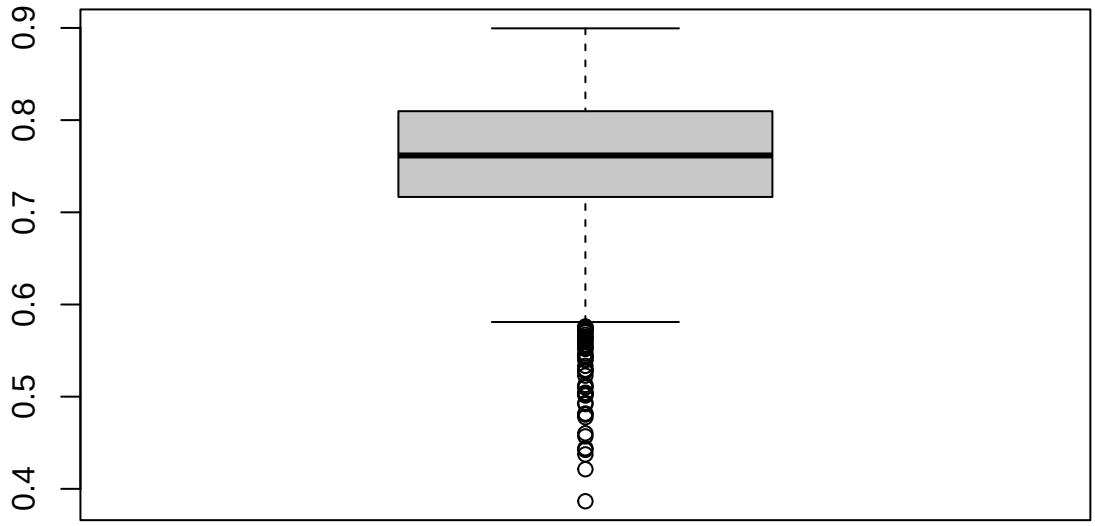
```
## Kurtosis :  4.195289
```

```
S <- skewness(Eccentricity)
cat("Skewness : ",S,"\n")
```

```
## Skewness :  -1.0937
```

The data are left-skewed, as observed from the skewness index. The high kurtosis indicates that the distribution is leptokurtic, meaning there is a great concentration of values around the mean, but also a high frequency of outliers that are far away from it.

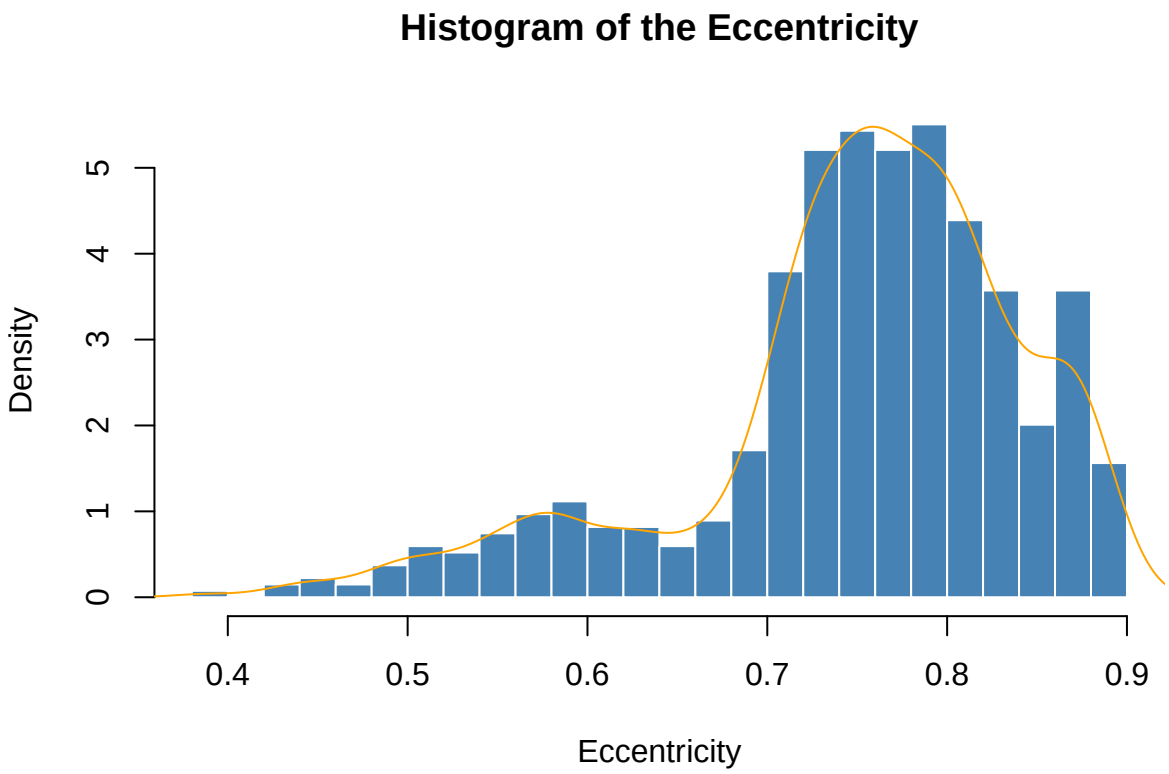
```
box <- boxplot(Eccentricity,main="Box plot of Eccentricity")
```




```
out <- length(box$out)/length(Area)
cat("Percentage of outliers : ",out*100,"%")
```

```
## Percentage of outliers :  7.589286 %
```

```
hist(Eccentricity,
     breaks = "FD",
     main = "Histogram of the Eccentricity",
     col = "steelblue",
     border = "white",
     freq = FALSE)
lines(density(Eccentricity),col = "orange")
```



Now we can try to fit different models to the data. First of all we observe that the support of the data is $[0, 1]$. So the distribution from which we could choose are:

- **Beta**
- **Simplex**

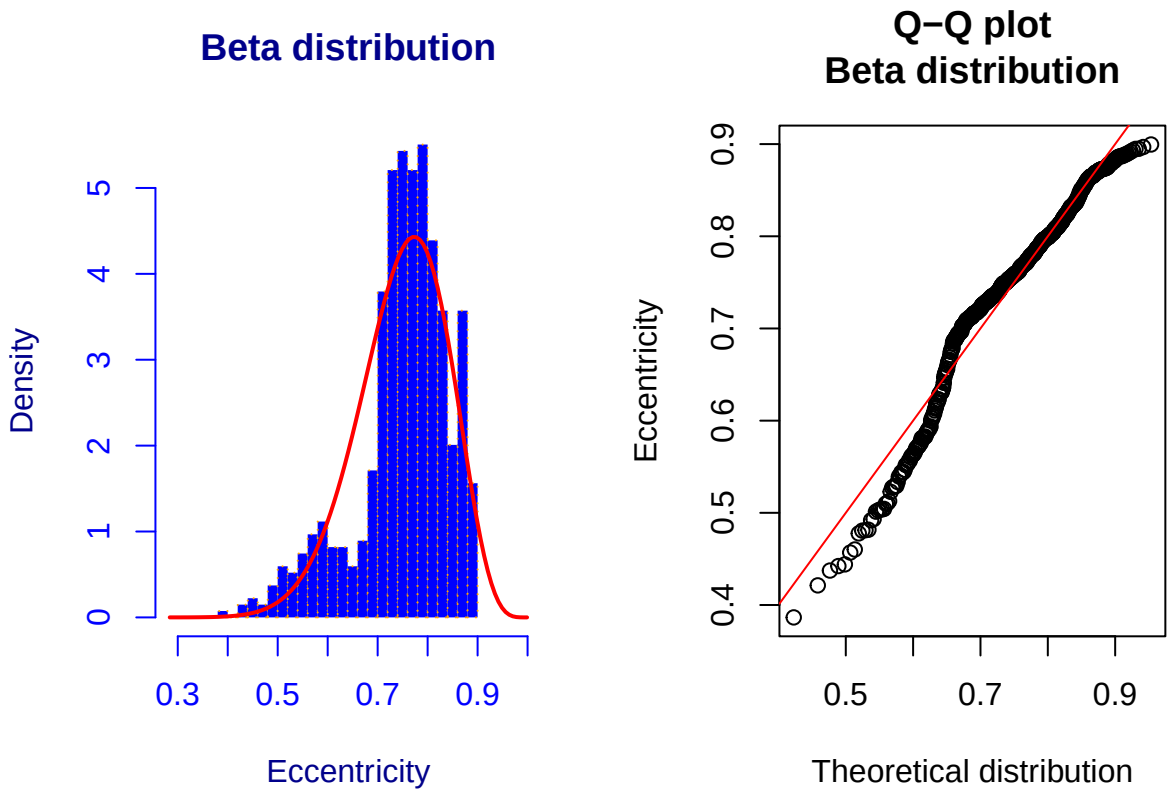
4.6.1 Beta

```
par(mfrow = c(1,2))
Beta.N1 <- histDist(y = Eccentricity, family = BE, nbins = "fd",
                    main = "Beta distribution",
                    col.hist="blue",border.hist="orange",line.col = "red"
                    ,ylab = "Density")

theoE <- ppoints(length(Eccentricity))
mu <- plogis(Beta.N1$mu.coefficients)
sigma <- plogis(Beta.N1$sigma.coefficients)
Beta.N1.Theoretical <- qBE(theoE,mu= mu, sigma = sigma)

qqplot(x = Beta.N1.Theoretical,y = Eccentricity,
       xlab = "Theoretical distribution",
       main="Q-Q plot\nBeta distribution")
```

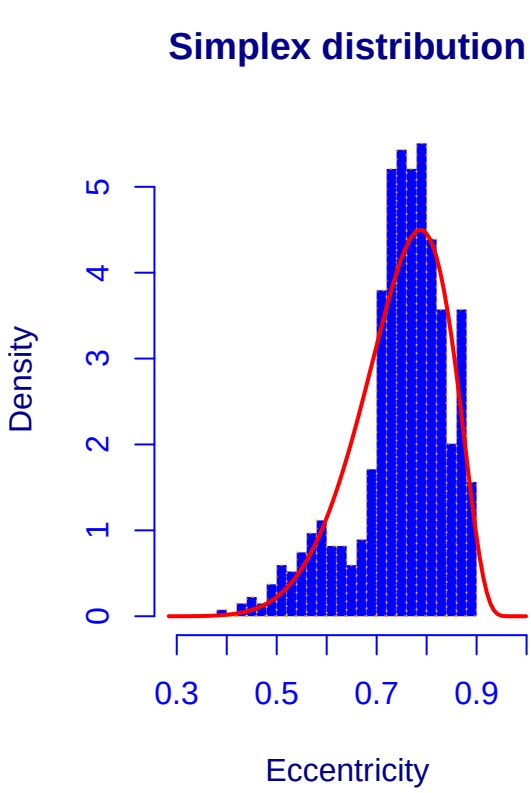
```
abline(a = 0, b=1,col="red")
```

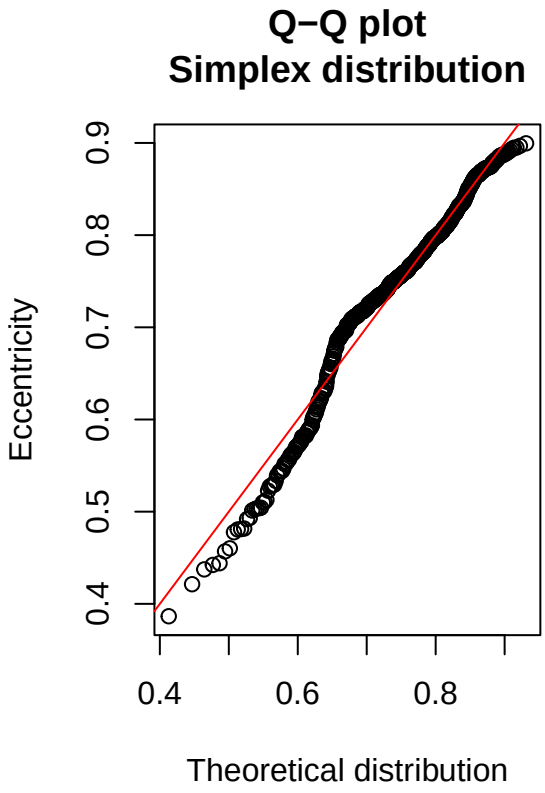


4.6.2 Simplex

```
par(mfrow = c(1,2))
Sx.N2<- histDist(y = Eccentricity, family = SIMPLEX, nbins = "fd",
                 main = "Simplex distribution",
                 col.hist="blue",border.hist="orange",line.col = "red"
                 ,ylab = "Density")
mu <- plogis(Sx.N2$mu.coefficients)
sigma <- exp(Sx.N2$sigma.coefficients)
Sx.N2.Theoretical <- qSIMPLEX(theoE,mu= mu, sigma = sigma)

qqplot(x = Sx.N2.Theoretical,y = Eccentricity,
       xlab = "Theoretical distribution",
       main="Q-Q plot\nSimplex distribution")
abline(a = 0, b=1,col="red")
```





4.6.3 Comparison

```
Eccentricity.comp <- data.frame(AIC=c(Beta.N1$aic,Sx.N2$aic),
                        BIC = c(Beta.N1$sbc,Sx.N2$sbc),
                        row.names=c("Beta","Simplex"))
Eccentricity.comp <- Eccentricity.comp[order(Eccentricity.comp$AIC,
                                             decreasing = FALSE), ]
Eccentricity.comp
```

```
##                AIC       BIC
## Simplex -1391.675 -1382.655
## Beta    -1361.839 -1352.818
```

From the AIC and BIC criteria the better model that fits the data in terms of parsimony and accuracy is the Simplex distribution.

4.7 ConvexArea

```
summary(ConvexArea)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
```

```
##   21877   35996   43531   52132   56783  240671
```

```
V<- var(ConvexArea)
cat("Variance : ",V,"\n")
```

```
## Variance :  845955317
```

```
Sd <- sd(ConvexArea)
cat("Standard deviation : ",Sd,"\n")
```

```
## Standard deviation :  29085.31
```

```
ConvexArea.table <- table(ConvexArea)
mode <- names(ConvexArea.table)[which.max(ConvexArea.table)]
cat("Mode : ",mode,"\n")
```

```
## Mode :  31977
```

```
K <- kurtosis(ConvexArea)
cat("Kurtosis : ",K,"\n")
```

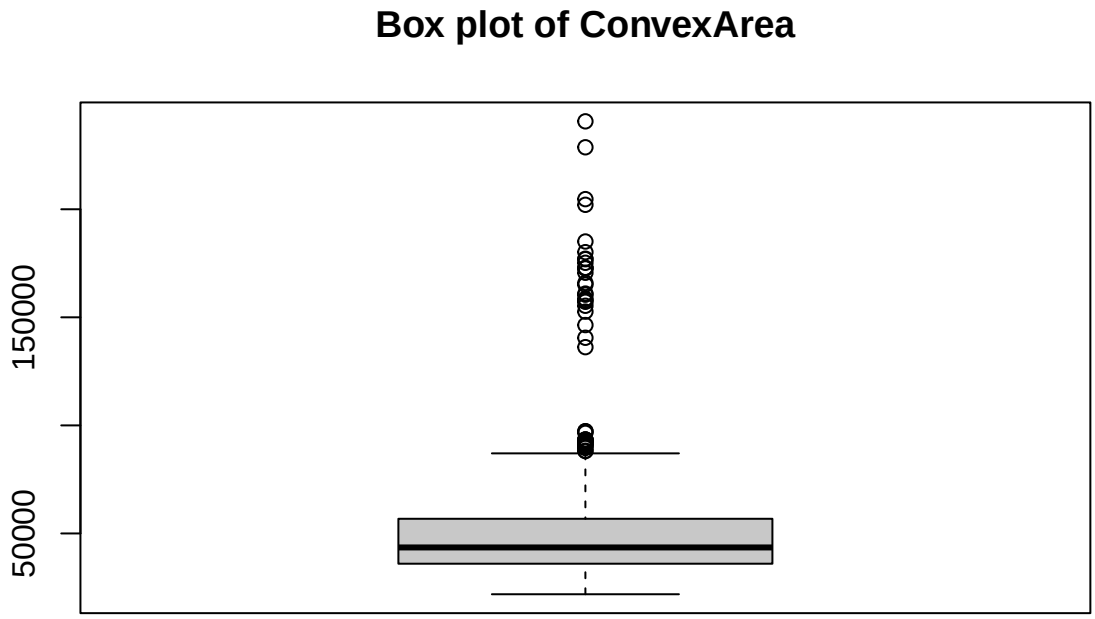
```
## Kurtosis :  14.45254
```

```
S <- skewness(ConvexArea)
cat("Skewness : ",S,"\n")
```

```
## Skewness :  3.036661
```

The data are right-skewed, as observed from the skewness index. The high kurtosis indicates that the distribution is leptokurtic, meaning there is a great concentration of values around the mean, but also a high frequency of outliers that are far away from it.

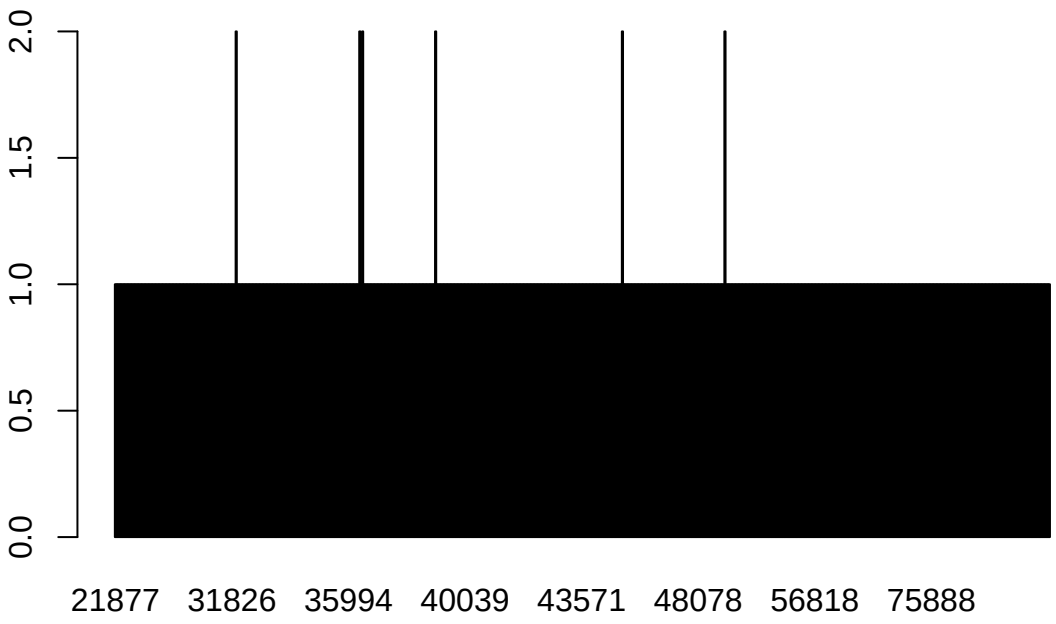
```
box <- boxplot(ConvexArea,main="Box plot of ConvexArea")
```




```
out <- length(box$out)/length(ConvexArea)
cat("Percentage of outliers : ",out*100,"%")
```

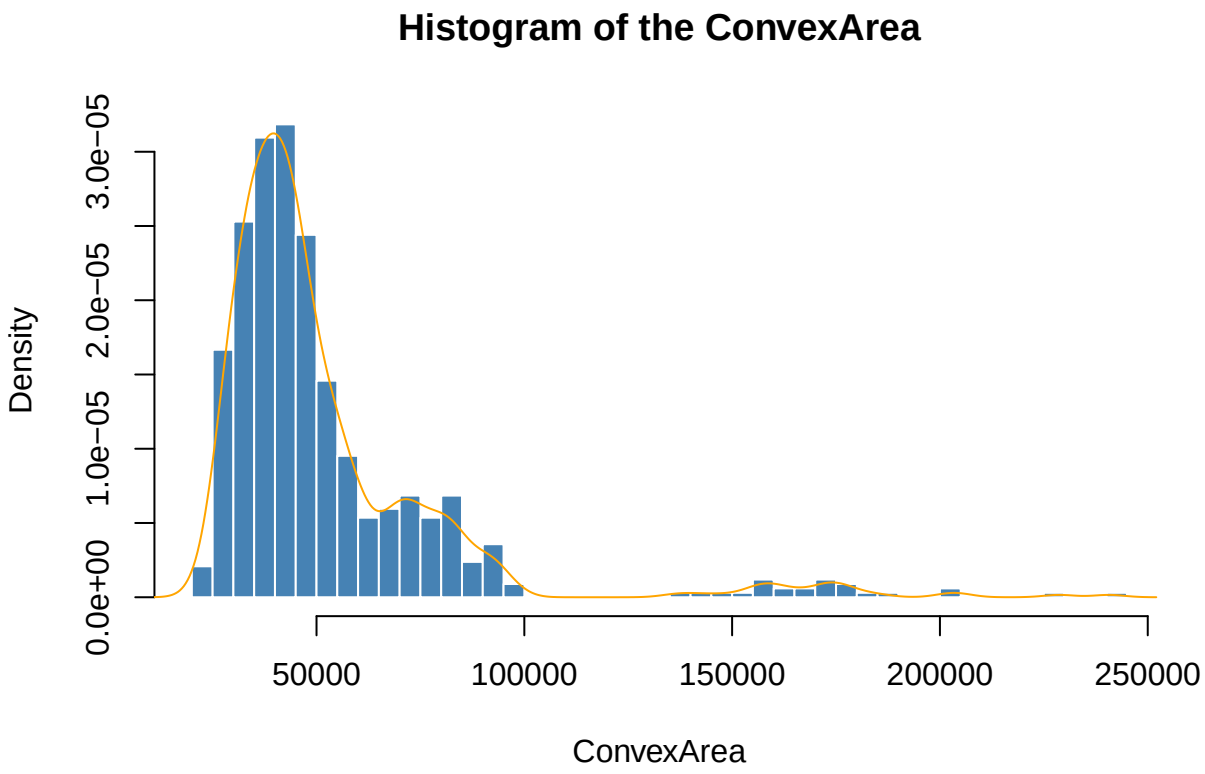
```
## Percentage of outliers :  6.39881 %
```

```
barplot(ConvexArea.table)
```



We can see from the bar plot that the data work as a continuous variable, not like an integer where values repeat multiple times. So in this case we treat all like a continuous variable.

```
hist(ConvexArea,
     breaks = "FD",
     main = "Histogram of the ConvexArea",
     col = "steelblue",
     border = "white",
     freq = FALSE)
lines(density(ConvexArea),col = "orange")
```



Now we can try to fit different models to the data. First of all we observe that the support of the data is $[0, +Inf[$. So the distribution from which we could choose are:

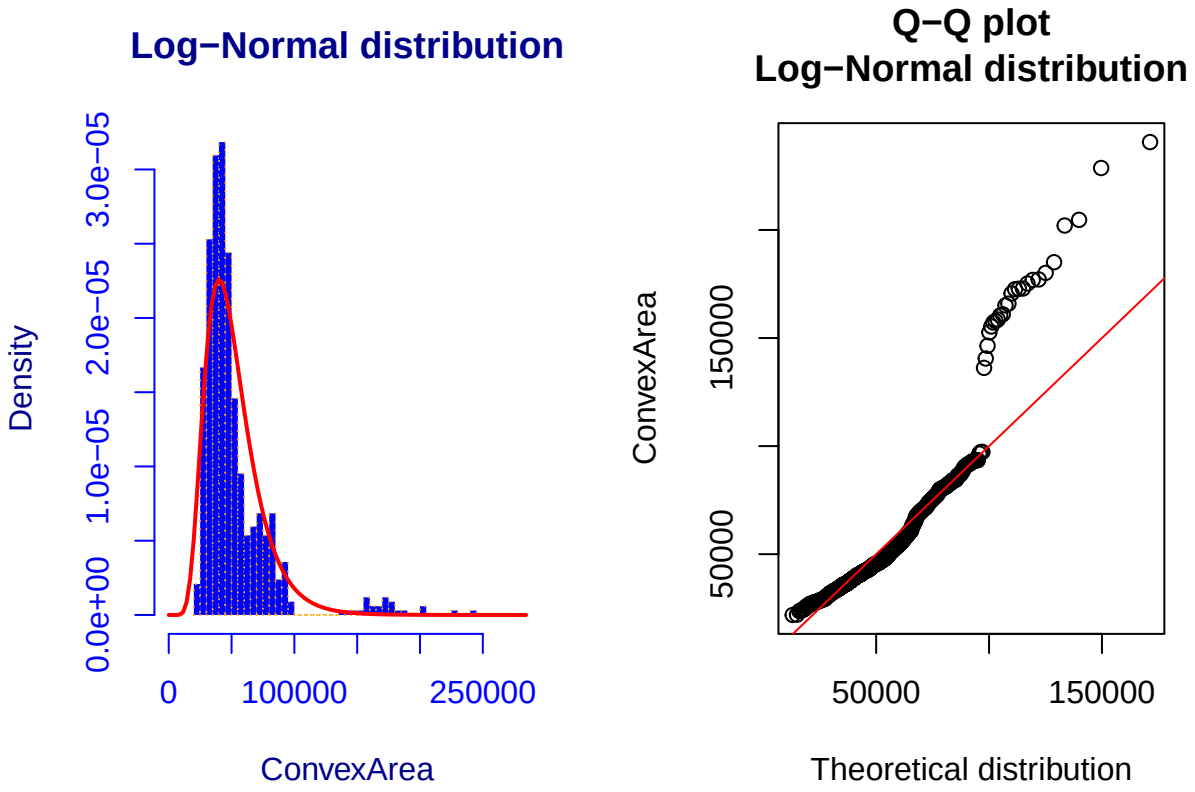
- **Log-Normal**
- **Gamma**
- **Weibull**
- **Exponential**
- **Inverse Gaussian**

4.7.1 Log-Normal

```
par(mfrow = c(1,2))
Log.N6 <- histDist(ConvexArea,family = LOGNO,nbins = "fd",
                   main = "Log-Normal distribution",
                   col.hist="blue",border.hist="orange",line.col = "red"
                   ,ylab = "Density")

theoCA <- ppoints(length(ConvexArea))
mu <- Log.N6$mu.coefficients
sigma <- exp(Log.N6$sigma.coefficients)
Log.N6.Theoretical <- qLOGNO(theoCA,mu= mu, sigma = sigma)
```

```
qqplot(x = Log.N6.Theoretical,y = ConvexArea,
       xlab = "Theoretical distribution",
       main="Q-Q plot\nLog-Normal distribution")
abline(a = 0, b=1,col="red")
```

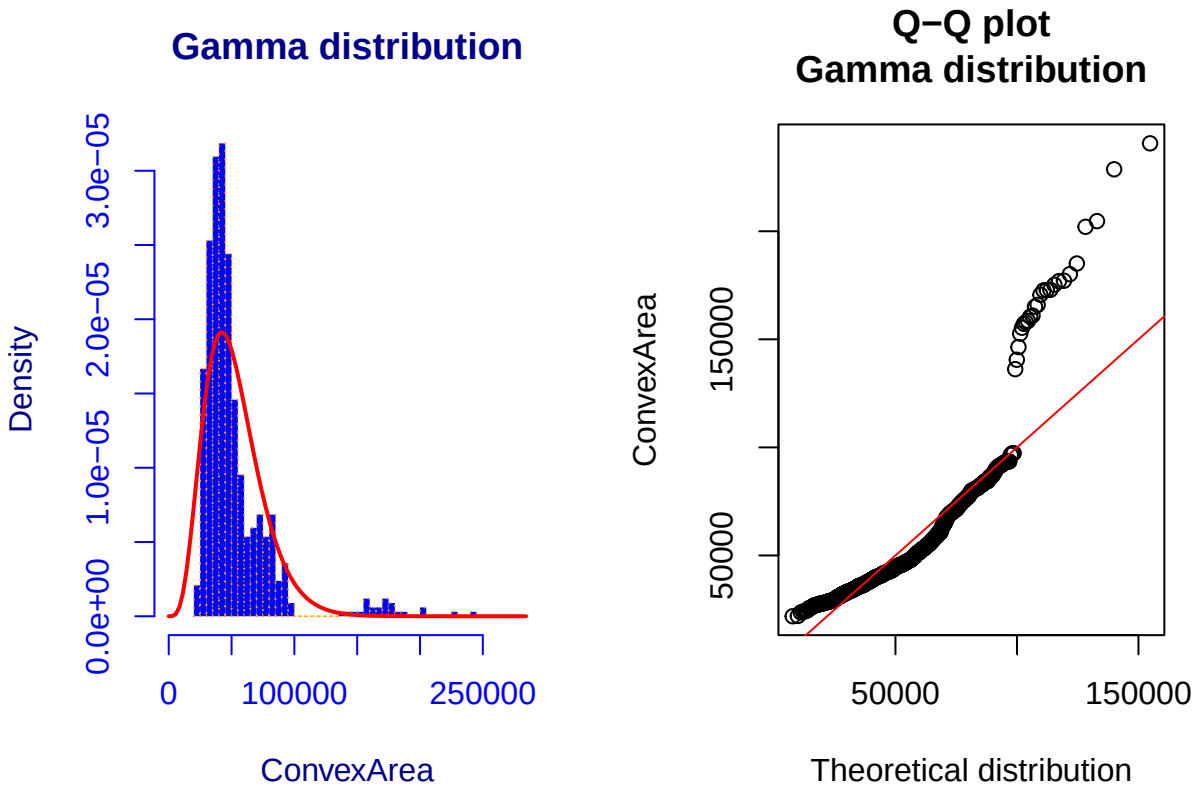


4.7.2 Gamma

```
par(mfrow = c(1,2))
Gamma.N6 <- histDist(ConvexArea,family = GA ,nbins = "fd",
                     main = "Gamma distribution",
                     col.hist="blue",border.hist="orange",line.col = "red"
                     ,ylab = "Density")

mu <- exp(Gamma.N6$mu.coefficients)
sigma <- exp(Gamma.N6$sigma.coefficients)
Gamma.N6.Theoretical <- qGA(theoCA,mu= mu, sigma = sigma)

qqplot(x = Gamma.N6.Theoretical,y = ConvexArea,
       xlab = "Theoretical distribution",
       main="Q-Q plot\nGamma distribution")
abline(a = 0, b=1,col="red")
```

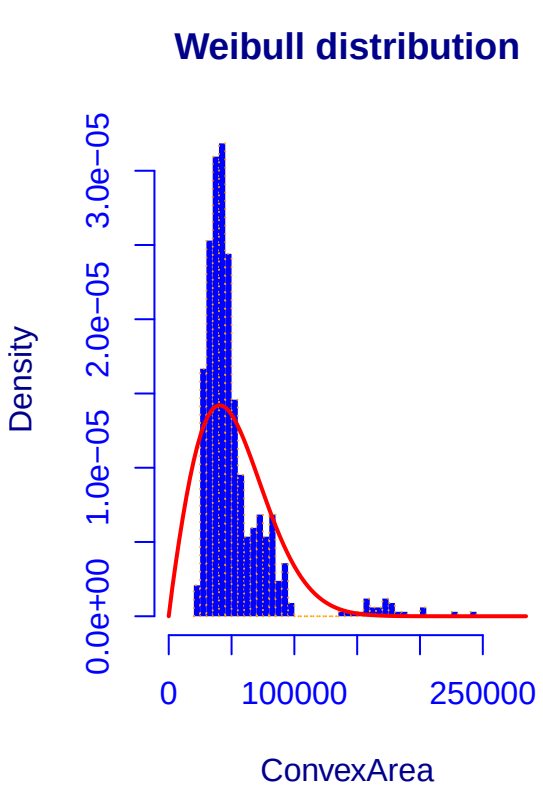


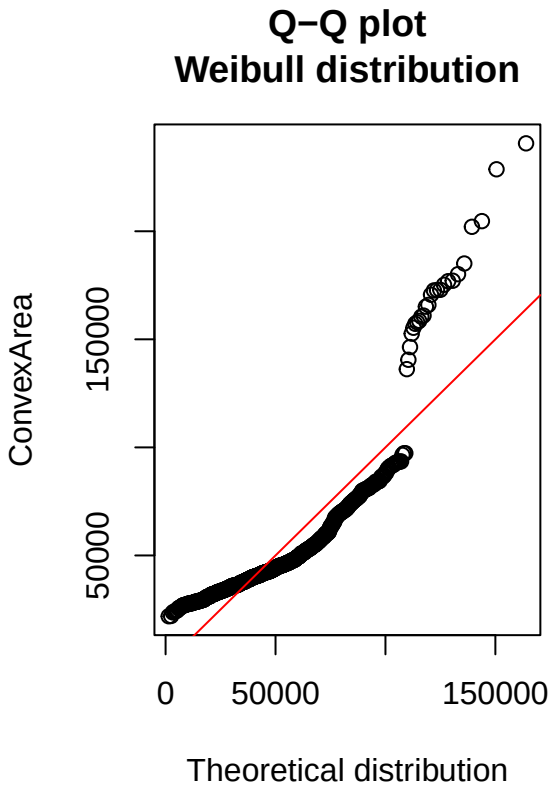
4.7.3 Weibull

```
par(mfrow = c(1,2))
Weibull.N6 <- histDist(ConvexArea,family = WEI ,nbins = "fd",
                  main = "Weibull distribution",
                  col.hist="blue",border.hist="orange",line.col = "red"
                  ,ylab = "Density")

mu <- exp(Weibull.N6$mu.coefficients)
sigma <- exp(Weibull.N6$sigma.coefficients)
Weibull.N6.Theoretical <- qWEI(theoCA,mu= mu, sigma = sigma)

qqplot(x = Weibull.N6.Theoretical,y = ConvexArea,
       xlab = "Theoretical distribution",
       main="Q-Q plot\nWeibull distribution")
abline(a = 0, b=1,col="red")
```



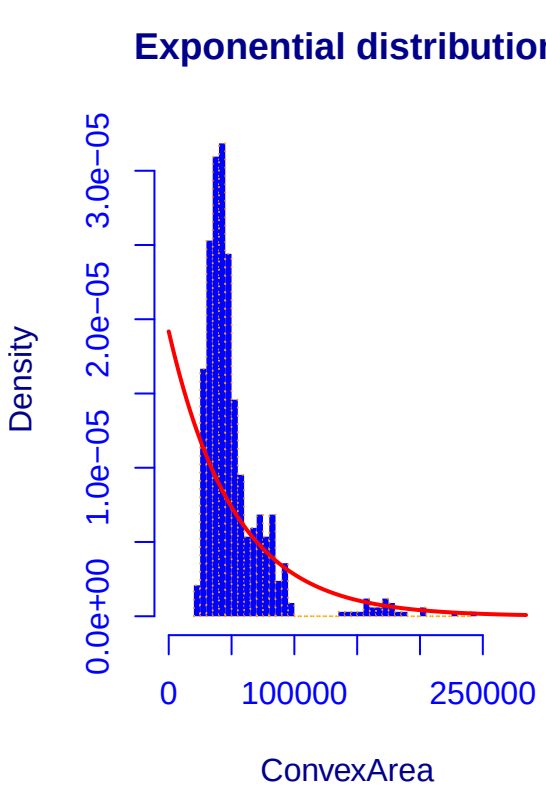


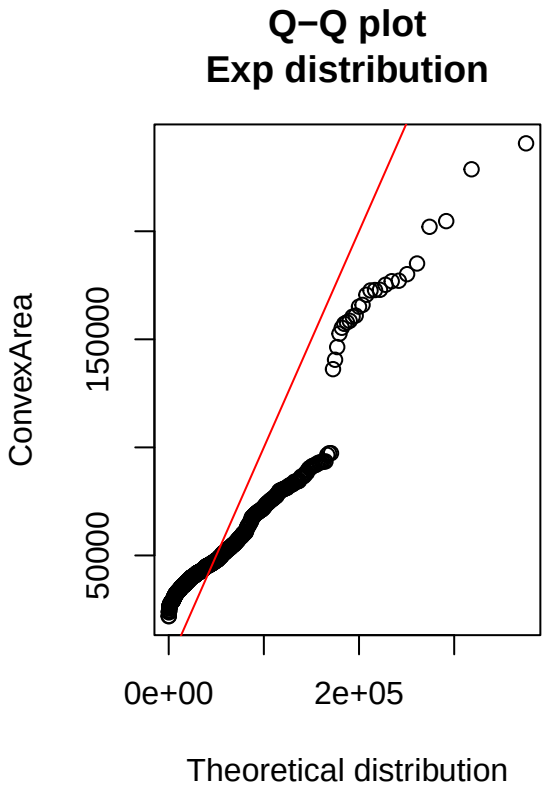
4.7.4 Exponential

```
par(mfrow = c(1,2))
Exp.N6 <- histDist(ConvexArea,family = EXP ,nbins = "fd",
                   main = "Exponential distribution",
                   col.hist="blue",border.hist="orange",line.col = "red"
                   ,ylab = "Density")

mu <- exp(Exp.N6$mu.coefficients)
Exp.N6.Theoretical <- qEXP(theoCA,mu= mu)

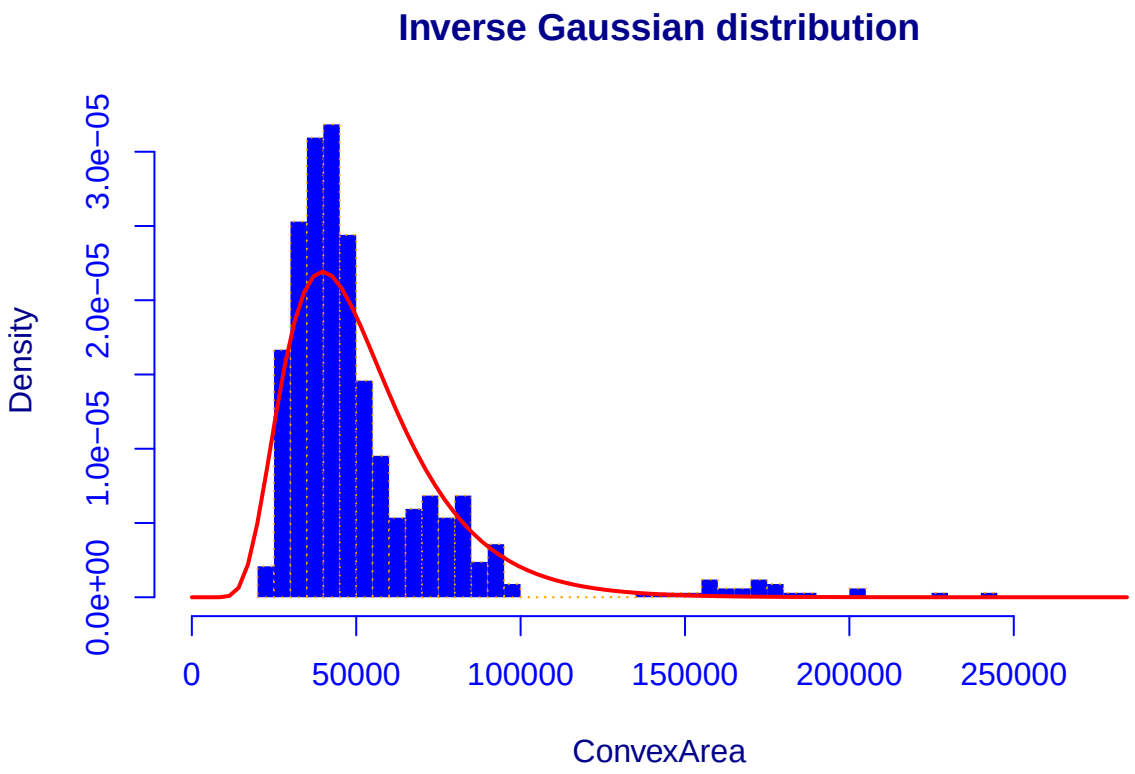
qqplot(x = Exp.N6.Theoretical,y = ConvexArea,
       xlab = "Theoretical distribution",
       main="Q-Q plot\nExp distribution")
abline(a = 0, b=1,col="red")
```





4.7.5 Inverse Gaussian

```
Ig.N6 <- histDist(ConvexArea,family = IG ,nbins = "fd",
                  main = "Inverse Gaussian distribution",
                  col.hist="blue",border.hist="orange",line.col = "red"
                  ,ylab = "Density")
```



In this case we can't use the Q-Q plot, because the variance is given by $\sigma^2\mu^3$, so R cannot generates easily value on that range.

4.7.6 Comparison

```
ConvexArea.comp <- data.frame(AIC=c(Log.N6$aic,Gamma.N6$aic,Weibull.N6$aic,
                                    Exp.N6$aic,Ig.N6$aic),
                              BIC = c(Log.N6$sbc,Gamma.N6$sbc,Weibull.N6$sbc,
                                      Exp.N6$sbc,Ig.N6$sbc),
                              row.names=c("Log-Normal","Gamma","Weibull",
                                          "Exponential","Inverse Gaussian"))
ConvexArea.comp <- ConvexArea.comp[order(ConvexArea.comp$AIC, decreasing = FALSE), ]
ConvexArea.comp
```

```
##                         AIC      BIC
## Log-Normal          15163.52 15172.54
## Inverse Gaussian    15172.25 15181.27
## Gamma               15302.88 15311.90
## Weibull             15507.79 15516.81
## Exponential         15943.91 15948.42
```

From the AIC and BIC criteria the better model that fits the data in terms of parsimony and accuracy is the Log-Normal distribution.

4.8 EquivDiameter

```
summary(EquivDiameter)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   166.1   212.9   234.3   249.2   267.5   549.6
```

```
V<- var(EquivDiameter)
cat("Variance : ",V,"\n")
```

```
## Variance :  3382.102
```

```
Sd <- sd(EquivDiameter)
cat("Standard deviation : ",Sd,"\n")
```

```
## Standard deviation :  58.15584
```

```
EquivDiameter.table <- table(EquivDiameter)
mode <- names(EquivDiameter.table)[which.max(EquivDiameter.table)]
cat("Mode : ",mode,"\n")
```

```
## Mode :  186.9741645
```

```
K <- kurtosis(EquivDiameter)
cat("Kurtosis : ",K,"\n")
```

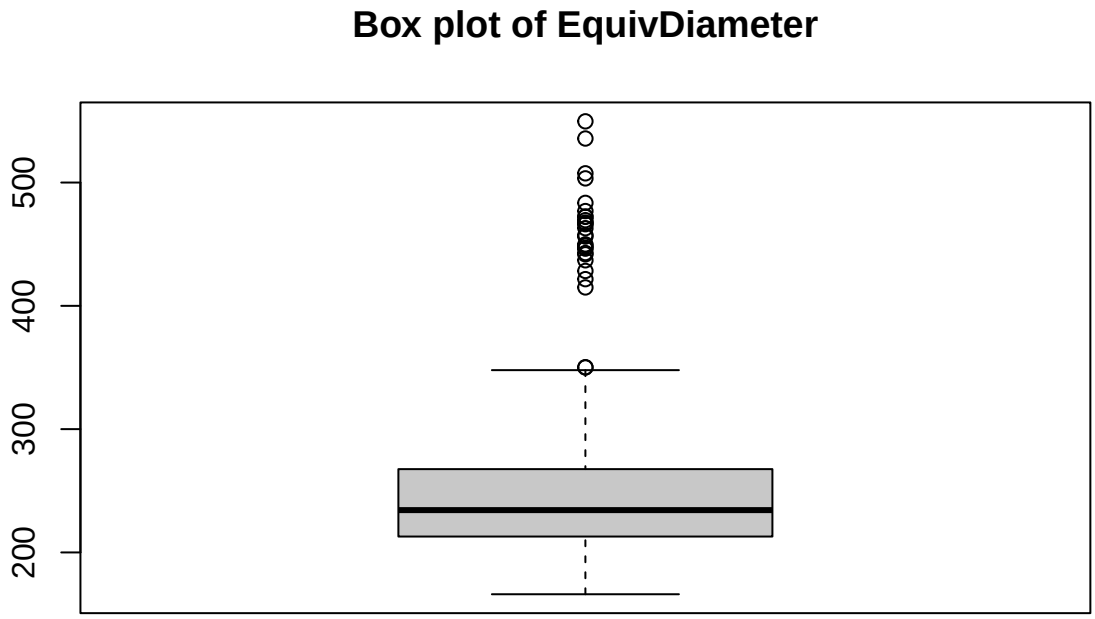
```
## Kurtosis :  8.628201
```

```
S <- skewness(EquivDiameter)
cat("Skewness : ",S,"\n")
```

```
## Skewness :  2.07335
```

The data are right-skewed, as observed from the skewness index. The high kurtosis indicates that the distribution is leptokurtic, meaning there is a great concentration of values around the mean, but also a high frequency of outliers that are far away from it.

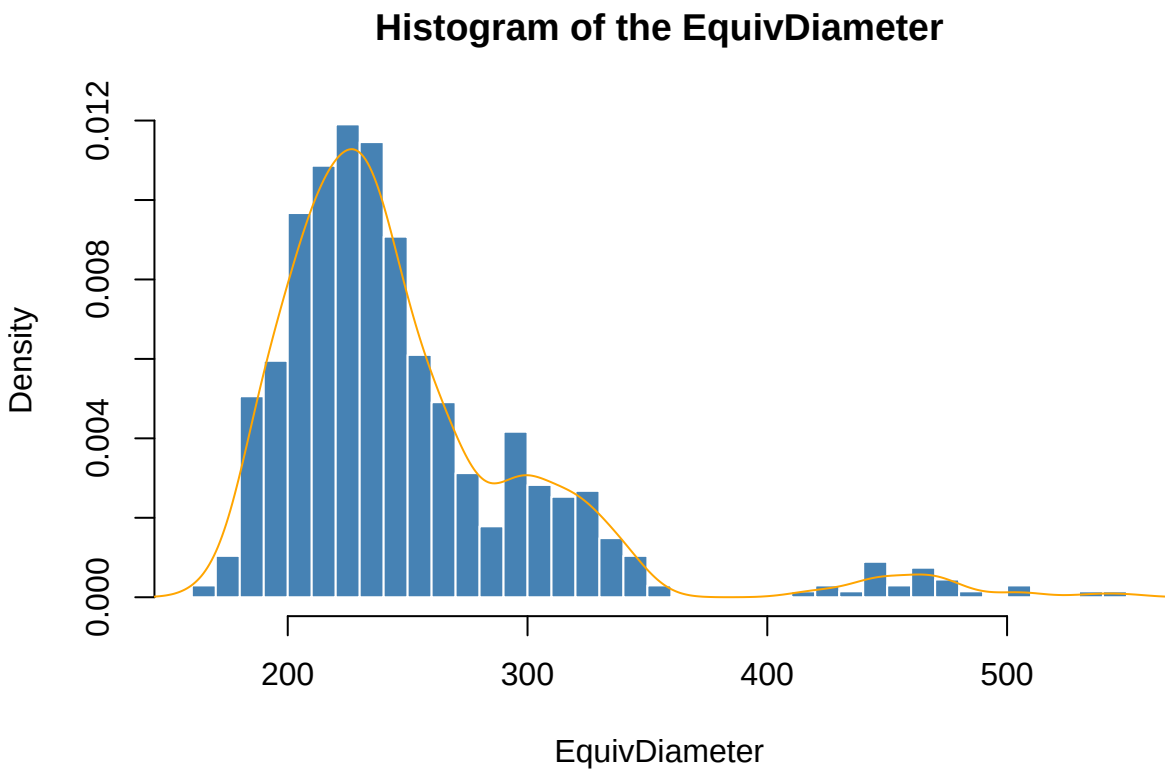
```
box <- boxplot(EquivDiameter,main="Box plot of EquivDiameter")
```




```
out <- length(box$out)/length(EquivDiameter)
cat("Percentage of outliers : ",out*100,"%")
```

```
## Percentage of outliers :  4.017857 %
```

```
hist(EquivDiameter,
     breaks = "FD",
     main = "Histogram of the EquivDiameter",
     col = "steelblue",
     border = "white",
     freq = FALSE)
lines(density(EquivDiameter),col = "orange")
```



Now we can try to fit different models to the data. First of all we observe that the support of the data is $[0, +Inf[$. So the distribution from which we could choose are:

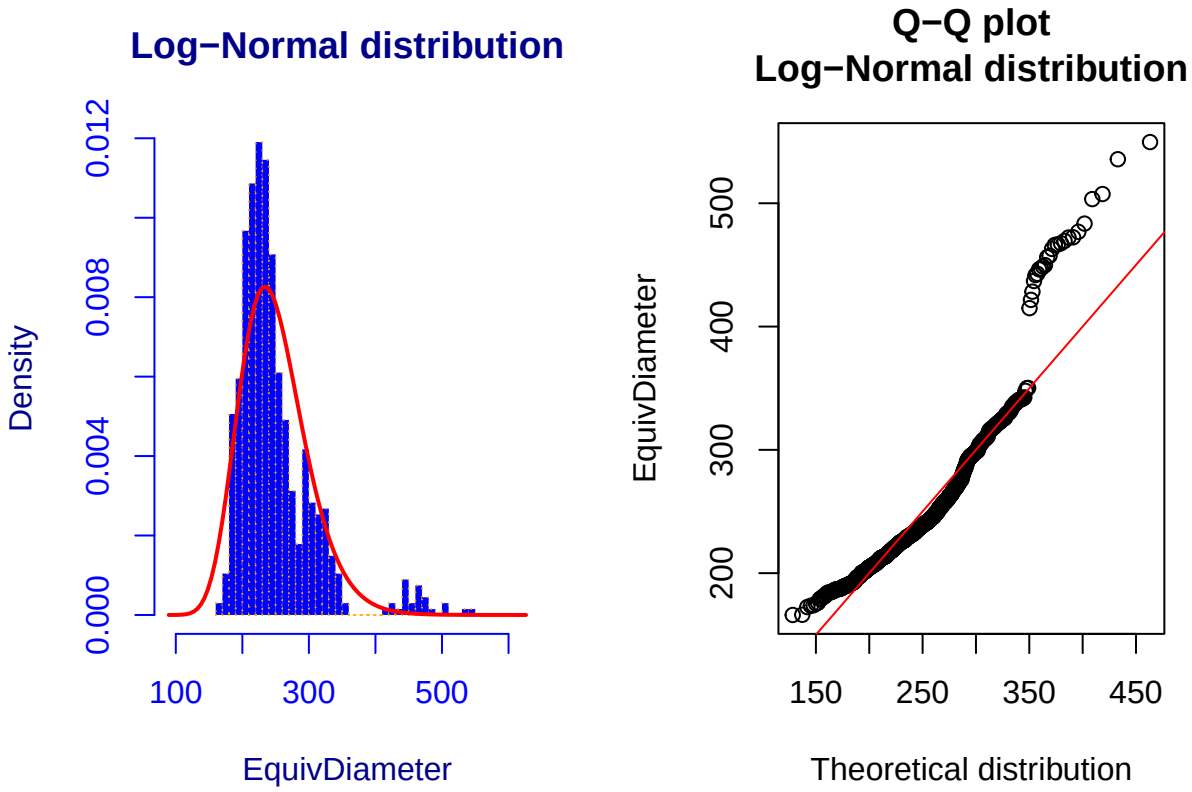
- **Log-Normal**
- **Gamma**
- **Weibull**
- **Exponential**
- **Inverse Gaussian**

4.8.1 Log-Normal

```
par(mfrow = c(1,2))
Log.N7 <- histDist(EquivDiameter,family = LOGNO,nbins = "fd",
                   main = "Log-Normal distribution",
                   col.hist="blue",border.hist="orange",line.col = "red"
                   ,ylab = "Density")

theoED <- ppoints(length(EquivDiameter))
mu <- Log.N7$mu.coefficients
sigma <- exp(Log.N7$sigma.coefficients)
Log.N7.Theoretical <- qLOGNO(theoED,mu= mu, sigma = sigma)
```

```
qqplot(x = Log.N7.Theoretical,y = EquivDiameter,
       xlab = "Theoretical distribution",
       main="Q-Q plot\nLog-Normal distribution")
abline(a = 0, b=1,col="red")
```

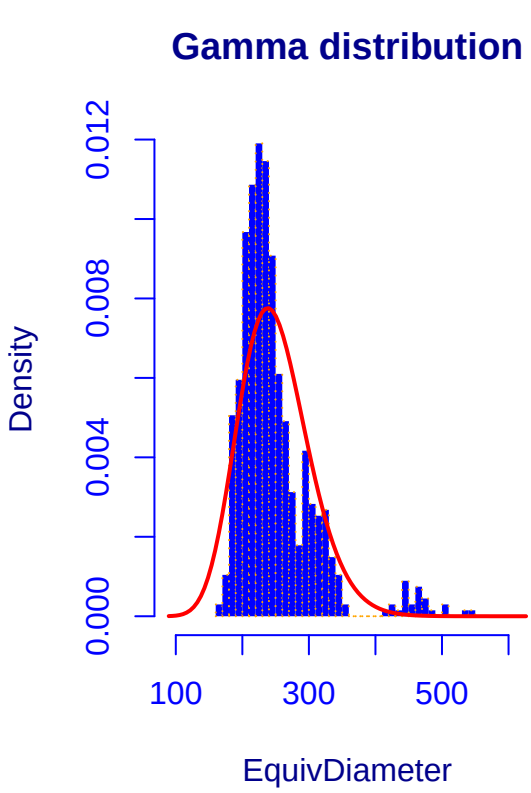


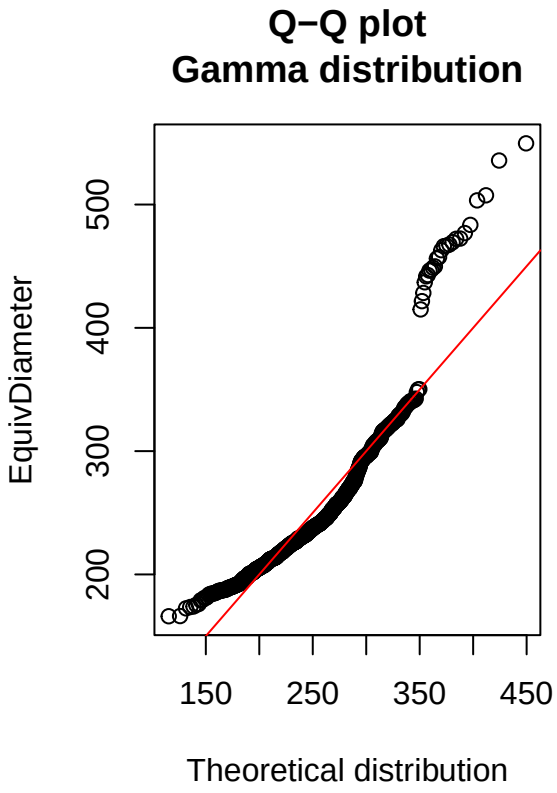
4.8.2 Gamma

```
par(mfrow = c(1,2))
Gamma.N7 <- histDist(EquivDiameter,family = GA ,nbins = "fd",
                     main = "Gamma distribution",
                     col.hist="blue",border.hist="orange",line.col = "red"
                     ,ylab = "Density")

mu <- exp(Gamma.N7$mu.coefficients)
sigma <- exp(Gamma.N7$sigma.coefficients)
Gamma.N7.Theoretical <- qGA(theoED,mu= mu, sigma = sigma)

qqplot(x = Gamma.N7.Theoretical,y = EquivDiameter,
       xlab = "Theoretical distribution",
       main="Q-Q plot\nGamma distribution")
abline(a = 0, b=1,col="red")
```



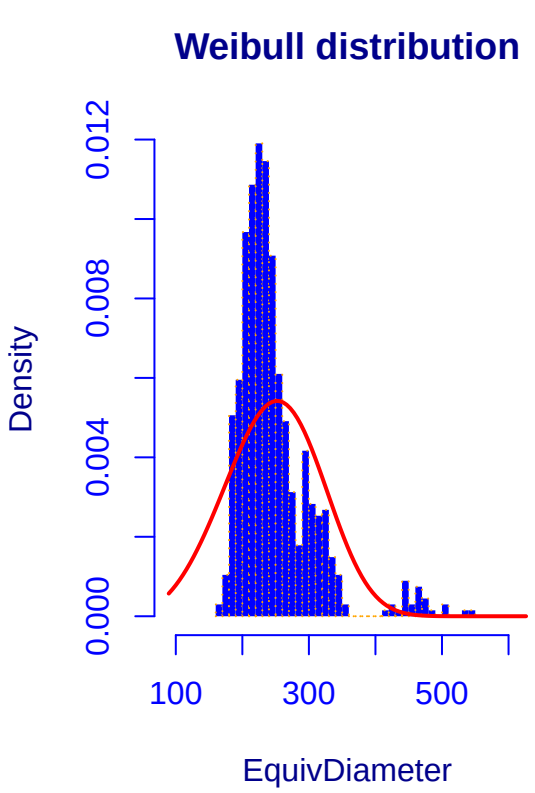


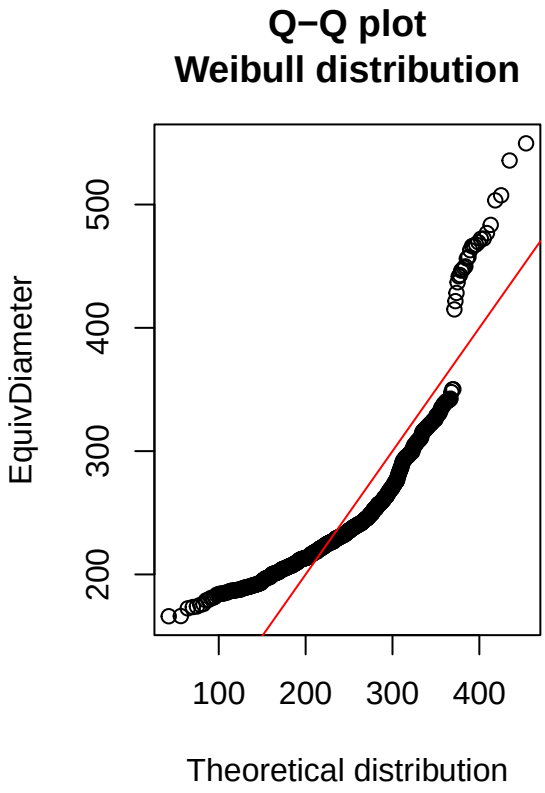
4.8.3 Weibull

```
par(mfrow = c(1,2))
Weibull.N7 <- histDist(EquivDiameter,family = WEI ,nbins = "fd",
                  main = "Weibull distribution",
                  col.hist="blue",border.hist="orange",line.col = "red"
                  ,ylab = "Density")

mu <- exp(Weibull.N7$mu.coefficients)
sigma <- exp(Weibull.N7$sigma.coefficients)
Weibull.N7.Theoretical <- qWEI(theoED,mu= mu, sigma = sigma)

qqplot(x = Weibull.N7.Theoretical,y = EquivDiameter,
       xlab = "Theoretical distribution",
       main="Q-Q plot\nWeibull distribution")
abline(a = 0, b=1,col="red")
```



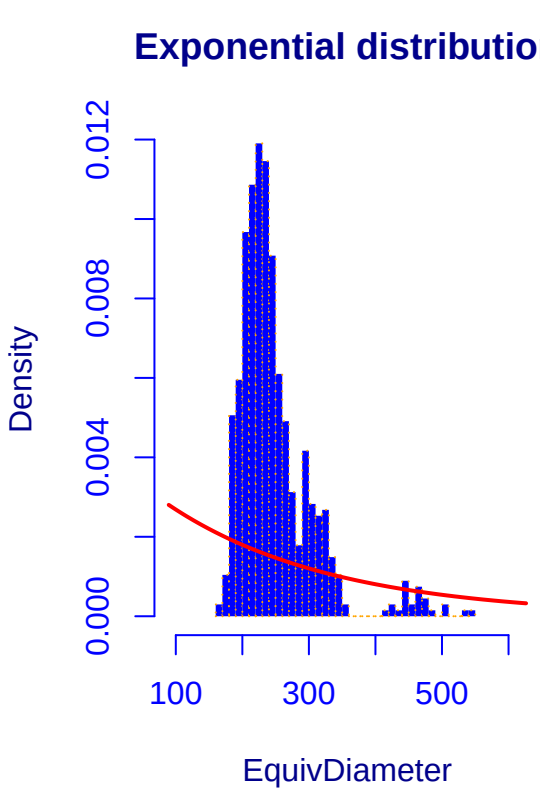


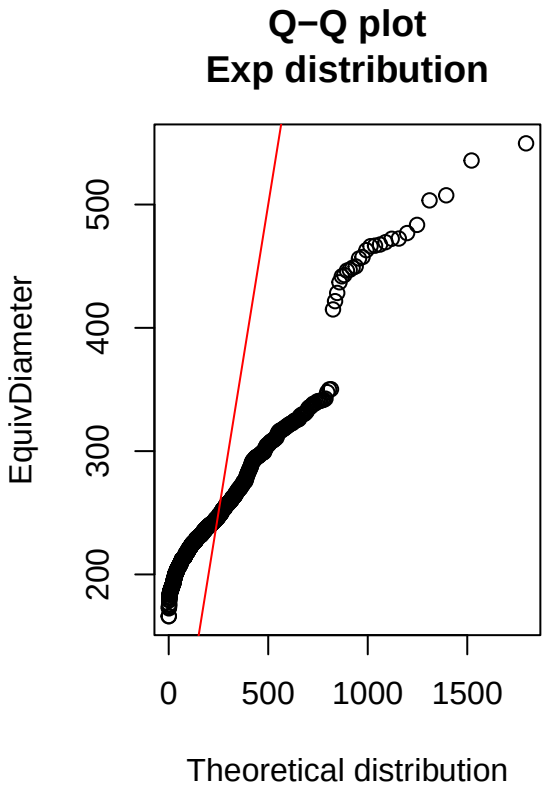
4.8.4 Exponential

```
par(mfrow = c(1,2))
Exp.N7 <- histDist(EquivDiameter,family = EXP ,nbins = "fd",
                   main = "Exponential distribution",
                   col.hist="blue",border.hist="orange",line.col = "red"
                   ,ylab = "Density")

mu <- exp(Exp.N7$mu.coefficients)
Exp.N7.Theoretical <- qEXP(theoED,mu= mu)

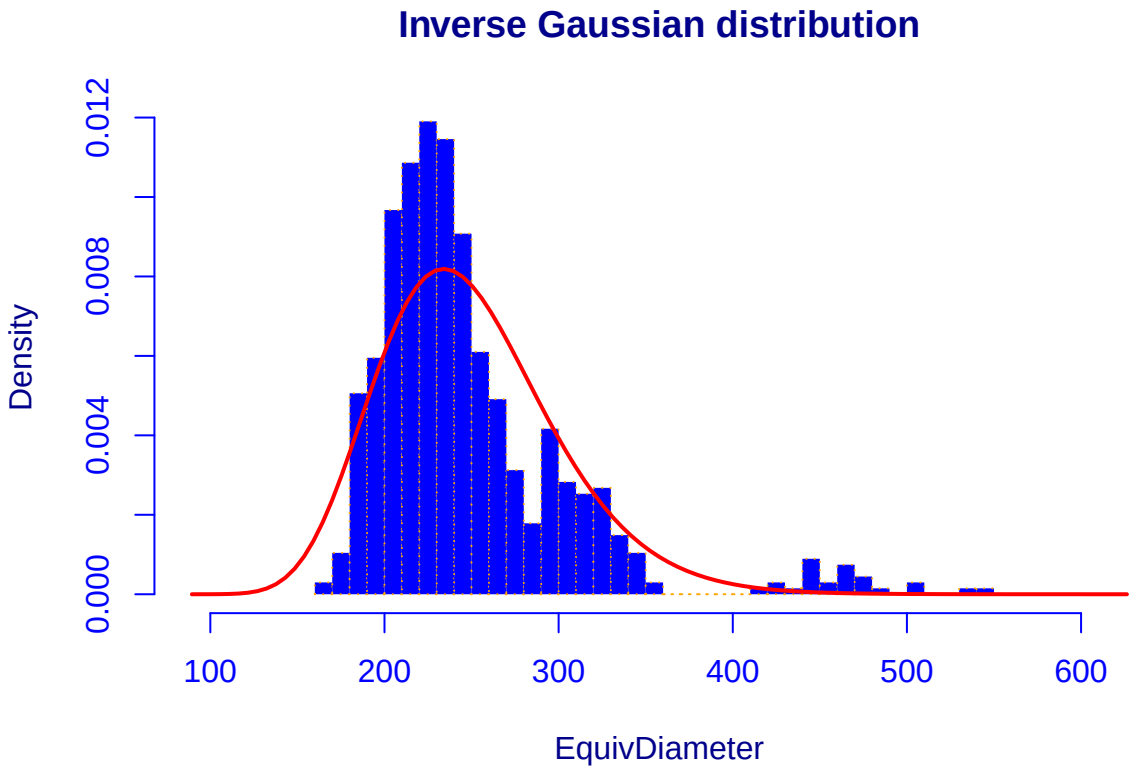
qqplot(x = Exp.N7.Theoretical,y = EquivDiameter,
       xlab = "Theoretical distribution",
       main="Q-Q plot\nExp distribution")
abline(a = 0, b=1,col="red")
```





4.8.5 Inverse Gaussian

```
Ig.N7 <- histDist(EquivDiameter,family = IG ,nbins = "fd",
                  main = "Inverse Gaussian distribution",
                  col.hist="blue",border.hist="orange",line.col = "red"
                  ,ylab = "Density")
```



In this case we can't use the Q-Q plot, because the variance is given by $\sigma^2\mu^3$, so R cannot generates easily value on that range.

4.8.6 Comparison

```
EquivDiameter.comp <- data.frame(AIC=c(Log.N7$aic,Gamma.N7$aic,Weibull.N7$aic,
                                       Exp.N7$aic,Ig.N7$aic),
                         BIC = c(Log.N7$sbc,Gamma.N7$sbc,Weibull.N7$sbc,
                                 Exp.N7$sbc,Ig.N7$sbc),
                         row.names=c("Log-Normal","Gamma","Weibull",
                                     "Exponential","Inverse Gaussian"))
EquivDiameter.comp <- EquivDiameter.comp[order(EquivDiameter.comp$AIC,
                                               decreasing = FALSE), ]
EquivDiameter.comp
```

```
##                         AIC      BIC
## Log-Normal         7148.671 7157.692
## Inverse Gaussian   7152.112 7161.132
## Gamma              7211.376 7220.397
## Weibull            7495.189 7504.210
## Exponential        8762.745 8767.255
```

From the AIC and BIC criteria the better model that fits the data in terms of parsimony

and accuracy is the Log-Normal distribution.

4.9 Extent

```
summary(Extent)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.5668  0.7137  0.7566  0.7490  0.7877  0.8519
```

```
V<- var(Extent)
cat("Variance : ",V,"\n")
```

```
## Variance :  0.002409926
```

```
Sd <- sd(Extent)
cat("Standard deviation : ",Sd,"\n")
```

```
## Standard deviation :  0.04909099
```

```
Extent.table <- table(Extent)
mode <- names(Extent.table)[which.max(Extent.table)]
cat("Mode : ",mode,"\n")
```

```
## Mode :  0.566767527
```

```
K <- kurtosis(Extent)
cat("Kurtosis : ",K,"\n")
```

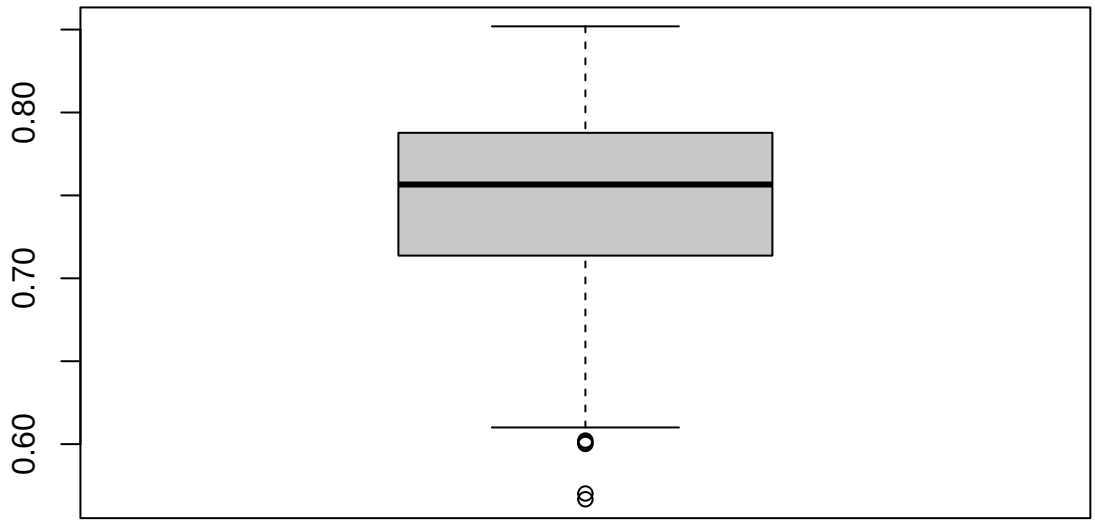
```
## Kurtosis :  3.140386
```

```
S <- skewness(Extent)
cat("Skewness : ",S,"\n")
```

```
## Skewness :  -0.6578401
```

The data are soft left-skewed, as observed from the skewness index. The high kurtosis indicates that the distribution is leptokurtic, meaning there is a great concentration of values around the mean, but also a high frequency of outliers that are far away from it.

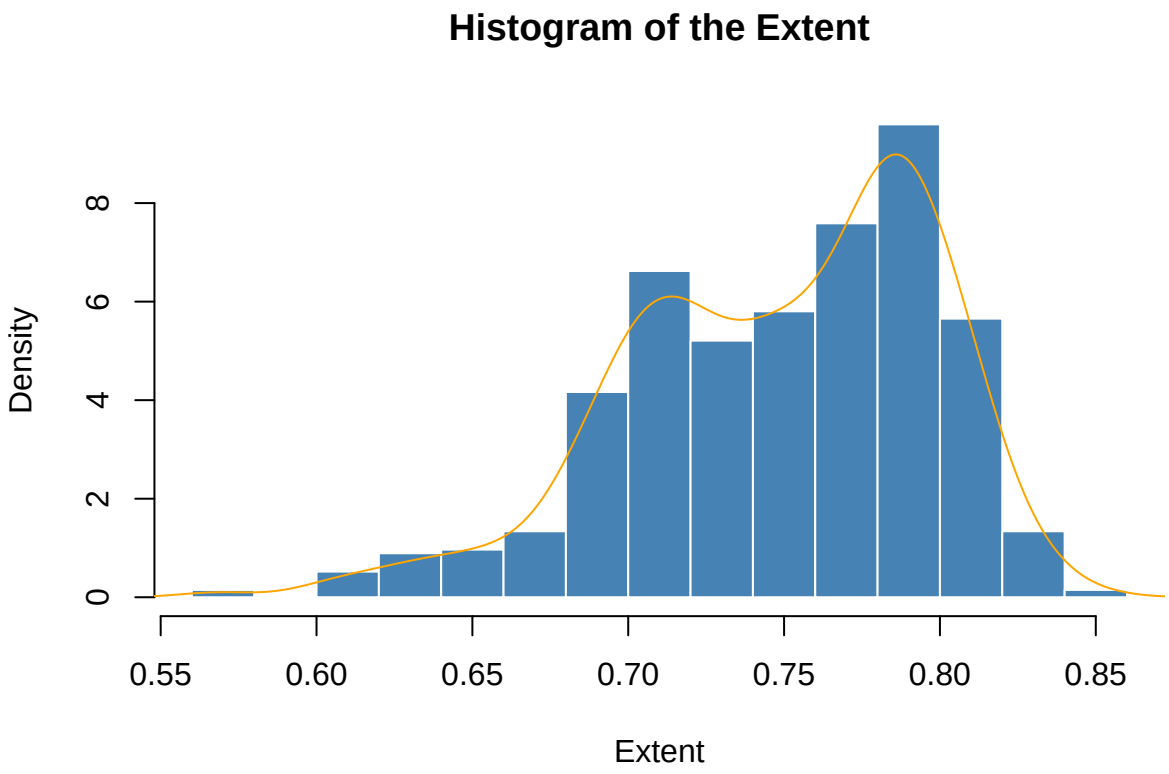
```
box <- boxplot(Extent,main="Box plot of Extent")
```




```
out <- length(box$out)/length(Extent)
cat("Percentage of outliers : ",out*100,"%")
```

```
## Percentage of outliers :  0.7440476 %
```

```
hist(Extent,
     breaks = "FD",
     main = "Histogram of the Extent",
     col = "steelblue",
     border = "white",
     freq = FALSE)
lines(density(Extent),col = "orange")
```



Now we can try to fit different models to the data. First of all we observe that the support of the data is $[0, 1]$. So the distribution from which we could choose are:

- **Beta**
- **Simplex**

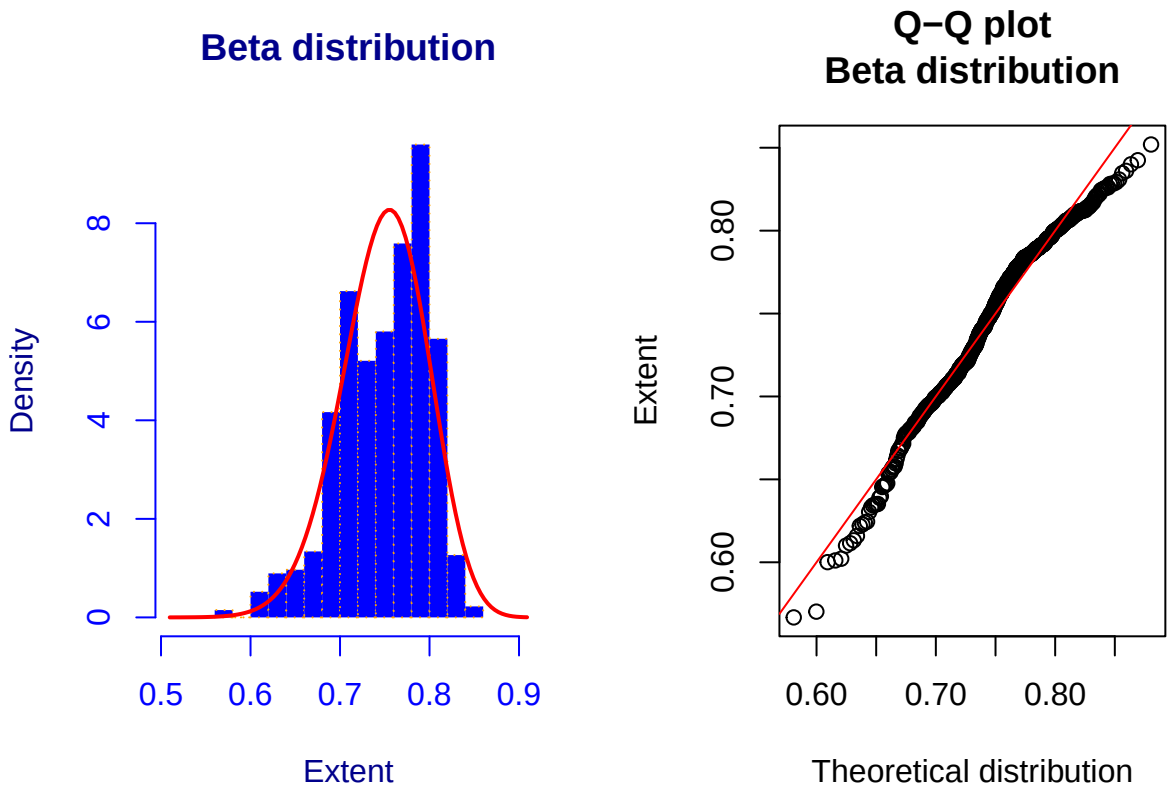
4.9.1 Beta

```
par(mfrow = c(1,2))
Beta.N2 <- histDist(y = Extent, family = BE, nbins = "fd",
                    main = "Beta distribution",
                    col.hist="blue",border.hist="orange",line.col = "red"
                    ,ylab = "Density")

theoEx <- ppoints(length(Extent))
mu <- plogis(Beta.N2$mu.coefficients)
sigma <- plogis(Beta.N2$sigma.coefficients)
Beta.N2.Theoretical <- qBE(theoEx,mu= mu, sigma = sigma)

qqplot(x = Beta.N2.Theoretical,y = Extent,
       xlab = "Theoretical distribution",
       main="Q-Q plot\nBeta distribution")
```

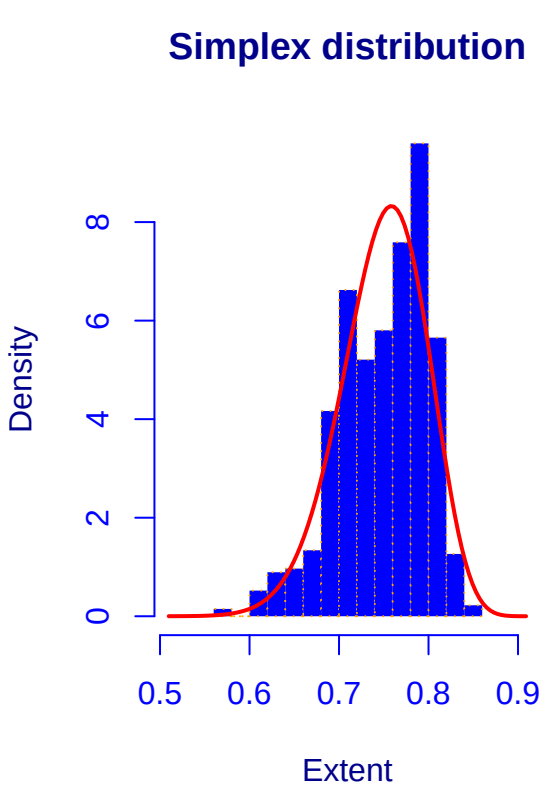
```
abline(a = 0, b=1,col="red")
```

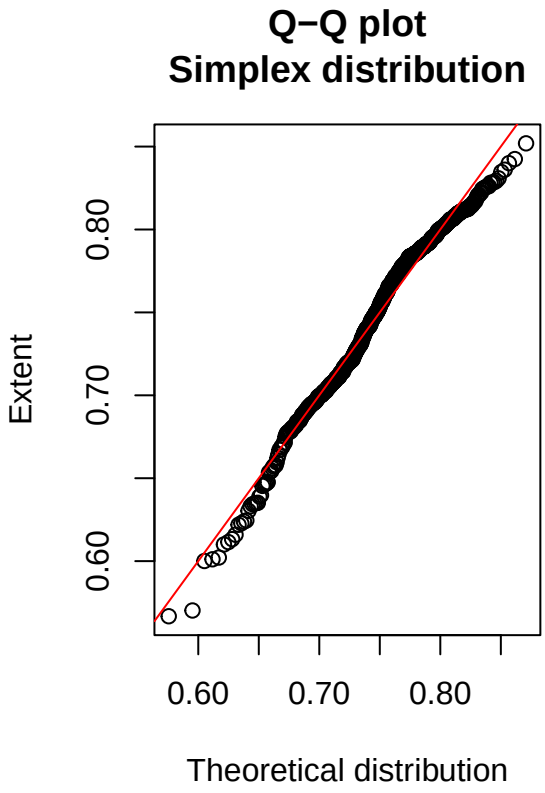


4.9.2 Simplex

```
par(mfrow = c(1,2))
Sx.N2<- histDist(y = Extent, family = SIMPLEX, nbins = "fd",
                 main = "Simplex distribution",
                 col.hist="blue",border.hist="orange",line.col = "red"
                 ,ylab = "Density")
mu <- plogis(Sx.N2$mu.coefficients)
sigma <- exp(Sx.N2$sigma.coefficients)
Sx.N2.Theoretical <- qSIMPLEX(theoEx,mu= mu, sigma = sigma)

qqplot(x = Sx.N2.Theoretical,y = Extent,
       xlab = "Theoretical distribution",
       main="Q-Q plot\nSimplex distribution")
abline(a = 0, b=1,col="red")
```





4.9.3 Comparison

```
Extent.comp <- data.frame(AIC=c(Beta.N2$aic,Sx.N2$aic),
                          BIC = c(Beta.N2$sbc,Sx.N2$sbc),
                          row.names=c("Beta","Simplex"))
Extent.comp <- Extent.comp[order(Extent.comp$AIC,
                                                    decreasing = FALSE), ]
Extent.comp
```

```
##                AIC       BIC
## Simplex -2189.114 -2180.093
## Beta    -2174.884 -2165.864
```

From the AIC and BIC criteria the better model that fits the data in terms of parsimony and accuracy is the Simplex distribution.

4.10 Solidity

```
summary(Solidity)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
```

```
##  0.9477  0.9858  0.9883  0.9872  0.9899  0.9937
```

```
V<- var(Solidity)
cat("Variance : ",V,"\n")
```

```
## Variance :  2.137441e-05
```

```
Sd <- sd(Solidity)
cat("Standard deviation : ",Sd,"\n")
```

```
## Standard deviation :  0.004623247
```

```
Solidity.table <- table(Solidity)
mode <- names(Solidity.table)[which.max(Solidity.table)]
cat("Mode : ",mode,"\n")
```

```
## Mode :  0.947666764
```

```
K <- kurtosis(Solidity)
cat("Kurtosis : ",K,"\n")
```

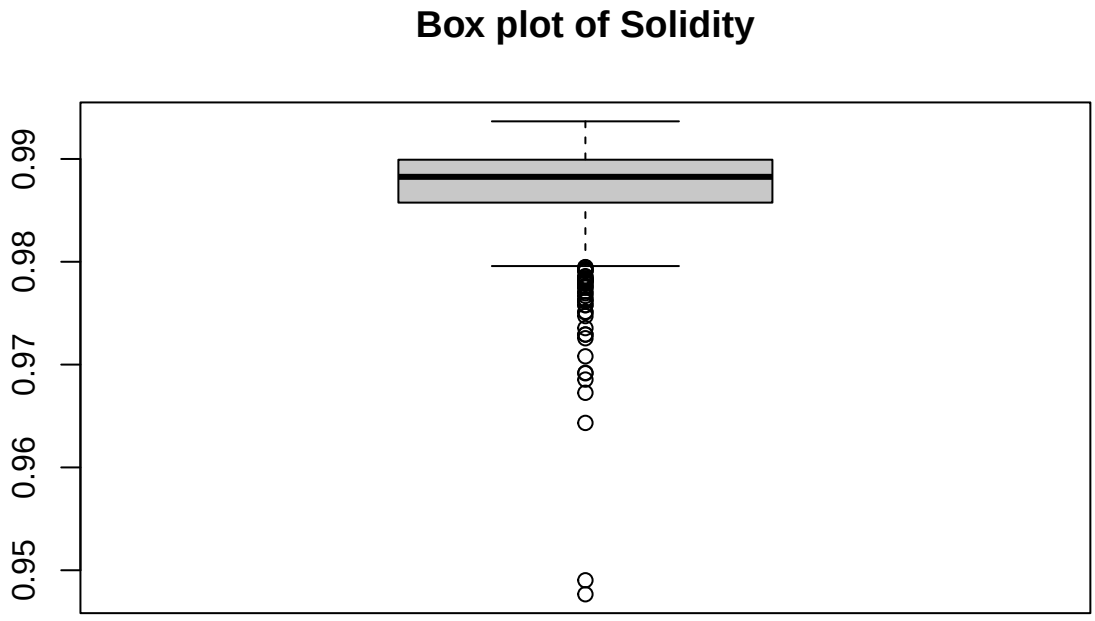
```
## Kurtosis :  19.4328
```

```
S <- skewness(Solidity)
cat("Skewness : ",S,"\n")
```

```
## Skewness :  -2.962927
```

The data are left-skewed, as observed from the skewness index. The high kurtosis indicates that the distribution is leptokurtic, meaning there is a great concentration of values around the mean, but also a high frequency of outliers that are far away from it.

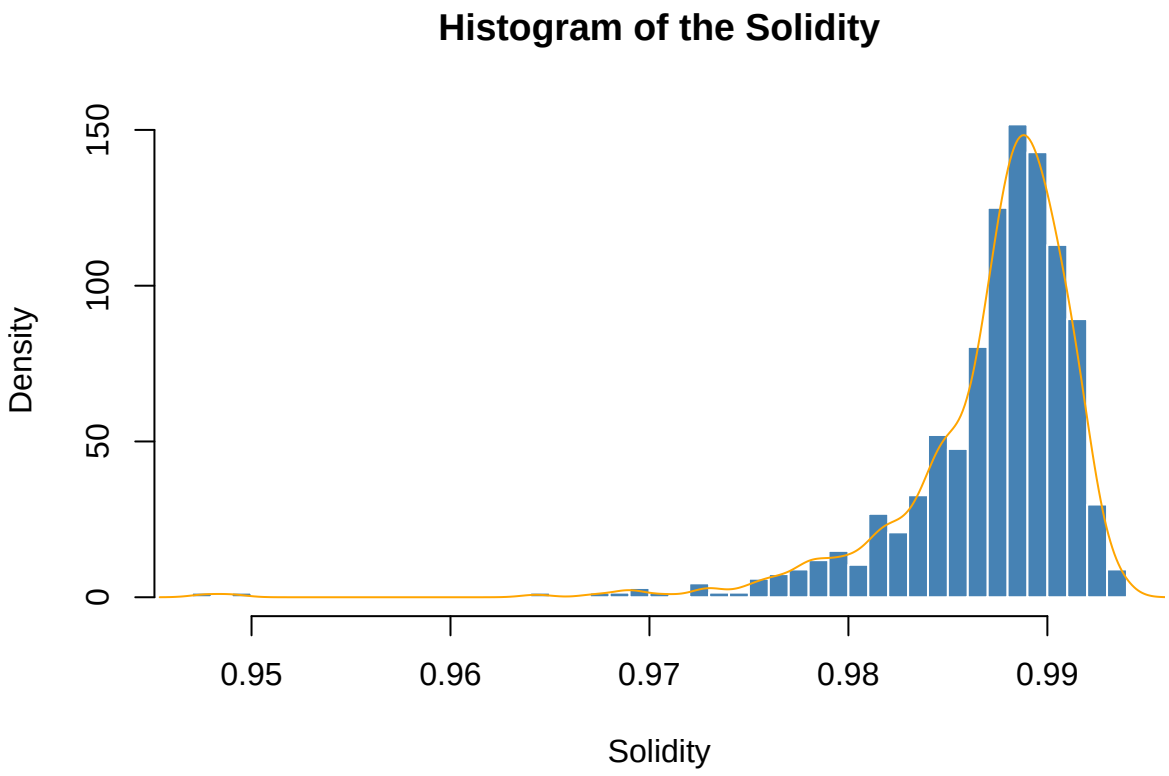
```
box <- boxplot(Solidity,main="Box plot of Solidity")
```




```
out <- length(box$out)/length(Solidity)
cat("Percentage of outliers : ",out*100,"%")
```

```
## Percentage of outliers :  6.25 %
```

```
hist(Solidity,
     breaks = "FD",
     main = "Histogram of the Solidity",
     col = "steelblue",
     border = "white",
     freq = FALSE)
lines(density(Solidity),col = "orange")
```



Now we can try to fit different models to the data. First of all we observe that the support of the data is $[0, 1]$. So the distribution from which we could choose are:

- **Beta**
- **Simplex**

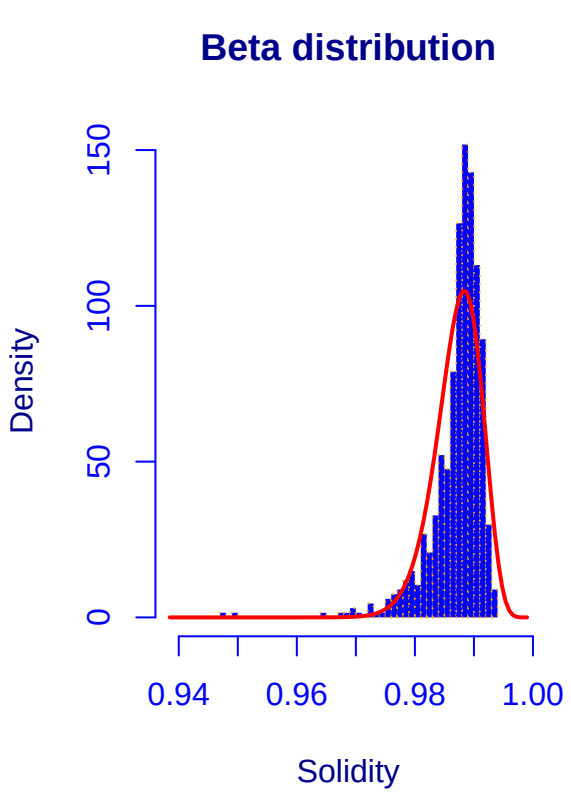
4.10.1 Beta

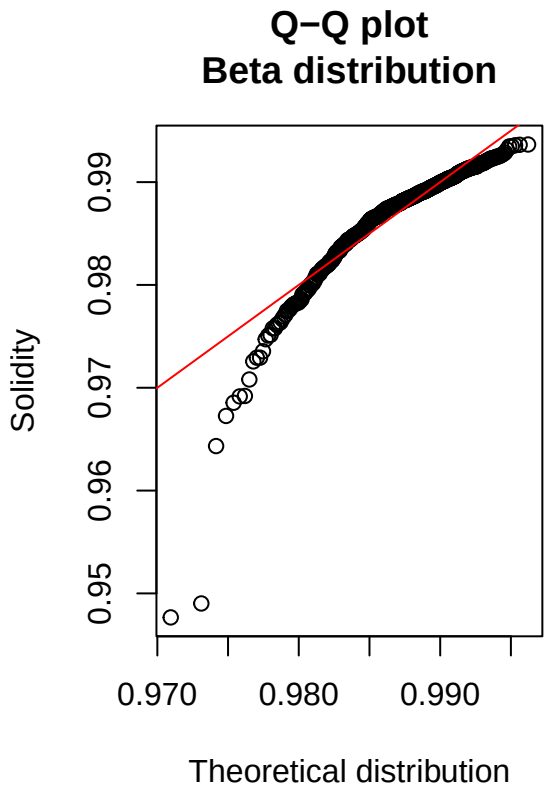
```
par(mfrow = c(1,2))
Beta.N3 <- histDist(y = Solidity, family = BE, nbins = "fd",
                    main = "Beta distribution",
                    col.hist="blue",border.hist="orange",line.col = "red"
                    ,ylab = "Density")

theoS <- ppoints(length(Solidity))
mu <- plogis(Beta.N3$mu.coefficients)
sigma <- plogis(Beta.N3$sigma.coefficients)
Beta.N3.Theoretical <- qBE(theoS,mu= mu, sigma = sigma)

qqplot(x = Beta.N3.Theoretical,y = Solidity,
       xlab = "Theoretical distribution",
       main="Q-Q plot\nBeta distribution")
```

```
abline(a = 0, b=1,col="red")
```

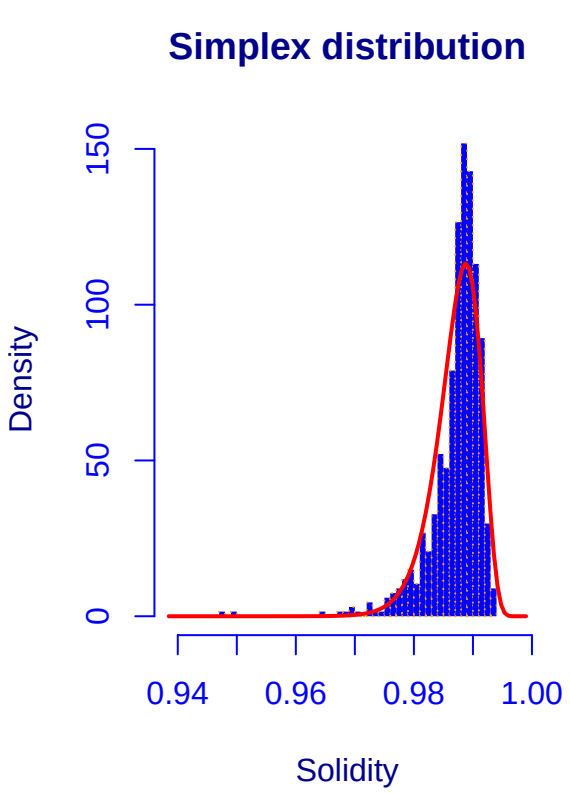


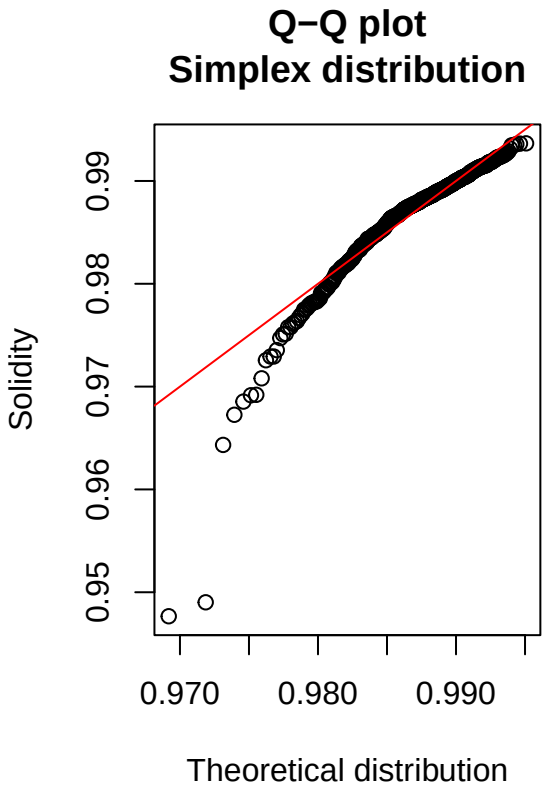


4.10.2 Simplex

```
par(mfrow = c(1,2))
Sx.N3<- histDist(y = Solidity, family = SIMPLEX, nbins = "fd",
                 main = "Simplex distribution",
                 col.hist="blue",border.hist="orange",line.col = "red"
                 ,ylab = "Density")
mu <- plogis(Sx.N3$mu.coefficients)
sigma <- exp(Sx.N3$sigma.coefficients)
Sx.N3.Theoretical <- qSIMPLEX(theoS,mu= mu, sigma = sigma)

qqplot(x = Sx.N3.Theoretical,y = Solidity,
       xlab = "Theoretical distribution",
       main="Q-Q plot\nSimplex distribution")
abline(a = 0, b=1,col="red")
```





4.10.3 Comparison

```
Solidity.comp <- data.frame(AIC=c(Beta.N3$aic,Sx.N3$aic),
                        BIC = c(Beta.N3$sbc,Sx.N3$sbc),
                        row.names=c("Beta","Simplex"))
Solidity.comp <- Solidity.comp[order(Solidity.comp$AIC,
                                                  decreasing = FALSE), ]
Solidity.comp
```

```
##                AIC       BIC
## Simplex -5642.795 -5633.775
## Beta    -5568.846 -5559.825
```

From the AIC and BIC criteria the better model that fits the data in terms of parsimony and accuracy is the Simplex distribution.

4.11 Roundness

```
summary(roundness)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
```

```
##  0.6690  0.8366  0.8854  0.8746  0.9157  0.9859
```

```
V<- var(roundness)
cat("Variance : ",V,"\n")
```

```
## Variance :  0.003377954
```

```
Sd <- sd(roundness)
cat("Standard deviation : ",Sd,"\n")
```

```
## Standard deviation :  0.05812017
```

```
roundness.table <- table(roundness)
mode <- names(roundness.table)[which.max(roundness.table)]
cat("Mode : ",mode,"\n")
```

```
## Mode :  0.668990042
```

```
K <- kurtosis(roundness)
cat("Kurtosis : ",K,"\n")
```

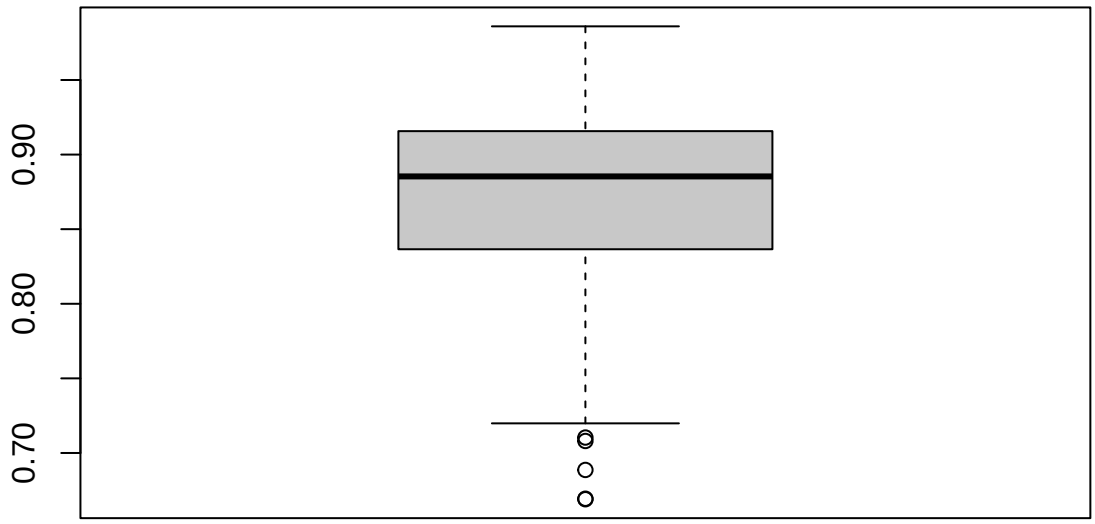
```
## Kurtosis :  2.983423
```

```
S <- skewness(roundness)
cat("Skewness : ",S,"\n")
```

```
## Skewness :  -0.6213146
```

The data are soft left-skewed, as observed from the skewness index. The high kurtosis indicates that the distribution is leptokurtic, meaning there is a great concentration of values around the mean, but also a high frequency of outliers that are far away from it.

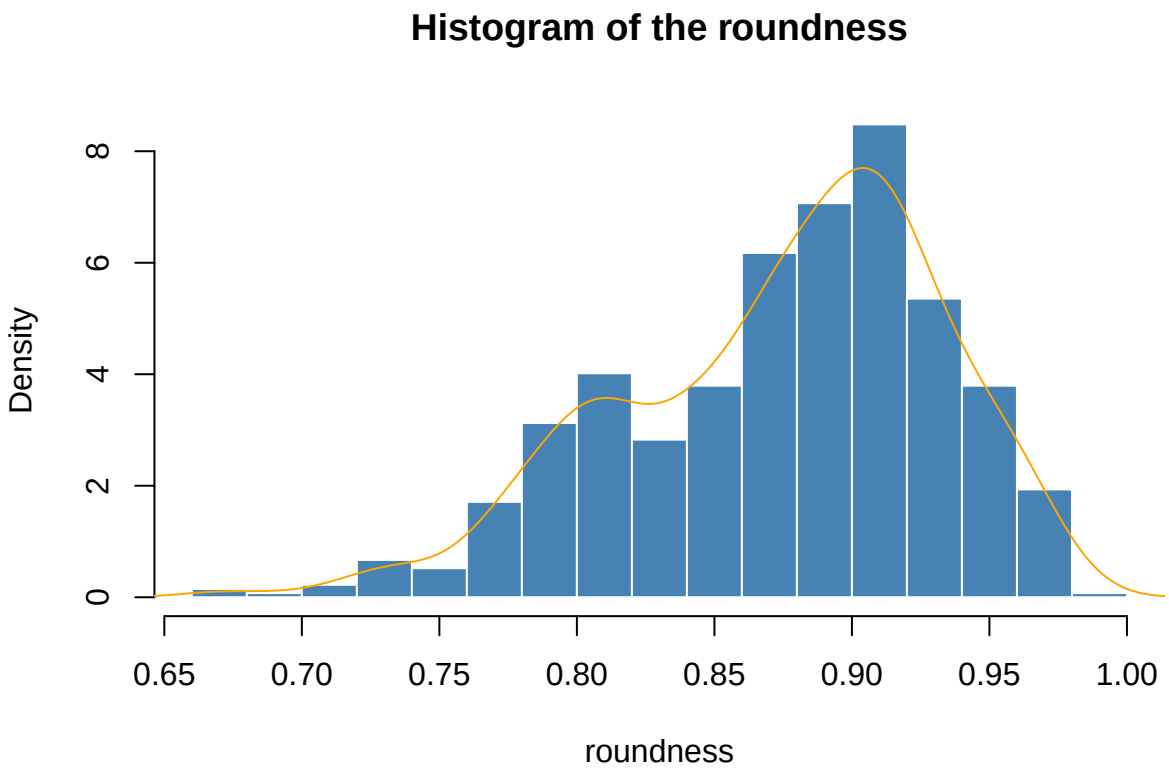
```
box <- boxplot(roundness,main="Box plot of roundness")
```




```
out <- length(box$out)/length(roundness)
cat("Percentage of outliers : ",out*100,"%")
```

```
## Percentage of outliers :  0.7440476 %
```

```
hist(roundness,
     breaks = "FD",
     main = "Histogram of the roundness",
     col = "steelblue",
     border = "white",
     freq = FALSE)
lines(density(roundness),col = "orange")
```



Now we can try to fit different models to the data. First of all we observe that the support of the data is $[0, 1]$. So the distribution from which we could choose are:

- **Beta**
- **Simplex**

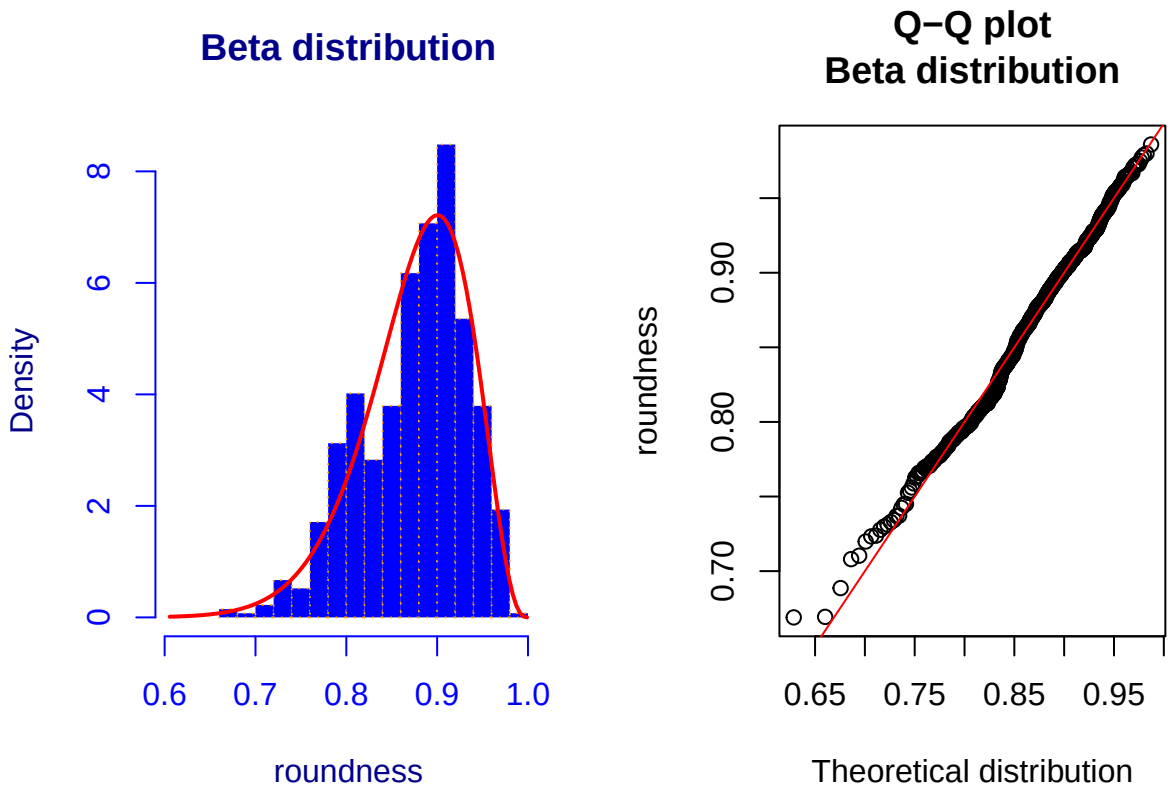
4.11.1 Beta

```
par(mfrow = c(1,2))
Beta.N4 <- histDist(y = roundness, family = BE, nbins = "fd",
                    main = "Beta distribution",
                    col.hist="blue",border.hist="orange",line.col = "red"
                    ,ylab = "Density")

theoR <- ppoints(length(roundness))
mu <- plogis(Beta.N4$mu.coefficients)
sigma <- plogis(Beta.N4$sigma.coefficients)
Beta.N4.Theoretical <- qBE(theoR,mu= mu, sigma = sigma)

qqplot(x = Beta.N4.Theoretical,y = roundness,
       xlab = "Theoretical distribution",
       main="Q-Q plot\nBeta distribution")
```

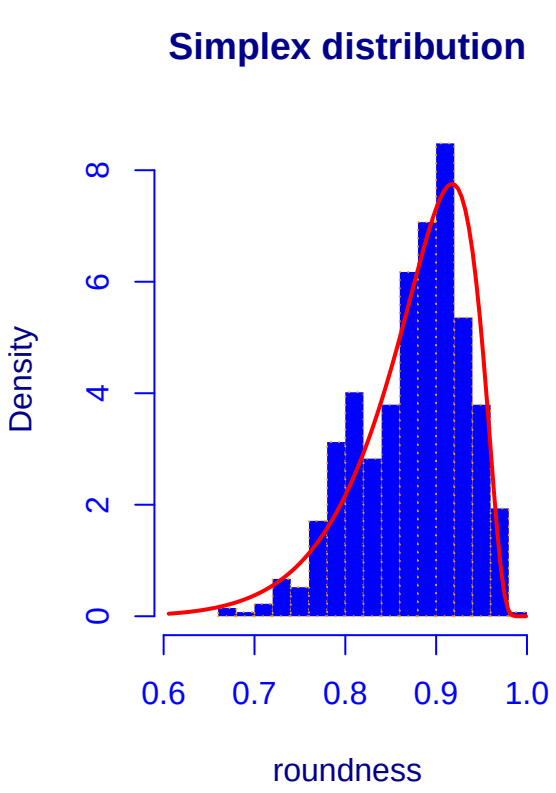
```
abline(a = 0, b=1,col="red")
```

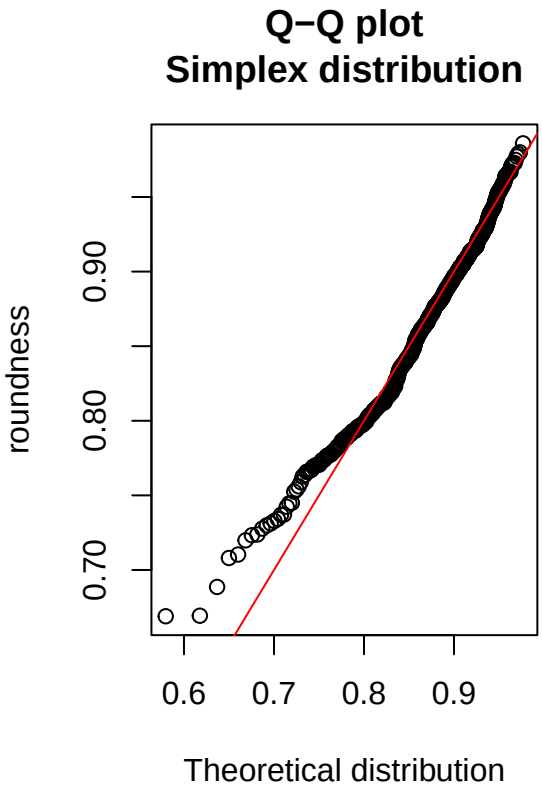


4.11.2 Simplex

```
par(mfrow = c(1,2))
Sx.N4<- histDist(y = roundness, family = SIMPLEX, nbins = "fd",
                 main = "Simplex distribution",
                 col.hist="blue",border.hist="orange",line.col = "red"
                 ,ylab = "Density")
mu <- plogis(Sx.N4$mu.coefficients)
sigma <- exp(Sx.N4$sigma.coefficients)
Sx.N4.Theoretical <- qSIMPLEX(theoR,mu= mu, sigma = sigma)

qqplot(x = Sx.N4.Theoretical,y = roundness,
       xlab = "Theoretical distribution",
       main="Q-Q plot\nSimplex distribution")
abline(a = 0, b=1,col="red")
```





4.11.3 Comparison

```
roundness.comp <- data.frame(AIC=c(Beta.N4$aic,Sx.N4$aic),
                      BIC = c(Beta.N4$sbc,Sx.N4$sbc),
                      row.names=c("Beta","Simplex"))
roundness.comp <- roundness.comp[order(roundness.comp$AIC,
                                                  decreasing = FALSE), ]
roundness.comp
```

```
##                AIC       BIC
## Beta    -1985.938 -1976.917
## Simplex -1949.654 -1940.633
```

From the AIC and BIC criteria the better model that fits the data in terms of parsimony and accuracy is the Beta distribution.

4.12 Compactness

```
summary(Compactness)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
```

```
##  0.6594  0.7633  0.8028  0.8005  0.8341  0.9596
```

```
V<- var(Compactness)
cat("Variance : ",V,"\n")
```

```
## Variance :  0.003844059
```

```
Sd <- sd(Compactness)
cat("Standard deviation : ",Sd,"\n")
```

```
## Standard deviation :  0.06200048
```

```
Compactness.table <- table(Compactness)
mode <- names(Compactness.table)[which.max(Compactness.table)]
cat("Mode : ",mode,"\n")
```

```
## Mode :  0.659401795
```

```
K <- kurtosis(Compactness)
cat("Kurtosis : ",K,"\n")
```

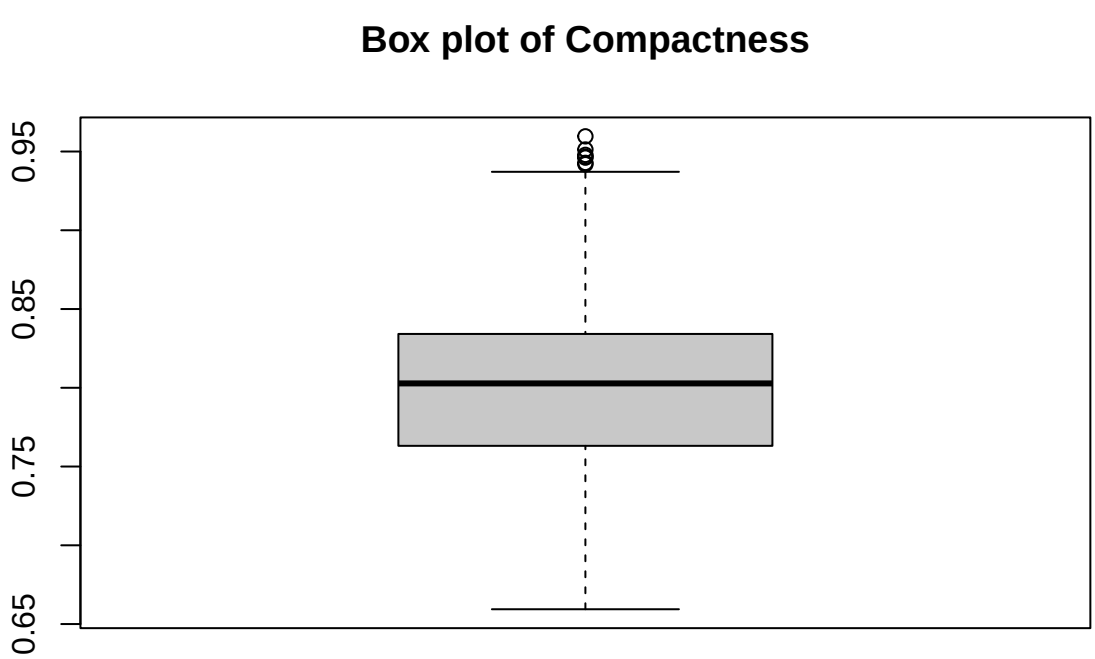
```
## Kurtosis :  2.766124
```

```
S <- skewness(Compactness)
cat("Skewness : ",S,"\n")
```

```
## Skewness :  0.1028096
```

The data are right-skewed, as observed from the skewness index. The high kurtosis indicates that the distribution is leptokurtic, meaning there is a great concentration of values around the mean, but also a high frequency of outliers that are far away from it.

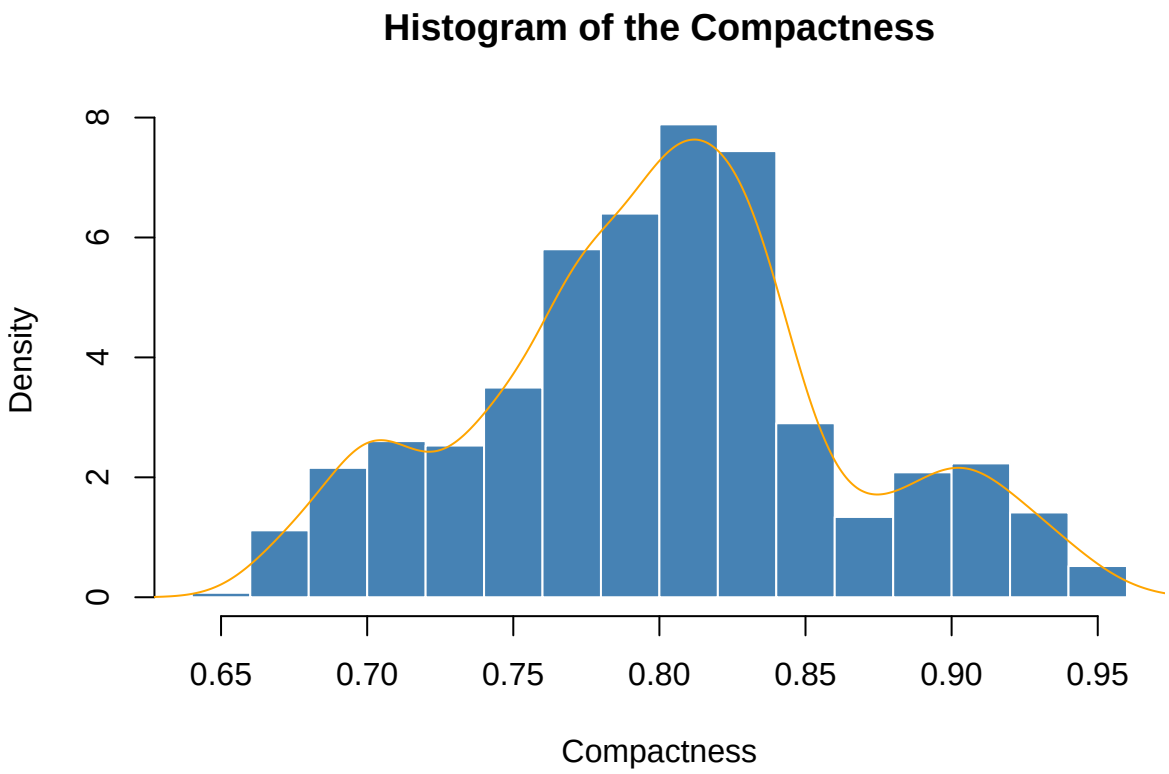
```
box <- boxplot(Compactness,main="Box plot of Compactness")
```




```
out <- length(box$out)/length(Compactness)
cat("Percentage of outliers : ",out*100,"%")
```

```
## Percentage of outliers :  1.041667 %
```

```
hist(Compactness,
     breaks = "FD",
     main = "Histogram of the Compactness",
     col = "steelblue",
     border = "white",
     freq = FALSE)
lines(density(Compactness),col = "orange")
```



Now we can try to fit different models to the data. First of all we observe that the support of the data is $[0, 1]$. So the distribution from which we could choose are:

- **Beta**
- **Simplex**

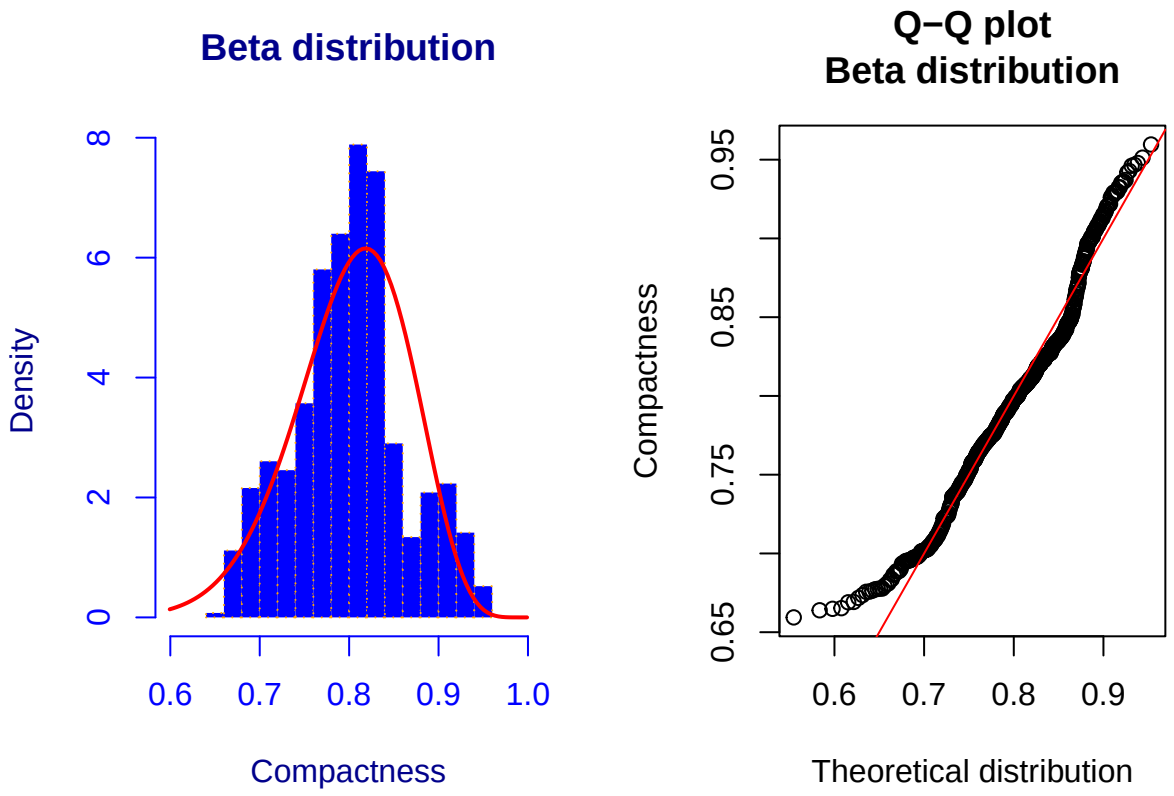
4.12.1 Beta

```
par(mfrow = c(1,2))
Beta.N5 <- histDist(y = Compactness, family = BE, nbins = "fd",
                    main = "Beta distribution",
                    col.hist="blue",border.hist="orange",line.col = "red"
                    ,ylab = "Density")

theoC <- ppoints(length(Compactness))
mu <- plogis(Beta.N5$mu.coefficients)
sigma <- plogis(Beta.N5$sigma.coefficients)
Beta.N5.Theoretical <- qBE(theoC,mu= mu, sigma = sigma)

qqplot(x = Beta.N5.Theoretical,y = Compactness,
       xlab = "Theoretical distribution",
       main="Q-Q plot\nBeta distribution")
```

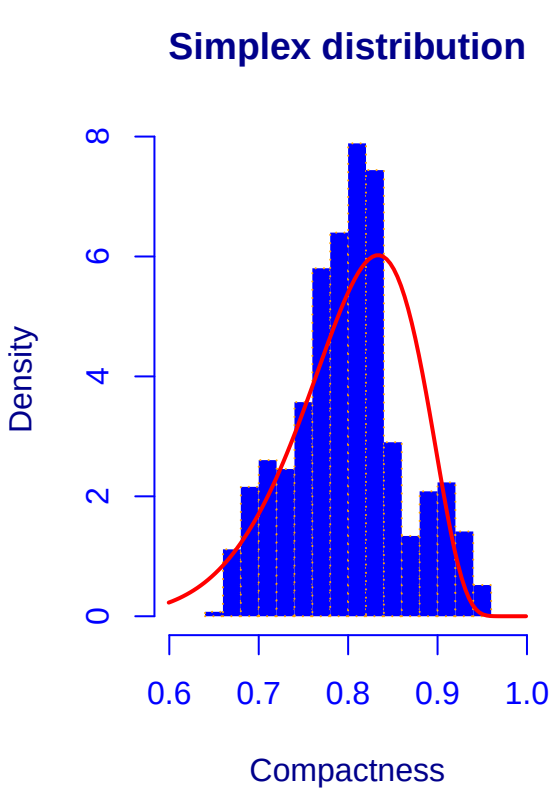
```
abline(a = 0, b=1,col="red")
```

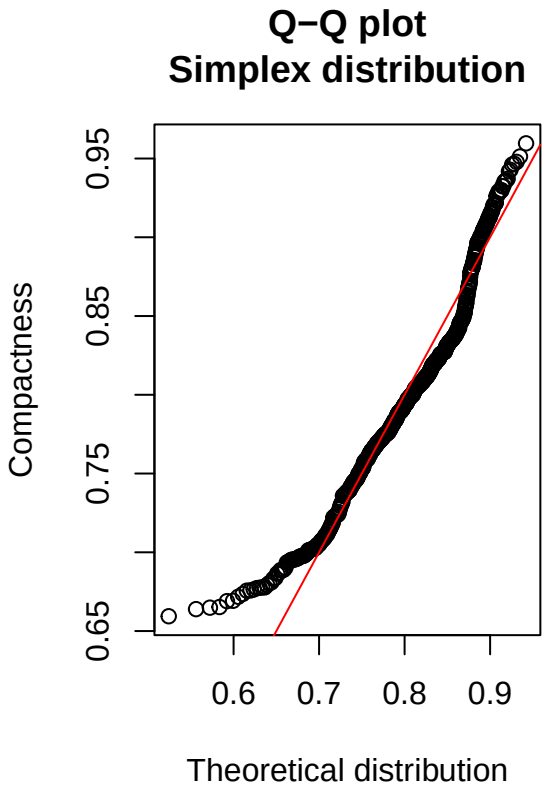


4.12.2 Simplex

```
par(mfrow = c(1,2))
Sx.N5<- histDist(y = Compactness, family = SIMPLEX, nbins = "fd",
                 main = "Simplex distribution",
                 col.hist="blue",border.hist="orange",line.col = "red"
                 ,ylab = "Density")
mu <- plogis(Sx.N5$mu.coefficients)
sigma <- exp(Sx.N5$sigma.coefficients)
Sx.N5.Theoretical <- qSIMPLEX(theoC,mu= mu, sigma = sigma)

qqplot(x = Sx.N5.Theoretical,y = Compactness,
       xlab = "Theoretical distribution",
       main="Q-Q plot\nSimplex distribution")
abline(a = 0, b=1,col="red")
```





4.12.3 Comparison

```
Compactness.comp <- data.frame(AIC=c(Beta.N5$aic,Sx.N5$aic),
                       BIC = c(Beta.N5$sbc,Sx.N5$sbc),
                       row.names=c("Beta","Simplex"))
Compactness.comp <- Compactness.comp[order(Compactness.comp$AIC,
                                           decreasing = FALSE), ]
Compactness.comp
```

```
##                AIC       BIC
## Beta     -1785.611 -1776.591
## Simplex -1719.385 -1710.364
```

From the AIC and BIC criteria the better model that fits the data in terms of parsimony and accuracy is the Beta distribution.

4.13 Class

```
summary(Class)
```

```
##    Length     Class      Mode
```

```
##       672 character character
```

```
cat("Frequency")
```

```
## Frequency
```

```
table(Class)
```

```
## Class
## BARBUNYA   BOMBAY     CALI DERMASON    HOROZ    SEKER     SIRA
##       47       25       67      188       97       96      152
```

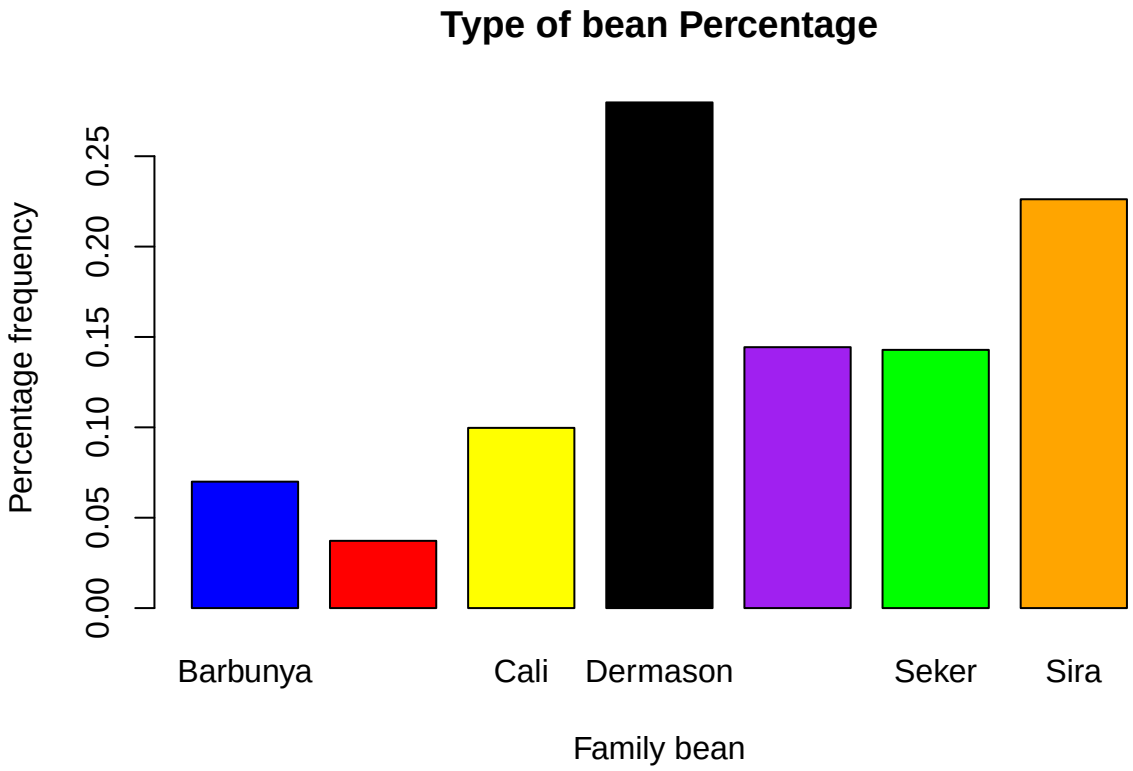
```
frequency <- prop.table(table(Class))*100
cat("Percentage frequency\n")
```

```
## Percentage frequency
```

```
frequency
```

```
## Class
##  BARBUNYA    BOMBAY      CALI  DERMASON     HOROZ     SEKER      SIRA
##  6.994048  3.720238  9.970238 27.976190 14.434524 14.285714 22.619048
```

```
barplot(table(Class)/length(Class),space = 0.3,
        col = c("blue","red","yellow","black","purple",
                "green","orange"),
        main="Type of bean Percentage",
        xlab="Family bean",
        ylab="Percentage frequency",
        names.arg = c("Barbunya","Bombay",
                      "Cali","Dermason",
                      "Horoz","Seker",
                      "Sira"))
```



5 PCA

5.1 Correlation

Before doing our analysis for the PCA approach we first of all analyze the correlation of the data. First of all we erase the class feature from the analysis, we will use it later in the clustering.

```
Beans.w.c <- Beans[,1:12]
```

We know from the description of the data that some features are correlated with others, for example Roundness is a ratio between the Area and the Perimeter. To emphasize the correlation between variables we use the function cor, but specifying the method "Spearman". This is different from the common Pearson, because is not parametric and it is used for variables that are not normally distributed and whene the correlation is possibly linear or not.

```
Beans.corr <- cor(Beans.w.c,method = "spearman")
Beans.corr
```

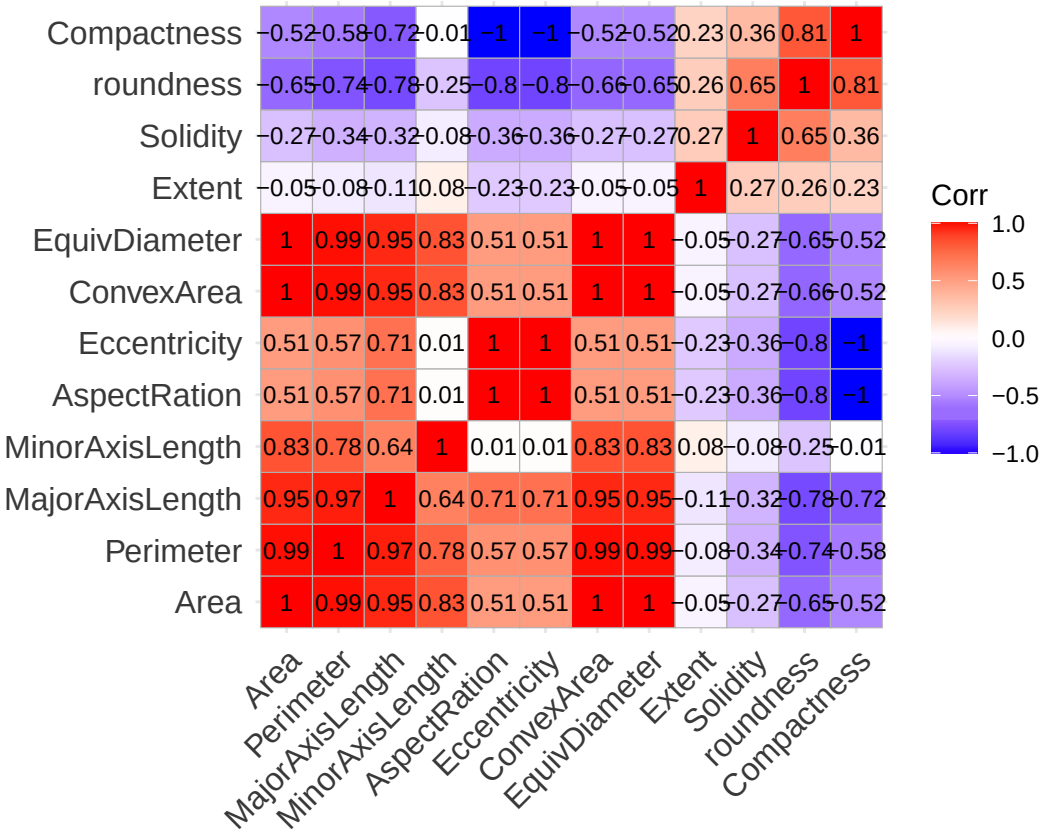
```
##                        Area   Perimeter MajorAxisLength MinorAxisLength
## Area              1.00000000  0.99020720       0.9523314     0.828670272
```

```
## Perimeter        0.99020720  1.00000000       0.9703465     0.776988263
## MajorAxisLength  0.95233142  0.97034649       1.0000000     0.644009953
## MinorAxisLength  0.82867027  0.77698826       0.6440100     1.000000000
## AspectRation     0.51281012  0.57103900       0.7147371     0.005729171
## Eccentricity     0.51281012  0.57103900       0.7147371     0.005729171
## ConvexArea       0.99988196  0.99134295       0.9531853     0.827188201
## EquivDiameter    1.00000000  0.99020720       0.9523314     0.828670272
## Extent          -0.04521157 -0.07939359      -0.1065503     0.078791519
## Solidity        -0.26527321 -0.33750784      -0.3237504    -0.084489412
## roundness       -0.65451050 -0.73723760      -0.7767718    -0.245677389
## Compactness     -0.51809887 -0.57648757      -0.7192827    -0.011772911
##                 AspectRation Eccentricity ConvexArea EquivDiameter      Extent
## Area             0.512810116  0.512810116  0.9998820    1.00000000 -0.04521157
## Perimeter        0.571038995  0.571038995  0.9913430    0.99020720 -0.07939359
## MajorAxisLength  0.714737103  0.714737103  0.9531853    0.95233142 -0.10655034
## MinorAxisLength  0.005729171  0.005729171  0.8271882    0.82867027  0.07879152
## AspectRation     1.000000000  1.000000000  0.5149879    0.51281012 -0.22592228
## Eccentricity     1.000000000  1.000000000  0.5149879    0.51281012 -0.22592228
## ConvexArea       0.514987862  0.514987862  1.0000000    0.99988196 -0.04817140
## EquivDiameter    0.512810116  0.512810116  0.9998820    1.00000000 -0.04521157
## Extent          -0.225922279 -0.225922279 -0.0481714   -0.04521157  1.00000000
## Solidity        -0.355210409 -0.355210409 -0.2740953   -0.26527321  0.26709340
## roundness       -0.804077585 -0.804077585 -0.6599135   -0.65451050  0.26295344
## Compactness     -0.999709435 -0.999709435 -0.5204238   -0.51809887  0.22619635
##                    Solidity  roundness Compactness
## Area            -0.26527321 -0.6545105 -0.51809887
## Perimeter       -0.33750784 -0.7372376 -0.57648757
## MajorAxisLength -0.32375044 -0.7767718 -0.71928270
## MinorAxisLength -0.08448941 -0.2456774 -0.01177291
## AspectRation    -0.35521041 -0.8040776 -0.99970943
## Eccentricity    -0.35521041 -0.8040776 -0.99970943
## ConvexArea      -0.27409529 -0.6599135 -0.52042376
## EquivDiameter   -0.26527321 -0.6545105 -0.51809887
## Extent           0.26709340  0.2629534  0.22619635
## Solidity         1.00000000  0.6541918  0.36345478
## roundness        0.65419183  1.0000000  0.80749655
## Compactness      0.36345478  0.8074965  1.00000000
```

```
ggcorrplot(Beans.corr, lab = TRUE, method = "square",
           lab_size = 3)
```

```
## Warning: `aes_string()` was deprecated in ggplot2 3.0.0.
## i Please use tidy evaluation idioms with `aes()`.
## i See also `vignette("ggplot2-in-packages")` for more information.
## i The deprecated feature was likely used in the ggcorrplot package.
```

```
##   Please report the issue at <https://github.com/kassambara/ggcorrplot/issues>.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



It's interesting to notice that the geometrical measure are positive corralated between them, but they are negatively or not correlated to the derived quantity.

5.2 PCA

First of we can see the variance of the data

```
Variance <- apply(Beans.w.c,2,var)
Variance
```

```
##            Area       Perimeter MajorAxisLength MinorAxisLength    AspectRation
##    8.227368e+08    4.436460e+04    7.068067e+03    1.977452e+03    6.008017e-02
##    Eccentricity      ConvexArea   EquivDiameter          Extent        Solidity
##    8.802923e-03    8.459553e+08    3.382102e+03    2.409926e-03    2.137441e-05
##       roundness     Compactness
##    3.377954e-03    3.844059e-03
```

We can see that there are bigger value like the variance of the Area and smaller value like

Solidity. For this reason we first standardize the data.

```
std.Beans <- apply(Beans.w.c,2,scale)
head(std.Beans)
```

```
##            Area   Perimeter MajorAxisLength MinorAxisLength AspectRation
## [1,]  1.3579903  1.53788849      1.70629835      1.21402703   0.93223866
## [2,] -0.5852250 -0.65534642     -0.58757531     -0.72567208   0.03427077
## [3,] -0.6768809 -0.85355269     -0.84378340     -0.71712669  -0.50506682
## [4,] -0.4179854 -0.50292681     -0.63848713     -0.15217988  -0.91477748
## [5,]  6.4786196  5.12779205      5.03315692      4.80177336   0.84361167
## [6,] -0.1119546 -0.07087815     -0.08940989      0.03860869  -0.21181606
##      Eccentricity ConvexArea EquivDiameter     Extent    Solidity   roundness
## [1,]   0.89158678  1.3680990    1.54779466  0.1359432 -0.82664386 -0.66070350
## [2,]   0.29289190 -0.5836078   -0.67372986 -0.8494951 -0.31872807  0.08872752
## [3,]  -0.23917891 -0.6774696   -0.81343768 -0.6031466  0.09443375  0.76079499
## [4,]  -0.78856340 -0.4188511   -0.43184167 -0.2564656  0.07953340  0.71283077
## [5,]   0.84369497  6.4822613    5.16541282  0.7149002 -0.28076384 -1.15541215
## [6,]   0.07149804 -0.1082819   -0.02461779  0.7536478 -0.54926671  0.21115444
##      Compactness
## [1,] -0.97582078
## [2,] -0.15515332
## [3,]  0.43371470
## [4,]  0.92202638
## [5,] -0.90127264
## [6,]  0.08751753
```

Then we can perform the eigenvalues of the covariance matrix.

```
Beans.cov <- cov(std.Beans)
Beans.eigen <- eigen(Beans.cov)

# Renaming rows and columns
colnames(Beans.eigen$vectors) <- paste("PC",1:ncol(Beans.eigen$vectors),sep="")
rownames(Beans.eigen$vectors) <- colnames(Beans.w.c)

Beans.PCs <- Beans.eigen$vectors

Beans.PCs
```

```
##                         PC1         PC2          PC3           PC4          PC5
## Area            -0.34769581  0.23236503 -0.023989934  0.0653058563  0.125672225
## Perimeter       -0.37167493  0.14316127  0.031528248  0.0058672668 -0.121677410
## MajorAxisLength -0.38130501  0.05299985 -0.078718575  0.0328525115  0.015089640
## MinorAxisLength -0.29973187  0.34517462  0.085169500  0.0482555492 -0.029830187
## AspectRation    -0.23036872 -0.41452009 -0.254884918 -0.0005951749 -0.043204206
## Eccentricity    -0.22770104 -0.40408267 -0.258835668 -0.0110300748  0.368829212
```

```
## ConvexArea      -0.34848390  0.23035353 -0.016707141  0.0602621598  0.129849030
## EquivDiameter   -0.36006658  0.19729799 -0.007595944  0.0442343738  0.001876809
## Extent           0.05211863  0.26964081 -0.465142072 -0.8413352795 -0.018224852
## Solidity         0.13970938  0.18795081 -0.723524562  0.4774881806 -0.419238467
## roundness        0.26685059  0.30960998 -0.227335312  0.2248312275  0.784887050
## Compactness      0.23723804  0.41541289  0.247294684  0.0139109383 -0.146208614
##                         PC6          PC7          PC8           PC9
## Area            -0.033389452 -0.552136768  0.036925468 -0.0306010325
## Perimeter       -0.007644607  0.247192119 -0.251180847  0.2303567441
## MajorAxisLength  0.224670636  0.233892734 -0.594082071 -0.0552958550
## MinorAxisLength -0.094943601  0.371908663  0.689387876 -0.0414933093
## AspectRation     0.599889604 -0.067055962  0.301949729  0.4983391631
## Eccentricity    -0.688457727  0.097652345 -0.007701261  0.2979291561
## ConvexArea      -0.031035706 -0.547832357  0.017150315 -0.0077972367
## EquivDiameter    0.021330775  0.315208721 -0.022472650 -0.0720638746
## Extent           0.003000554 -0.002573809  0.005182918  0.0007644613
## Solidity        -0.129583351 -0.017448511 -0.009687053 -0.0246246140
## roundness        0.264015434  0.159403785 -0.017561360  0.0595088358
## Compactness     -0.132554626 -0.042299263 -0.122042355  0.7711770620
##                        PC10          PC11          PC12
## Area            -5.590081e-02 -0.0542554257  7.001014e-01
## Perimeter        8.013697e-01  0.0233554155  7.396969e-02
## MajorAxisLength -4.350585e-01  0.4402040091  8.036926e-03
## MinorAxisLength -1.086813e-01  0.3798505971  1.171379e-02
## AspectRation    -4.992968e-02 -0.0434176707 -3.521310e-03
## Eccentricity    -9.266496e-02  0.0282593324 -6.170716e-04
## ConvexArea       5.687195e-02  0.0566335885 -7.048720e-01
## EquivDiameter   -2.540252e-01 -0.8076321601 -8.425802e-02
## Extent          -7.553933e-05 -0.0002322495  5.194365e-05
## Solidity         1.115996e-03  0.0082095905 -6.854724e-03
## roundness        1.246803e-01 -0.0031256763  1.165960e-02
## Compactness     -2.433204e-01 -0.0143823829 -6.713356e-03
```

We can see that the PC1 is highly correlated to the geometrical features of a bean, instead the PC2 is negative correlated to al the features except for AspectRation and Eccentricity. The PC3 is highly correlated to Solidity and positive correlated to extend and roundness.

First of all we choose the number of principal components using three methods:

- **Cumulative proportion of variance explained**
- **Kaiser's rule**
- **Scree plot**

5.2.1 Cumulative proportion of variance explained

```
CPVE <- 0
d <- length(Beans.eigen$values)
n.comp <- 0
PVE <- numeric(d)

for (i in 1:d){
   PVE[i] <-  Beans.eigen$values[i]/d
   CPVE <- CPVE + Beans.eigen$values[i]

   if (CPVE/d <= 0.8) n.comp <- n.comp+1
}

cat("Number of components are the first: ",n.comp+1)
```

```
## Number of components are the first:  2
```

5.2.2 Kaiser's rule

```
n.comp.K <- 0
mean.lambda <- mean(Beans.eigen$values)
for (i in 1:d){
  if (Beans.eigen$values[i] > mean.lambda)
    n.comp.K <- n.comp.K +1
  else
    break
}

cat("Number of components are the first: ",n.comp.K)
```

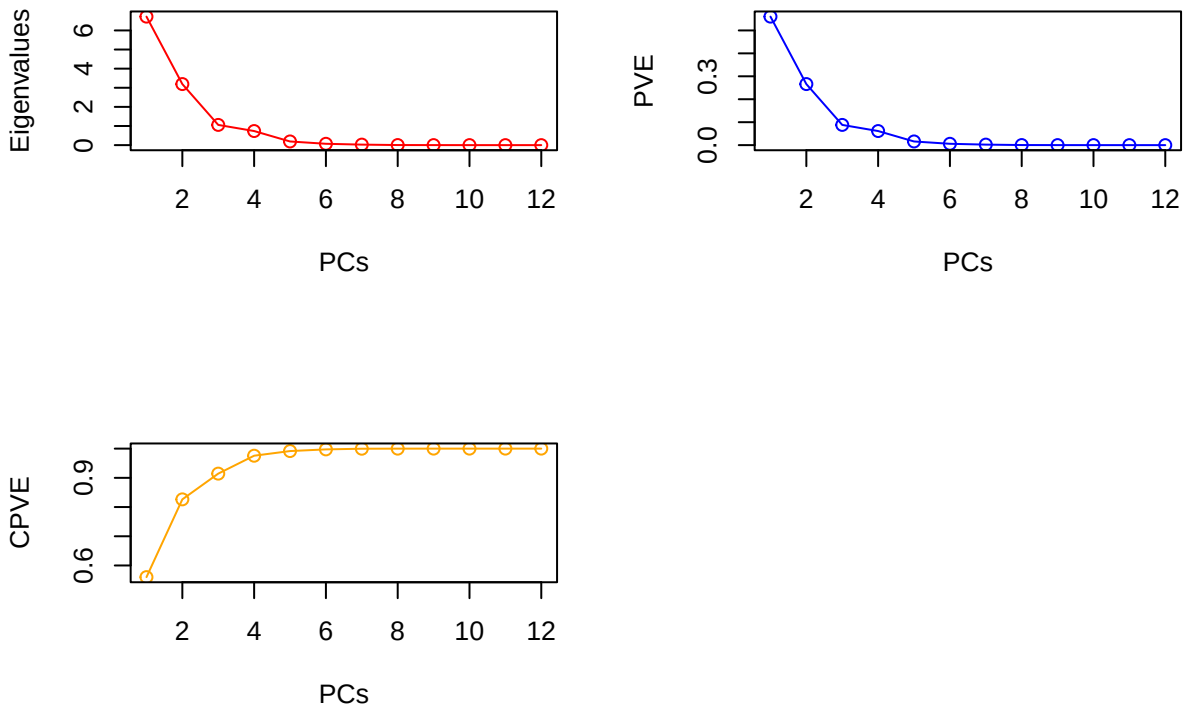
```
## Number of components are the first:  3
```

5.2.3 Scree plot

```
par(mfrow=c(2,2))

plot(seq(from = 1,to = d, by = 1),Beans.eigen$values,xlab = "PCs",
     ylab = "Eigenvalues",type = "o",
     lty = 1,col="red")
plot(seq(from = 1,to = d, by = 1),PVE,xlab = "PCs",
     ylab = "PVE",type = "o",
     lty = 1,col="blue")
plot(seq(from = 1,to = d, by = 1),cumsum(PVE),xlab = "PCs",
     ylab = "CPVE",type = "o",
```

```
     lty = 1,col="orange")
```

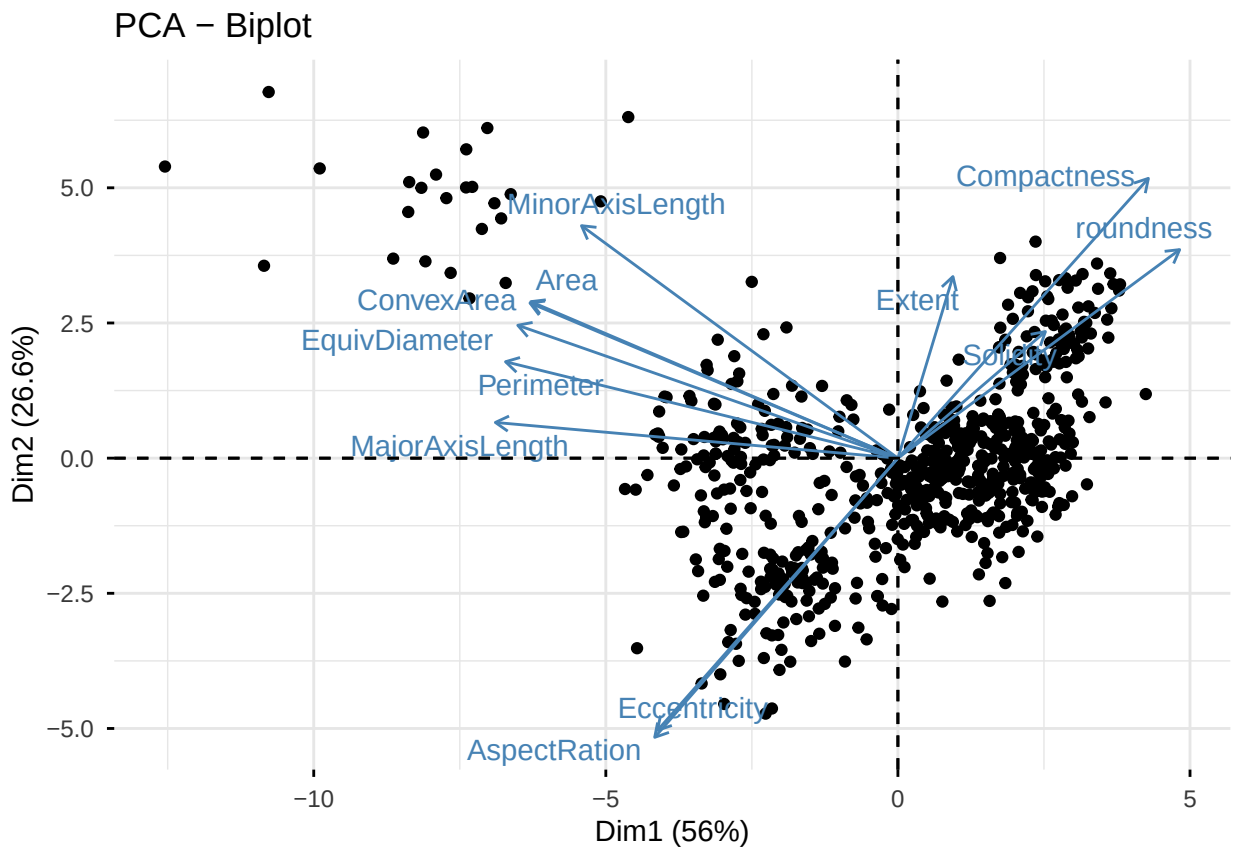


We can see that the elbow occur at the third component, so in this case we take only two components.

5.3 PCs plot

Following from the fact that the majority of the methods say that is better 2 PCs, then we represent all in a 2-D dimensional space given by the first two eigenvectors.

```
# We can do the biplot only using prcomp for the PCA, the result is the same.
pca <- prcomp(std.Beans,  scale = TRUE)
fviz_pca_biplot(pca,label = "var",repel = TRUE)
```



In the graph we can see the dependency that we have shown in the correlation matrix. The first PC explain the 56% of the variability of the data and the second PC the 26.6%.

6 Multivariate analysis : Cluster analysis

6.1 Clustering tendency

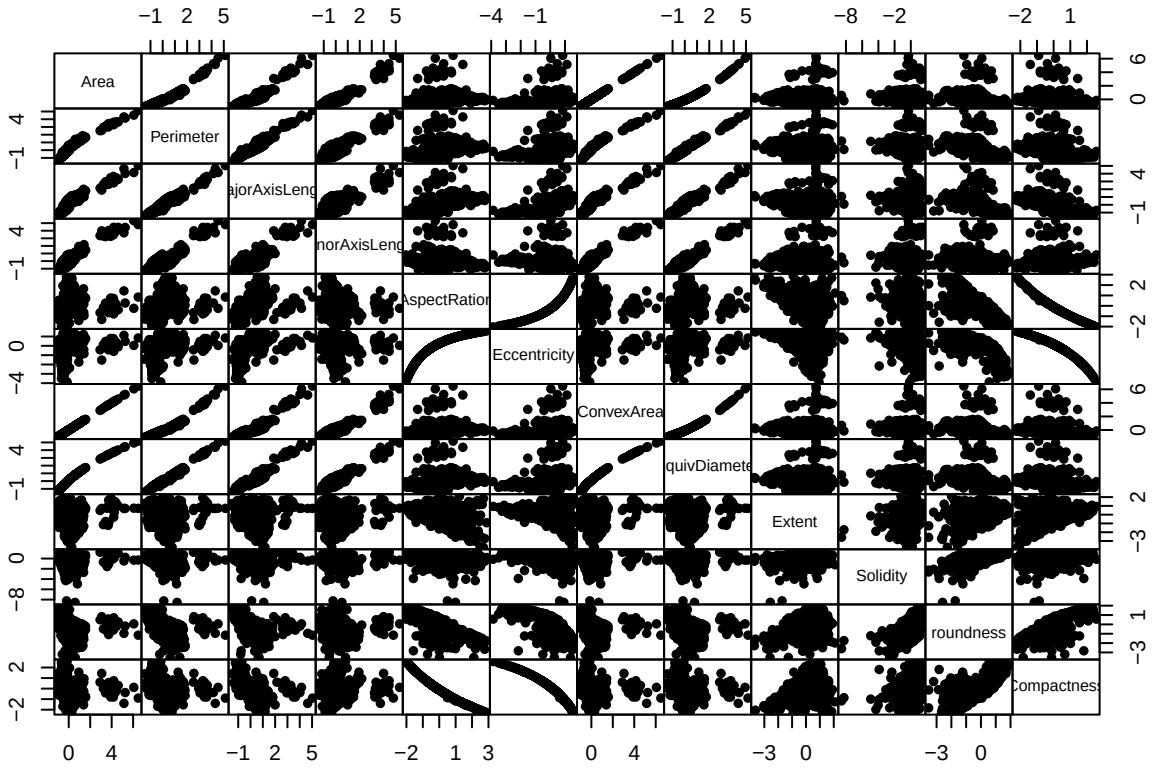
We first analyze our data to establish if they can be clustered or not. This is done by comparing our data to random generated data.

```
# Generates data randomly based and Beans.w.c based on columns (the 2 inside apply).
random.data <- apply(Beans.w.c,2,
                     function(x){runif(length(x),min(x),max(x))})


random.data <- as.data.frame(random.data)
std.random.data <- scale(random.data)
```

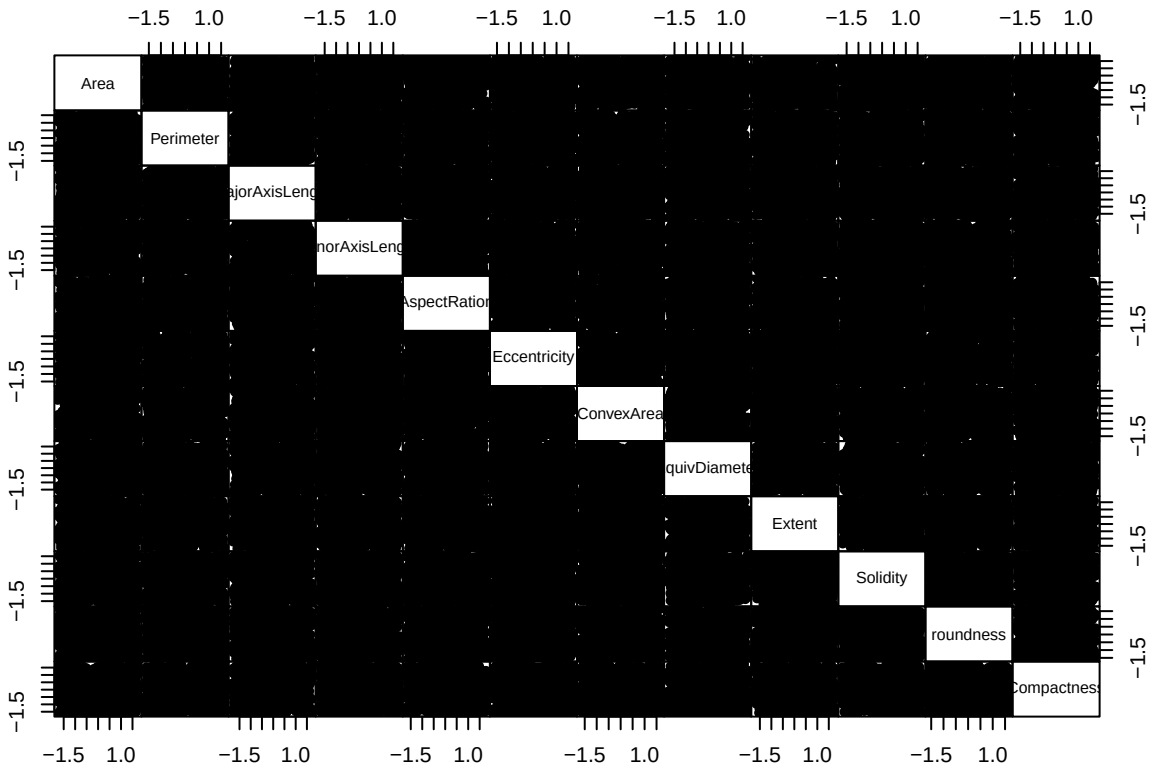
We can plot for both the pair plot.

```
pairs(std.Beans,gap = 0,pch =16)
```



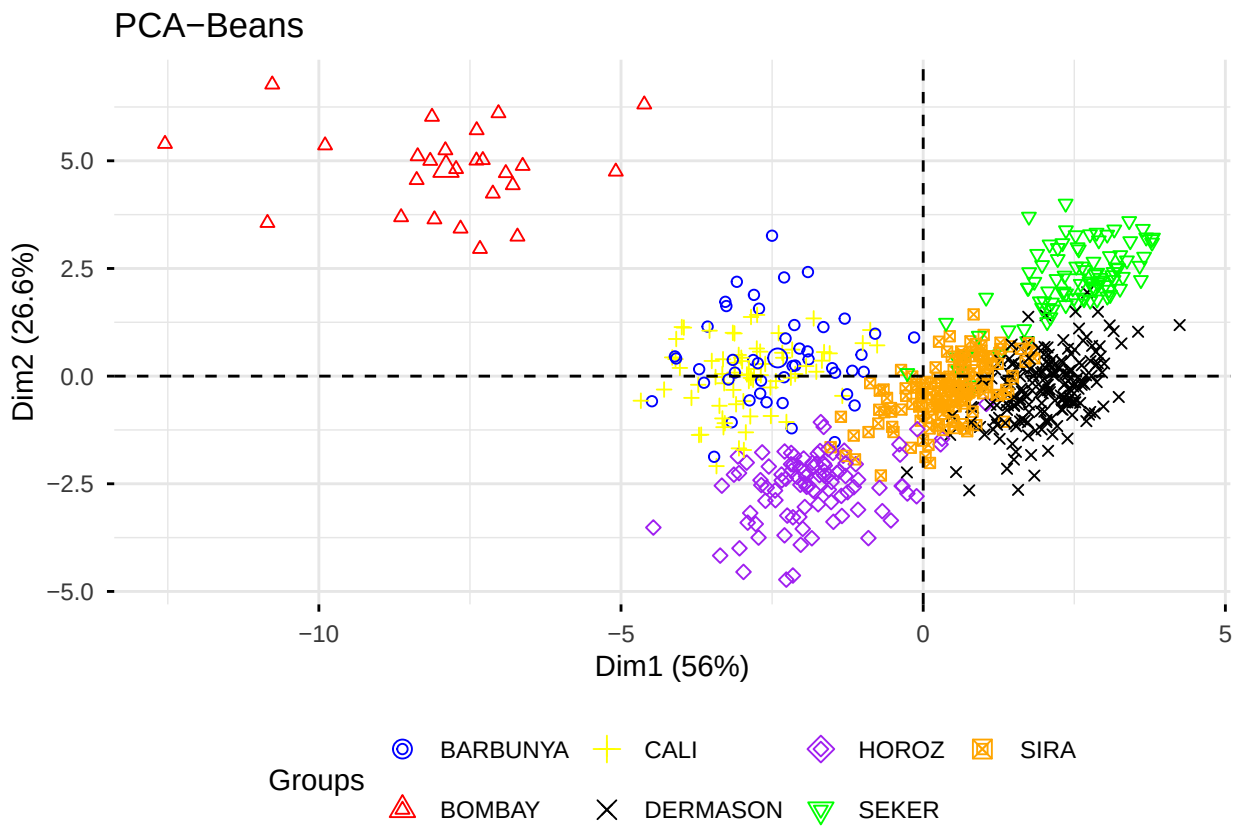
We can see that in different rectangles there are possible clusters, for example in Compactness-MinorAxisLength. In other we can see a correlation instead of a cluster, for example ConvexArea and EquivDiameter.

```
pairs(std.random.data,gap=0,pch=16)
```



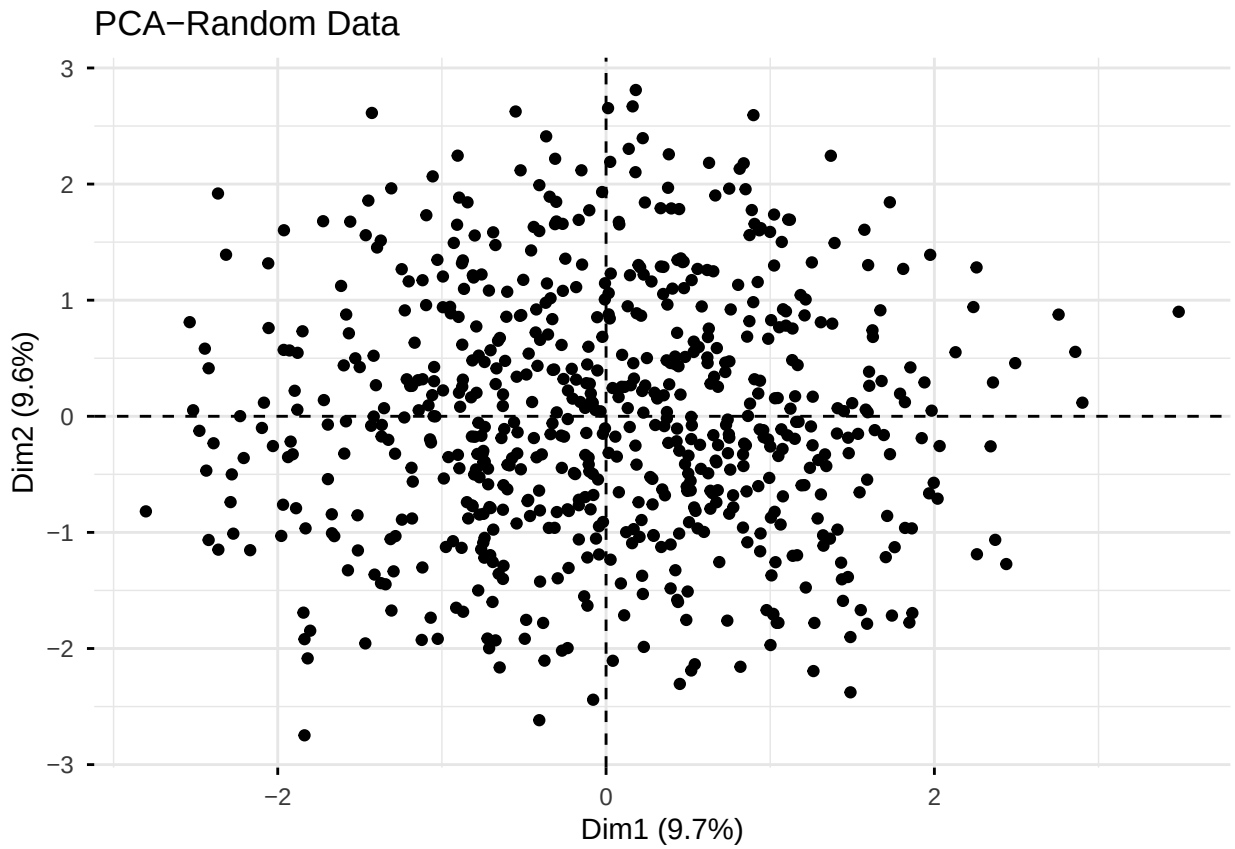
In this case is not evident a clustering division of the data, but only a big clustering that contains all the data in a rectangle.

```
fviz_pca_ind(pca, title = "PCA-Beans",habillage = Beans$Class,
             palette = c("blue","red","yellow","black","purple",
                "green","orange"),
             geom = "point", legend = "bottom" )
```



We can see a clustering tendency in the data. Remember that this graph is made by using the specific class contained in the Beans dataset.

```
fviz_pca_ind(prcomp(std.random.data), title = "PCA-Random Data",
             geom = "point", legend = "bottom" )
```



Clearly there is not a clustering tendency.

We now use two methods to evaluate the clustering tendency.

6.1.1 Hopkins statistic

```
hopkins(std.Beans, n =nrow(Beans.w.c)-1)
```

```
## $H
## [1] 0.0885169
```

```
hopkins(std.random.data, n =nrow(Beans.w.c)-1)
```

```
## $H
## [1] 0.5005727
```

As we can see the statistic for the Beans dataset is near the zero, so they can be clustered. In the case of random variable, they cannot be clustered because the value is near 0.5.

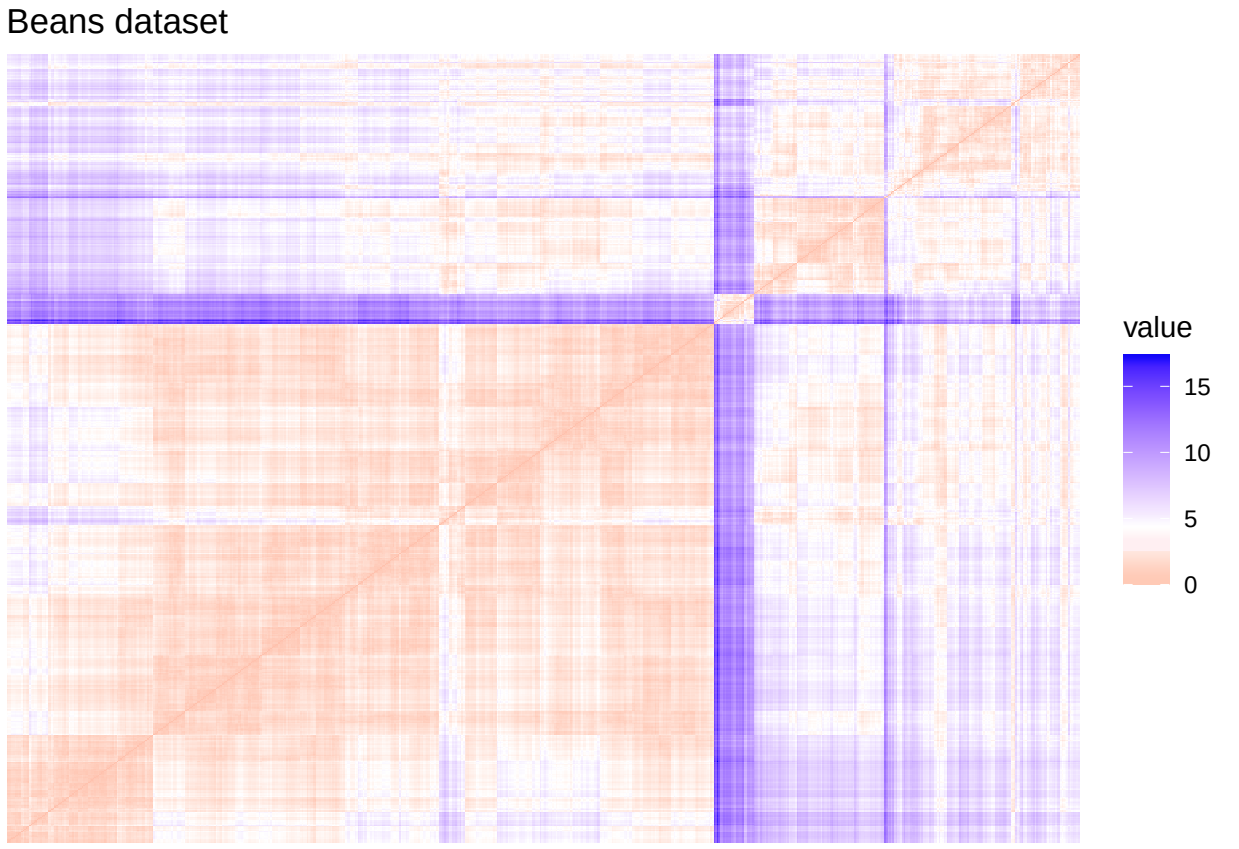
6.1.2 VAT Algorithm

First we crate the dissimilarity matrix of both the data, the features are all numerical and continuous, so we can use the function dist to compute the dissimilarity matrix. We use the euclidean distance.

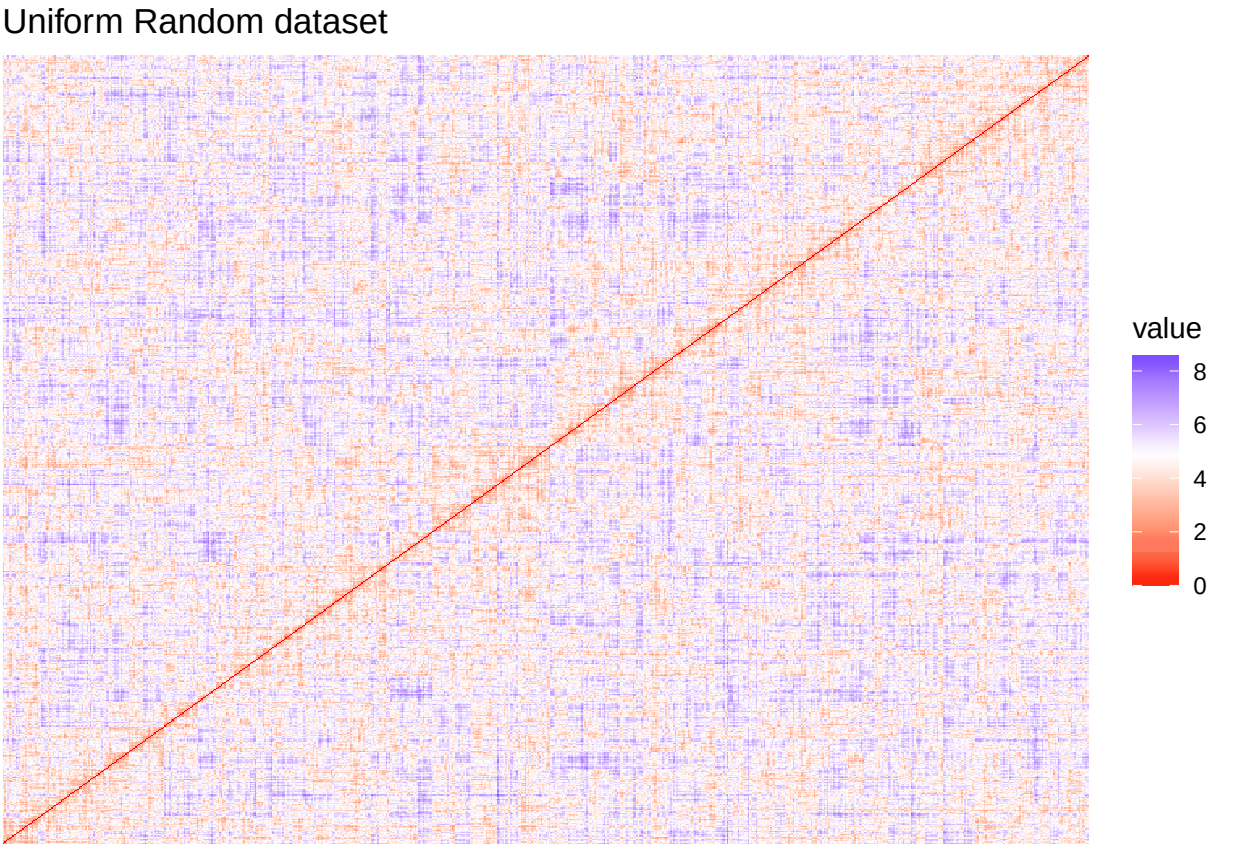
```
Beans.diss <- dist(std.Beans)
Random.diss <- dist(std.random.data)
```

Then we create the two different plots and compare them.

```
par(mfrow=c(1,2))
fviz_dist(Beans.diss, show_labels = FALSE)+ labs(title="Beans dataset")
```




```
fviz_dist(Random.diss, show_labels = FALSE)+labs(title="Uniform Random dataset")
```



We can see that there are different rectangles on the diagonal in the graph of the dataset, instead in the random uniform dataset we can see no rectangles. All the previous methods demonstrate that the data has some clustering tendency.

6.2 Hierarchial clustering

The hierarchical agglomerative clustering is a method that starts from the single unit considered as clusters and then "goes up", in other words merge them together to for a bigger cluster until all the units are in a single big cluster. The procedure depends on two factor:

- The **dissimilarity measure** used for the dissimilarity matrix (In our case the Euclidean distance and the Manhattan distance)
- The **linkage method** that we will use to merge two different clusters (In our case we can use single linkage method, complete linkage method, average linkage method, centroid linkage method and ward's method. The last two method can be used only because all the features are numerical)

The structure of the analysis is as follow: 1. Doing the hierarchical algorithm 2. Doing the correlation between the dissimilarity matrix and the cophenetic distance to see if the clustering is valid. If It's a value close to 1, it's a good tree. 3. Evaluating the optimal number of cluster doing the average silhouette width, the gap statistics and the WSS analysis. 4. Computing the internal validation through the analysis of the silhouette width, connectivity

and Dunn index. 5. Computing the external validation of the data through Rand index and Meila's index. The first measures the agreement and the second the disagreement with the Classes.

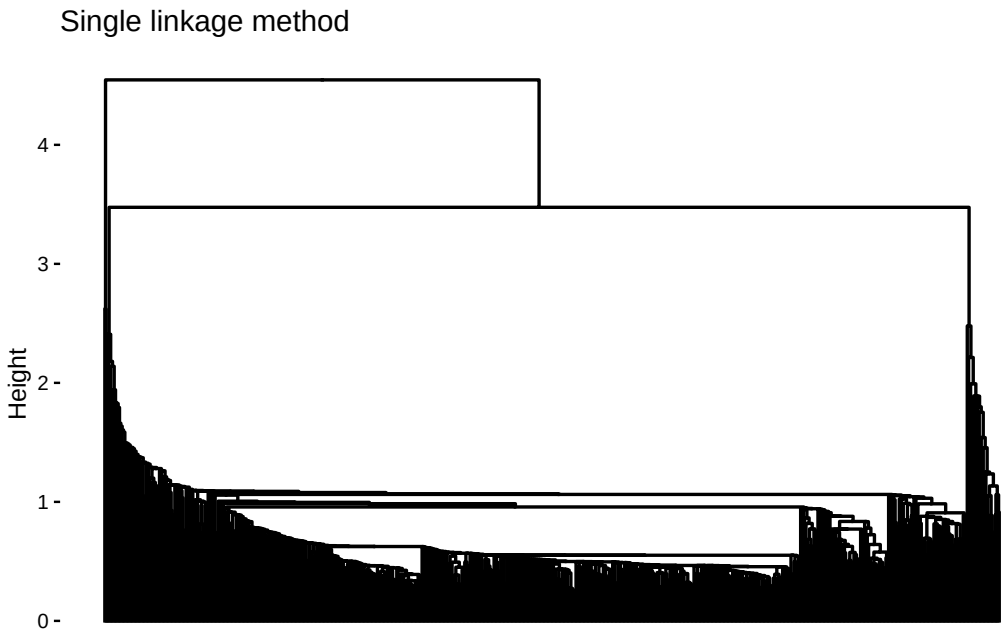
6.2.1 Euclidean distance

First of all we create the new dissimilarity matrix. (Remember to use standardized data)

```
Beans.eu <- dist(std.Beans, method = "euclidean")
```

6.2.1.1 Single linkage method .

```
Beans.hc.s.e <- hclust(Beans.eu, method = "single")
fviz_dend(Beans.hc.s.e, main = "Single linkage method",
          show_labels = FALSE)        # show_labels = FALSE delete the
```



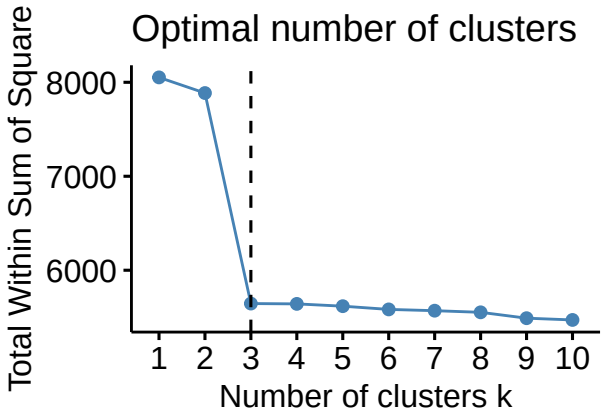

```
                                    # name of different beans
```

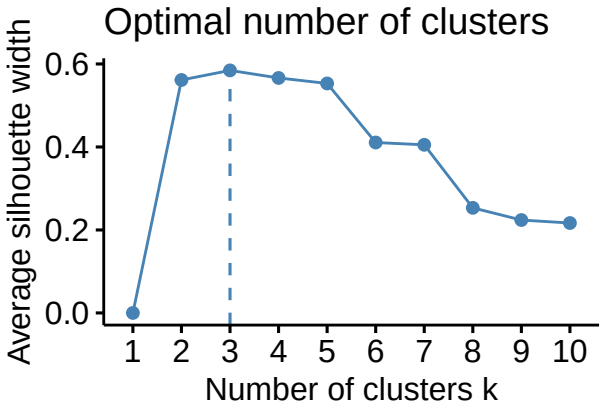
```
corr.s.e <- cor(Beans.eu,cophenetic(Beans.hc.s.e))
corr.s.e
```

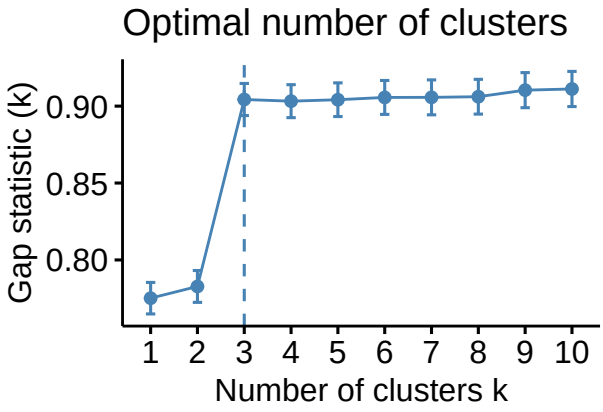
```
## [1] 0.7954828
```

```
p1 <- fviz_nbclust(std.Beans, hcut, method = "wss", hc_metric = "euclidean"
             ,hc_method = "single") +
  geom_vline(xintercept = 3, linetype = 2)
p2 <- fviz_nbclust(std.Beans, hcut, method = "silhouette", hc_metric = "euclidean"
```

```
              ,hc_method = "single")
p3 <- fviz_nbclust(std.Beans, hcut, method = "gap_stat", hc_metric = "euclidean"
              ,hc_method = "single", nboot = 50)
grid.arrange(p1, p2, p3, ncol = 2, nrow = 2)
```





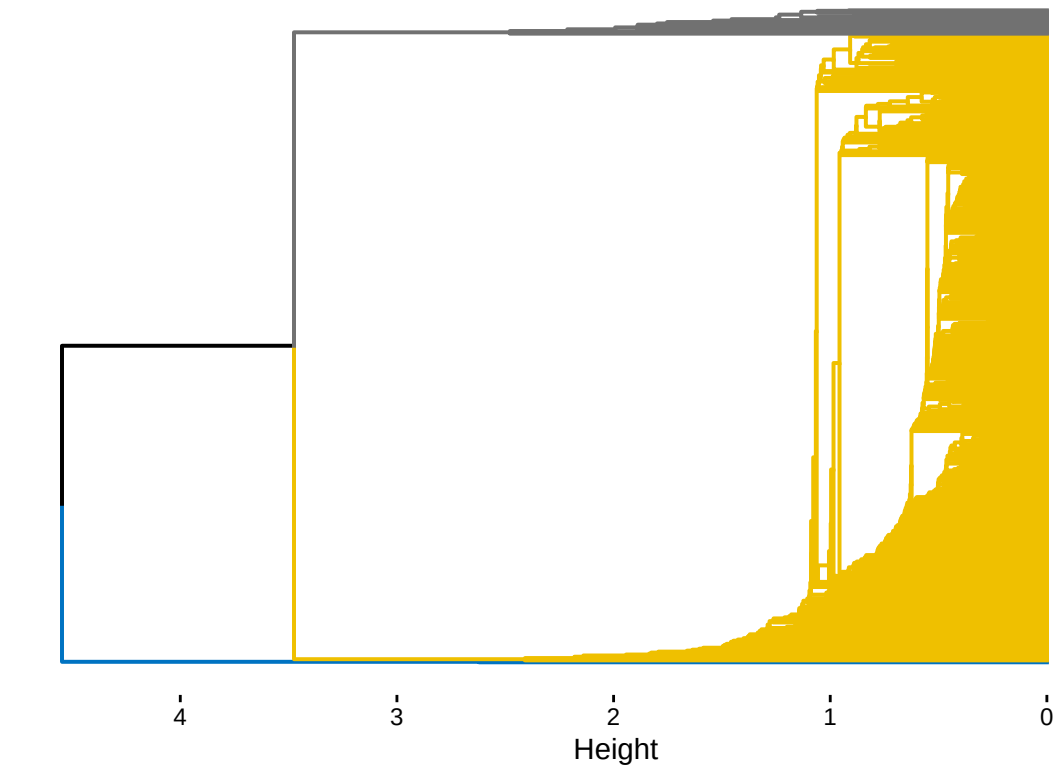


With this method 3 is the optimal number of clusters.

We can now perform the Internal validation.

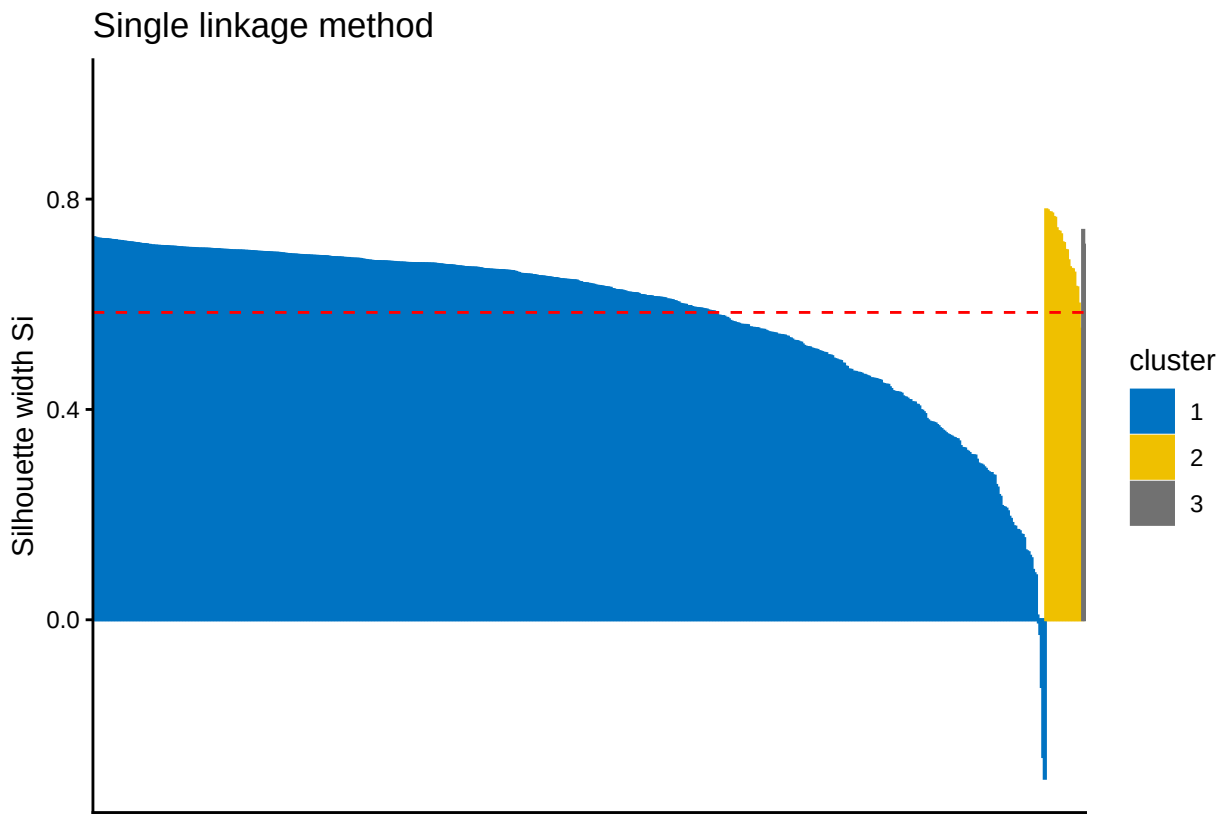
```
Beans.e.eu.s<-eclust(std.Beans, FUNcluster = "hclust",
                     hc_metric = "euclidean", k = 3,
                    hc_method = "single", graph = FALSE)

fviz_dend(Beans.e.eu.s, show_labels=FALSE, horiz = TRUE,
          palette = "jco",main = "Single linkage method")
```




```
# Silhouette width
fviz_silhouette(Beans.e.eu.s, ggtheme = theme_classic(),
                palette = "jco", k= 3, main = "Single linkage method")
```

```
##   cluster size ave.sil.width
## 1       1  645          0.58
## 2       2   25          0.70
## 3       3    2          0.73
```




```
sil <- Beans.e.eu.s$silinfo$widths[,1:3]

clusters <- unique(sil[,"cluster"])
for (i in clusters){
  neg.sil <- which(sil[,"sil_width"]<0 & sil[,"cluster"] == i)
  cat("Number of misassigned units for cluster",i,": ",
      nrow(sil[neg.sil, , drop = FALSE]),"\n")
}
```

```
## Number of misassigned units for cluster 1 :  5
## Number of misassigned units for cluster 2 :  0
## Number of misassigned units for cluster 3 :  0
```

```
val.e.h.s <- clValid(std.Beans, nClust = 2:7,
                     clMethods = "hierarchical", validation ="internal",
                     method="single", maxitems = 1000)
summary(val.e.h.s)
```

```
##
## Clustering Methods:
##  hierarchical
##
## Cluster sizes:
```

```
##  2 3 4 5 6 7
##
## Validation Measures:
##                                  2       3       4       5       6       7
##
## hierarchical Connectivity   3.8579  3.8579  5.8579  9.6202 12.8032 15.7321
##              Dunn           0.2615  0.3273  0.2468  0.2334  0.2269  0.2085
##              Silhouette     0.5614  0.5845  0.5663  0.5530  0.4107  0.4052
##
## Optimal Scores:
##
##              Score  Method       Clusters
## Connectivity 3.8579 hierarchical 2
## Dunn         0.3273 hierarchical 3
## Silhouette   0.5845 hierarchical 3
```

We can now perform the external validation.

```
table(Beans$Class, Beans.e.eu.s$cluster)
```

```
##
##               1   2   3
##   BARBUNYA   47   0   0
##   BOMBAY      0  25   0
##   CALI       67   0   0
##   DERMASON  188   0   0
##   HOROZ      95   0   2
##   SEKER      96   0   0
##   SIRA      152   0   0
```

```
# Numeric counter part of classes

classes <- as.numeric(as.factor(Beans$Class))
clust_stats <- cluster.stats(d = Beans.eu,clustering = classes,
                             alt.clustering = Beans.e.eu.s$cluster )

cat("Rand index: ", clust_stats$corrected.rand)
```

```
## Rand index:  0.03537183
```

```
cat("Meila's VI index: ", clust_stats$vi)
```

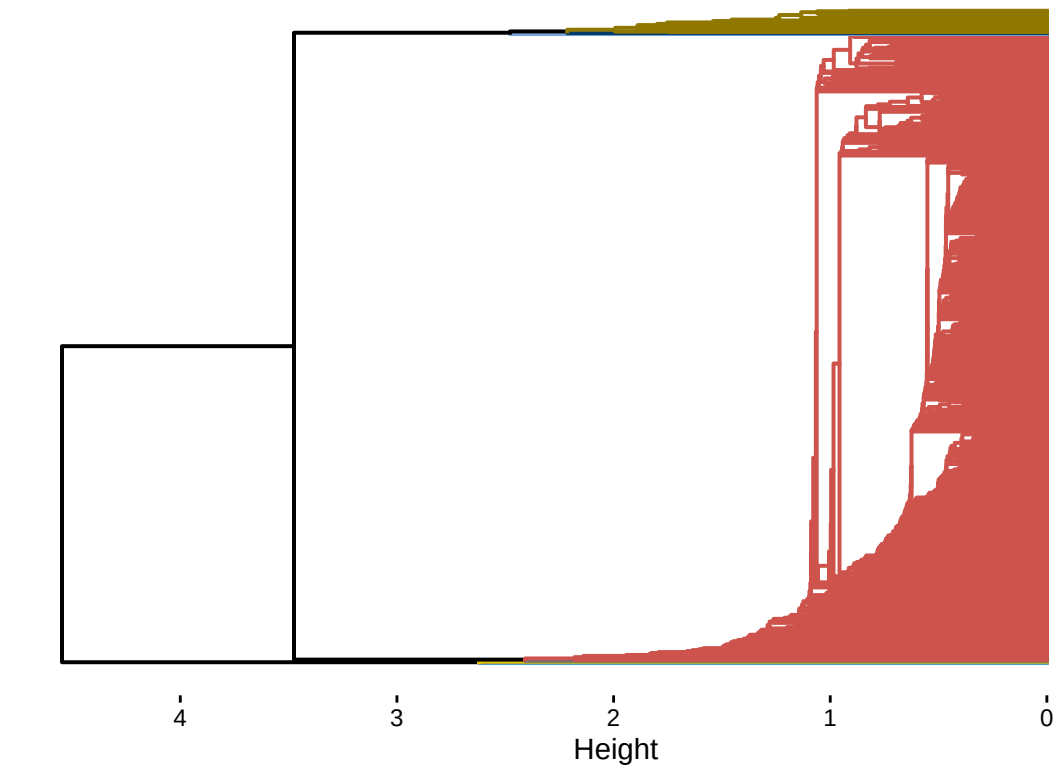
```
## Meila's VI index:  1.638186
```

The Rand index indicates a week agreement. The Meila's index indicates strong disagreement.

We can try forcing the system to give us 7 clusters.

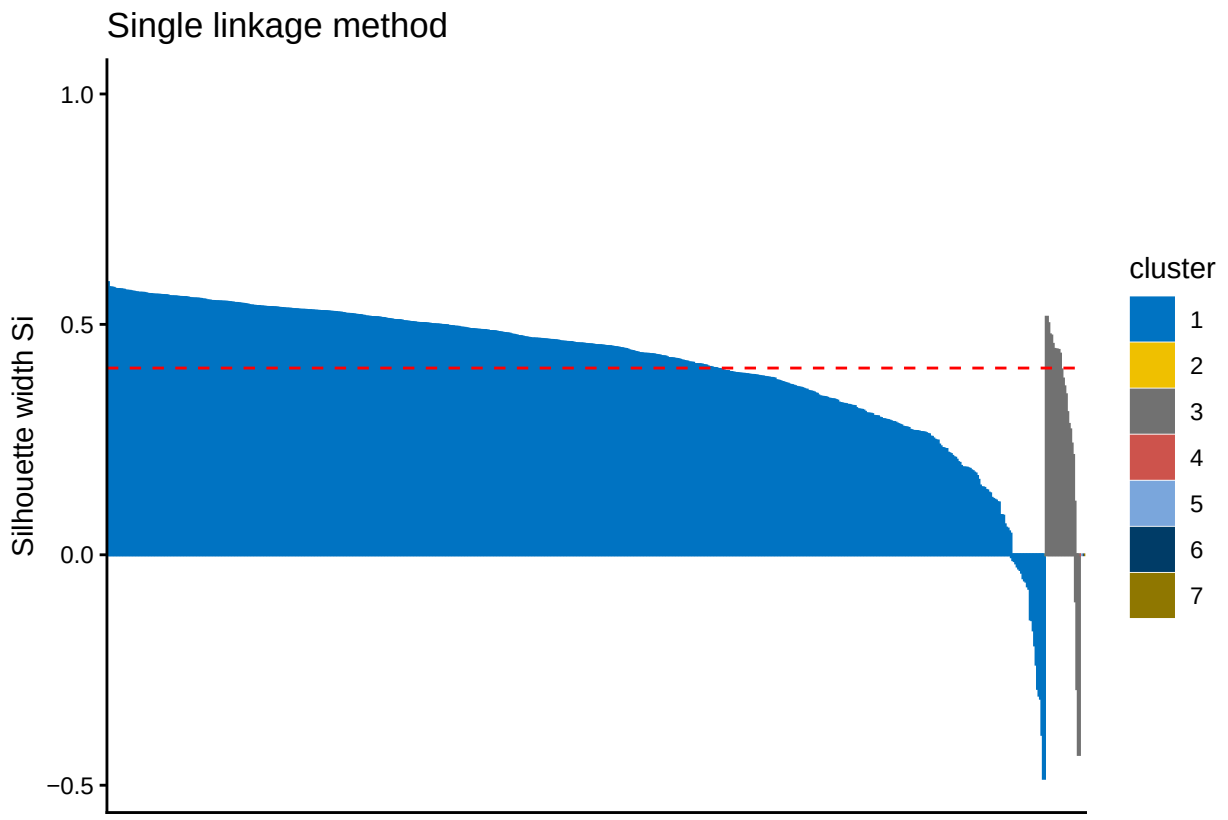
```
Beans.e.eu.s7<-eclust(std.Beans, FUNcluster = "hclust",
                      hc_metric = "euclidean", k = 7,
                     hc_method = "single", graph = FALSE)

fviz_dend(Beans.e.eu.s7, show_labels=FALSE, horiz = TRUE,
          palette = "jco",main = "Single linkage method")
```




```
# Silhouette width
fviz_silhouette(Beans.e.eu.s7, ggtheme = theme_classic(),
                palette = "jco", k= 3, main = "Single linkage method")
```

```
##   cluster size ave.sil.width
## 1       1  644          0.41
## 2       2    1          0.00
## 3       3   23          0.29
## 4       4    1          0.00
## 5       5    1          0.00
## 6       6    1          0.00
## 7       7    1          0.00
```




```
sil <- Beans.e.eu.s7$silinfo$widths[,1:3]
clusters <- unique(sil[,"cluster"])
for (i in clusters){
  neg.sil <- which(sil[,"sil_width"]<0 & sil[,"cluster"] == i)
  cat("Number of misassigned units for cluster",i,": ",
      nrow(sil[neg.sil, , drop = FALSE]),"\n")
}
```

```
## Number of misassigned units for cluster 1 :  23
## Number of misassigned units for cluster 2 :  0
## Number of misassigned units for cluster 3 :  3
## Number of misassigned units for cluster 4 :  0
## Number of misassigned units for cluster 5 :  0
## Number of misassigned units for cluster 6 :  0
## Number of misassigned units for cluster 7 :  0
```

```
table(Beans$Class, Beans.e.eu.s7$cluster)
```

```
##
##              1   2   3   4   5   6   7
##   BARBUNYA  47   0   0   0   0   0   0
##   BOMBAY     0   1  23   0   0   0   1
##   CALI      67   0   0   0   0   0   0
```

```
##    DERMASON 188   0   0   0   0   0   0
##    HOROZ     95   0   0   0   1   1   0
##    SEKER     95   0   0   1   0   0   0
##    SIRA     152   0   0   0   0   0   0
```

```
clust_stats <- cluster.stats(d = Beans.eu,clustering = classes,
                             alt.clustering = Beans.e.eu.s7$cluster )

cat("Rand index: ", clust_stats$corrected.rand)
```

```
## Rand index:  0.03529264
```

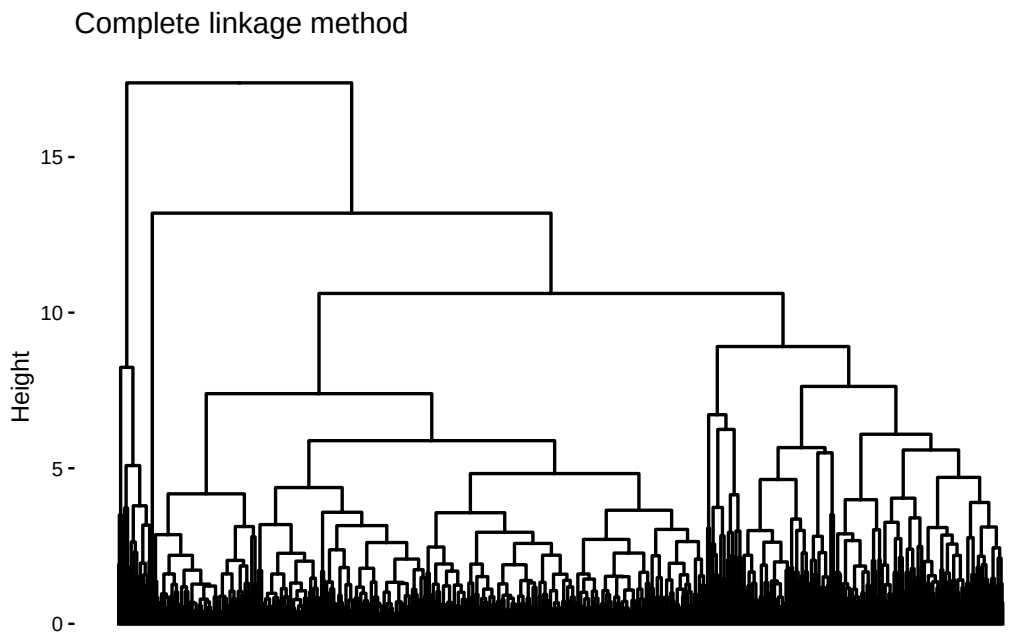
```
cat("Meila's VI index: ", clust_stats$vi)
```

```
## Meila's VI index:  1.658114
```

Also in this case both the indices don't give a good result.

6.2.1.2 Complete linkage mathod .

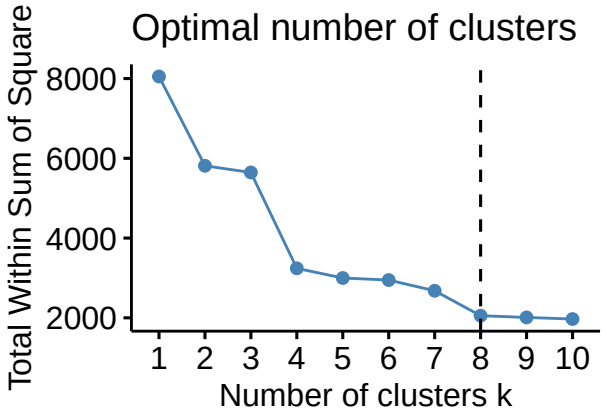
```
Beans.hc.c.e <- hclust(Beans.eu, method = "complete")
fviz_dend(Beans.hc.c.e, main = "Complete linkage method",
          show_labels = FALSE)
```

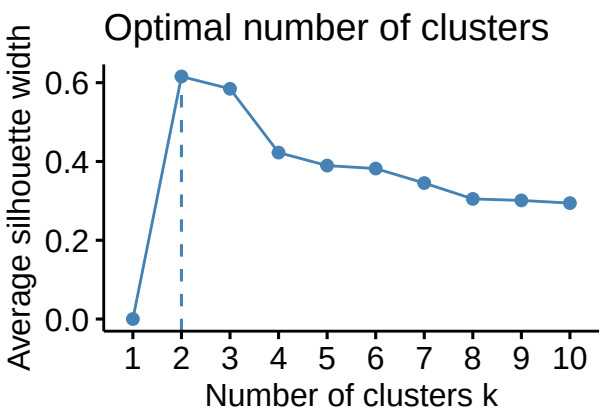


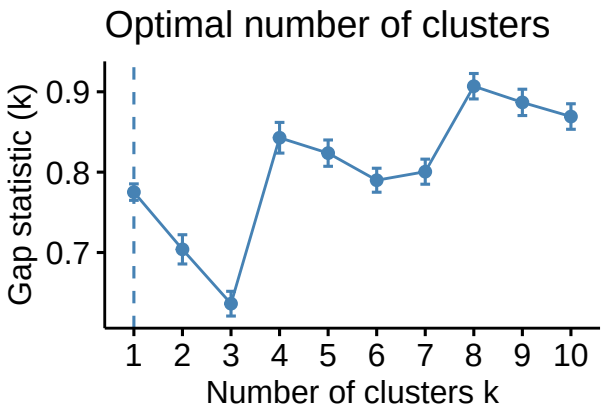

```
corr.c.e <- cor(Beans.eu,cophenetic(Beans.hc.c.e))
corr.c.e
```

```
## [1] 0.8338768
```

```
p1 <- fviz_nbclust(std.Beans, hcut, method = "wss", hc_metric = "euclidean"
             ,hc_method = "complete") +
  geom_vline(xintercept = 8, linetype = 2)
p2 <- fviz_nbclust(std.Beans, hcut, method = "silhouette", hc_metric = "euclidean"
             ,hc_method = "complete")
p3 <- fviz_nbclust(std.Beans, hcut, method = "gap_stat", hc_metric = "euclidean"
             ,hc_method = "complete", nboot = 50)
grid.arrange(p1, p2, p3, ncol = 2, nrow = 2)
```



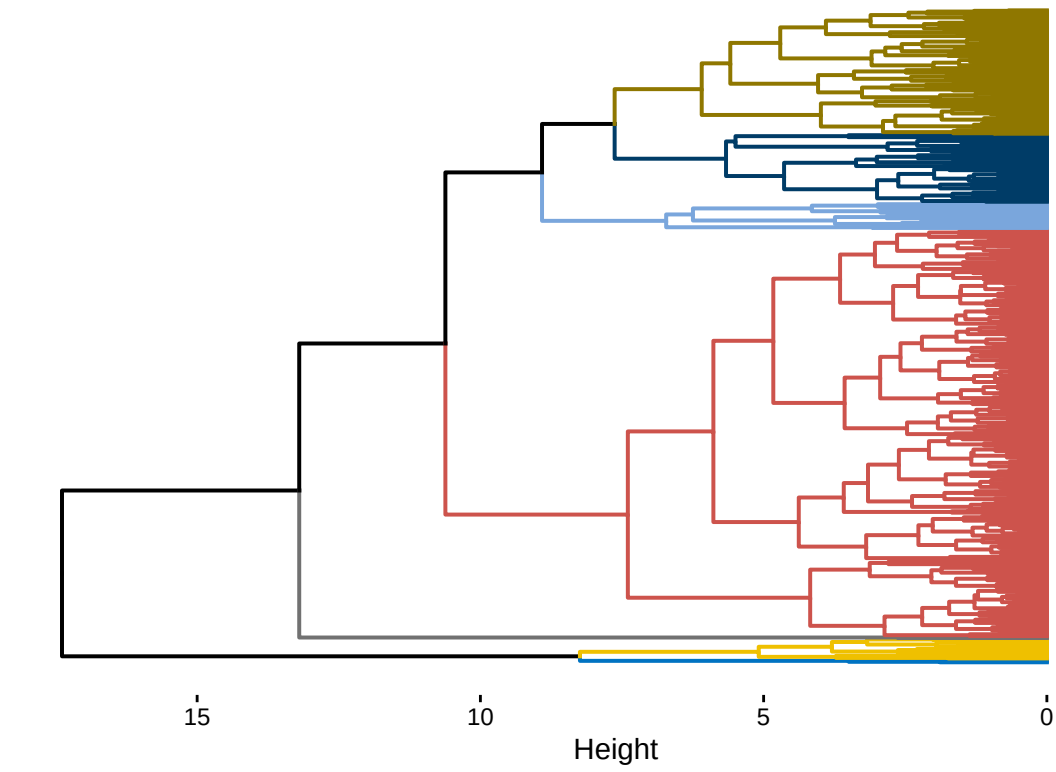




The three different method gives us three different result. It's impossible to choose one based on these three method. In this case we will use 7 clusters as the number of classes.

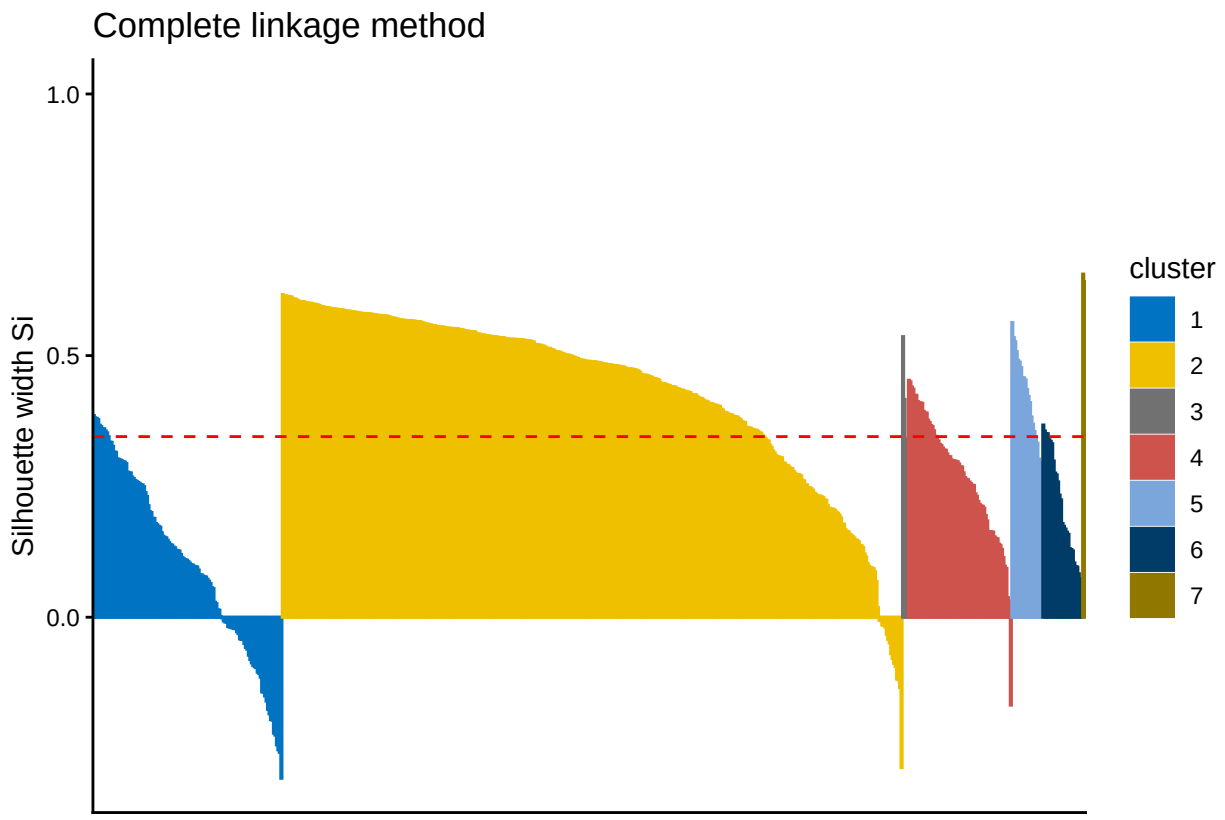
```
Beans.e.eu.c<-eclust(std.Beans, FUNcluster = "hclust",
                     hc_metric = "euclidean", k = 7,
                     hc_method = "complete", graph = FALSE)

fviz_dend(Beans.e.eu.c, show_labels=FALSE, horiz = TRUE,
          palette = "jco",main = "Complete linkage method")
```




```
# Silhouette width
fviz_silhouette(Beans.e.eu.c, ggtheme = theme_classic(),
                palette = "jco",main = "Complete linkage method")
```

```
##   cluster size ave.sil.width
## 1       1  128          0.10
## 2       2  420          0.44
## 3       3    4          0.39
## 4       4   70          0.27
## 5       5   21          0.42
## 6       6   27          0.21
## 7       7    2          0.65
```




```
sil <- Beans.e.eu.c$silinfo$widths[,1:3]

clusters <- unique(sil[,"cluster"])
for (i in clusters){
  neg.sil <- which(sil[,"sil_width"]<0 & sil[,"cluster"] == i)
  cat("Number of misassigned units for cluster",i,": ",
      nrow(sil[neg.sil, , drop = FALSE]),"\n")
}
```

```
## Number of misassigned units for cluster 1 :  41
## Number of misassigned units for cluster 2 :  16
## Number of misassigned units for cluster 3 :  0
## Number of misassigned units for cluster 4 :  1
## Number of misassigned units for cluster 5 :  0
## Number of misassigned units for cluster 6 :  0
## Number of misassigned units for cluster 7 :  0
```

```
val.e.h.c <- clValid(std.Beans, nClust = 2:7, clMethods = "hierarchical",
                     validation ="internal", method="complete",
                     maxitems = 1000)
summary(val.e.h.c)
```

```
##
```

```
## Clustering Methods:
##  hierarchical
##
## Cluster sizes:
##  2 3 4 5 6 7
##
## Validation Measures:
##                                  2        3        4        5        6        7
##
## hierarchical Connectivity    0.0000   3.8579  32.7806  66.4266  72.9063 114.8238
##              Dunn            0.2633   0.3273   0.0618   0.0668   0.0722   0.0664
##              Silhouette      0.6157   0.5845   0.4225   0.3896   0.3820   0.3452
##
## Optimal Scores:
##
##              Score  Method       Clusters
## Connectivity 0.0000 hierarchical 2
## Dunn         0.3273 hierarchical 3
## Silhouette   0.6157 hierarchical 2
```

We can now perform the external validation.

```
table(Beans$Class, Beans.e.eu.c$cluster)
```

```
##
##              1   2   3   4   5   6   7
##   BARBUNYA  26   1   0   2   0  18   0
##   BOMBAY     0   0   4   0  21   0   0
##   CALI      60   1   0   0   0   6   0
##   DERMASON   0 185   0   3   0   0   0
##   HOROZ     42   5   0  48   0   0   2
##   SEKER      0  94   0   0   0   2   0
##   SIRA       0 134   0  17   0   1   0
```

```
clust_stats <- cluster.stats(d = Beans.eu,clustering = classes,
                             alt.clustering = Beans.e.eu.c$cluster )

cat("Rand index: ", clust_stats$corrected.rand)
```

```
## Rand index:  0.3172234
```

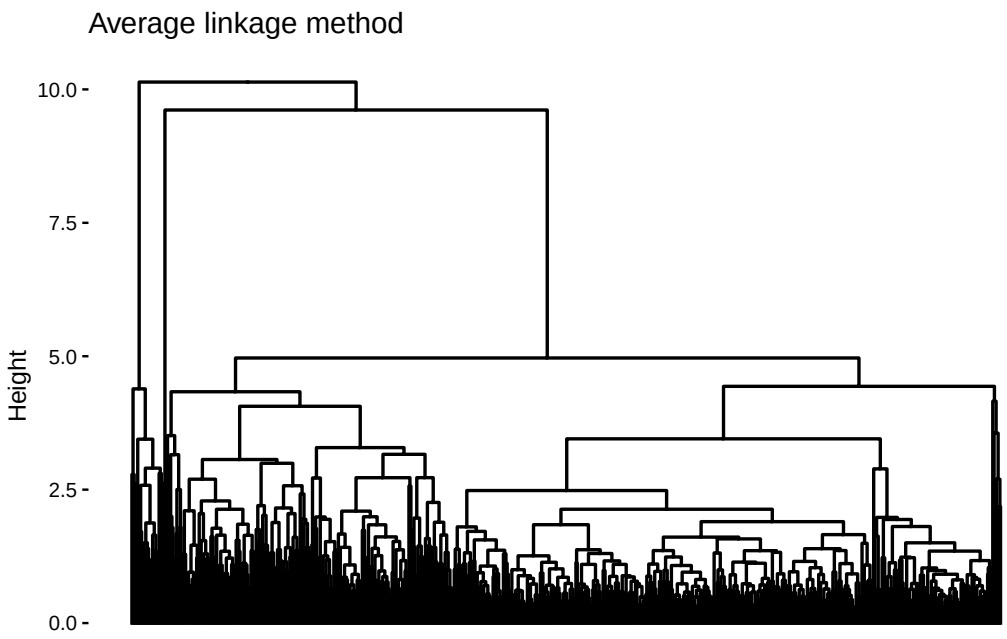
```
cat("Meila's VI index: ", clust_stats$vi)
```

```
## Meila's VI index:  1.416491
```

The Rand index indicates a week agreement. The Meila's index indicates strong disagreement.

6.2.1.3 Average linkage method .

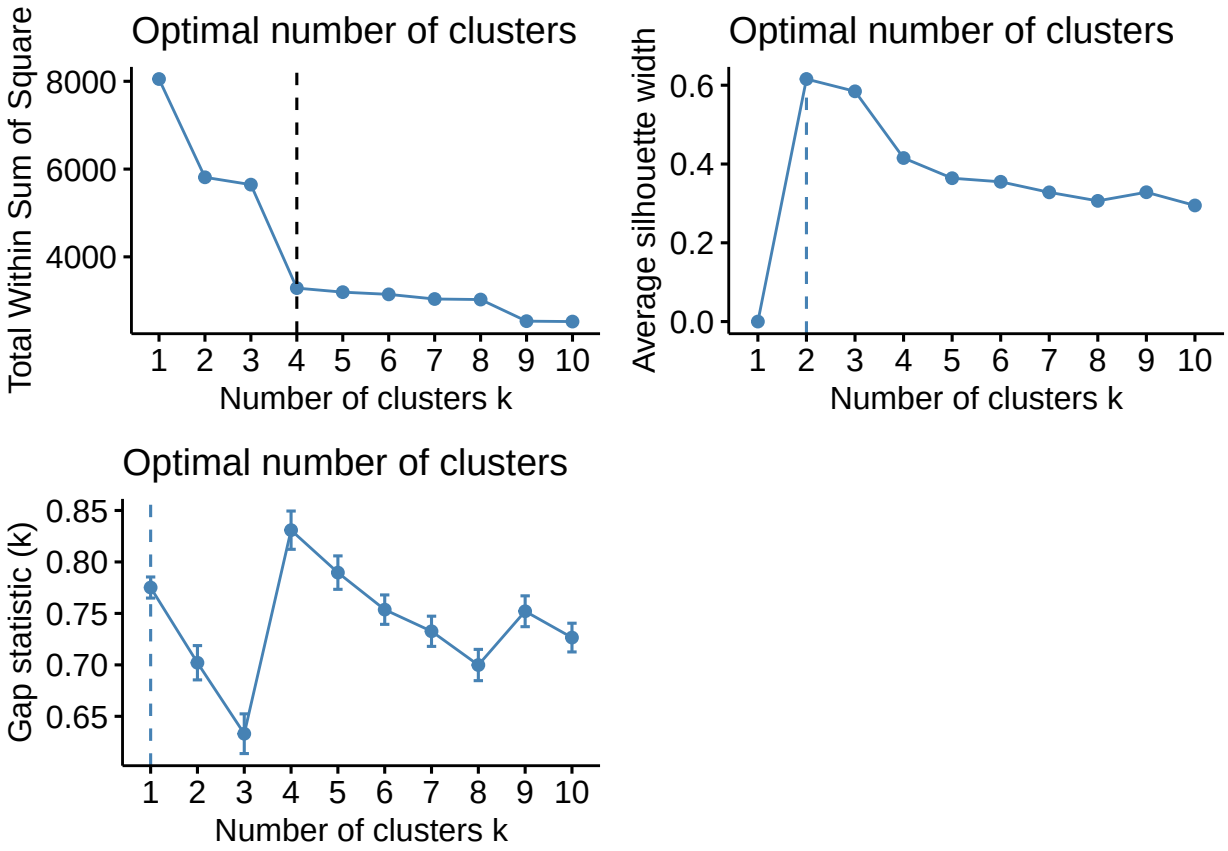
```
Beans.hc.a.e <- hclust(Beans.eu, method = "average")
fviz_dend(Beans.hc.a.e, main = "Average linkage method",
          show_labels = FALSE)
```




```
corr.a.e <- cor(Beans.eu,cophenetic(Beans.hc.a.e))
corr.a.e
```

```
## [1] 0.8596034
```

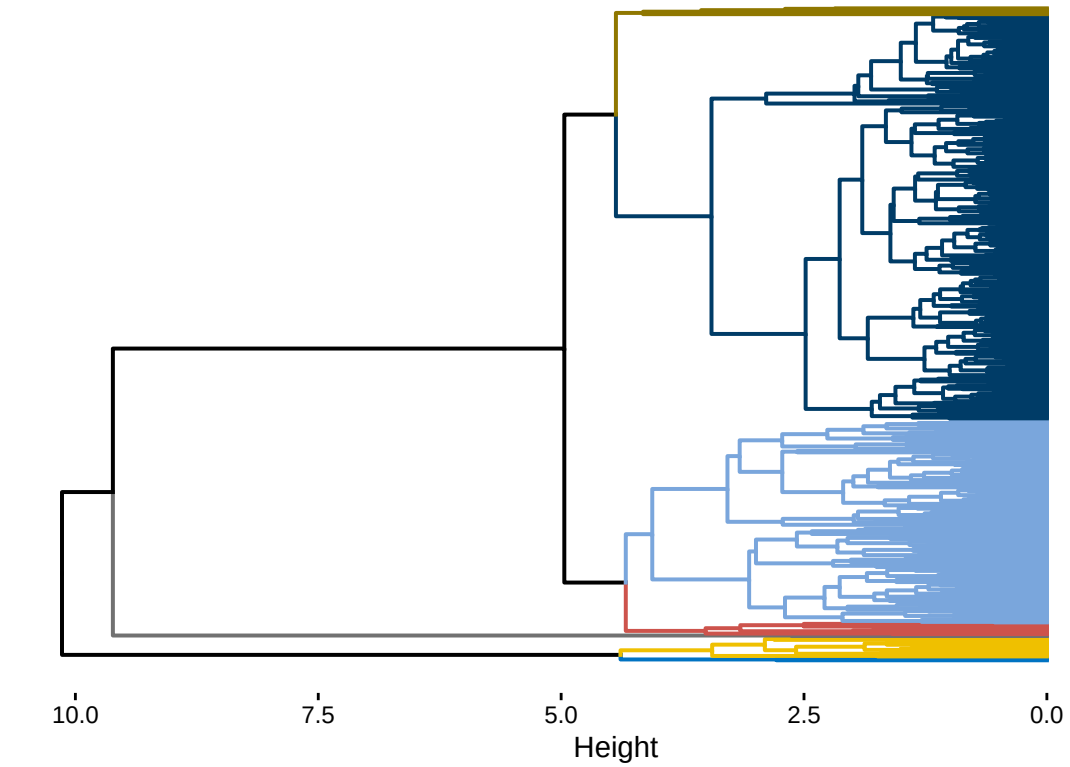
```
p1 <- fviz_nbclust(std.Beans, hcut, method = "wss", hc_metric = "euclidean"
             ,hc_method = "average") +
  geom_vline(xintercept = 4, linetype = 2)
p2 <- fviz_nbclust(std.Beans, hcut, method = "silhouette", hc_metric = "euclidean"
             ,hc_method = "average")
p3 <- fviz_nbclust(std.Beans, hcut, method = "gap_stat", hc_metric = "euclidean"
             ,hc_method = "average", nboot = 50)
grid.arrange(p1, p2, p3, ncol = 2, nrow = 2)
```



The three different method gives us three different result. It's impossible to choose one based on these three method. In this case we will use 7 clusters as the number of classes.

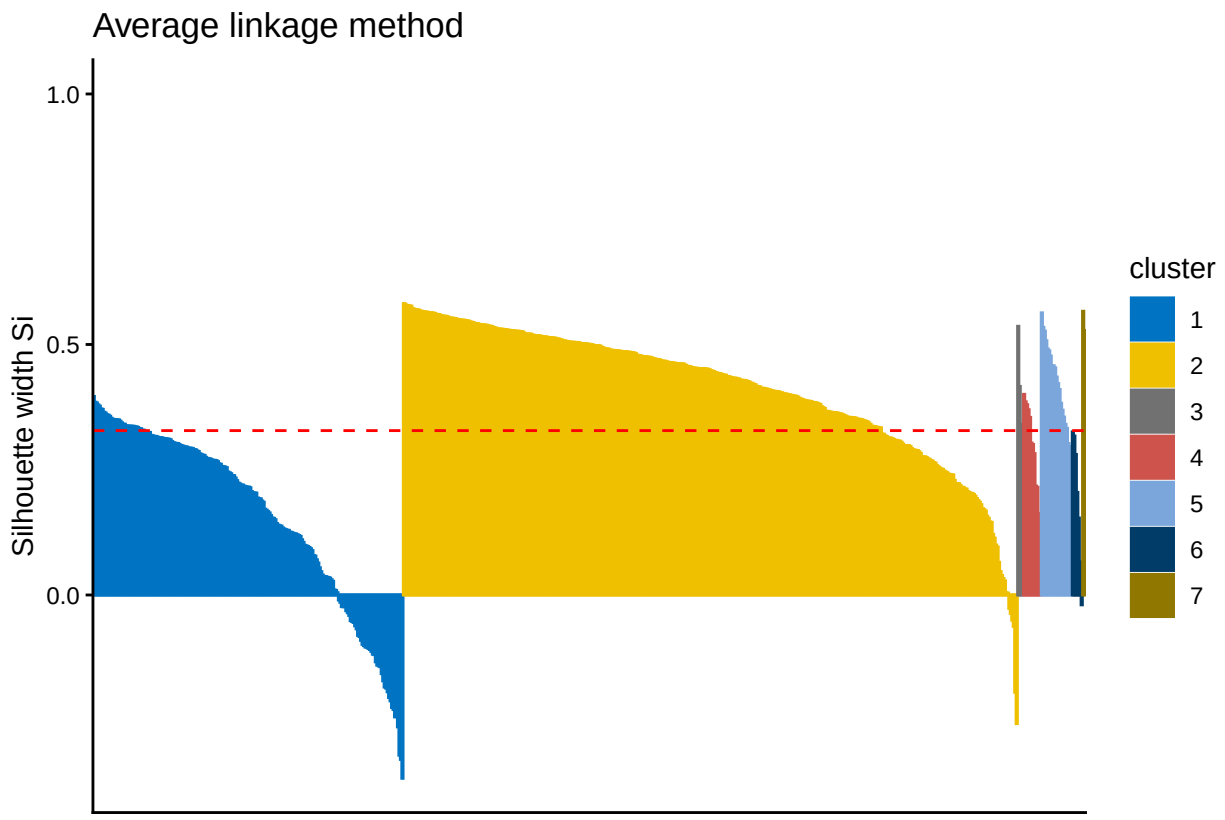
```
Beans.e.eu.al<-eclust(std.Beans, FUNcluster = "hclust",
                      hc_metric = "euclidean", k = 7, hc_method = "average",
                      graph = FALSE)

fviz_dend(Beans.e.eu.al, show_labels=FALSE, horiz = TRUE, palette = "jco",
          main = "Average linkage method")
```




```
# Silhouette width
fviz_silhouette(Beans.e.eu.al, ggtheme = theme_classic(),
                palette = "jco",main = "Average linkage method")
```

```
##   cluster size ave.sil.width
## 1       1  210          0.16
## 2       2  416          0.41
## 3       3    4          0.39
## 4       4   12          0.29
## 5       5   21          0.42
## 6       6    7          0.19
## 7       7    2          0.55
```




```
sil <- Beans.e.eu.al$silinfo$widths[,1:3]

clusters <- unique(sil[,"cluster"])
for (i in clusters){
  neg.sil <- which(sil[,"sil_width"]<0 & sil[,"cluster"] == i)
  cat("Number of misassigned units for cluster",i,": ",
      nrow(sil[neg.sil, , drop = FALSE]),"\n")
}
```

```
## Number of misassigned units for cluster 1 :  44
## Number of misassigned units for cluster 2 :  6
## Number of misassigned units for cluster 3 :  0
## Number of misassigned units for cluster 4 :  0
## Number of misassigned units for cluster 5 :  0
## Number of misassigned units for cluster 6 :  1
## Number of misassigned units for cluster 7 :  0
```

```
val.e.h.al <- clValid(std.Beans, nClust = 2:7, clMethods = "hierarchical",
                      validation ="internal", method="average",
                      maxitems = 1000)
summary(val.e.h.al)
```

```
##
```

```
## Clustering Methods:
##  hierarchical
##
## Cluster sizes:
##  2 3 4 5 6 7
##
## Validation Measures:
##                                  2       3       4       5       6       7
##
## hierarchical Connectivity   0.0000  3.8579 43.6028 48.5825 55.0623 73.9865
##              Dunn           0.2633  0.3273  0.0649  0.0649  0.0649  0.0671
##              Silhouette     0.6157  0.5845  0.4152  0.3640  0.3548  0.3281
##
## Optimal Scores:
##
##              Score  Method       Clusters
## Connectivity 0.0000 hierarchical 2
## Dunn         0.3273 hierarchical 3
## Silhouette   0.6157 hierarchical 2
```

We can now perform the external validation.

```
table(Beans$Class, Beans.e.eu.al$cluster)
```

```
##
##              1   2   3   4   5   6   7
##   BARBUNYA  41   1   0   4   0   1   0
##   BOMBAY     0   0   4   0  21   0   0
##   CALI      61   1   0   5   0   0   0
##   DERMASON   6 181   0   1   0   0   0
##   HOROZ     88   5   0   2   0   0   2
##   SEKER      0  91   0   0   0   5   0
##   SIRA      14 137   0   0   0   1   0
```

```
clust_stats <- cluster.stats(d = Beans.eu,clustering = classes,
                             alt.clustering = Beans.e.eu.al$cluster )

cat("Rand index: ", clust_stats$corrected.rand)
```

```
## Rand index:  0.2975155
```

```
cat("Meila's VI index: ", clust_stats$vi)
```

```
## Meila's VI index:  1.450716
```

The Rand index indicates a week agreement. The Meila's index indicates strong disagreement.

6.2.1.4 Centroid linkage method .

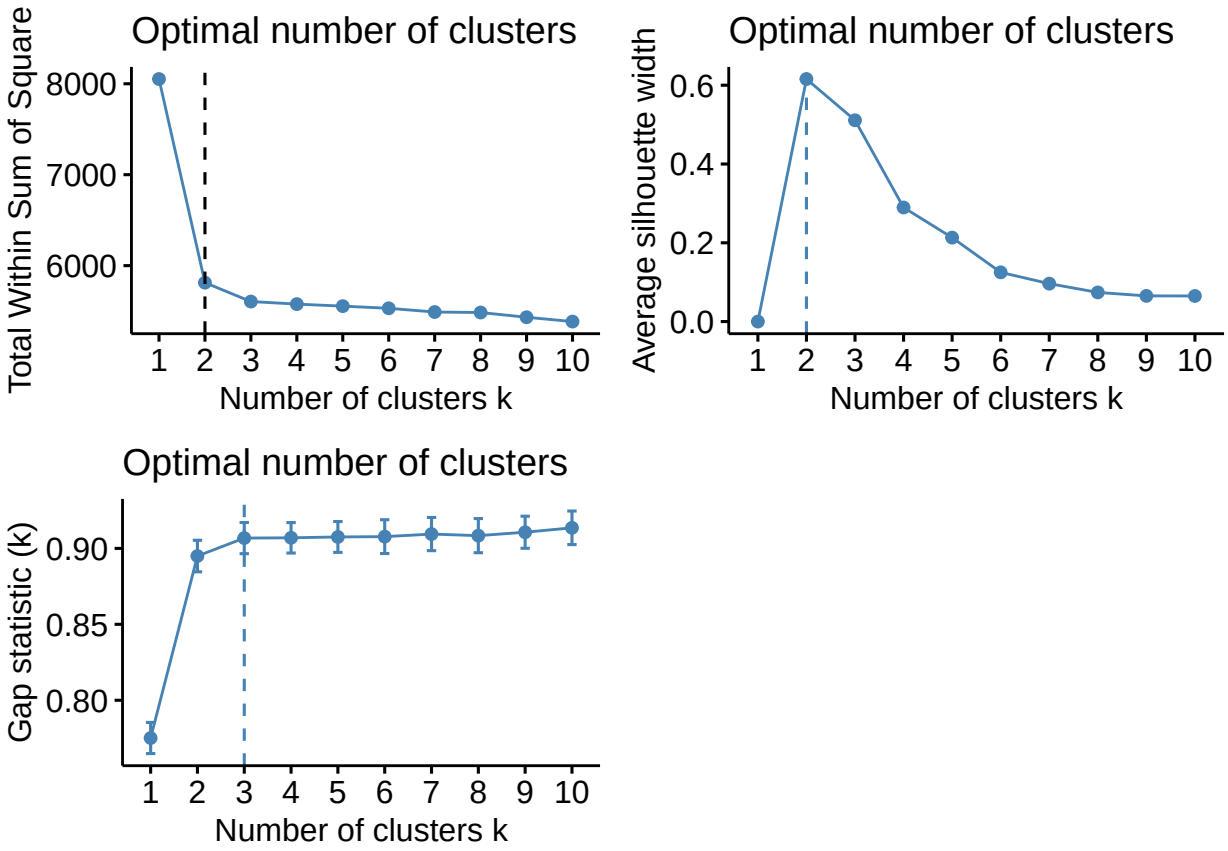
```
Beans.hc.ce.e <- hclust(Beans.eu, method = "centroid")
fviz_dend(Beans.hc.ce.e, main = "Centroid linkage method",
          show_labels = FALSE)
```




```
corr.ce.e <- cor(Beans.eu,cophenetic(Beans.hc.ce.e))
corr.ce.e
```

```
## [1] 0.8291048
```

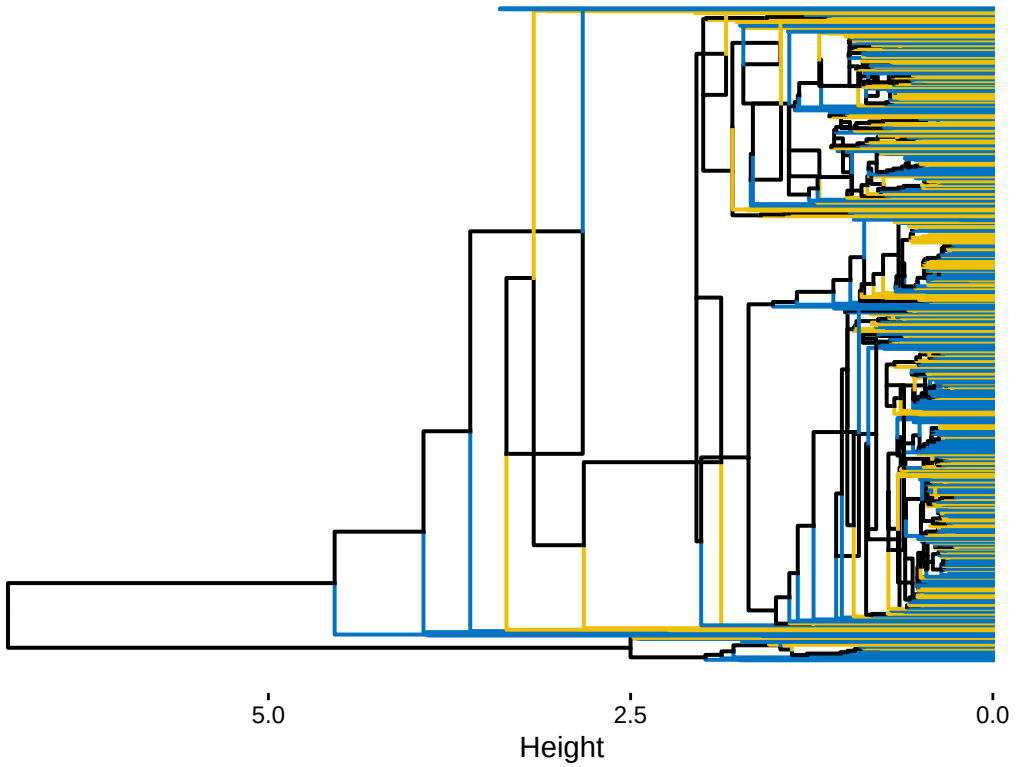
```
p1 <- fviz_nbclust(std.Beans, hcut, method = "wss", hc_metric = "euclidean"
             ,hc_method = "centroid") +
  geom_vline(xintercept = 2, linetype = 2)
p2 <- fviz_nbclust(std.Beans, hcut, method = "silhouette", hc_metric = "euclidean"
             ,hc_method = "centroid")
p3 <- fviz_nbclust(std.Beans, hcut, method = "gap_stat", hc_metric = "euclidean"
             ,hc_method = "centroid", nboot = 50)
grid.arrange(p1, p2, p3, ncol = 2, nrow = 2)
```



With this methods 2 is the optimal number of clusters.

```
Beans.e.eu.cl<-eclust(std.Beans, FUNcluster = "hclust",
                      hc_metric = "euclidean", k = 2,
                      hc_method = "centroid", graph = FALSE)

fviz_dend(Beans.e.eu.cl, show_labels=FALSE, horiz = TRUE,
          palette = "jco",main = "Centroid linkage method")
```

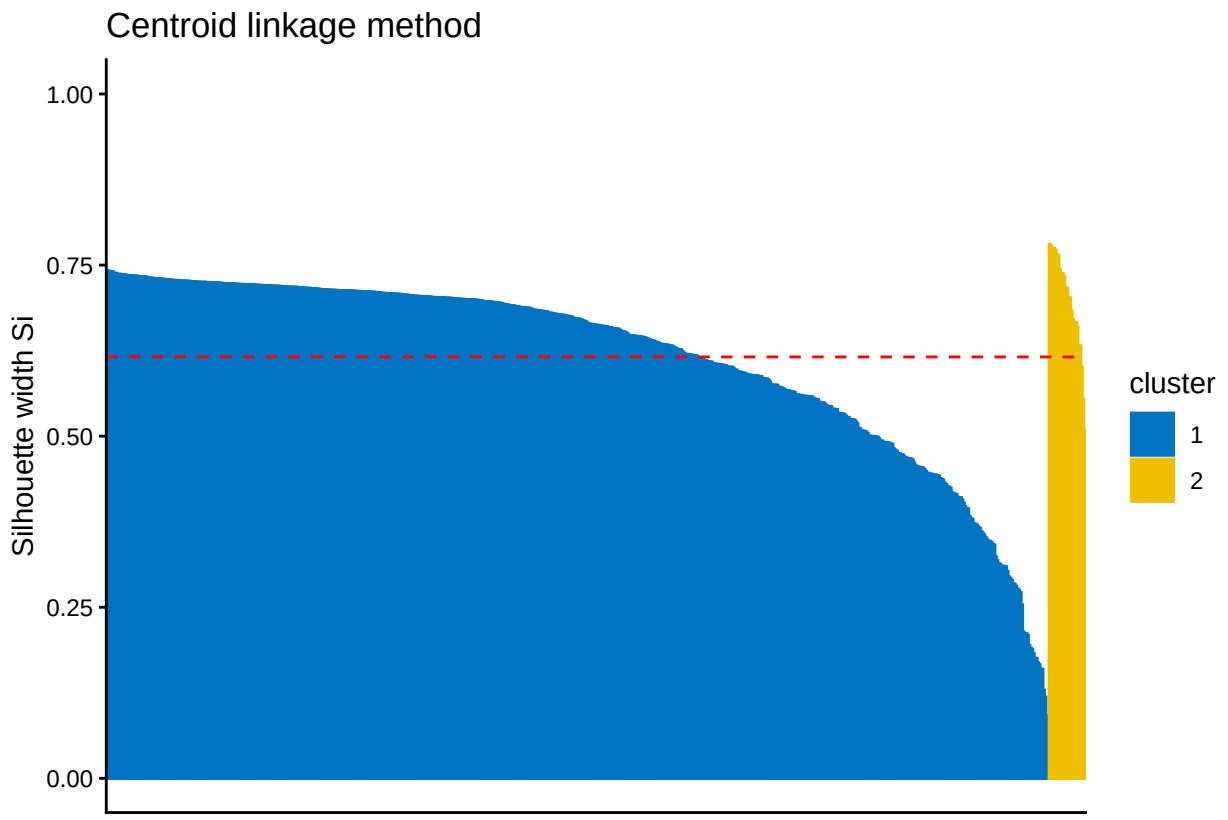



```
# Dunn index
Dunn.hc.cl <- cluster.stats(d = Beans.eu, clustering = Beans.e.eu.cl$cluster)
cat("Dunn index: ", Dunn.hc.cl$dunn)
```

```
## Dunn index:  0.2633058
```

```
# Silhouette width
fviz_silhouette(Beans.e.eu.cl, ggtheme = theme_classic(),
                palette = "jco",main = "Centroid linkage method")
```

```
##   cluster size ave.sil.width
## 1       1  647          0.61
## 2       2   25          0.70
```




```
cat("\nAverage silouette widith: ",Beans.e.eu.cl$silinfo$avg.width)
```

```
##
## Average silouette widith:  0.6157432
```

```
sil <- Beans.e.eu.cl$silinfo$widths[,1:3]

clusters <- unique(sil["cluster"])
for (i in clusters){
  neg.sil <- which(sil["sil_width"]<0 & sil["cluster"] == i)
  cat("Number of misassigned units for cluster",i,": ",
      nrow(sil[neg.sil, , drop = FALSE]),"\n")
}
```

```
## Number of misassigned units for cluster 1 :  0
## Number of misassigned units for cluster 2 :  0
```

We can now perform the external validation.

```
table(Beans$Class, Beans.e.eu.cl$cluster)
```

```
##
##               1   2
##   BARBUNYA   47   0
```

```
##   BOMBAY     0  25
##   CALI      67   0
##   DERMASON 188   0
##   HOROZ     97   0
##   SEKER     96   0
##   SIRA     152   0
```

```
clust_stats <- cluster.stats(d = Beans.eu,clustering = classes,
                             alt.clustering = Beans.e.eu.cl$cluster )

cat("Rand index: ", clust_stats$corrected.rand)
```

```
## Rand index:  0.03463531
```

```
cat("Meila's VI index: ", clust_stats$vi)
```

```
## Meila's VI index:  1.629362
```

The Rand index indicates that choosing at random a cluster for each data is the same of this result. The Meila's index indicates an high disagreement.

We can see that in this case that the Bombay's bean belongs to the cluster 2. So we can try to force the system to have 7 clusters and see the result.

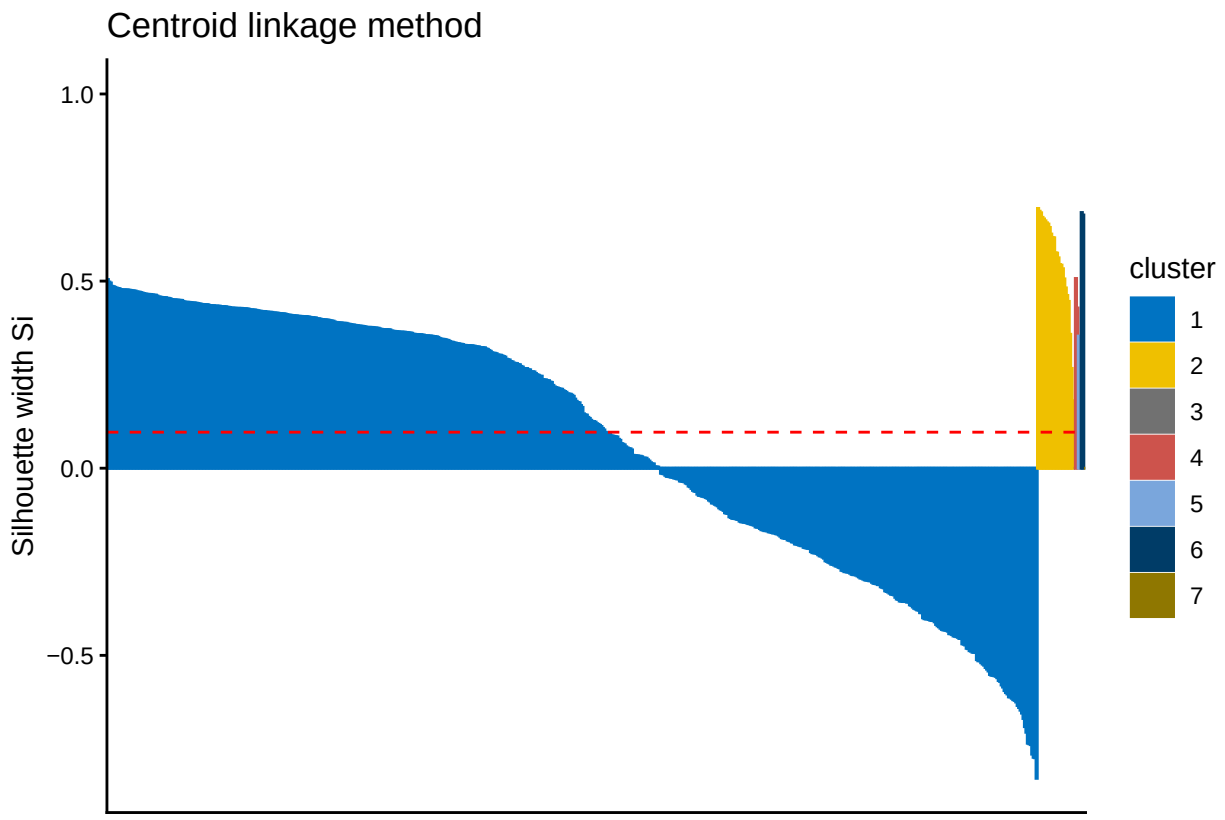
```
Beans.e.eu.cl7<-eclust(std.Beans, FUNcluster = "hclust",
                       hc_metric = "euclidean", k = 7,
                       hc_method = "centroid", graph = FALSE)
```

```
# Dunn index
Dunn.hc.cl7 <- cluster.stats(d = Beans.eu, clustering = Beans.e.eu.cl$cluster)
cat("Dunn index: ", Dunn.hc.cl$dunn)
```

```
## Dunn index:  0.2633058
```

```
# Silhouette width
fviz_silhouette(Beans.e.eu.cl7, ggtheme = theme_classic(),
                palette = "jco",main = "Centroid linkage method")
```

```
##   cluster size ave.sil.width
## 1       1  639          0.07
## 2       2   25          0.56
## 3       3    1          0.00
## 4       4    2          0.47
## 5       5    2          0.34
## 6       6    2          0.68
## 7       7    1          0.00
```




```
cat("Average silouette widith: ",Beans.e.eu.cl$silinfo$avg.width)
```

```
## Average silouette widith:  0.6157432
```

```
sil <- Beans.e.eu.cl7$silinfo$widths[,1:3]

clusters <- unique(sil["cluster"])
for (i in clusters){
  neg.sil <- which(sil["sil_width"]<0 & sil["cluster"] == i)
  cat("Number of misassigned units for cluster",i,": ",
      nrow(sil[neg.sil, , drop = FALSE]),"\n")
}
```

```
## Number of misassigned units for cluster 1 :  260
## Number of misassigned units for cluster 2 :  0
## Number of misassigned units for cluster 3 :  0
## Number of misassigned units for cluster 4 :  0
## Number of misassigned units for cluster 5 :  0
## Number of misassigned units for cluster 6 :  0
## Number of misassigned units for cluster 7 :  0
```

```r
table(Beans$Class, Beans.e.eu.cl7$cluster)
```

```
##
##               1   2   3   4   5   6   7
##    BARBUNYA  45   0   0   0   2   0   0
##    BOMBAY     0  25   0   0   0   0   0
##    CALI      67   0   0   0   0   0   0
##    DERMASON 188   0   0   0   0   0   0
##    HOROZ     92   0   1   2   0   2   0
##    SEKER     95   0   0   0   0   0   1
##    SIRA     152   0   0   0   0   0   0
```

```r
clust_stats <- cluster.stats(d = Beans.eu,clustering = classes,
                             alt.clustering = Beans.e.eu.cl7$cluster )

cat("Rand index: ", clust_stats$corrected.rand)
```

```
## Rand index:  0.03882751
```

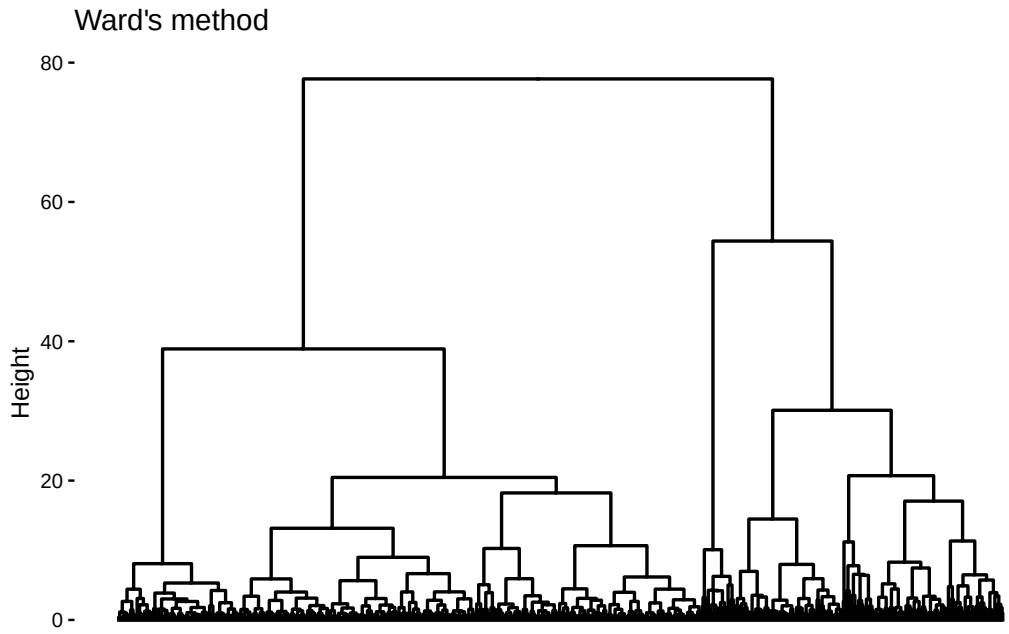
```r
cat("Meila's VI index: ", clust_stats$vi)
```

```
## Meila's VI index:  1.662144
```

7 clusters doesn't change the result. Only Bombay's bean are well described.

6.2.1.5 Ward's method .

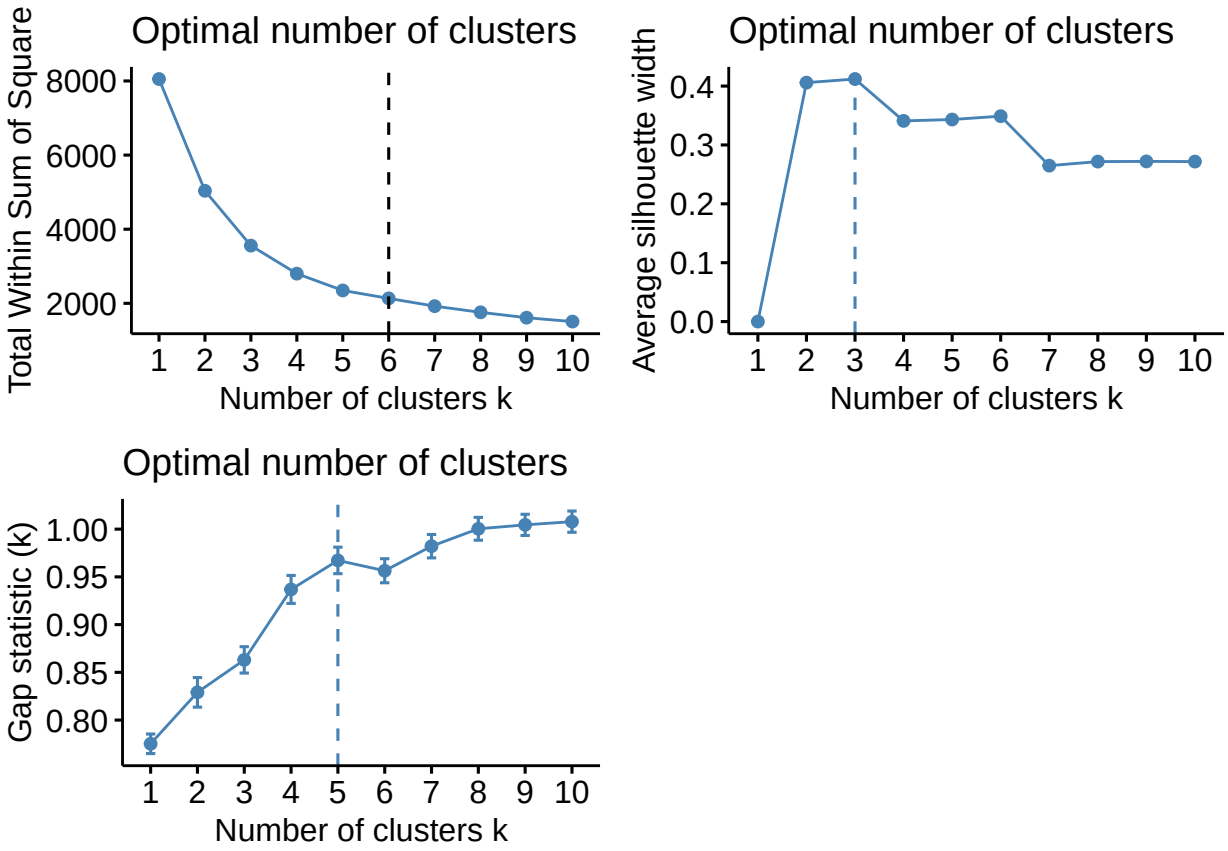
```r
Beans.hc.w.e <- hclust(Beans.eu, method = "ward.D2")
fviz_dend(Beans.hc.w.e, main = "Ward's method",
          show_labels = FALSE)
```




```
corr.w.e <- cor(Beans.eu,cophenetic(Beans.hc.w.e))
corr.w.e
```

```
## [1] 0.6313787
```

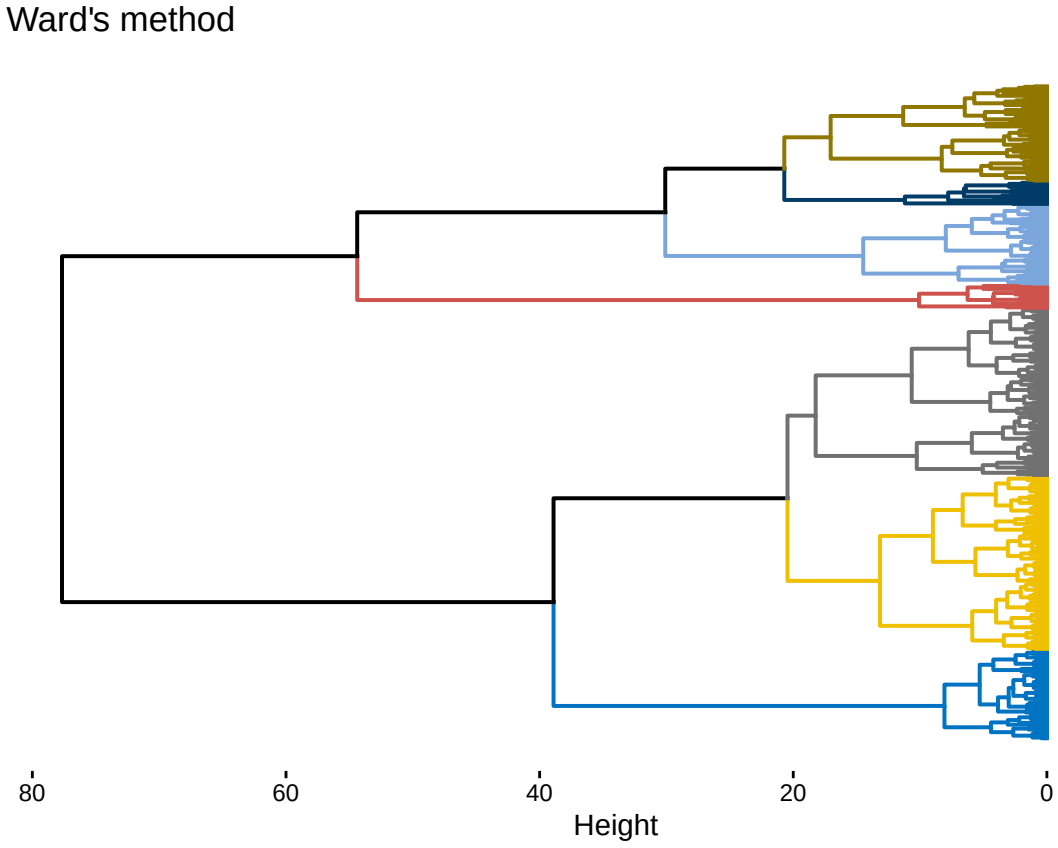
```
p1 <- fviz_nbclust(std.Beans, hcut, method = "wss", hc_metric = "euclidean"
             ,hc_method = "ward.D2") +
  geom_vline(xintercept = 6, linetype = 2)
p2 <- fviz_nbclust(std.Beans, hcut, method = "silhouette", hc_metric = "euclidean"
             ,hc_method = "ward.D2")
p3 <- fviz_nbclust(std.Beans, hcut, method = "gap_stat", hc_metric = "euclidean"
             ,hc_method = "ward.D2", nboot = 50)
grid.arrange(p1, p2, p3, ncol = 2, nrow = 2)
```



The three different method gives us three different result. It's impossible to choose one based on these three method. In this case we will use 7 clusters as the number of classes.

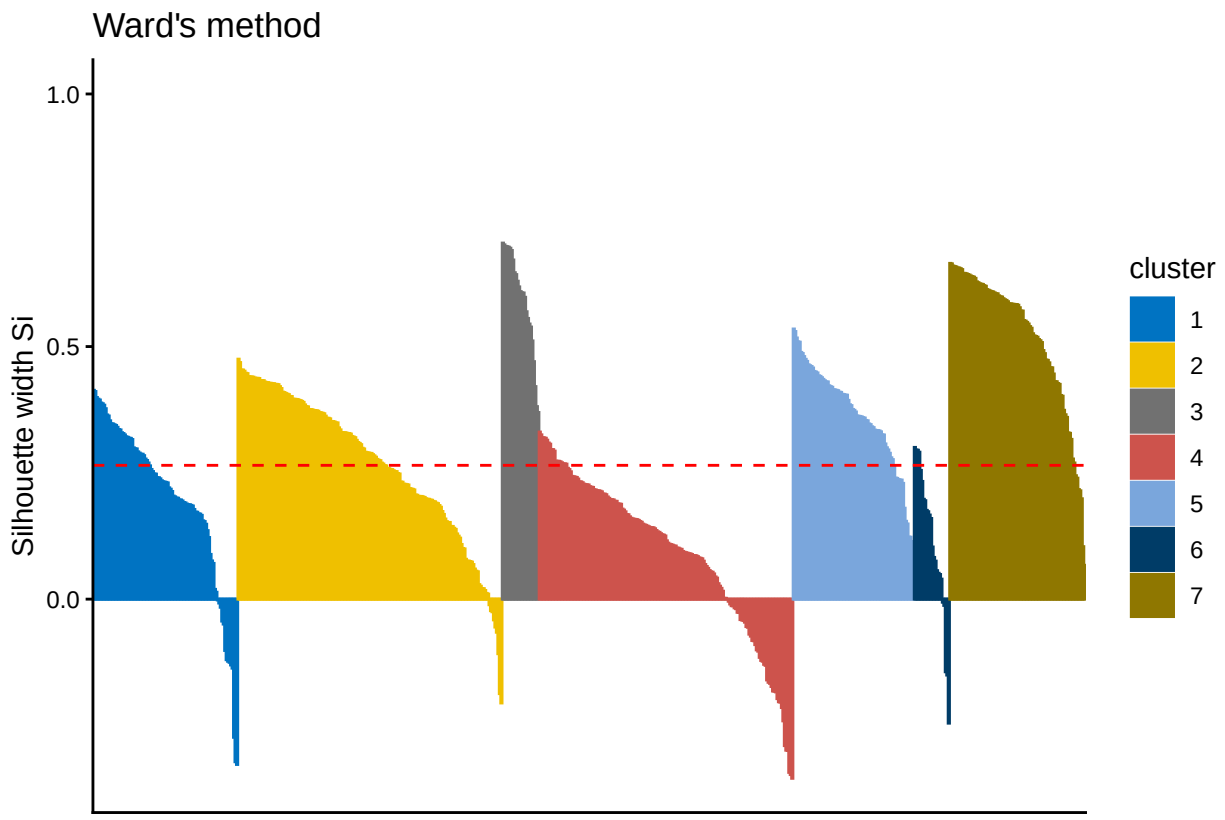
```
Beans.e.eu.w<-eclust(std.Beans, FUNcluster = "hclust",
                     hc_metric = "euclidean",k = 7,
                     hc_method = "ward.D2", graph = FALSE)

fviz_dend(Beans.e.eu.w, show_labels=FALSE, horiz = TRUE,
          palette = "jco",main = "Ward's method")
```




```
# Silhouette width
fviz_silhouette(Beans.e.eu.w, ggtheme = theme_classic(),
                palette = "jco",main = "Ward's method")
```

```
##   cluster size ave.sil.width
## 1       1   98          0.20
## 2       2  179          0.27
## 3       3   25          0.59
## 4       4  172          0.09
## 5       5   82          0.36
## 6       6   24          0.11
## 7       7   92          0.52
```




```
sil <- Beans.e.eu.w$silinfo$widths[,1:3]

clusters <- unique(sil[,"cluster"])
for (i in clusters){
  neg.sil <- which(sil[,"sil_width"]<0 & sil[,"cluster"] == i)
  cat("Number of misassigned units for cluster",i,": ",
      nrow(sil[neg.sil, , drop = FALSE]),"\n")
}
```

```
## Number of misassigned units for cluster 1 :  14
## Number of misassigned units for cluster 2 :  9
## Number of misassigned units for cluster 3 :  0
## Number of misassigned units for cluster 4 :  45
## Number of misassigned units for cluster 5 :  0
## Number of misassigned units for cluster 6 :  5
## Number of misassigned units for cluster 7 :  0
```

```
val.e.h.w <- clValid(std.Beans, nClust = 2:7, clMethods = "hierarchical",
                     validation ="internal", method="ward",
                     maxitems = 1000)
```

```
## Il metodo "ward" è stato rinominato in "ward.D"; nota il nuovo "ward.D2"
```

```
summary(val.e.h.w)
```

```
##
## Clustering Methods:
##  hierarchical
##
## Cluster sizes:
##  2 3 4 5 6 7
##
## Validation Measures:
##                                   2        3        4        5        6        7
##
## hierarchical Connectivity   31.3413  44.2306  44.2306  67.8706 124.6841 141.6393
##              Dunn            0.0488   0.0294   0.0437   0.0447   0.0338   0.0338
##              Silhouette      0.4185   0.3055   0.3464   0.3507   0.2645   0.2604
##
## Optimal Scores:
##
##              Score   Method       Clusters
## Connectivity 31.3413 hierarchical 2
## Dunn          0.0488 hierarchical 2
## Silhouette    0.4185 hierarchical 2
```

We can now perform the external validation.

```
table(Beans$Class, Beans.e.eu.w$cluster)
```

```
##
##              1   2   3   4   5   6   7
##   BARBUNYA  40   0   0   1   0   6   0
##   BOMBAY     0   0  25   0   0   0   0
##   CALI      52   0   0   1   3  11   0
##   DERMASON   0 153   0  24   0   1  10
##   HOROZ      0   0   0  13  78   6   0
##   SEKER      5   4   0   5   0   0  82
##   SIRA       1  22   0 128   1   0   0
```

```
clust_stats <- cluster.stats(d = Beans.eu,clustering = classes,
                             alt.clustering = Beans.e.eu.w$cluster )

cat("Rand index: ", clust_stats$corrected.rand)
```

```
## Rand index:  0.6115831
```

```
cat("Meila's VI index: ", clust_stats$vi)
```

```
## Meila's VI index:  1.150003
```

This method is more interesting: it divides the beans in different clusters. As we can see for example 1 belongs to Barbunya and Cali, 2 belongs to Dermason, 3 bolngs to Bombay, 4 belongs to Siria, 5 belogns to Horoz, 6 is not well defined and 7 belongs to Seker.

The Rand index indicates a good agreement. The Meila's is still an high value.

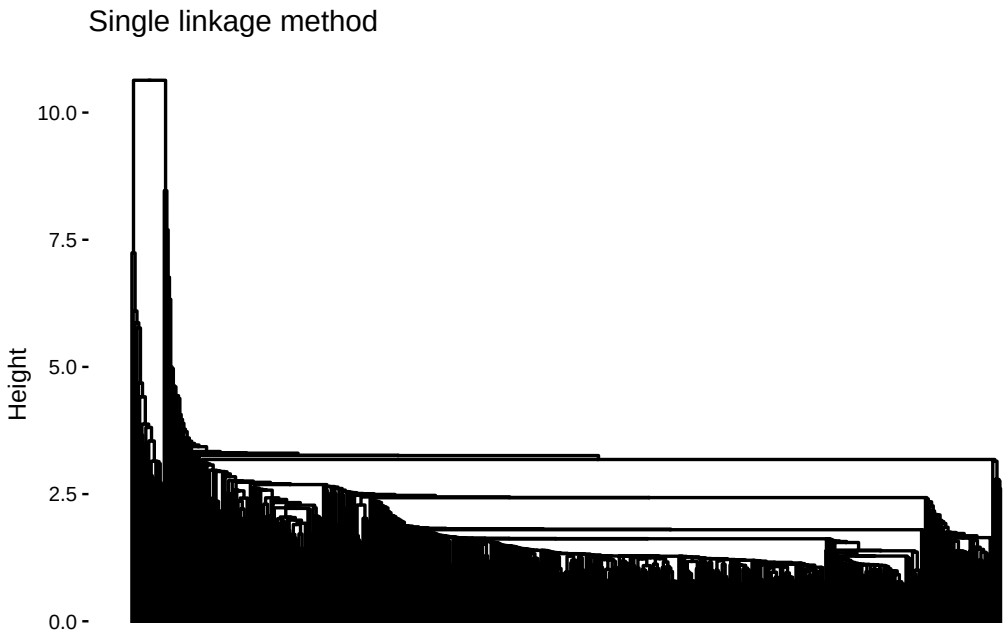
6.2.2 Manhattan distance

Now we will use the Manhattan distance. This distance is lower subject to outlayers.

```
Beans.ma <- dist(std.Beans, method = "manhattan")
```

6.2.2.1 Single linkage method .

```
Beans.hc.s.m <- hclust(Beans.ma, method = "single")
fviz_dend(Beans.hc.s.m, main = "Single linkage method",
          show_labels = FALSE)
```

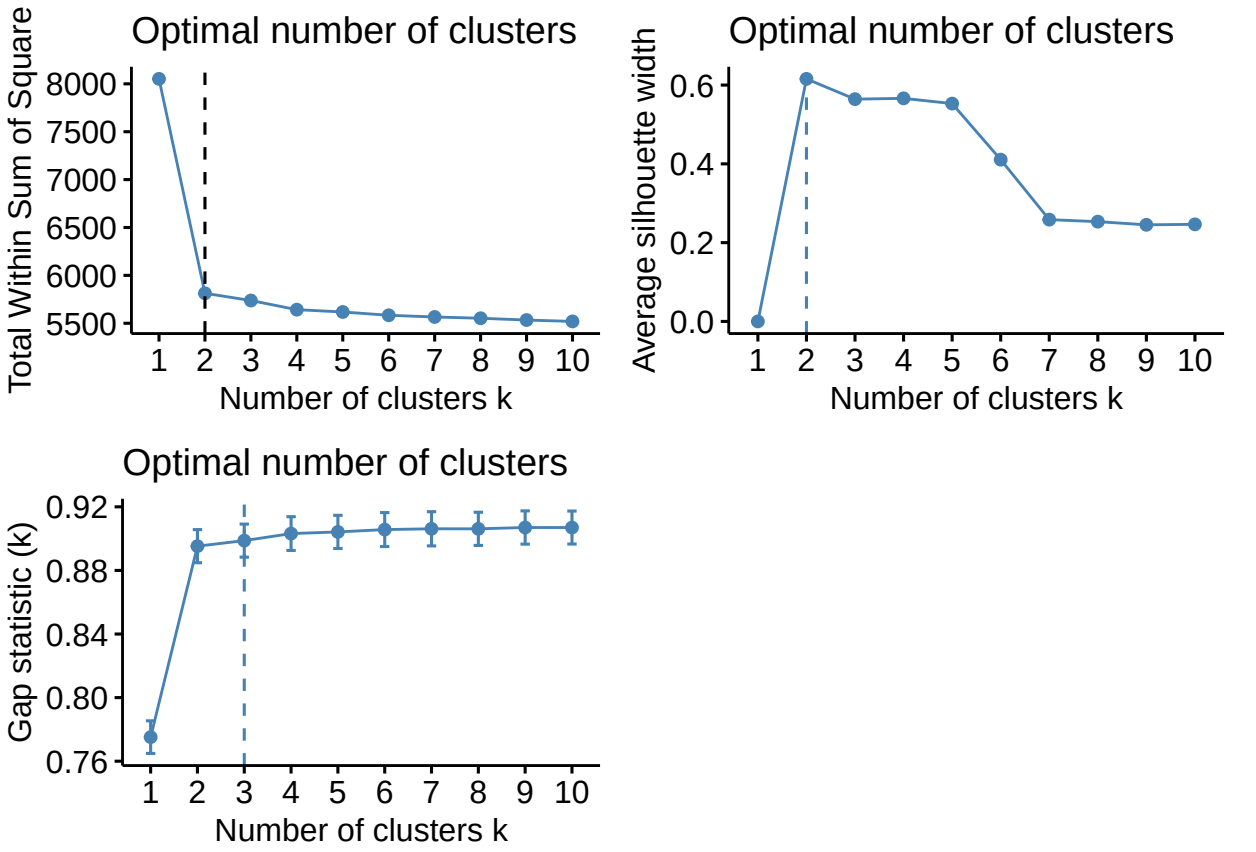



```
corr.s.m <- cor(Beans.ma,cophenetic(Beans.hc.s.m))
corr.s.m
```

```
## [1] 0.7525378
```

```
p1 <- fviz_nbclust(std.Beans, hcut, method = "wss", hc_metric = "manhattan"
             ,hc_method = "single") +
  geom_vline(xintercept = 2, linetype = 2)
p2 <- fviz_nbclust(std.Beans, hcut, method = "silhouette", hc_metric = "manhattan"
             ,hc_method = "single")
```

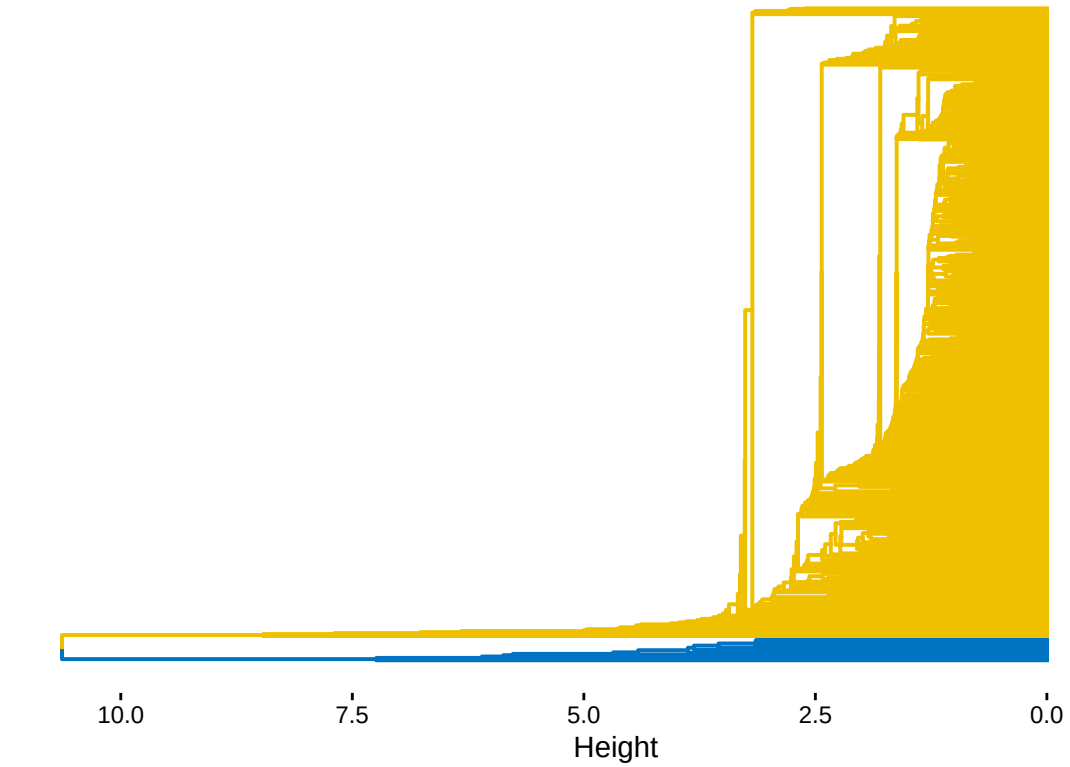
```
p3 <- fviz_nbclust(std.Beans, hcut, method = "gap_stat", hc_metric = "manhattan"
             ,hc_method = "single", nboot = 50)
grid.arrange(p1, p2, p3, ncol = 2, nrow = 2)
```



With this method 2 is the optimal number of clusters.

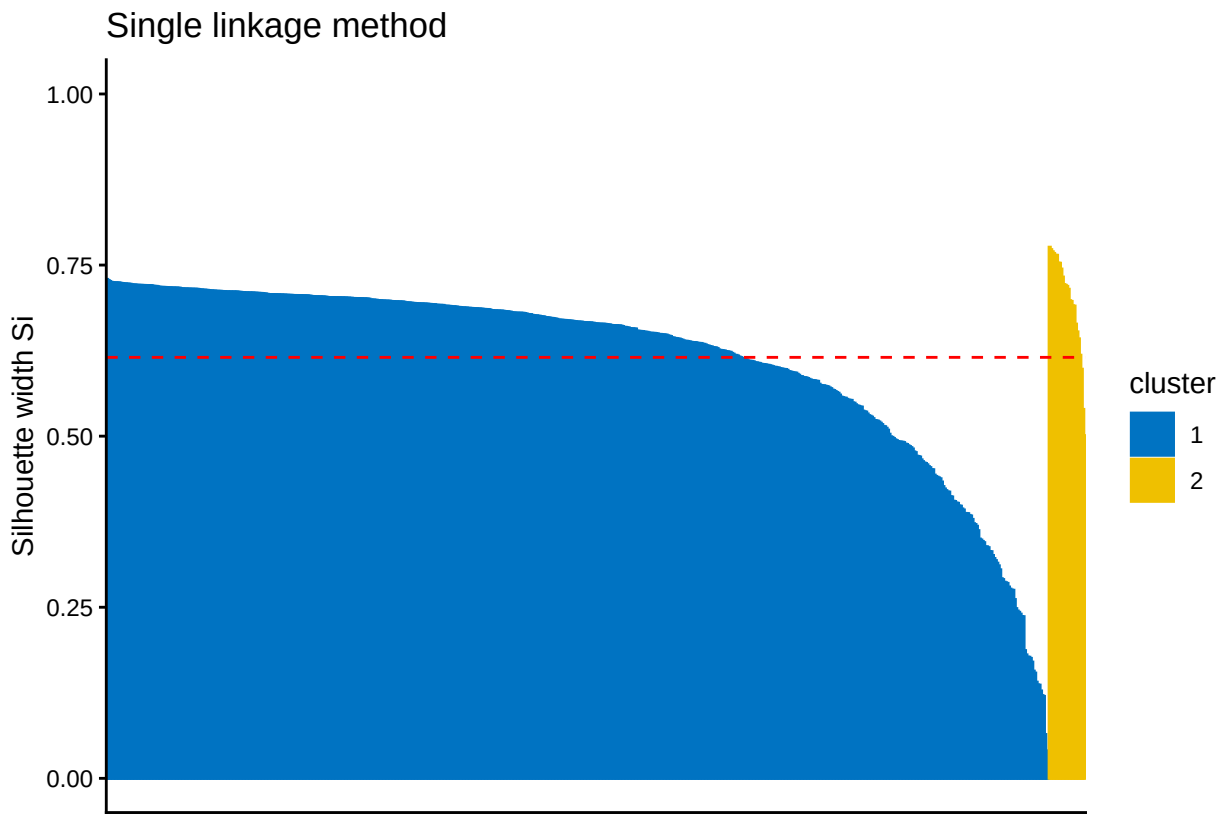
```
Beans.e.m.s<-eclust(std.Beans, FUNcluster = "hclust",
                    hc_metric = "manhattan",k = 2,
                    hc_method = "single", graph = FALSE)

fviz_dend(Beans.e.m.s, show_labels=FALSE, horiz = TRUE,
          palette = "jco",main = "Single linkage method")
```




```
# Silhouette width
fviz_silhouette(Beans.e.m.s, ggtheme = theme_classic(), palette = "jco",
                main = "Single linkage method")
```

```
##   cluster size ave.sil.width
## 1       1  647          0.61
## 2       2   25          0.70
```




```
sil <- Beans.e.m.s$silinfo$widths[,1:3]

clusters <- unique(sil[,"cluster"])
for (i in clusters){
  neg.sil <- which(sil[,"sil_width"]<0 & sil[,"cluster"] == i)
  cat("Number of misassigned units for cluster",i,": ",
      nrow(sil[neg.sil, , drop = FALSE]),"\n")
}
```

```
## Number of misassigned units for cluster 1 :  0
## Number of misassigned units for cluster 2 :  0
```

```
val.m.h.s <- clValid(std.Beans, nClust = 2:7, clMethods = "hierarchical",
                     validation ="internal", method="single",
                     maxitems = 1000)
summary(val.m.h.s)
```

```
##
## Clustering Methods:
##  hierarchical
##
## Cluster sizes:
##  2 3 4 5 6 7
```

```
##
## Validation Measures:
##                                  2       3       4       5       6       7
##
## hierarchical Connectivity   3.8579  3.8579  5.8579  9.6202 12.8032 15.7321
##              Dunn           0.2615  0.3273  0.2468  0.2334  0.2269  0.2085
##              Silhouette     0.5614  0.5845  0.5663  0.5530  0.4107  0.4052
##
## Optimal Scores:
##
##              Score  Method       Clusters
## Connectivity 3.8579 hierarchical 2
## Dunn         0.3273 hierarchical 3
## Silhouette   0.5845 hierarchical 3
```

We can now perform the external validation.

```
table(Beans$Class, Beans.e.m.s$cluster)
```

```
##
##               1   2
##   BARBUNYA   47   0
##   BOMBAY      0  25
##   CALI       67   0
##   DERMASON  188   0
##   HOROZ      97   0
##   SEKER      96   0
##   SIRA      152   0
```

```
clust_stats <- cluster.stats(d = Beans.eu,clustering = classes,
                             alt.clustering = Beans.e.m.s$cluster )

cat("Rand index: ", clust_stats$corrected.rand)
```

```
## Rand index:  0.03463531
```

```
cat("Meila's VI index: ", clust_stats$vi)
```

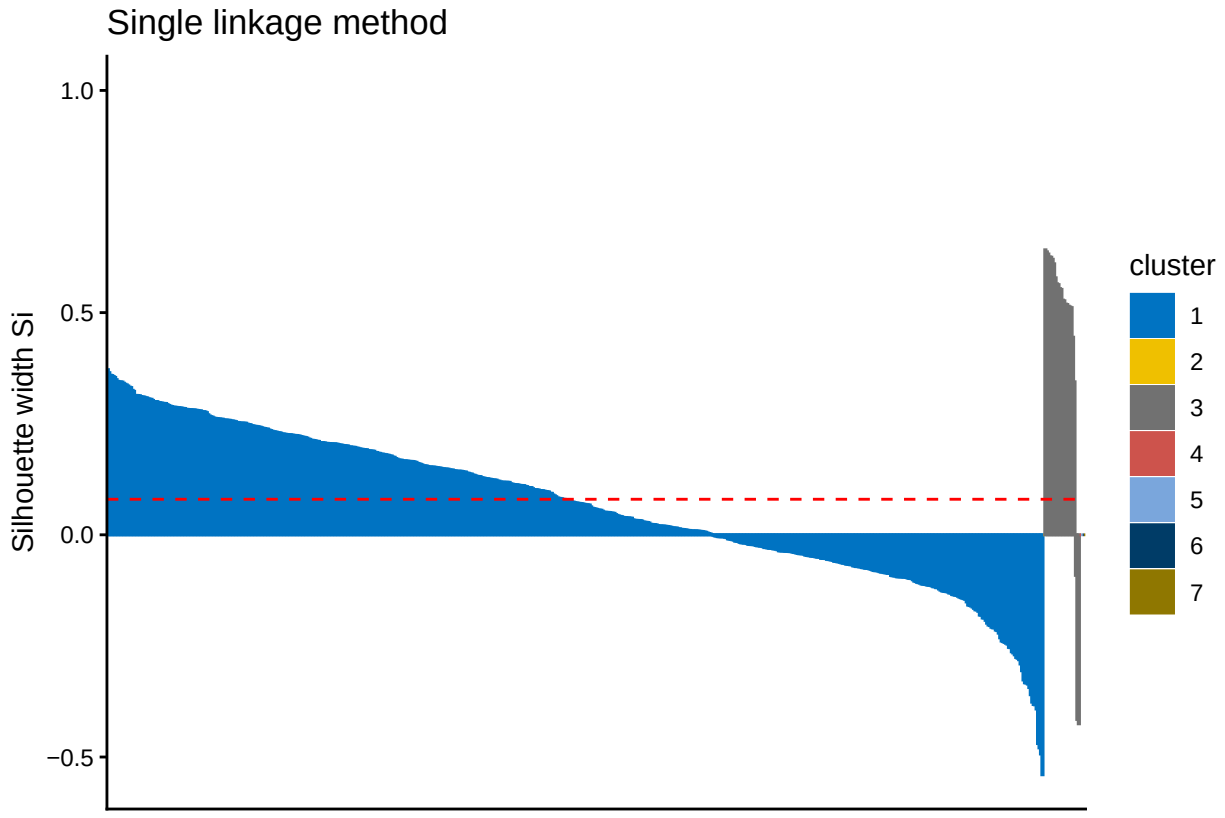
```
## Meila's VI index:  1.629362
```

The Rand index indicates that choosing at random a cluster for each data is the same of this result. The Meila's index indicates an high disagreement. We can see that in this case that the Bombay's bean belongs to the cluster 2. So we can try to force the system to have 7 clusters and see the result.

```
Beans.e.m.s7<-eclust(std.Beans, FUNcluster = "hclust",
                     hc_metric = "manhattan",k = 7,
                     hc_method = "single", graph = FALSE)
```

```
# Silhouette width
fviz_silhouette(Beans.e.m.s7, ggtheme = theme_classic(), palette = "jco",
                main = "Single linkage method")
```

```
##   cluster size ave.sil.width
## 1       1  643          0.07
## 2       2    1          0.00
## 3       3   24          0.45
## 4       4    1          0.00
## 5       5    1          0.00
## 6       6    1          0.00
## 7       7    1          0.00
```




```
sil <- Beans.e.m.s7$silinfo$widths[,1:3]

clusters <- unique(sil["cluster"])
for (i in clusters){
  neg.sil <- which(sil["sil_width"]<0 & sil["cluster"] == i)
  cat("Number of misassigned units for cluster",i,": ",
      nrow(sil[neg.sil, , drop = FALSE]),"\n")
}
```

```
## Number of misassigned units for cluster 1 :  227
## Number of misassigned units for cluster 2 :  0
## Number of misassigned units for cluster 3 :  3
## Number of misassigned units for cluster 4 :  0
## Number of misassigned units for cluster 5 :  0
## Number of misassigned units for cluster 6 :  0
## Number of misassigned units for cluster 7 :  0
```

```
table(Beans$Class, Beans.e.m.s7$cluster)
```

```
##
##               1   2   3   4   5   6   7
##    BARBUNYA  47   0   0   0   0   0   0
##    BOMBAY     0   1  24   0   0   0   0
##    CALI      67   0   0   0   0   0   0
##    DERMASON 188   0   0   0   0   0   0
##    HOROZ     95   0   0   0   1   1   0
##    SEKER     95   0   0   1   0   0   0
##    SIRA     151   0   0   0   0   0   1
```

```
clust_stats <- cluster.stats(d = Beans.eu,clustering = classes,
                             alt.clustering = Beans.e.m.s7$cluster )

cat("Rand index: ", clust_stats$corrected.rand)
```

```
## Rand index:  0.03522276
```

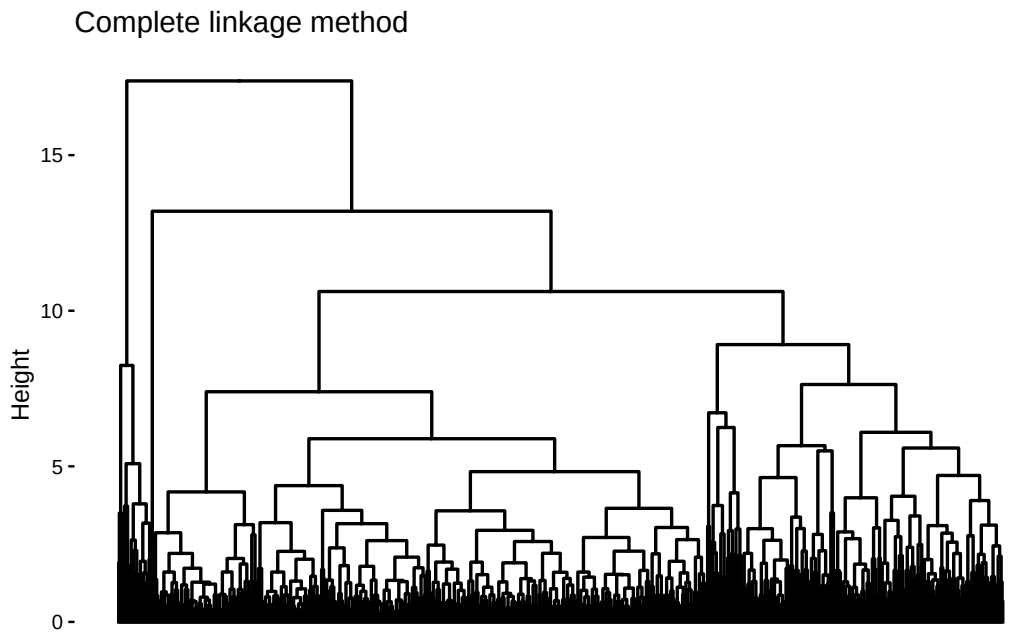
```
cat("Meila's VI index: ", clust_stats$vi)
```

```
## Meila's VI index:  1.658735
```

7 clusters doesn't change the result. Only Bombay's bean are well described.

6.2.2.2 Complete linkage mathod .

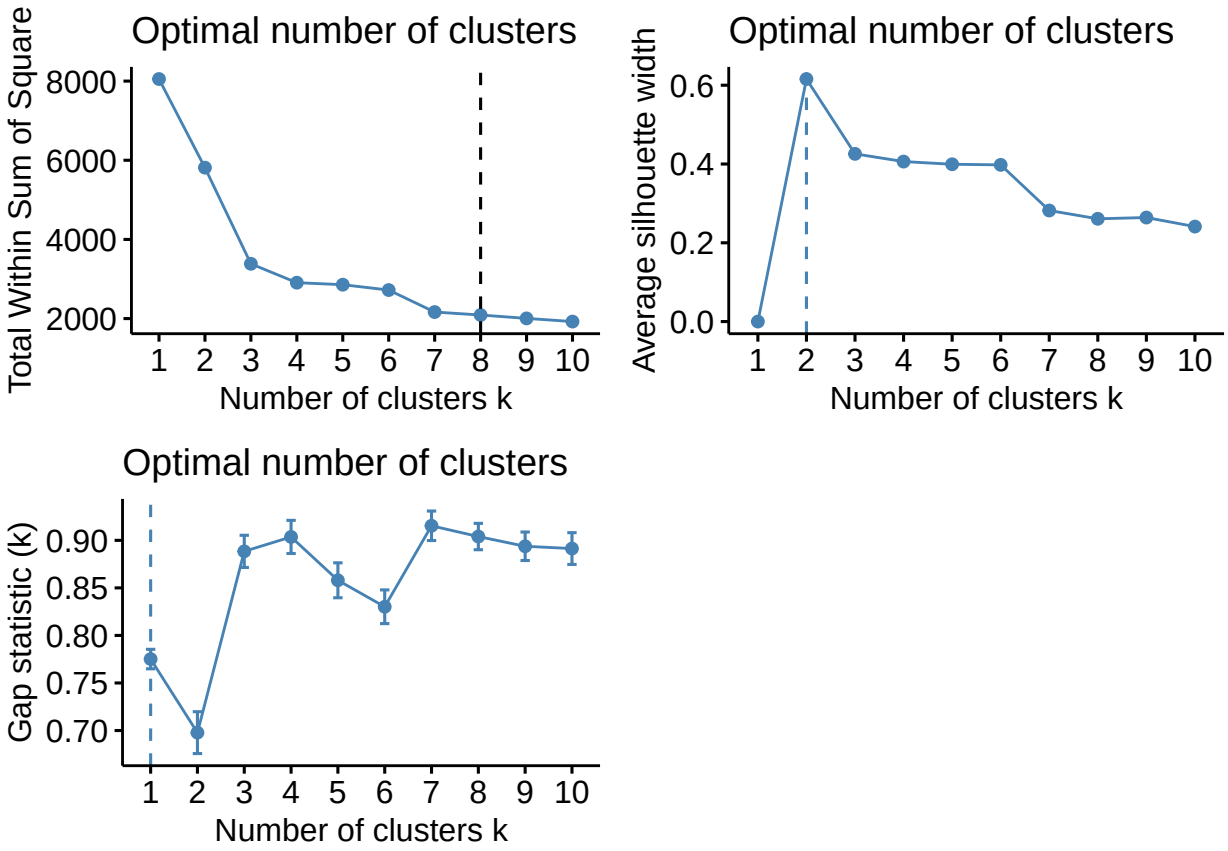
```
Beans.hc.c.m <- hclust(Beans.ma, method = "complete")
fviz_dend(Beans.hc.c.e, main = "Complete linkage method",
          show_labels = FALSE)
```




```
corr.c.m <- cor(Beans.ma,cophenetic(Beans.hc.c.m))
corr.c.m
```

```
## [1] 0.8090778
```

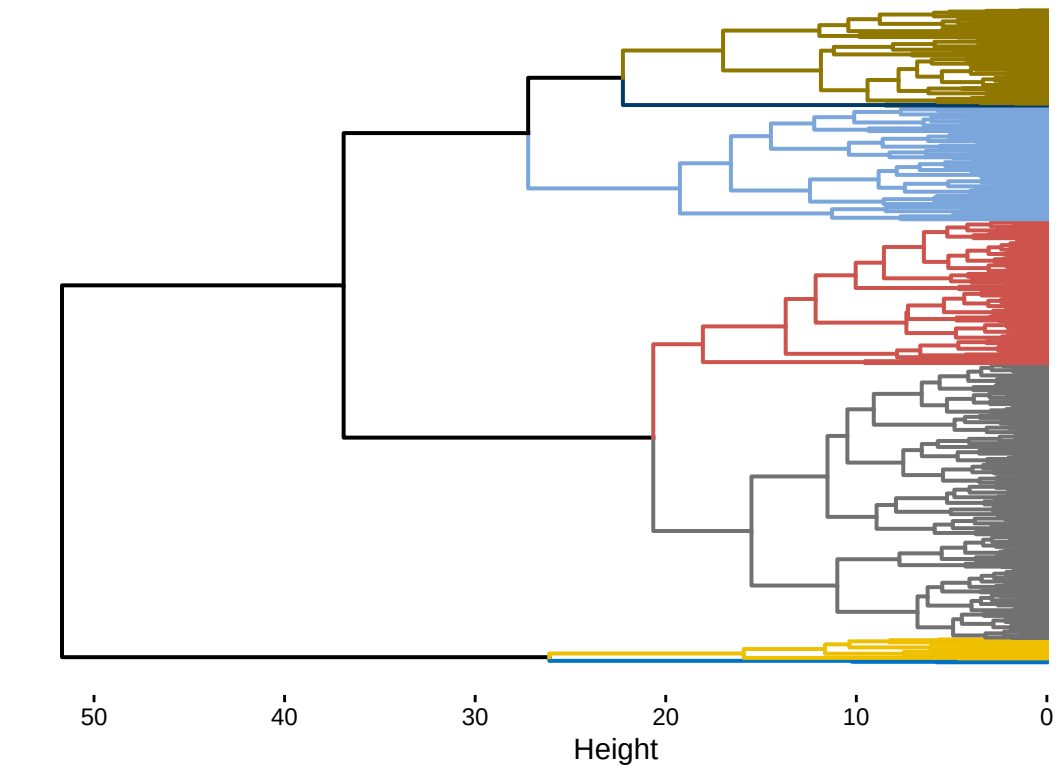
```
p1 <- fviz_nbclust(std.Beans, hcut, method = "wss", hc_metric = "manhattan"
             ,hc_method = "complete") +
  geom_vline(xintercept = 8, linetype = 2)
p2 <- fviz_nbclust(std.Beans, hcut, method = "silhouette", hc_metric = "manhattan"
             ,hc_method = "complete")
p3 <- fviz_nbclust(std.Beans, hcut, method = "gap_stat", hc_metric = "manhattan"
             ,hc_method = "complete", nboot = 50)
grid.arrange(p1, p2, p3, ncol = 2, nrow = 2)
```



The three different method gives us three different result. It's impossible to choose one based on these three method. In this case we will use 7 clusters as the number of classes.

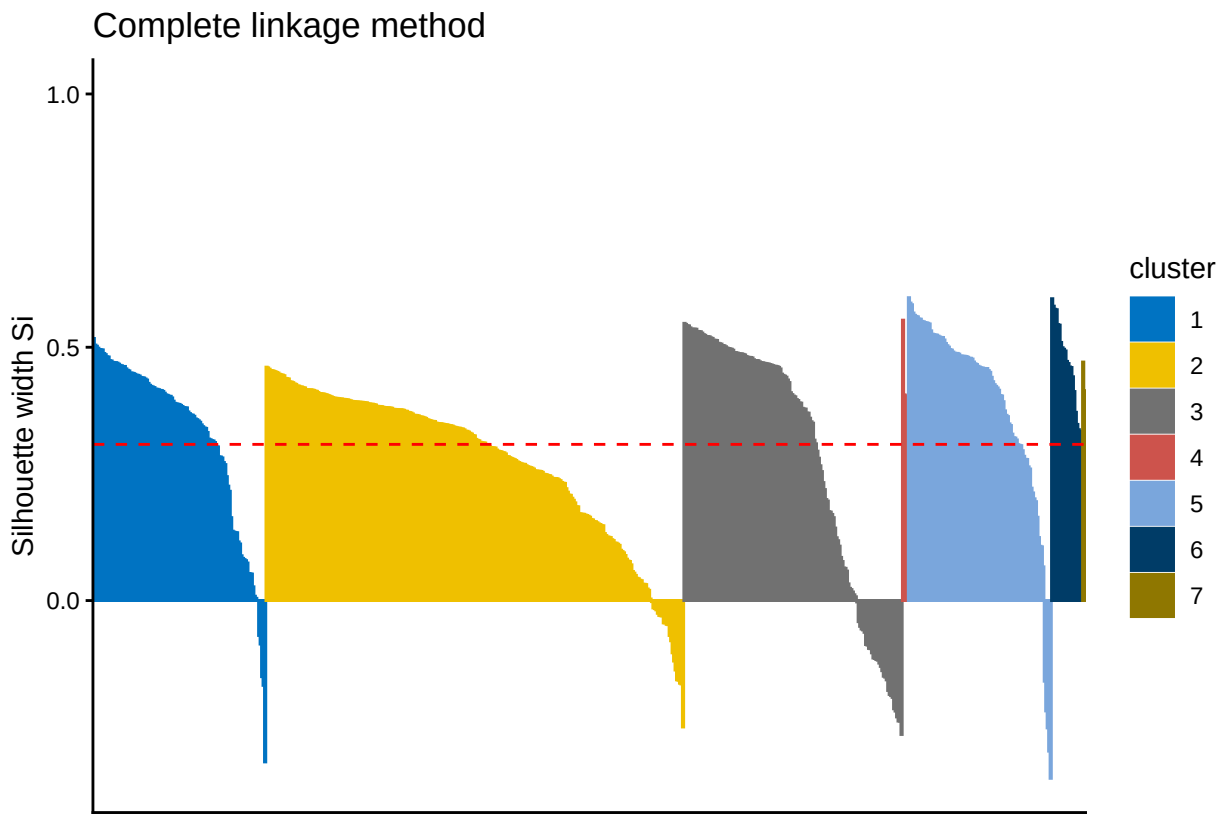
```
Beans.e.m.c<-eclust(std.Beans, FUNcluster = "hclust", hc_metric = "manhattan",
                    k = 7, hc_method = "complete", graph = FALSE)

fviz_dend(Beans.e.m.c, show_labels=FALSE, horiz = TRUE, palette = "jco",
          main = "Complete linkage method")
```




```
# Silhouette width
fviz_silhouette(Beans.e.m.c, ggtheme = theme_classic(), palette = "jco",
                main = "Complete linkage method")
```

```
##   cluster size ave.sil.width
## 1       1  117          0.33
## 2       2  283          0.27
## 3       3  148          0.28
## 4       4    4          0.42
## 5       5   97          0.39
## 6       6   21          0.46
## 7       7    2          0.44
```




```
sil <- Beans.e.m.c$silinfo$widths[,1:3]

clusters <- unique(sil[,"cluster"])
for (i in clusters){
  neg.sil <- which(sil[,"sil_width"]<0 & sil[,"cluster"] == i)
  cat("Number of misassigned units for cluster",i,": ",
      nrow(sil[neg.sil, , drop = FALSE]),"\n")
}
```

```
## Number of misassigned units for cluster 1 :  5
## Number of misassigned units for cluster 2 :  22
## Number of misassigned units for cluster 3 :  31
## Number of misassigned units for cluster 4 :  0
## Number of misassigned units for cluster 5 :  5
## Number of misassigned units for cluster 6 :  0
## Number of misassigned units for cluster 7 :  0
```

```
val.m.h.c <- clValid(std.Beans, nClust = 2:7, clMethods = "hierarchical",
                     validation ="internal", method="complete",
                     maxitems = 1000)
summary(val.m.h.c)
```

```
##
```

```
## Clustering Methods:
##  hierarchical
##
## Cluster sizes:
##  2 3 4 5 6 7
##
## Validation Measures:
##                                  2        3        4        5        6        7
##
## hierarchical Connectivity    0.0000   3.8579  32.7806  66.4266  72.9063 114.8238
##              Dunn            0.2633   0.3273   0.0618   0.0668   0.0722   0.0664
##              Silhouette      0.6157   0.5845   0.4225   0.3896   0.3820   0.3452
##
## Optimal Scores:
##
##              Score  Method       Clusters
## Connectivity 0.0000 hierarchical 2
## Dunn         0.3273 hierarchical 3
## Silhouette   0.6157 hierarchical 2
```

We can now perform the external validation.

```
table(Beans$Class, Beans.e.m.c$cluster)
```

```
##
##              1   2   3   4   5   6   7
##   BARBUNYA  46   1   0   0   0   0   0
##   BOMBAY     0   0   0   4   0  21   0
##   CALI      66   0   0   0   1   0   0
##   DERMASON   0 146  39   0   3   0   0
##   HOROZ      3   5   0   0  87   0   2
##   SEKER      0   3  93   0   0   0   0
##   SIRA       2 128  16   0   6   0   0
```

```
clust_stats <- cluster.stats(d = Beans.eu,clustering = classes,
                             alt.clustering = Beans.e.m.c$cluster )

cat("Rand index: ", clust_stats$corrected.rand)
```

```
## Rand index:  0.4791077
```

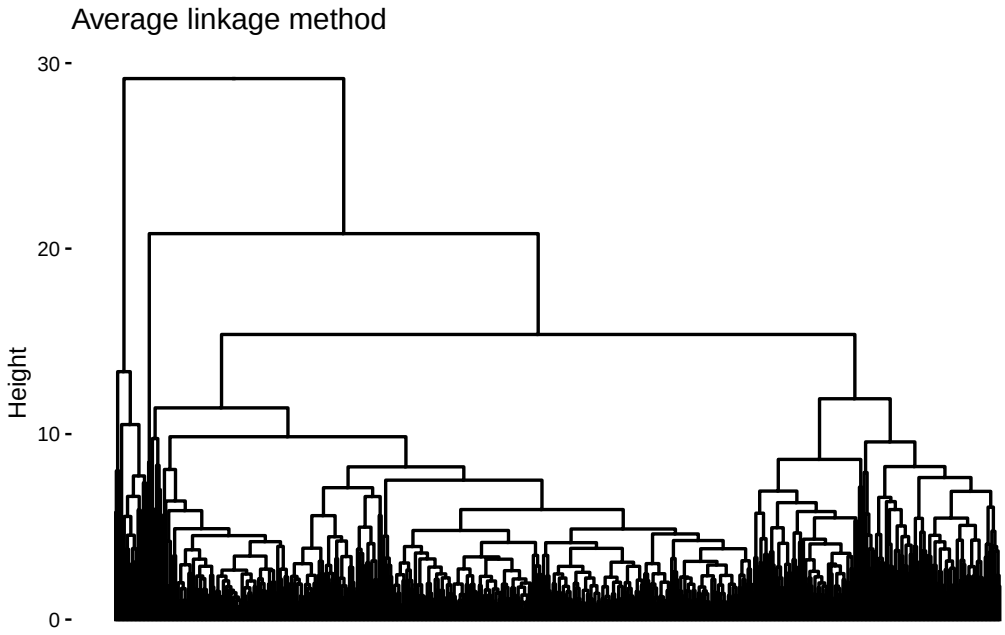
```
cat("Meila's VI index: ", clust_stats$vi)
```

```
## Meila's VI index:  1.16532
```

The Rand index indicates a middle agreement. The Meila's index indicates an high disagreement.

6.2.2.3 Average linkage method .

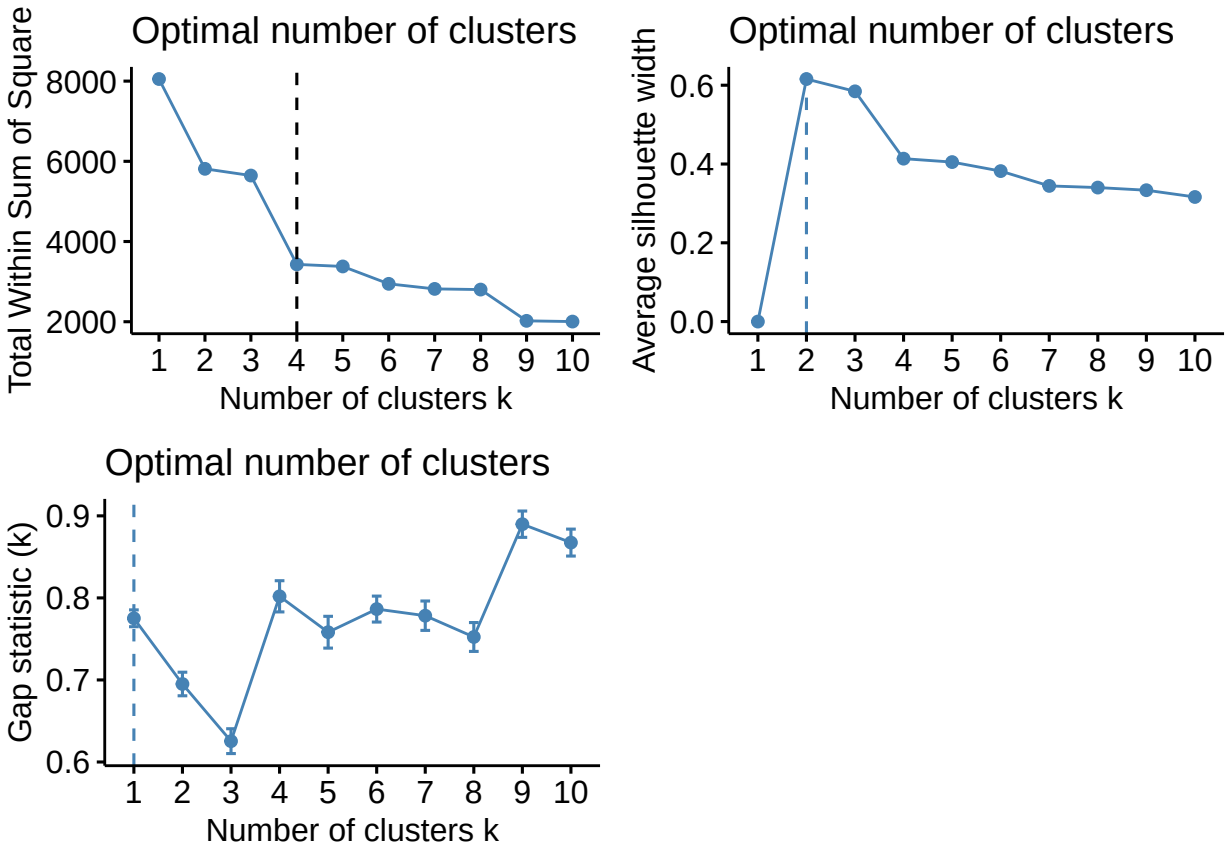
```
Beans.hc.a.m <- hclust(Beans.ma, method = "average")
fviz_dend(Beans.hc.a.m, main = "Average linkage method",
          show_labels = FALSE)
```




```
corr.a.m <- cor(Beans.ma,cophenetic(Beans.hc.a.m))
corr.a.m
```

```
## [1] 0.857768
```

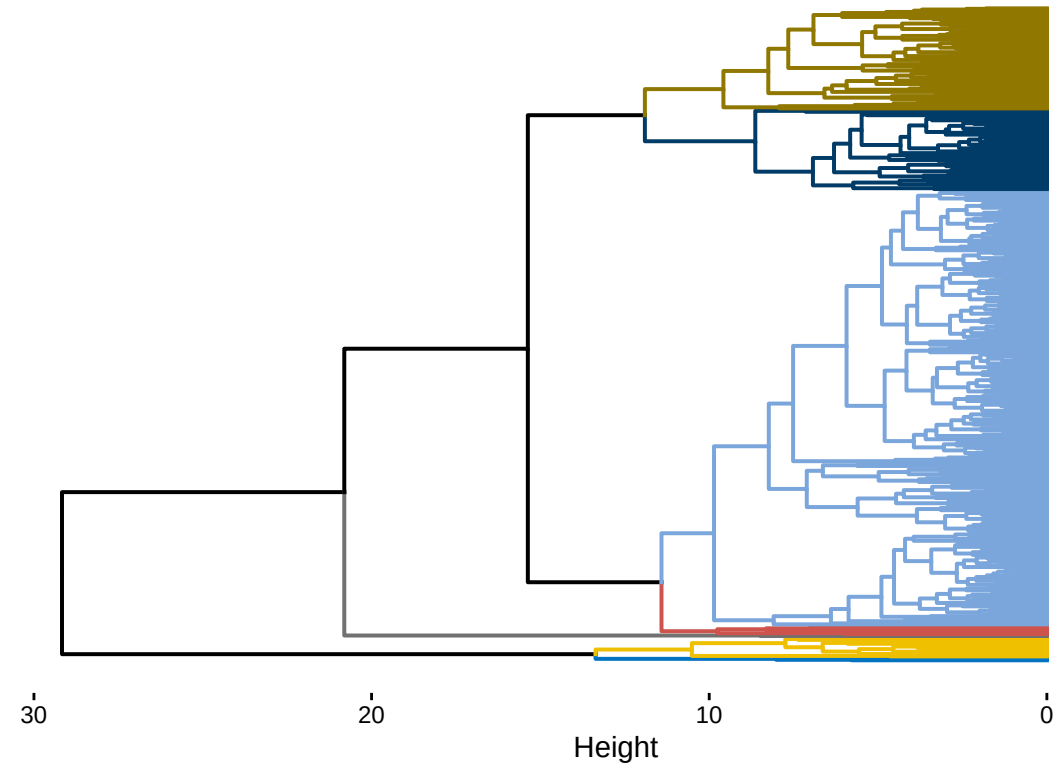
```
p1 <- fviz_nbclust(std.Beans, hcut, method = "wss", hc_metric = "manhattan"
             ,hc_method = "average") +
  geom_vline(xintercept = 4, linetype = 2)
p2 <- fviz_nbclust(std.Beans, hcut, method = "silhouette", hc_metric = "manhattan"
             ,hc_method = "average")
p3 <- fviz_nbclust(std.Beans, hcut, method = "gap_stat", hc_metric = "manhattan"
             ,hc_method = "average", nboot = 50)
grid.arrange(p1, p2, p3, ncol = 2, nrow = 2)
```



The three different method gives us three different result. It's impossible to choose one based on these three method. In this case we will use 7 clusters as the number of classes.

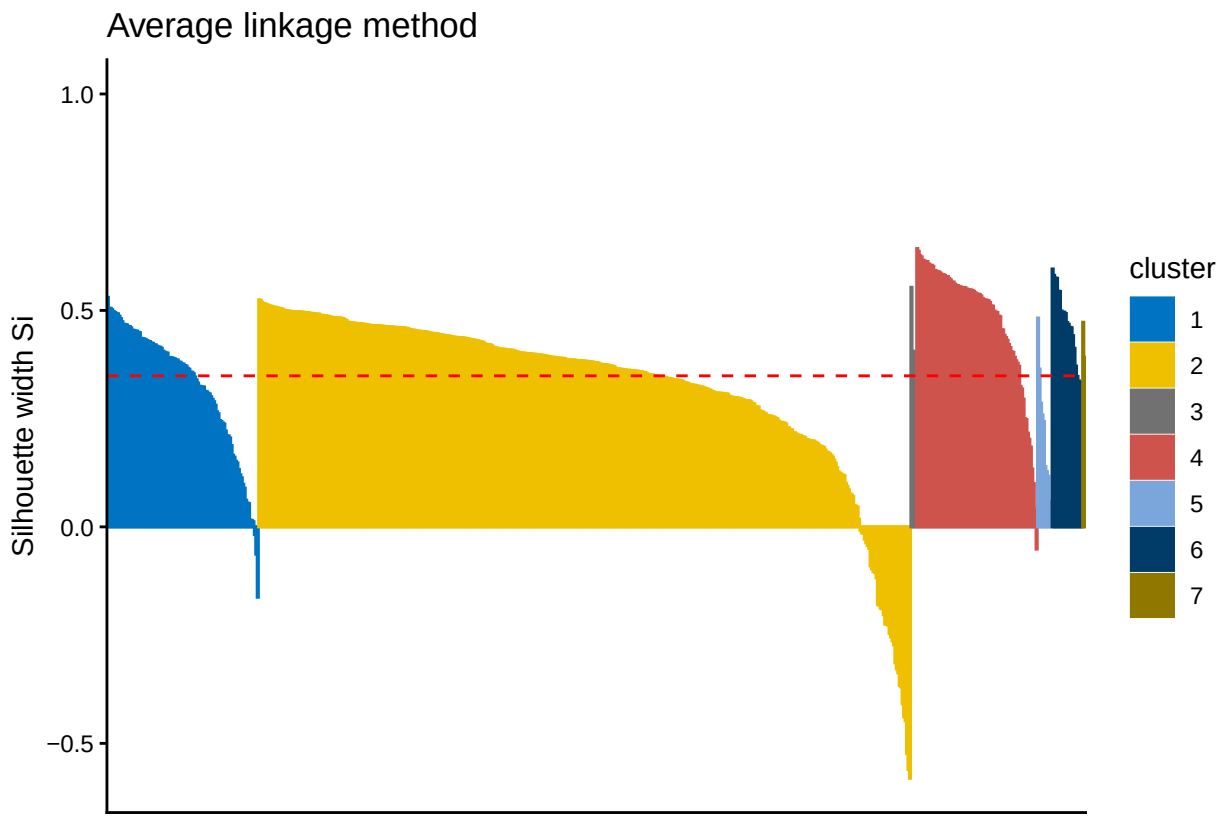
```
Beans.e.m.al<-eclust(std.Beans, FUNcluster = "hclust", hc_metric = "manhattan",
                     k = 7, hc_method = "average", graph = FALSE)

fviz_dend(Beans.e.m.al, show_labels=FALSE, horiz = TRUE, palette = "jco",
          main = "Average linkage method")
```




```
# Silhouette width
fviz_silhouette(Beans.e.m.al, ggtheme = theme_classic(), palette = "jco",
                main = "Average linkage method")
```

```
##   cluster size ave.sil.width
## 1       1  104          0.33
## 2       2  448          0.33
## 3       3    4          0.42
## 4       4   83          0.48
## 5       5   10          0.21
## 6       6   21          0.46
## 7       7    2          0.43
```




```
sil <- Beans.e.m.al$silinfo$widths[,1:3]

clusters <- unique(sil[,"cluster"])
for (i in clusters){
  neg.sil <- which(sil[,"sil_width"]<0 & sil[,"cluster"] == i)
  cat("Number of misassigned units for cluster",i,": ",
      nrow(sil[neg.sil, , drop = FALSE]),"\n")
}
```

```
## Number of misassigned units for cluster 1 :  3
## Number of misassigned units for cluster 2 :  35
## Number of misassigned units for cluster 3 :  0
## Number of misassigned units for cluster 4 :  1
## Number of misassigned units for cluster 5 :  0
## Number of misassigned units for cluster 6 :  0
## Number of misassigned units for cluster 7 :  0
```

```
val.m.h.a <- clValid(std.Beans, nClust = 2:7, clMethods = "hierarchical",
                     validation ="internal", method="average",
                     maxitems = 1000)
summary(val.m.h.a)
```

```
##
```

```
## Clustering Methods:
##  hierarchical
##
## Cluster sizes:
##  2 3 4 5 6 7
##
## Validation Measures:
##                                  2       3       4       5       6       7
##
## hierarchical Connectivity   0.0000  3.8579 43.6028 48.5825 55.0623 73.9865
##              Dunn           0.2633  0.3273  0.0649  0.0649  0.0649  0.0671
##              Silhouette     0.6157  0.5845  0.4152  0.3640  0.3548  0.3281
##
## Optimal Scores:
##
##              Score  Method       Clusters
## Connectivity 0.0000 hierarchical 2
## Dunn         0.3273 hierarchical 3
## Silhouette   0.6157 hierarchical 2
```

We can now perform the external validation.

```
table(Beans$Class, Beans.e.m.al$cluster)
```

```
##
##              1   2   3   4   5   6   7
##   BARBUNYA  41   3   0   0   3   0   0
##   BOMBAY     0   0   4   0   0  21   0
##   CALI      63   1   0   3   0   0   0
##   DERMASON   0 187   0   0   1   0   0
##   HOROZ      0  19   0  76   0   0   2
##   SEKER      0  91   0   0   5   0   0
##   SIRA       0 147   0   4   1   0   0
```

```
clust_stats <- cluster.stats(d = Beans.eu,clustering = classes,
                             alt.clustering = Beans.e.m.al$cluster )

cat("Rand index: ", clust_stats$corrected.rand)
```

```
## Rand index:  0.3256128
```

```
cat("Meila's VI index: ", clust_stats$vi)
```

```
## Meila's VI index:  1.222872
```

The Rand index indicates a week agreement. The Meila's index indicates strong disagreement.

6.3 Partitioning clustering

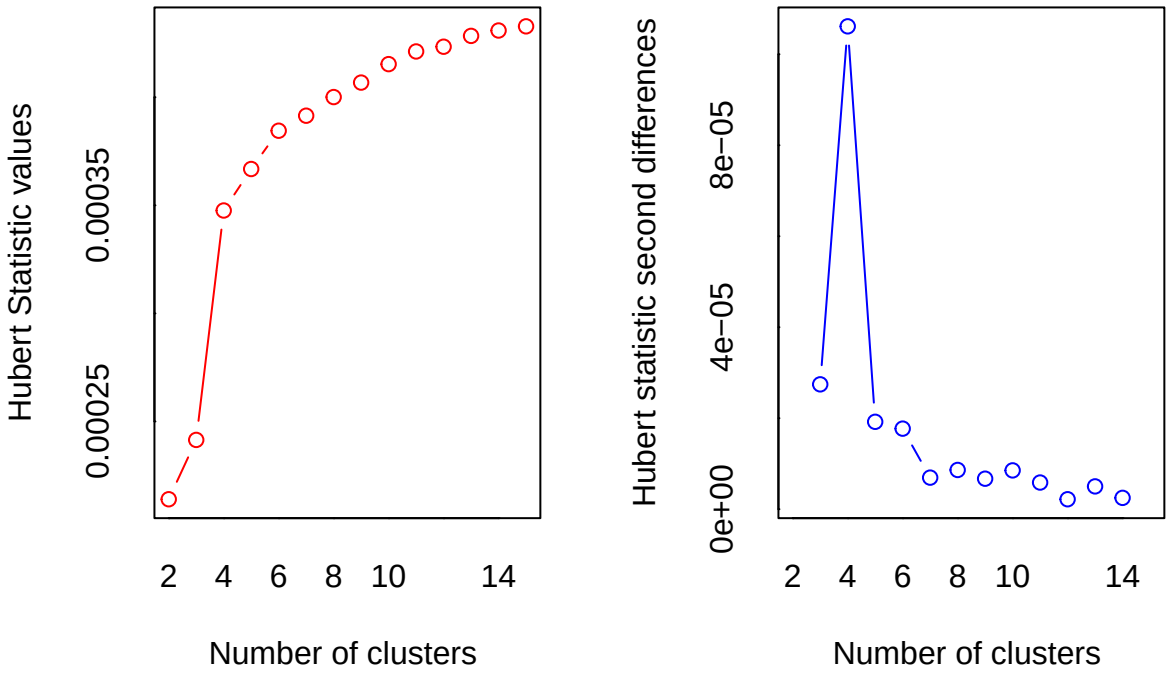
There are two different types:

- **K-means**: Starts from random value, but evaluates a mean center and then aggregate the clusters that are closest to it. It computational favorable respect to hierarchical clustering, but it is high sensitive to outlayers. It can be used only with numerical value.
- **K-medoids**: Similar to K-means but the center now is defined by a unit. It less sensitive to outlayers and can be used also in case of categorical or logical value.

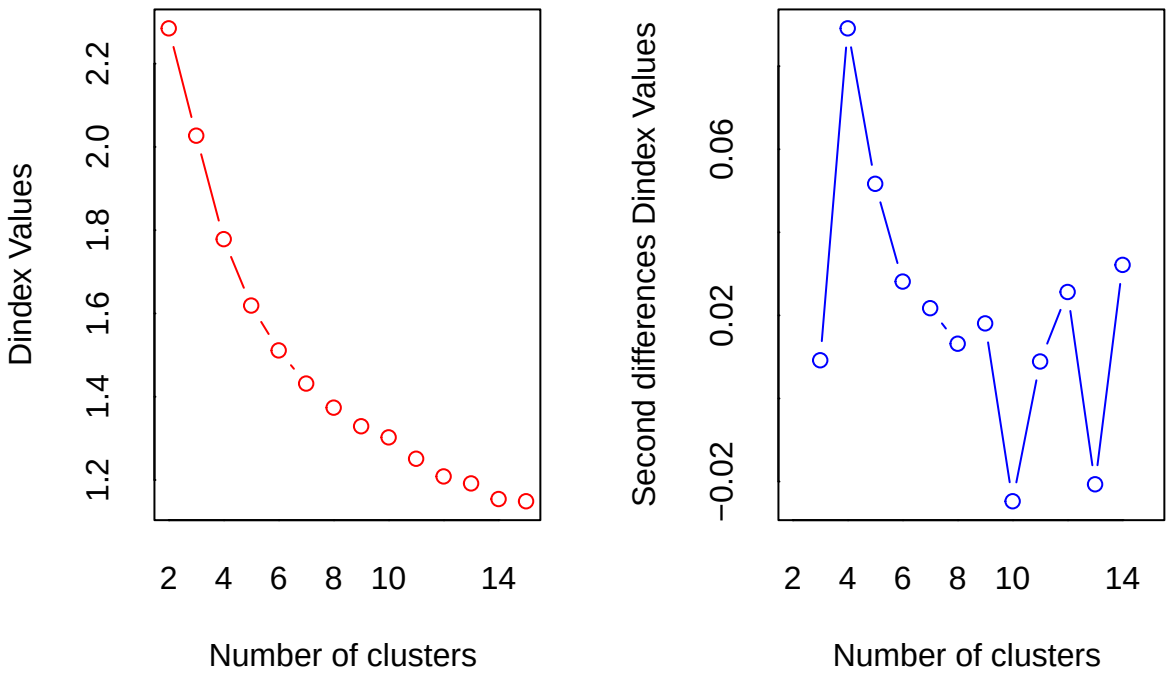
The structure of the analysis is as follow: 1. Choosing the number of clusters using the WSS or average silhouette width or gap statistics or other statistics. 2. Doing the k algorithm. 3. Computing the internal validation through the analysis of the silhouette width, connectivity and Dunn index. 4. Computing the external validation of the data through Rand index and Meila's index. The first measures the agreement and the second the disagreement with the Classes. 5. Doing the two plots of the data in the PCA space or on the pair plot. ### K-means

First of all we need to evaluate the number of optimal cluster. In this case we will use a method that evaluate the optimal number using 24 different methods.

```
nb <- NbClust(data = std.Beans, diss = Beans.eu,
              distance = NULL, min.nc = 2,
              max.nc = 15, method = "kmeans")
```




```
## *** : The Hubert index is a graphical method of determining the number of clusters.
##                 In the plot of Hubert index, we seek a significant knee that correspo
##                 significant increase of the value of the measure i.e the significant
##                 index second differences plot.
##
```




```
## *** : The D index is a graphical method of determining the number of clusters.
##                 In the plot of D index, we seek a significant knee (the significant p
##                 second differences plot) that corresponds to a significant increase o
##                 the measure.
##
## *******************************************************************
## * Among all indices:
## * 8 proposed 2 as the best number of clusters
## * 3 proposed 3 as the best number of clusters
## * 7 proposed 4 as the best number of clusters
## * 3 proposed 5 as the best number of clusters
## * 1 proposed 12 as the best number of clusters
## * 1 proposed 13 as the best number of clusters
## * 1 proposed 15 as the best number of clusters
##
##                   ***** Conclusion *****
##
## * According to the majority rule, the best number of clusters is  2
##
##
## *******************************************************************
```

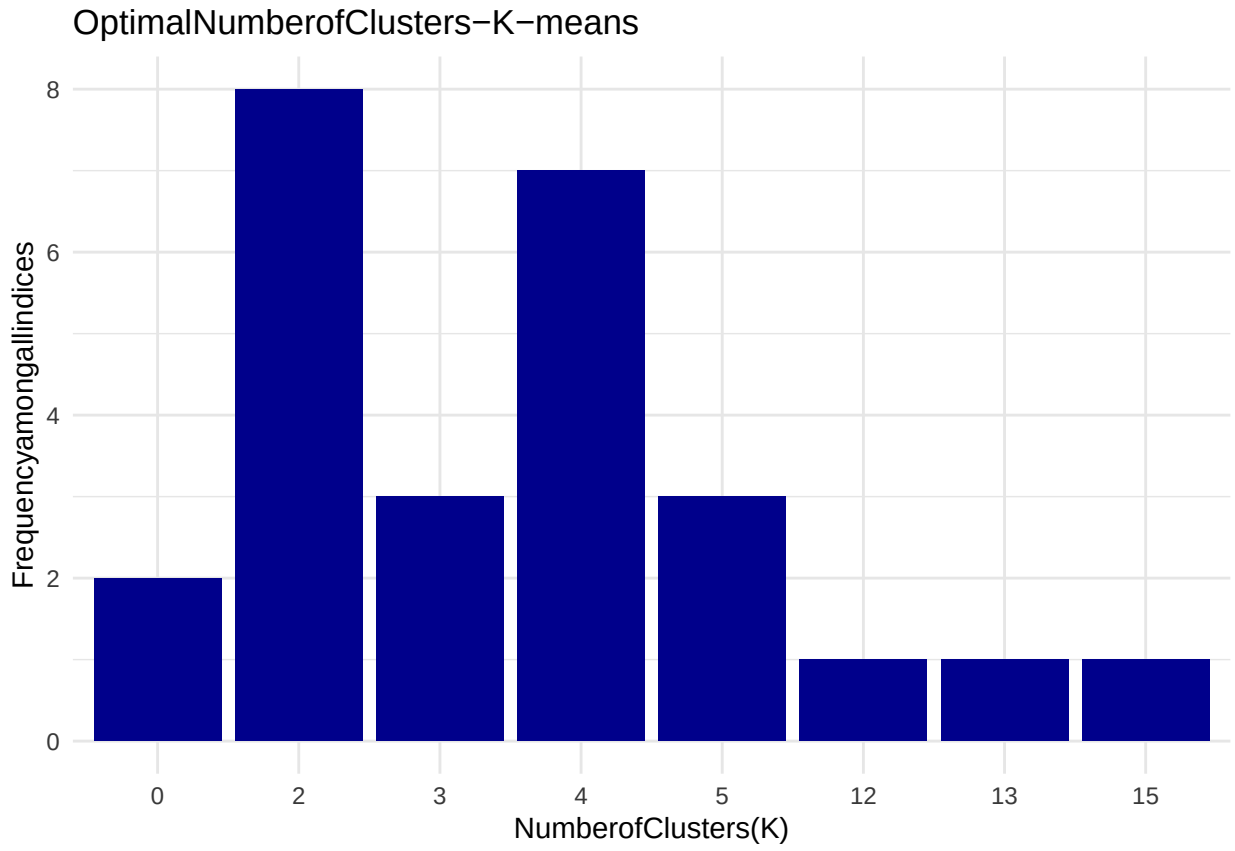
```r
# Creating a dataframe
selected_clusters<-as.data.frame(table(nb$Best.nc[1,]))

# renaming all the rows and the columns

colnames(selected_clusters)<-c("NumberofClusters","Frequency")

# Creating the plot

ggplot(selected_clusters, aes(x= `NumberofClusters`
,y=Frequency)) +
geom_bar(stat="identity",fill= "darkblue")+
theme_minimal()+
labs(title="OptimalNumberofClusters-K-means",
x="NumberofClusters(K)",
y="Frequencyamongallindices")
```




```r
km.res <- kmeans(std.Beans,2,nstart = 50)
km.res
```

```
## K-means clustering with 2 clusters of sizes 422, 250
##
```

```
## Cluster means:
##         Area  Perimeter MajorAxisLength MinorAxisLength AspectRation
## 1 -0.4837659 -0.5827091      -0.6198309      -0.4002003   -0.5137485
## 2  0.8165968  0.9836130       1.0462745       0.6755380    0.8672075
##   Eccentricity ConvexArea EquivDiameter     Extent   Solidity  roundness
## 1   -0.4384282 -0.4860460    -0.5402858  0.1520089  0.3181190  0.5733236
## 2    0.7400668  0.8204457     0.9120024 -0.2565910 -0.5369848 -0.9677702
##   Compactness
## 1   0.4996786
## 2  -0.8434575
##
## Clustering vector:
##   [1] 2 1 1 1 2 1 2 2 2 1 1 2 1 1 1 2 2 1 1 2 2 2 1 2 1 1 1 1 1 1 1 2 1 2 1 2 1
##  [38] 1 2 2 1 2 1 1 1 1 1 1 1 1 1 1 2 1 1 2 2 2 1 2 1 1 1 1 1 2 1 1 1 1 1 1 2 1 1
##  [75] 2 2 2 2 1 1 1 1 2 1 1 1 1 2 1 1 1 1 1 2 1 1 2 2 2 1 1 1 2 1 2 2 2 1 2 1 1
## [112] 1 2 2 2 1 1 2 2 2 1 1 1 2 1 1 2 1 1 1 1 2 1 1 1 1 2 2 1 2 1 2 1 1 2 1 1 2
## [149] 1 1 2 1 2 1 1 1 1 2 1 2 2 1 2 1 1 1 2 1 2 1 2 1 2 2 2 2 1 1 1 2 1 1 1 2 2
## [186] 2 2 2 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 1 1 1 1 1 2 1 1 1 1 1 1 2 1 1 1 1 1 1
## [223] 1 1 2 1 2 1 2 2 1 1 2 1 1 1 1 1 1 1 2 2 2 1 1 1 1 2 1 1 2 1 2 1 2 2 1 1 1
## [260] 1 1 1 2 2 1 2 1 2 1 2 1 2 2 1 1 1 1 1 1 2 1 1 1 1 1 1 2 2 2 1 1 2 2 2 2 1
## [297] 2 2 2 1 1 2 1 1 1 2 1 1 2 1 1 2 2 2 2 1 2 2 1 1 2 1 2 1 1 1 1 2 2 1 1 2 1
## [334] 2 2 2 1 1 1 1 1 1 1 2 1 2 2 1 2 2 2 1 1 1 2 1 2 1 1 1 2 1 2 1 1 1 1 1 1 1
## [371] 1 2 1 1 1 2 2 1 2 1 2 2 1 1 2 1 2 1 2 1 2 2 1 1 1 2 1 1 1 2 2 2 1 2 1 1 2
## [408] 1 1 1 1 2 2 1 1 2 1 1 2 1 1 1 1 1 2 2 2 1 1 1 1 1 2 1 1 1 1 2 1 2 1 2 1 1
## [445] 1 1 1 1 1 2 1 1 2 1 1 1 1 1 2 2 2 2 1 2 1 1 1 1 2 1 1 1 1 1 2 2 1 1 1 2 1
## [482] 1 2 1 1 1 1 1 2 2 2 2 1 1 1 2 1 2 1 1 1 1 2 2 1 2 2 2 1 1 1 1 2 1 1 1 1 1
## [519] 1 1 2 1 1 1 1 1 1 2 1 2 2 1 1 2 1 1 2 1 2 1 1 1 1 1 1 1 1 1 1 1 1 2 1 2
## [556] 1 1 1 1 1 2 1 2 1 2 1 2 2 1 1 2 2 1 1 1 1 2 1 2 1 2 1 1 1 2 1 2 2 2 2 1 2
## [593] 1 1 2 2 2 1 2 1 2 1 1 1 1 2 2 2 2 2 1 1 2 1 2 2 2 2 1 1 2 1 2 2 2 1 1 1 2
## [630] 2 2 1 1 2 1 1 2 1 1 1 2 1 2 1 1 1 2 1 2 1 1 1 2 1 1 1 1 1 1 1 1 1 2 2 2 1
## [667] 1 2 2 1 2 1
##
## Within cluster sum of squares by cluster:
## [1] 1608.066 3262.211
##  (between_SS / total_SS =  39.5 %)
##
## Available components:
##
## [1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
## [6] "betweenss"    "size"         "iter"         "ifault"
```

As we can see only two clusters can't explain well the data. This can be seen on the value of WSS that are really high and on the value of BSS/TSS that is only the 40 %. We can try the second better number of clusters.

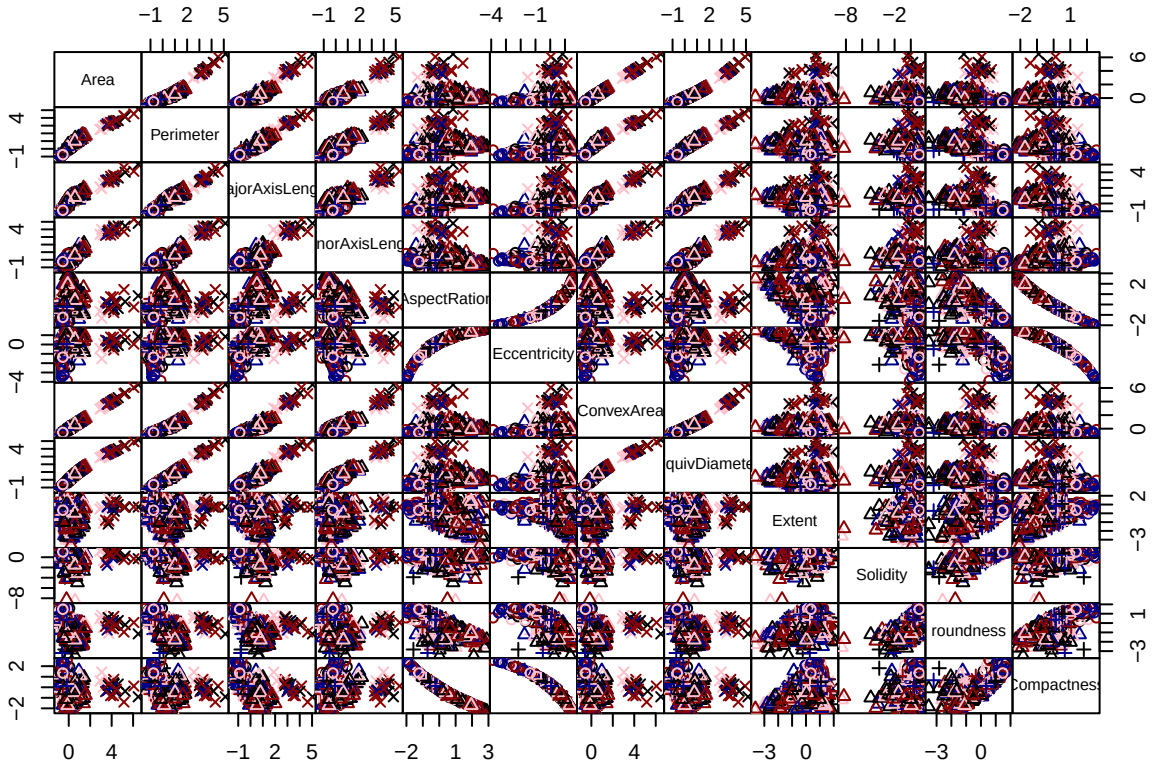
```r
km.res <- kmeans(std.Beans,4,nstart = 50)
km.res
```

```
## K-means clustering with 4 clusters of sizes 97, 225, 325, 25
##
## Cluster means:
##         Area  Perimeter MajorAxisLength MinorAxisLength AspectRation
## 1 -0.4766798 -0.6454764      -0.8653212     -0.03140542  -1.45001261
## 2  0.4429777  0.7072712       0.8030496      0.31313244   0.96748271
## 3 -0.4840826 -0.5631475      -0.5441344     -0.50891243  -0.23217181
## 4  4.1557922  3.4599251       3.2037481      3.91952264  -0.06306197
##   Eccentricity ConvexArea EquivDiameter      Extent   Solidity  roundness
## 1 -1.918808348 -0.4810825    -0.5241069  0.52450284  0.6589100  1.2789296
## 2  0.805010261  0.4486653     0.5979730 -0.34753697 -0.6396259 -1.0719652
## 3  0.006170949 -0.4858202    -0.5430130  0.04037595  0.2266558  0.3703314
## 4  0.119661703  4.1442757     3.7109474  0.56787431  0.2535366 -0.1288676
##   Compactness
## 1  1.66232165
## 2 -0.93053867
## 3  0.15022269
## 4 -0.02785492
##
## Clustering vector:
##   [1] 2 3 3 3 4 3 2 2 2 1 3 2 3 3 3 2 4 3 3 2 2 2 3 2 3 3 3 3 3 3 2 3 4 3 4 3
##  [38] 3 2 2 1 3 1 3 3 3 3 3 3 3 1 2 1 3 2 2 4 3 2 3 3 3 3 3 2 3 3 1 1 3 1 2 1 3
##  [75] 2 2 2 2 3 3 3 1 2 3 1 3 3 2 3 1 3 3 3 2 3 3 2 2 4 3 3 3 2 3 2 2 2 3 2 3 3
## [112] 3 2 2 2 3 3 2 2 2 3 3 3 2 3 3 2 3 3 3 3 2 3 3 1 1 2 2 1 2 1 2 3 3 2 3 3 2
## [149] 3 3 2 3 2 3 3 1 3 2 3 2 2 3 2 3 1 3 2 1 4 1 2 1 2 2 2 2 3 3 1 2 3 3 3 2 2
## [186] 2 4 2 3 3 3 3 3 1 3 3 3 3 3 2 2 2 2 3 3 3 3 3 2 3 3 3 1 3 3 2 3 3 1 1 1 3
## [223] 3 3 2 3 2 3 2 2 3 3 4 1 3 3 3 1 3 3 2 2 2 3 3 3 1 2 3 3 2 3 2 1 4 2 1 1 3
## [260] 1 3 3 2 2 1 2 3 2 3 2 3 2 4 3 1 3 3 1 3 4 1 3 3 3 3 1 2 2 2 1 1 4 2 2 2 3
## [297] 2 2 2 1 1 2 1 3 3 2 3 3 2 3 1 2 4 2 4 1 2 2 3 3 2 3 2 3 1 1 3 4 2 3 3 4 3
## [334] 2 2 2 3 3 3 3 3 3 3 2 3 2 2 3 2 2 2 3 3 1 2 3 2 3 1 1 2 3 2 3 3 3 3 3 1 3
## [371] 1 2 3 3 3 2 2 3 2 3 2 2 3 3 2 3 2 3 2 1 2 2 1 3 3 2 3 3 1 2 2 2 3 2 1 3 2
## [408] 3 3 3 3 2 2 3 1 2 3 1 2 1 1 3 3 3 2 2 2 3 3 3 3 3 4 1 3 3 1 2 3 2 2 4 3 3
## [445] 1 3 3 3 3 2 3 3 2 3 3 3 3 3 2 2 2 2 1 2 3 3 3 1 4 3 3 1 3 1 2 2 1 3 3 2 1
## [482] 3 4 3 3 3 1 1 2 2 2 2 3 1 3 2 3 2 1 3 3 3 2 2 3 4 2 2 1 3 3 3 2 3 3 3 3 3
## [519] 3 1 2 3 3 3 1 3 3 2 1 2 2 3 3 2 3 3 2 3 2 1 3 3 3 3 3 3 3 3 1 3 1 3 2 3 2
## [556] 3 3 3 3 1 2 3 2 3 2 1 2 2 3 3 2 2 3 3 1 3 2 3 2 3 2 3 1 1 2 3 4 2 2 2 3 2
## [593] 3 3 2 2 2 3 2 3 2 3 3 3 3 2 2 2 2 2 3 3 2 1 2 2 2 2 3 3 2 1 4 2 2 3 3 3 2
## [630] 2 4 3 3 2 3 3 2 3 3 1 2 1 2 3 1 1 2 3 2 3 1 3 2 3 3 3 3 3 1 3 3 1 2 2 2 1
## [667] 3 2 2 3 2 1
##
## Within cluster sum of squares by cluster:
```

```
## [1]  135.3016 1604.1658  786.6992  131.5906
##  (between_SS / total_SS =  67.0 %)
##
## Available components:
##
## [1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
## [6] "betweenss"    "size"         "iter"         "ifault"
```

This give us a better representation of the data. Let us see the representation on the biplot.

```
cl <- km.res$cluster
pairs(std.Beans, gap = 0, pch = cl ,col =c("black","darkblue","darkred","pink") )
```



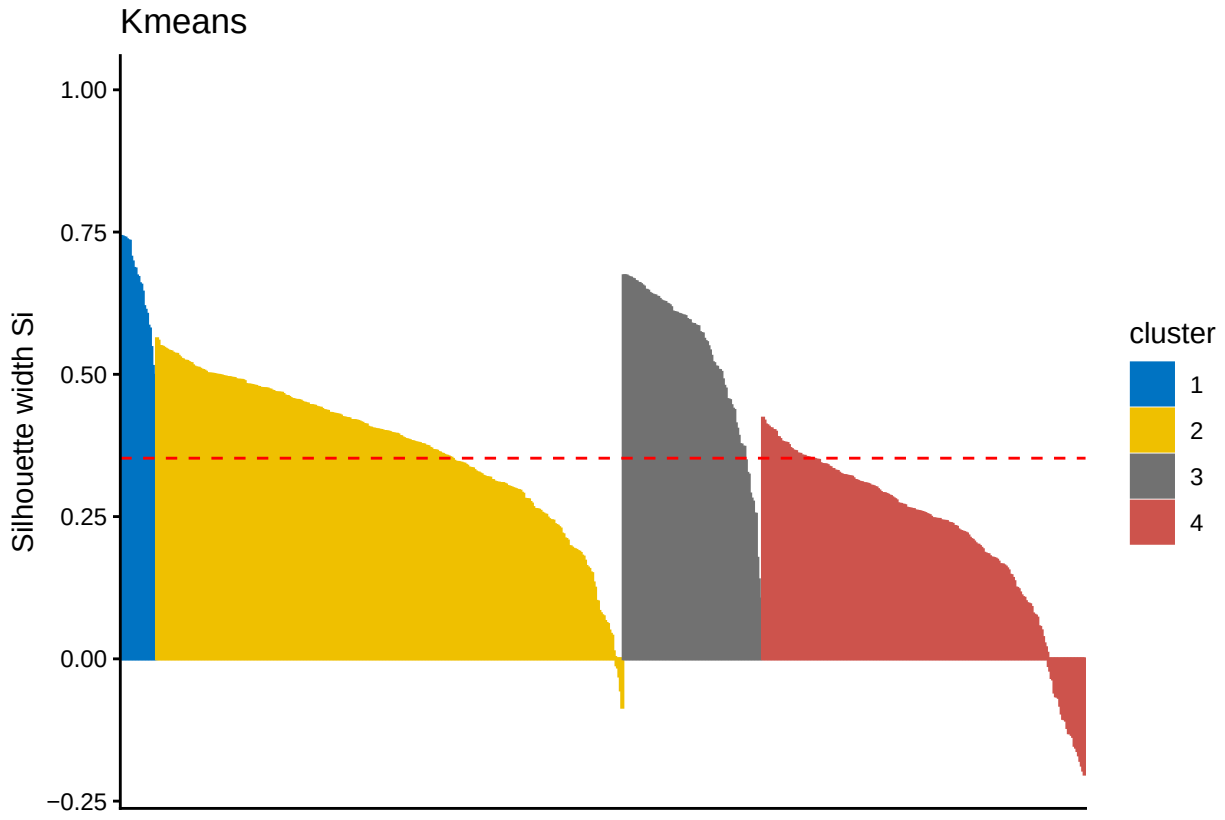
We can now do the internal clustering validation.

```
Beans.km<-eclust(std.Beans, FUNcluster = "kmeans",k = 4, graph = FALSE)


# Silhouette width
fviz_silhouette(Beans.km, ggtheme = theme_classic(), palette = "jco",
                main = "Kmeans")
```

```
##   cluster size ave.sil.width
## 1       1   25          0.65
```

```
## 2       2  325          0.37
## 3       3   97          0.53
## 4       4  225          0.22
```




```
sil <- Beans.km$silinfo$widths[,1:3]

clusters <- unique(sil["cluster"])
for (i in clusters){
  neg.sil <- which(sil["sil_width"]<0 & sil["cluster"] == i)
  cat("Number of misassigned units for cluster",i,": ",
      nrow(sil[neg.sil, , drop = FALSE]),"\n")
}
```

```
## Number of misassigned units for cluster 1 :  0
## Number of misassigned units for cluster 2 :  5
## Number of misassigned units for cluster 3 :  0
## Number of misassigned units for cluster 4 :  26
```

```
val.km <- clValid(std.Beans, nClust = 2:7, clMethods = "kmeans",
                  validation ="internal", maxitems = 1000)
summary(val.km)
```

```
##
```

```
## Clustering Methods:
##  kmeans
##
## Cluster sizes:
##  2 3 4 5 6 7
##
## Validation Measures:
##                              2        3        4        5        6        7
##
## kmeans Connectivity    0.0000  56.8746  77.9151 104.6690 111.1488 140.1448
##        Dunn            0.2633   0.0363   0.0363   0.0424   0.0424   0.0523
##        Silhouette      0.6157   0.4210   0.3866   0.3764   0.3701   0.3581
##
## Optimal Scores:
##
##              Score  Method Clusters
## Connectivity 0.0000 kmeans 2
## Dunn         0.2633 kmeans 2
## Silhouette   0.6157 kmeans 2
```

We can now perform the external validation.

```
table(Beans$Class, Beans.km$cluster)
```

```
##
##              1   2   3   4
##   BARBUNYA   0   1   0  46
##   BOMBAY    25   0   0   0
##   CALI       0   1   0  66
##   DERMASON   0 174  13   1
##   HOROZ      0   4   0  93
##   SEKER      0  11  84   1
##   SIRA       0 134   0  18
```

```
clust_stats <- cluster.stats(d = Beans.eu,clustering = classes,
                             alt.clustering = Beans.km$cluster )

cat("Rand index: ", clust_stats$corrected.rand)
```

```
## Rand index:  0.4285207
```

```
cat("Meila's VI index: ", clust_stats$vi)
```

```
## Meila's VI index:  1.189763
```

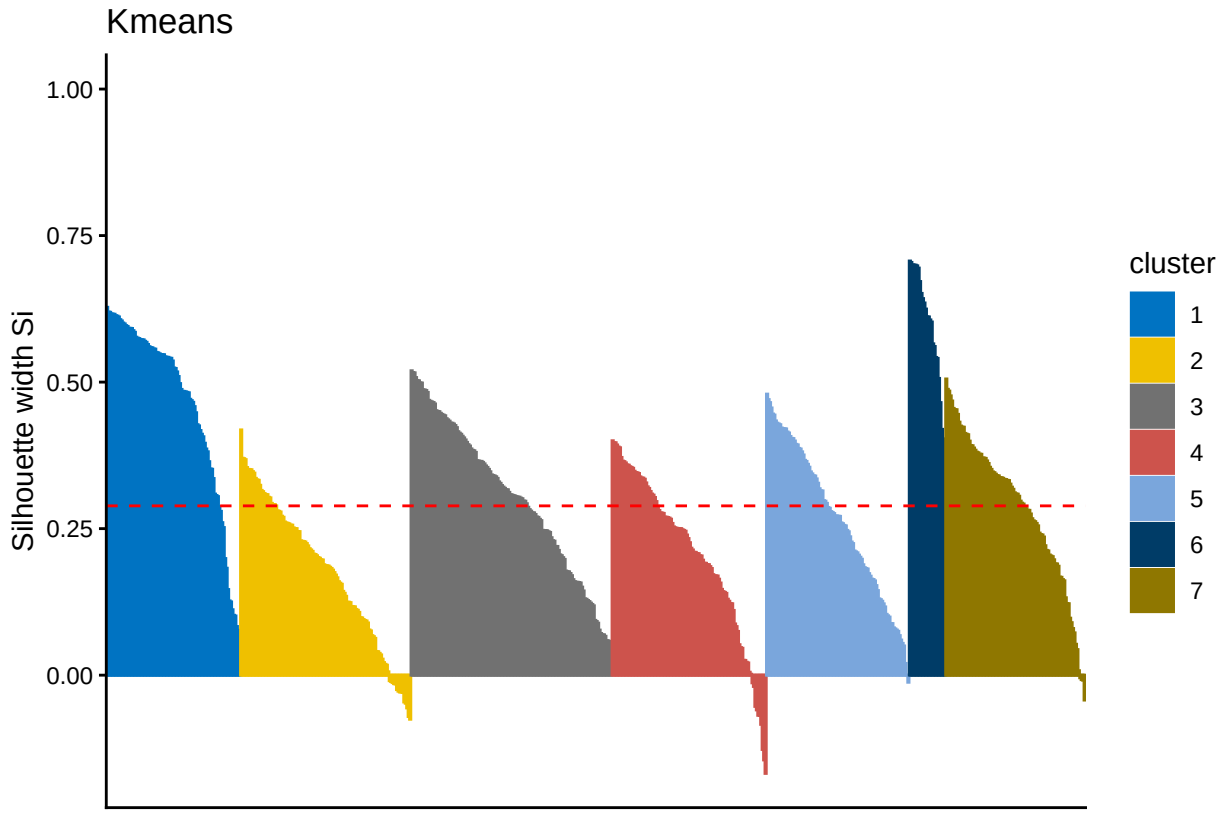
The Rand index indicates a middle agreement. The Meila's index indicates an high disagreement.

We can try to force the system to have 7 clsuters.

```
Beans.km7<-eclust(std.Beans, FUNcluster = "kmeans",k = 7, graph = FALSE)
```

```
# Silhouette width
fviz_silhouette(Beans.km7, ggtheme = theme_classic(), palette = "jco",
                main = "Kmeans")
```

```
##   cluster size ave.sil.width
## 1       1   92          0.46
## 2       2  117          0.17
## 3       3  138          0.30
## 4       4  106          0.20
## 5       5   98          0.26
## 6       6   25          0.60
## 7       7   96          0.29
```




```
sil <- Beans.km7$silinfo$widths[,1:3]

clusters <- unique(sil[,"cluster"])
for (i in clusters){
  neg.sil <- which(sil[,"sil_width"]<0 & sil[,"cluster"] == i)
  cat("Number of misassigned units for cluster",i,": ",
```

```
      nrow(sil[neg.sil, , drop = FALSE]),"\n")
}
```

```
## Number of misassigned units for cluster 1 :  0
## Number of misassigned units for cluster 2 :  15
## Number of misassigned units for cluster 3 :  0
## Number of misassigned units for cluster 4 :  10
## Number of misassigned units for cluster 5 :  2
## Number of misassigned units for cluster 6 :  0
## Number of misassigned units for cluster 7 :  4
```

```
table(Beans$Class, Beans.km7$cluster)
```

```
##
##              1   2   3   4   5   6   7
##   BARBUNYA   0   2   0  44   1   0   0
##   BOMBAY     0   0   0   0   0  25   0
##   CALI       0   0   0  58   5   0   4
##   DERMASON   8  44 122   0  14   0   0
##   HOROZ      0   3   0   2   5   0  87
##   SEKER     84   4   4   1   3   0   0
##   SIRA       0  64  12   1  70   0   5
```

```
clust_stats <- cluster.stats(d = Beans.eu,clustering = classes,
                             alt.clustering = Beans.km7$cluster )

cat("Rand index: ", clust_stats$corrected.rand)
```

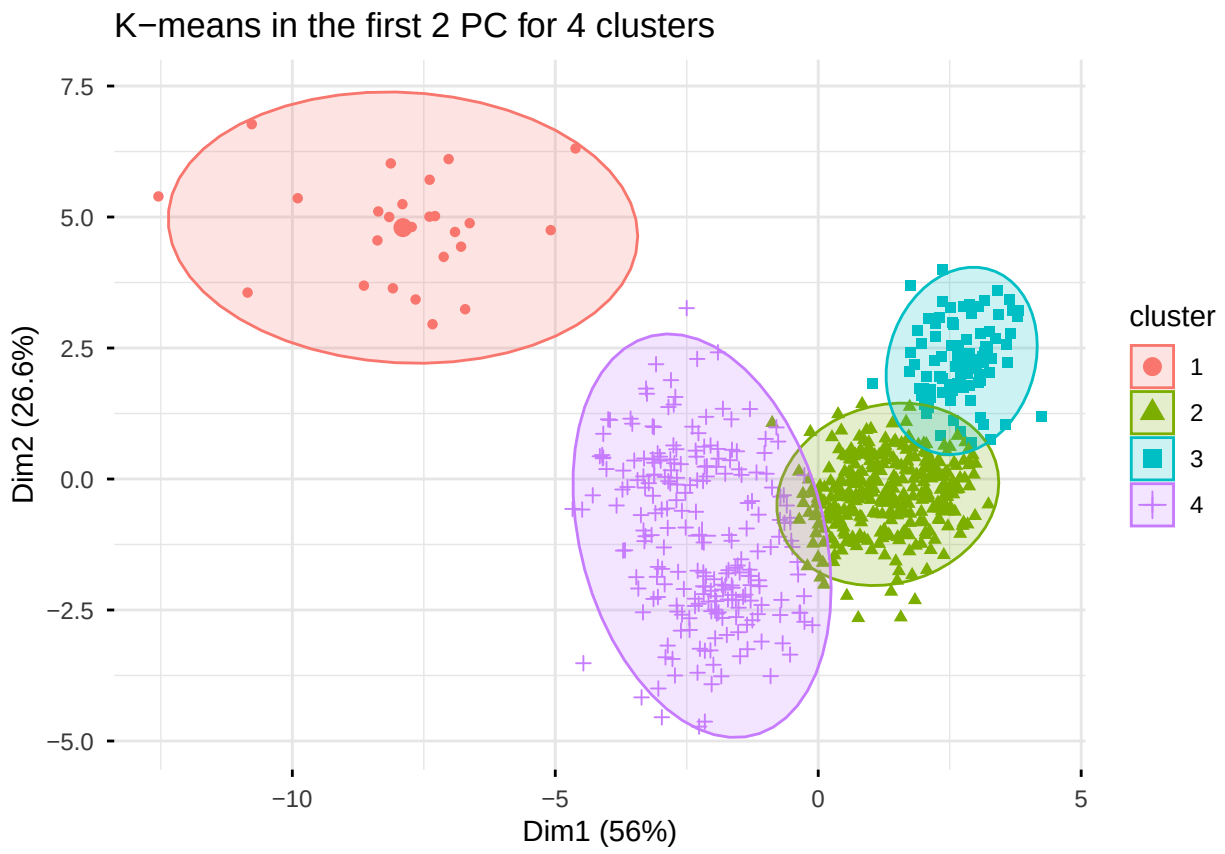
```
## Rand index:  0.5141156
```

```
cat("Meila's VI index: ", clust_stats$vi)
```

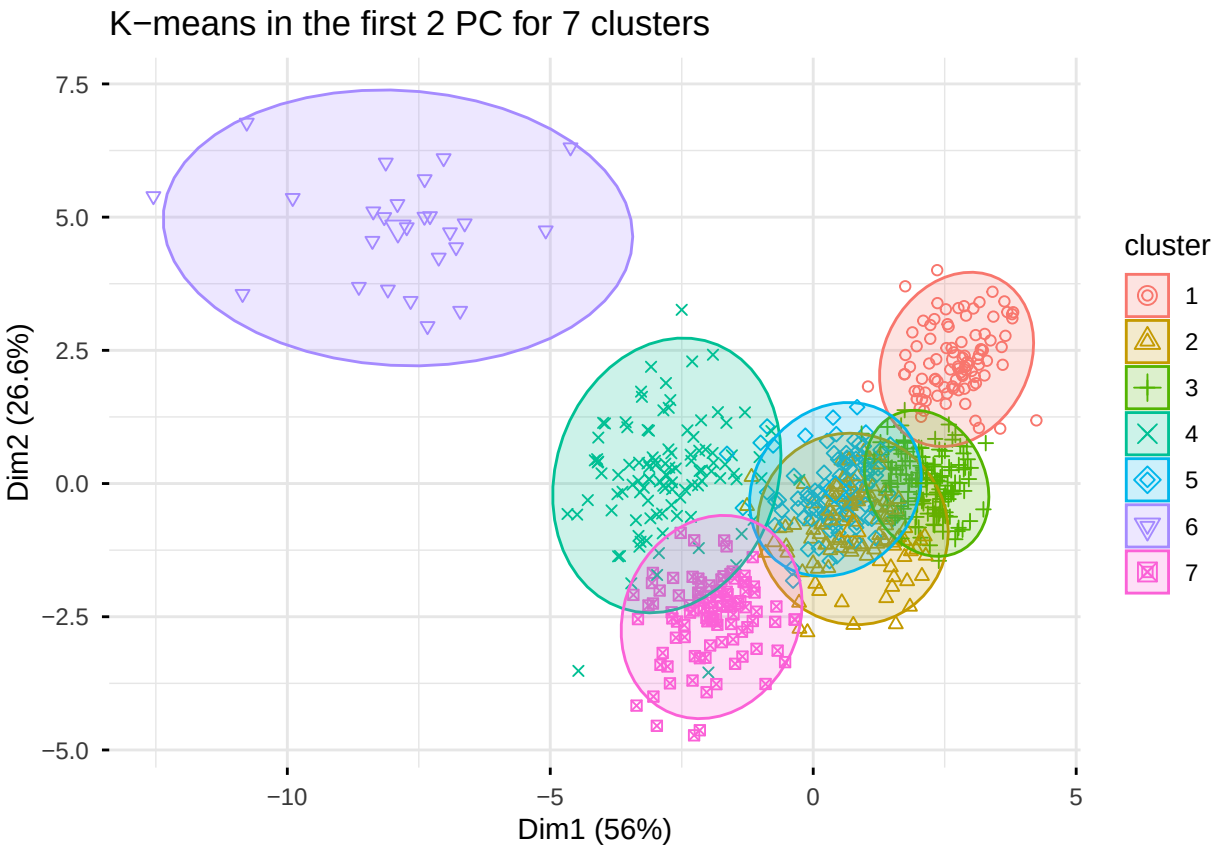
```
## Meila's VI index:  1.343421
```

The Rand index indicates a middle agreement. The Meila's index indicates an high disagreement. We can project all in the two frist PC.

```
fviz_cluster(Beans.km,data = std.Beans, geom = "point", ellipse.type = "norm",
             ggtheme = theme_minimal(),
             main = "K-means in the first 2 PC for 4 clusters")
```



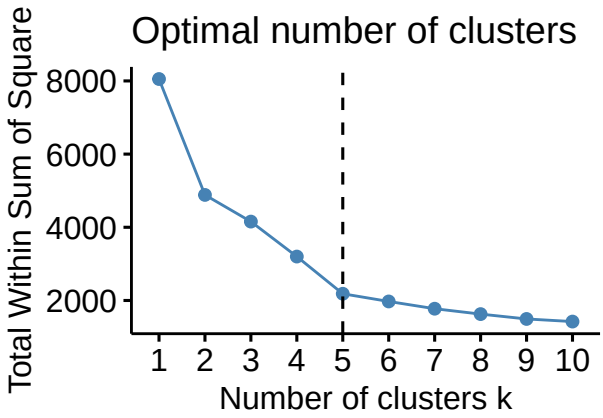

```
fviz_cluster(Beans.km7,data = std.Beans, geom = "point", ellipse.type = "norm",
             ggtheme = theme_minimal(),
             main = "K-means in the first 2 PC for 7 clusters")
```

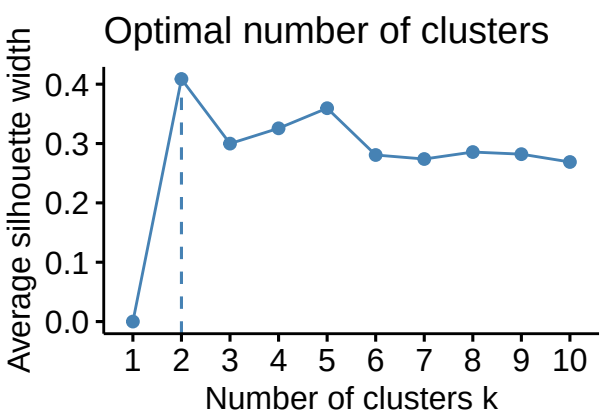


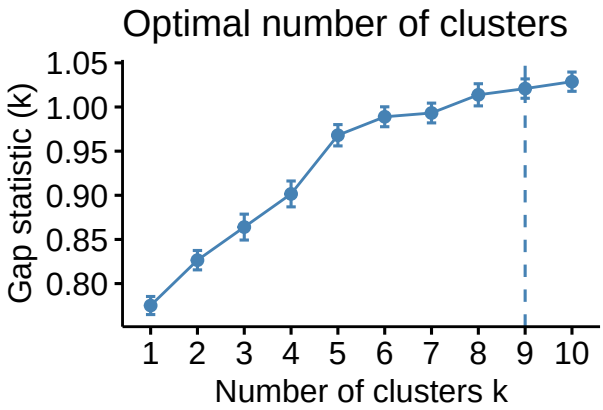
6.3.1 K-medoids

Now we compute the K-medoids. First of all we need the number of clusters usign the euclidean distance.

```
p1 <- fviz_nbclust(std.Beans, pam, method = "wss", metric = "euclidean" ) +
  geom_vline(xintercept = 5, linetype = 2)
p2 <- fviz_nbclust(std.Beans, pam, method = "silhouette", metric = "euclidean")
p3 <- fviz_nbclust(std.Beans, pam, method = "gap_stat", metric = "euclidean"
            , nboot = 50)
grid.arrange(p1, p2, p3, ncol = 2, nrow = 2)
```

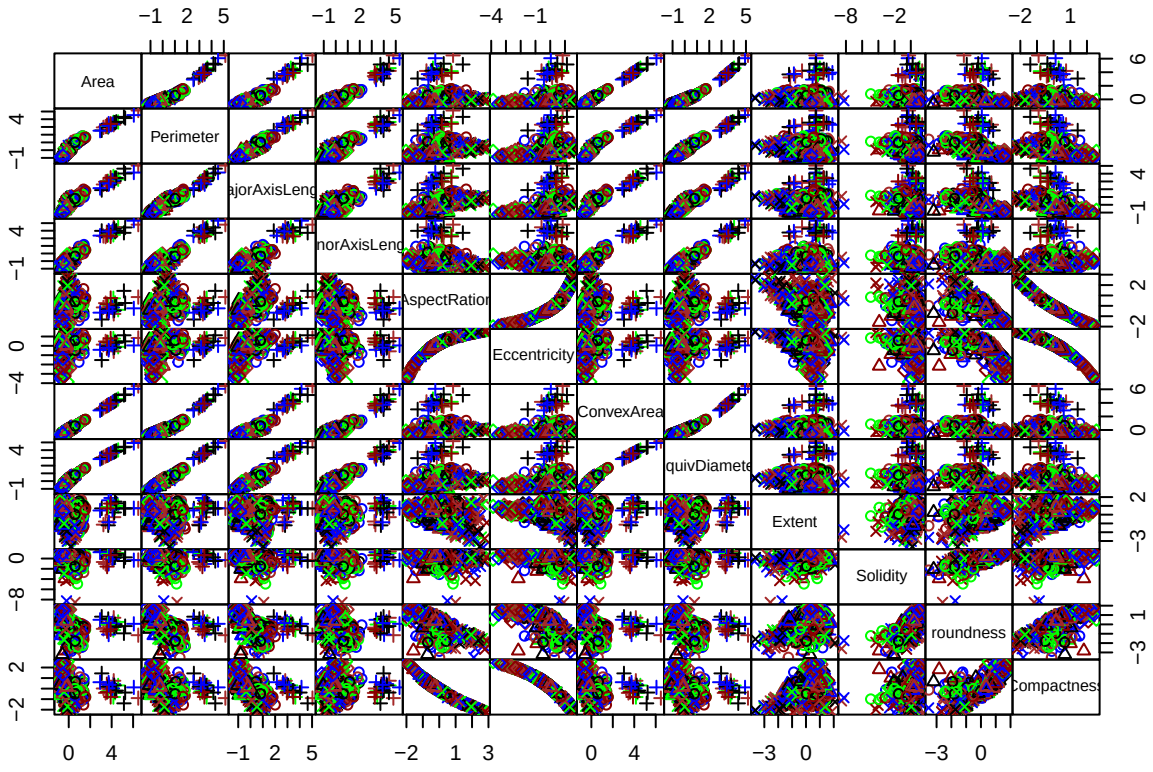






All the methods give us a different result. So we will try with 5 cluster, because this is not a so high number like 9 or a low number like 2.

```
pam.res <- pam(std.Beans,5,metric="euclidean")
cl <- pam.res$clustering
pairs(std.Beans,gap = 0, pch = cl, col=c("darkred","blue","black","green","brown"))
```

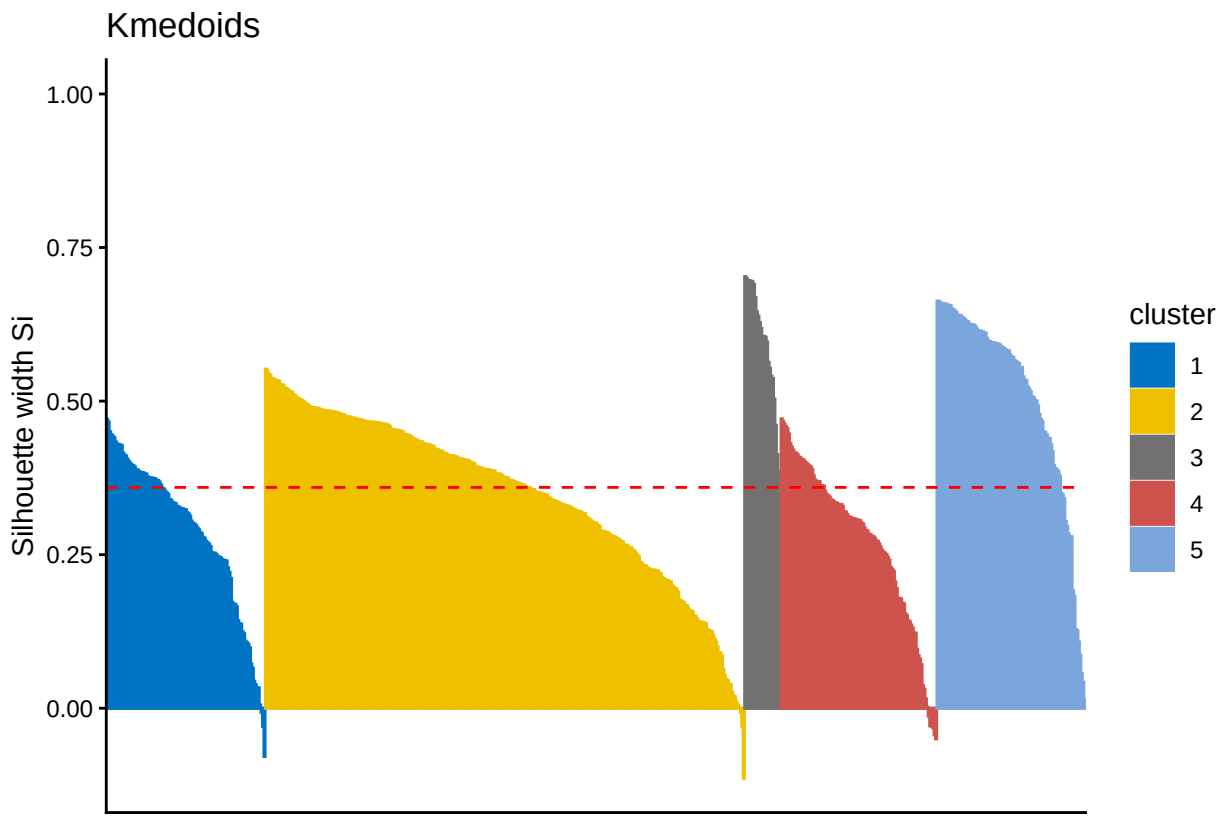


We can now do the internal clustering validation.

```
Beans.kmd<-eclust(std.Beans, FUNcluster = "pam",k = 5, graph = FALSE)


# Silhouette width
fviz_silhouette(Beans.kmd, ggtheme = theme_classic(), palette = "jco",
                main = "Kmedoids")
```

```
##   cluster size ave.sil.width
## 1       1  109          0.29
## 2       2  329          0.35
## 3       3   25          0.59
## 4       4  107          0.27
## 5       5  102          0.51
```




```
sil <- Beans.kmd$silinfo$widths[,1:3]

clusters <- unique(sil[,"cluster"])
for (i in clusters){
  neg.sil <- which(sil[,"sil_width"]<0 & sil[,"cluster"] == i)
  cat("Number of misassigned units for cluster",i,": ",
      nrow(sil[neg.sil, , drop = FALSE]),"\n")
}
```

```
## Number of misassigned units for cluster 1 :  3
## Number of misassigned units for cluster 2 :  4
## Number of misassigned units for cluster 3 :  0
## Number of misassigned units for cluster 4 :  6
## Number of misassigned units for cluster 5 :  0
```

```
val.PCA.kmd <- clValid(std.Beans, nClust = 2:7, clMethods = "pam",
                       validation ="internal",
                       maxitems = 1000)
summary(val.PCA.kmd)
```

```
##
## Clustering Methods:
##  pam
```

```
##
## Cluster sizes:
##  2 3 4 5 6 7
##
## Validation Measures:
##                            2        3        4        5        6        7
##
## pam Connectivity   51.8310  81.4496  91.3433  91.3433 171.9905 200.9583
##     Dunn            0.0245   0.0253   0.0320   0.0406   0.0237   0.0272
##     Silhouette      0.4087   0.2999   0.3256   0.3595   0.2807   0.2739
##
## Optimal Scores:
##
##              Score   Method Clusters
## Connectivity 51.8310 pam    2
## Dunn          0.0406 pam    5
## Silhouette    0.4087 pam    2
```

We can now perform the external validation.

```
table(Beans$Class, Beans.kmd$cluster)
```

```
##
##              1   2   3   4   5
##   BARBUNYA  44   1   0   2   0
##   BOMBAY     0   0  25   0   0
##   CALI      63   0   0   4   0
##   DERMASON   0 169   0   1  18
##   HOROZ      0   4   0  93   0
##   SEKER      0  12   0   0  84
##   SIRA       2 143   0   7   0
```

```
clust_stats <- cluster.stats(d = Beans.eu,clustering = classes,
                             alt.clustering = Beans.kmd$cluster )

cat("Rand index: ", clust_stats$corrected.rand)
```

```
## Rand index:  0.5178392
```

```
cat("Meila's VI index: ", clust_stats$vi)
```

```
## Meila's VI index:  0.9944241
```

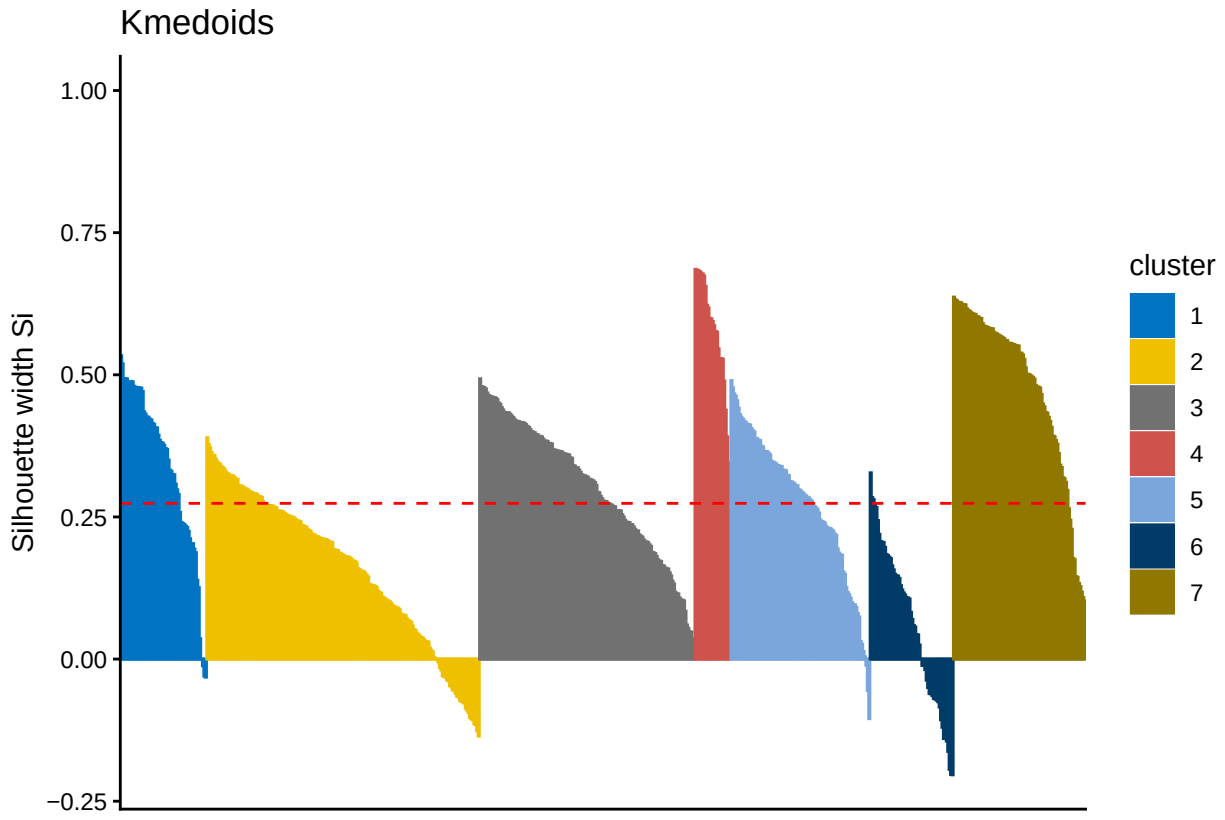
The Rand index indicates a middle agreement. The Meila's index indicates an high disagreement (but the est encountered until now).

We can try to force the system to have 7 clusters.

```
Beans.kmd7<-eclust(std.Beans, FUNcluster = "pam",k = 7, graph = FALSE)
```

```
# Silhouette width
fviz_silhouette(Beans.kmd7, ggtheme = theme_classic(), palette = "jco",
                main = "Kmedoids")
```

```
##   cluster size ave.sil.width
## 1       1   60          0.34
## 2       2  190          0.16
## 3       3  150          0.30
## 4       4   25          0.57
## 5       5   97          0.27
## 6       6   58          0.06
## 7       7   92          0.48
```




```
sil <- Beans.kmd7$silinfo$widths[,1:3]

clusters <- unique(sil["cluster"])
for (i in clusters){
  neg.sil <- which(sil["sil_width"]<0 & sil["cluster"] == i)
  cat("Number of misassigned units for cluster",i,": ",
      nrow(sil[neg.sil, , drop = FALSE]),"\n")
```

```
}
```

```
## Number of misassigned units for cluster 1 :  4
## Number of misassigned units for cluster 2 :  30
## Number of misassigned units for cluster 3 :  0
## Number of misassigned units for cluster 4 :  0
## Number of misassigned units for cluster 5 :  3
## Number of misassigned units for cluster 6 :  22
## Number of misassigned units for cluster 7 :  0
```

```
table(Beans$Class, Beans.kmd7$cluster)
```

```
##
##               1   2   3   4   5   6   7
##   BARBUNYA   10   1   0   0   0  36   0
##   BOMBAY      0   0   0  25   0   0   0
##   CALI       50   1   0   0   3  13   0
##   DERMASON    0  44 135   0   0   1   8
##   HOROZ       0   5   0   0  90   2   0
##   SEKER       0   6   4   0   0   2  84
##   SIRA        0 133  11   0   4   4   0
```

```
clust_stats <- cluster.stats(d = Beans.eu,clustering = classes,
                             alt.clustering = Beans.kmd7$cluster )

cat("Rand index: ", clust_stats$corrected.rand)
```

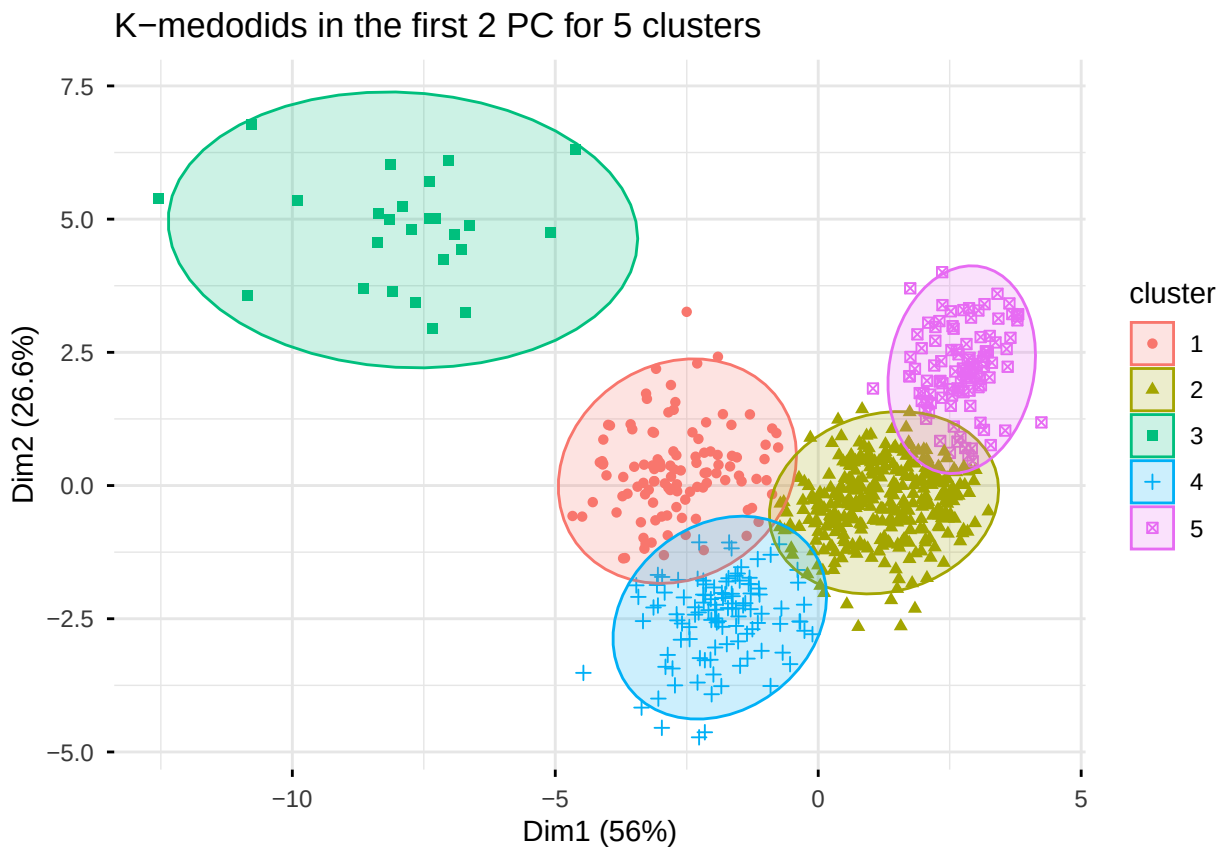
```
## Rand index:  0.6148422
```

```
cat("Meila's VI index: ", clust_stats$vi)
```

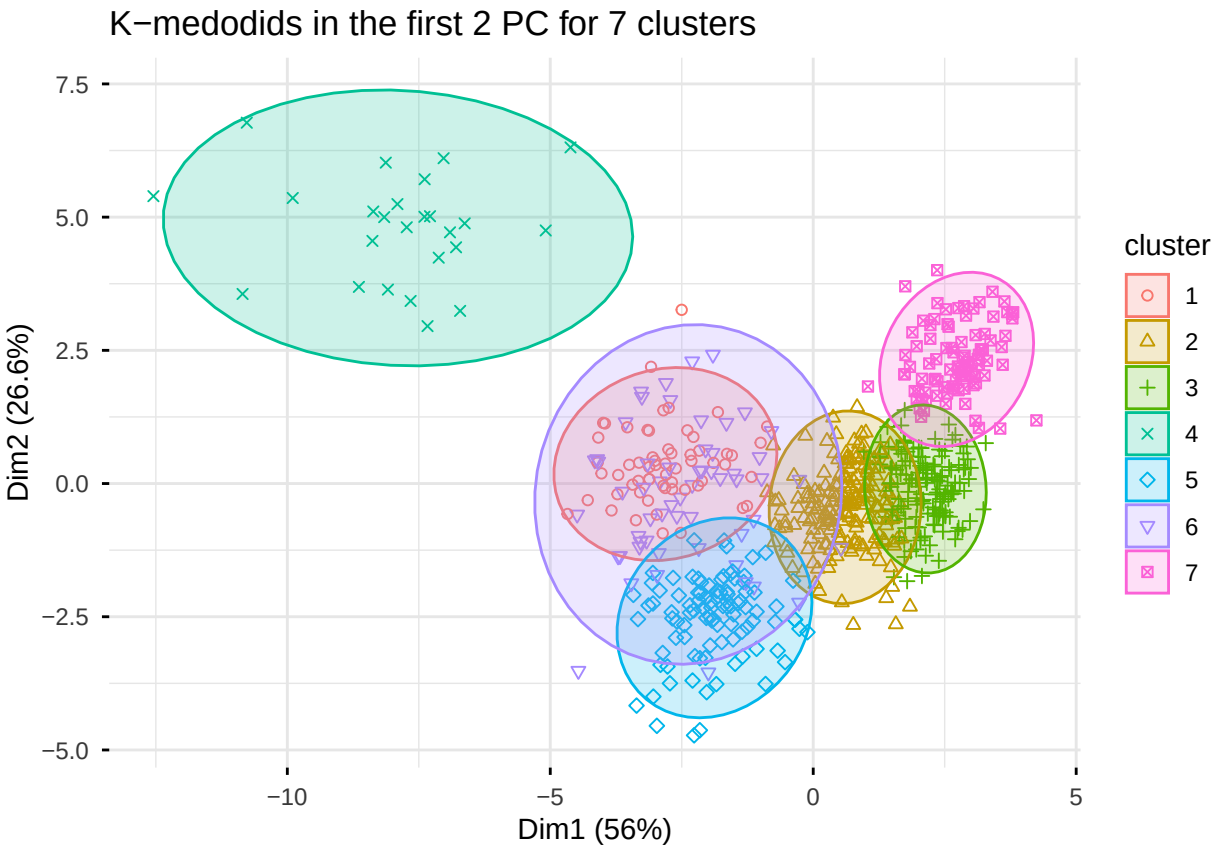
```
## Meila's VI index:  1.099005
```

The Rand index indicates an high agreement. The Meila's index indicates an high disagreement. We can project in the first two PC to see a better graphically representation.

```
fviz_cluster(Beans.kmd,data = std.Beans, geom = "point", ellipse.type = "norm",
             ggtheme = theme_minimal(),
             main = "K-medodids in the first 2 PC for 5 clusters")
```




```
fviz_cluster(Beans.kmd7,data = std.Beans, geom = "point", ellipse.type = "norm",
             ggtheme = theme_minimal(),
             main = "K-mededids in the first 2 PC for 7 clusters")
```



6.4 Stability measure

This section is useful to compare different algorithms and choose the best one. This contains four different indices:

- The **APN(Average proportion of non-overlap)**: measures the average proportion of observation not placed in the same cluster by clustering based on the full data and clustering based on the data with a single column removed.
- The **AD(Average distance)**: measures the average distance between observations placed in the same cluster under both cases (full data and data without one column).
- The **ADM(Average distance between means)** : measures the average distance between cluster centers for observations placed in the same cluster under both cases.
- The **FOM(Figure of merit)**: measure the average intra-cluster variance of the deleted column, where the clustering is based on the remaining (undeleted) columns.

In our case we will compare each hierarchical clustering procedure with kmeans and pam algorithm.

```
clmethods <- c("hierarchical","kmeans","pam")
val.e.s <- clValid(std.Beans,nClust = 2:7,clMethods = clmethods,
                   metric = "euclidean" ,validation = "stability",
```

```
                  method = "single", maxitems = 1000)

summary(val.e.s)
```

```
##
## Clustering Methods:
##  hierarchical kmeans pam
##
## Cluster sizes:
##  2 3 4 5 6 7
##
## Validation Measures:
##                         2      3      4      5      6      7
##
## hierarchical APN  0.0060 0.0002 0.0009 0.0025 0.0007 0.0030
##              AD   4.1928 3.7053 3.7023 3.6973 3.6778 3.6738
##              ADM  0.0597 0.0057 0.0096 0.0262 0.0073 0.0242
##              FOM  0.9991 0.8308 0.8308 0.8294 0.8280 0.8282
## kmeans       APN  0.0279 0.0216 0.0465 0.0887 0.0789 0.1875
##              AD   3.3286 2.9428 2.7274 2.6788 2.5983 2.6080
##              ADM  0.2925 0.2363 0.2973 0.3577 0.3137 0.6533
##              FOM  0.7903 0.6316 0.5896 0.5642 0.5556 0.5184
## pam          APN  0.0228 0.0317 0.0515 0.0987 0.0721 0.1533
##              AD   3.2542 2.9077 2.6872 2.3703 2.1654 2.1416
##              ADM  0.1064 0.1272 0.4438 0.3347 0.1495 0.3886
##              FOM  0.7836 0.7192 0.6397 0.5033 0.4672 0.4715
##
## Optimal Scores:
##
##     Score  Method       Clusters
## APN 0.0002 hierarchical 3
## AD  2.1416 pam          7
## ADM 0.0057 hierarchical 3
## FOM 0.4672 pam          6
```

```
clmethods <- c("hierarchical","kmeans","pam")
val.e.c <- clValid(std.Beans,nClust = 2:7,clMethods = clmethods,
                  metric = "euclidean" ,validation = "stability",
                  method = "complete", maxitems = 1000)

summary(val.e.c)
```

```
##
## Clustering Methods:
##  hierarchical kmeans pam
```

```
##
## Cluster sizes:
##  2 3 4 5 6 7
##
## Validation Measures:
##                         2      3      4      5      6      7
##
## hierarchical APN  0.0000 0.0756 0.0956 0.1523 0.1904 0.2306
##              AD   3.7438 3.7080 2.9371 2.8888 2.8598 2.7766
##              ADM  0.0000 0.3006 0.3610 0.5684 0.6434 0.7842
##              FOM  0.8320 0.8227 0.6330 0.6096 0.5949 0.5727
## kmeans       APN  0.0000 0.0227 0.0319 0.0474 0.0893 0.0984
##              AD   3.7438 2.8702 2.6585 2.6118 2.5953 2.3952
##              ADM  0.0000 0.0886 0.1225 0.2072 0.3068 0.5702
##              FOM  0.8320 0.6240 0.5707 0.5682 0.5616 0.5483
## pam          APN  0.0228 0.0317 0.0515 0.0987 0.0721 0.1533
##              AD   3.2542 2.9077 2.6872 2.3703 2.1654 2.1416
##              ADM  0.1064 0.1272 0.4438 0.3347 0.1495 0.3886
##              FOM  0.7836 0.7192 0.6397 0.5033 0.4672 0.4715
##
## Optimal Scores:
##
##     Score  Method       Clusters
## APN 0.0000 hierarchical 2
## AD  2.1416 pam          7
## ADM 0.0000 hierarchical 2
## FOM 0.4672 pam          6
```

```
clmethods <- c("hierarchical","kmeans","pam")
val.e.al <- clValid(std.Beans,nClust = 2:7,clMethods = clmethods,
                    metric = "euclidean" ,validation = "stability",
                    method = "average", maxitems = 1000)

summary(val.e.al)
```

```
##
## Clustering Methods:
##  hierarchical kmeans pam
##
## Cluster sizes:
##  2 3 4 5 6 7
##
## Validation Measures:
##                         2      3      4      5      6      7
##
```

```
## hierarchical APN  0.0025 0.0339 0.0192 0.0673 0.0802 0.0916
##              AD   3.9468 3.7051 3.5080 2.8957 2.8809 2.8385
##              ADM  0.2986 0.1407 1.3816 0.2924 0.3350 0.3639
##              FOM  0.9813 0.8245 0.7871 0.6261 0.6243 0.6197
## kmeans       APN  0.1815 0.0227 0.0325 0.0437 0.1711 0.2608
##              AD   3.9041 2.8702 2.6769 2.5959 2.6196 2.6075
##              ADM  0.9207 0.0886 0.1624 0.1798 0.5849 0.9062
##              FOM  0.8803 0.6240 0.5715 0.5676 0.5510 0.5205
## pam          APN  0.0228 0.0317 0.0515 0.0987 0.0721 0.1533
##              AD   3.2542 2.9077 2.6872 2.3703 2.1654 2.1416
##              ADM  0.1064 0.1272 0.4438 0.3347 0.1495 0.3886
##              FOM  0.7836 0.7192 0.6397 0.5033 0.4672 0.4715
##
## Optimal Scores:
##
##     Score  Method       Clusters
## APN 0.0025 hierarchical 2
## AD  2.1416 pam          7
## ADM 0.0886 kmeans       3
## FOM 0.4672 pam          6
```

```
clmethods <- c("hierarchical","kmeans","pam")
val.e.w <- clValid(std.Beans,nClust = 2:7,clMethods = clmethods,
                   metric = "euclidean" ,validation = "stability",
                   method = "ward", maxitems = 1000)
```

```
summary(val.e.w)
```

```
##
## Clustering Methods:
##  hierarchical kmeans pam
##
## Cluster sizes:
##  2 3 4 5 6 7
##
## Validation Measures:
##                         2      3      4      5      6      7
##
## hierarchical APN  0.0384 0.0682 0.0449 0.0855 0.2600 0.2428
##              AD   3.3043 3.0815 2.5694 2.4172 2.3594 2.2667
##              ADM  0.1712 0.6421 0.1687 0.3402 0.6947 0.7068
##              FOM  0.7768 0.7669 0.5611 0.5132 0.5068 0.4996
## kmeans       APN  0.0227 0.0787 0.0255 0.0262 0.1929 0.1864
##              AD   3.2535 3.0681 2.5097 2.2969 2.2503 2.1668
##              ADM  0.1095 0.6846 0.0860 0.0920 0.4656 0.5074
##              FOM  0.7831 0.7553 0.5612 0.5012 0.4917 0.4803
```

```
## pam          APN  0.0228 0.0317 0.0515 0.0987 0.0721 0.1533
##              AD   3.2542 2.9077 2.6872 2.3703 2.1654 2.1416
##              ADM  0.1064 0.1272 0.4438 0.3347 0.1495 0.3886
##              FOM  0.7836 0.7192 0.6397 0.5033 0.4672 0.4715
##
## Optimal Scores:
##
##     Score  Method Clusters
## APN 0.0227 kmeans 2
## AD  2.1416 pam    7
## ADM 0.0860 kmeans 4
## FOM 0.4672 pam    6
```

```r
clmethods <- c("hierarchical","kmeans","pam")
val.m.s <- clValid(std.Beans,nClust = 2:7,
                   clMethods = clmethods, metric = "manhattan" ,
                   validation = "stability", method = "single", maxitems = 1000)

summary(val.m.s)
```

```
##
## Clustering Methods:
##  hierarchical kmeans pam
##
## Cluster sizes:
##  2 3 4 5 6 7
##
## Validation Measures:
##                         2       3       4       5       6       7
##
## hierarchical APN   0.0000  0.0015  0.0007  0.0017  0.0017  0.0016
##              AD   10.8450 10.8146 10.7659 10.7425 10.7071 10.6777
##              ADM   0.0000  0.0160  0.0102  0.0194  0.0162  0.0123
##              FOM   0.8320  0.8318  0.8315  0.8312  0.8296  0.8283
## kmeans       APN   0.0000  0.0216  0.0410  0.0550  0.1182  0.2826
##              AD   10.8450  8.1468  7.3146  7.1155  7.0939  7.2221
##              ADM   0.0000  0.2363  0.2372  0.2067  0.4463  1.0121
##              FOM   0.8320  0.6316  0.5784  0.5666  0.5471  0.5165
## pam          APN   0.0446  0.1848  0.1336  0.1773  0.0366  0.1286
##              AD    9.1997  8.7551  7.3660  6.6035  5.5397  5.5132
##              ADM   0.2693  0.8852  0.4736  0.5501  0.0827  0.3007
##              FOM   0.8102  0.7558  0.6634  0.5860  0.4584  0.4538
##
## Optimal Scores:
##
```

```
##     Score  Method       Clusters
## APN 0.0000 hierarchical 2
## AD  5.5132 pam          7
## ADM 0.0000 hierarchical 2
## FOM 0.4538 pam          7
```

```
clmethod <- c("hierarchical","kmeans","pam")
val.m.c <- clValid(std.Beans,nClust = 2:7,clMethods = clmethods,
                   metric = "manhattan" ,validation = "stability",
                   method = "complete", maxitems = 1000)

summary(val.m.c)
```

```
##
## Clustering Methods:
##  hierarchical kmeans pam
##
## Cluster sizes:
##  2 3 4 5 6 7
##
## Validation Measures:
##                          2       3       4       5       6       7
##
## hierarchical APN    0.0000  0.0943  0.1153  0.1183  0.1417  0.2282
##              AD    10.8450  8.3262  7.6735  7.5000  7.4265  6.7498
##              ADM    0.0000  0.4333  0.5357  0.4778  0.5145  0.6693
##              FOM    0.8320  0.6557  0.5880  0.5686  0.5655  0.5127
## kmeans       APN    0.0000  0.0237  0.0303  0.0482  0.0711  0.0381
##              AD    10.8450  7.9042  7.2701  7.1129  7.0046  5.9815
##              ADM    0.0000  0.0920  0.1971  0.1814  0.2540  0.1314
##              FOM    0.8320  0.6241  0.5822  0.5644  0.5597  0.4887
## pam          APN    0.0446  0.1848  0.1336  0.1773  0.0366  0.1286
##              AD     9.1997  8.7551  7.3660  6.6035  5.5397  5.5132
##              ADM    0.2693  0.8852  0.4736  0.5501  0.0827  0.3007
##              FOM    0.8102  0.7558  0.6634  0.5860  0.4584  0.4538
##
## Optimal Scores:
##
##     Score  Method       Clusters
## APN 0.0000 hierarchical 2
## AD  5.5132 pam          7
## ADM 0.0000 hierarchical 2
## FOM 0.4538 pam          7
```

```
clmethods <- c("hierarchical","kmeans","pam")
val.m.al <- clValid(std.Beans,nClust = 2:7,clMethods = clmethods,
```

```
                    metric = "manhattan" ,validation = "stability",
                    method = "average", maxitems = 1000)

summary(val.m.al)
```

```
##
## Clustering Methods:
##  hierarchical kmeans pam
##
## Cluster sizes:
##  2 3 4 5 6 7
##
## Validation Measures:
##                          2       3       4       5       6       7
##
## hierarchical APN   0.0000  0.0331  0.0648  0.0788  0.0752  0.0924
##              AD   10.8450 10.7724  8.4176  8.1326  7.5320  7.3412
##              ADM   0.0000  0.1359  0.4317  0.3523  0.3964  0.3958
##              FOM   0.8320  0.8249  0.6424  0.6127  0.5682  0.5560
## kmeans       APN   0.0000  0.0227  0.0330  0.0376  0.0442  0.0943
##              AD   10.8450  7.9008  7.2147  7.0859  6.9557  6.9370
##              ADM   0.0000  0.0886  0.1649  0.1439  0.1599  0.3431
##              FOM   0.8320  0.6240  0.5716  0.5623  0.5570  0.5587
## pam          APN   0.0446  0.1848  0.1336  0.1773  0.0366  0.1286
##              AD    9.1997  8.7551  7.3660  6.6035  5.5397  5.5132
##              ADM   0.2693  0.8852  0.4736  0.5501  0.0827  0.3007
##              FOM   0.8102  0.7558  0.6634  0.5860  0.4584  0.4538
##
## Optimal Scores:
##
##     Score  Method       Clusters
## APN 0.0000 hierarchical 2
## AD  5.5132 pam          7
## ADM 0.0000 hierarchical 2
## FOM 0.4538 pam          7
```

We can see that in general for APN and ADM the algorithms indicates the hierarchical as the best clustering method with 2 or 3 clusters. Instead the AD and FOM in the majority of cases indicates the pam algorithm as the best in the case of high number of clusters, in general 6 or 7 clusters.

7 Fuzzy Clustering (Fuzzy K-means)

In this section we analysis the data trough the lent of the fuzzy clustering. This type of clustering is a soft clustering approach, this means that each unit in the data doesn't belongs to only one cluster, but for each cluster it has a certain probability to belong to it. First of all we need to derive the optimal number of clusters this is done by comparing two indices:

- **fuzzy silhouette(FS)**: with this index the optimal value of K corresponds to the maximum of the index.
- **Xien and Beni index (XB)**: with this index the optimal value of K is obtained by minimizing the index.

Then we choose the best index between the two chosen. Then we report the centroids and then compare the data with the external value. At the end we plot on the pair plot and on the space spanned by the two first PCs.

7.1 Algorithms

```
Beans.FKM.SIL <- FKM(X = Beans.w.c,k = 2:8 ,RS = 10, stand = 1)
```

```
## The default index SIL.F has been set
## The default value alpha=1 has been set for computing SIL.F
```

```
Beans.FKM.SIL$criterion
```

```
## SIL.F k=2 SIL.F k=3 SIL.F k=4 SIL.F k=5 SIL.F k=6 SIL.F k=7 SIL.F k=8
## 0.6545989 0.4924353 0.5598633 0.4783865 0.5503549 0.5620822 0.5753026
```

```
Beans.FKM.SIL$k
```

```
## Number of clusters
##                  2
```

```
Beans.FKM.XB <- FKM(X = Beans.w.c,k = 2:8 ,RS = 10, index = "XB",stand = 1)

Beans.FKM.XB$criterion
```

```
##    XB k=2    XB k=3    XB k=4    XB k=5    XB k=6    XB k=7    XB k=8
## 0.3181709 0.4527556 0.3915006 0.3828668 0.7961600 0.6674818 0.7225723
```

```
Beans.FKM.XB$k
```

```
## Number of clusters
##                  2
```

In both the situations the algorithm give us only 2 clusters, this means that it can't recognize the differences between the units based on the geometrical characteristics. The only thing that we can see is that in both cases the second best value for FS and XB is 5 clusters. We can try with that and then force the system to have 7 clusters.

```
Beans.FKM5 <- FKM(X = Beans.w.c,k = 5 ,RS = 10,stand = 1)
Beans.FKM7 <- FKM(X = Beans.w.c,k = 7 ,RS = 10,stand = 1)
```

7.2 Clusters and centroids

```
cl.size(Beans.FKM.SIL$U)
```

```
## Clus 1 Clus 2
##    251    421
```

```
cl.size(Beans.FKM.XB$U)
```

```
## Clus 1 Clus 2
##    251    421
```

```
cl.size(Beans.FKM5$U)
```

```
## Clus 1 Clus 2 Clus 3 Clus 4 Clus 5
##    117    116    106    308     25
```

```
cl.size(Beans.FKM7$U)
```

```
## Clus 1 Clus 2 Clus 3 Clus 4 Clus 5 Clus 6 Clus 7
##     90    104     96     92     25    102    163
```

Each unit is assigned to a cluster according to the maximum membership degree getting to so-called closest hard clustering partition. We can see that for FS algorithm e XB algorithm the result is the same.

```
Hraw(Beans.FKM.SIL$X,Beans.FKM.SIL$H)
```

```
##            Area Perimeter MajorAxisLength MinorAxisLength AspectRation
## Clus 1 71008.54 1023.5227        392.9468        224.5127     1.775745
## Clus 2 39194.56  729.8613        267.5770        183.5654     1.462580
##        Eccentricity ConvexArea EquivDiameter    Extent  Solidity roundness
## Clus 1    0.8141193   72080.63      295.3305 0.7378515 0.9849782 0.8224536
## Clus 2    0.7121210   39651.40      220.8815 0.7563689 0.9885694 0.9071684
##        Compactness
## Clus 1   0.7521991
## Clus 2   0.8295104
```

```
Hraw(Beans.FKM.XB$X,Beans.FKM.XB$H)
```

```
##            Area Perimeter MajorAxisLength MinorAxisLength AspectRation
## Clus 1 71008.54 1023.5227        392.9468        224.5127     1.775745
## Clus 2 39194.56  729.8613        267.5770        183.5654     1.462580
##        Eccentricity ConvexArea EquivDiameter    Extent  Solidity roundness
## Clus 1    0.8141193   72080.63      295.3305 0.7378515 0.9849782 0.8224536
## Clus 2    0.7121210   39651.40      220.8815 0.7563689 0.9885694 0.9071684
```

```
##        Compactness
## Clus 1   0.7521991
## Clus 2   0.8295104
```

```
Hraw(Beans.FKM5$X,Beans.FKM5$H)
```

```
##             Area Perimeter MajorAxisLength MinorAxisLength AspectRation
## Clus 1  71237.69 1037.7004        391.3445        231.3512     1.699754
## Clus 2  53596.12  905.8118        360.2110        189.3490     1.911498
## Clus 3  37789.48  708.2109        246.0038        194.9457     1.266012
## Clus 4  37579.00  722.0727        269.2493        176.7142     1.524144
## Clus 5 167925.67 1555.9336        579.7125        369.5119     1.572531
##        Eccentricity ConvexArea EquivDiameter    Extent  Solidity roundness
## Clus 1    0.8007947   72402.35      299.4117 0.7463429 0.9839513 0.8239197
## Clus 2    0.8454959   54358.72      259.8929 0.7225659 0.9860599 0.8146485
## Clus 3    0.5969547   38177.26      218.5628 0.7731166 0.9898691 0.9414645
## Clus 4    0.7501734   38024.35      217.6222 0.7481455 0.9883381 0.8983475
## Clus 5    0.7647550  169895.11      460.7132 0.7793307 0.9883119 0.8658764
##        Compactness
## Clus 1   0.7669804
## Clus 2   0.7237439
## Clus 3   0.8904350
## Clus 4   0.8098679
## Clus 5   0.7966326
```

```
Hraw(Beans.FKM7$X,Beans.FKM7$H)
```

```
##             Area Perimeter MajorAxisLength MinorAxisLength AspectRation
## Clus 1  45642.05  809.6479        305.5825        189.2189     1.617290
## Clus 2  75811.69 1074.6270        405.4089        238.6011     1.706694
## Clus 3  55129.63  925.1620        371.7732        189.0282     1.976093
## Clus 4  38014.30  708.2974        242.9643        198.6949     1.224318
## Clus 5 169788.66 1565.4941        583.6119        371.8592     1.572583
## Clus 6  42694.54  773.5715        290.2318        186.4912     1.557782
## Clus 7  33541.51  677.7968        250.1038        169.8317     1.473783
##        Eccentricity ConvexArea EquivDiameter    Extent  Solidity roundness
## Clus 1    0.7798637   46258.53      239.6616 0.7164204 0.9868249 0.8674331
## Clus 2    0.8040233   77073.71      309.4920 0.7482523 0.9835763 0.8199281
## Clus 3    0.8585167   55901.79      263.9249 0.7228981 0.9862502 0.8042433
## Clus 4    0.5669364   38391.37      219.4197 0.7751458 0.9901961 0.9481690
## Clus 5    0.7654691  171751.99      463.7856 0.7816749 0.9885189 0.8668444
## Clus 6    0.7610847   43198.30      231.9785 0.7736583 0.9884523 0.8897738
## Clus 7    0.7299505   33936.83      205.6790 0.7480341 0.9884112 0.9105081
##        Compactness
## Clus 1   0.7864757
## Clus 2   0.7647444
```

```
## Clus 3   0.7107085
## Clus 4   0.9043914
## Clus 5   0.7964604
## Clus 6   0.8012265
## Clus 7   0.8237314
```

Forcing now the units to be an hard partitioning cluster, we can compare the data with the given classes.

```
table(Beans$Class,Beans.FKM.SIL$clus[,1])
```

```
##
##               1   2
##   BARBUNYA   46   1
##   BOMBAY     25   0
##   CALI       67   0
##   DERMASON    1 187
##   HOROZ      93   4
##   SEKER       0  96
##   SIRA       19 133
```

```
table(Beans$Class,Beans.FKM.XB$clus[,1])
```

```
##
##               1   2
##   BARBUNYA   46   1
##   BOMBAY     25   0
##   CALI       67   0
##   DERMASON    1 187
##   HOROZ      93   4
##   SEKER       0  96
##   SIRA       19 133
```

```
table(Beans$Class,Beans.FKM5$clus[,1])
```

```
##
##               1   2   3   4   5
##   BARBUNYA   46   0   0   1   0
##   BOMBAY      0   0   0   0  25
##   CALI       64   3   0   0   0
##   DERMASON    0   1  20 167   0
##   HOROZ       2  92   0   3   0
##   SEKER       1   0  86   9   0
##   SIRA        4  20   0 128   0
```

```
table(Beans$Class,Beans.FKM7$clus[,1])
```

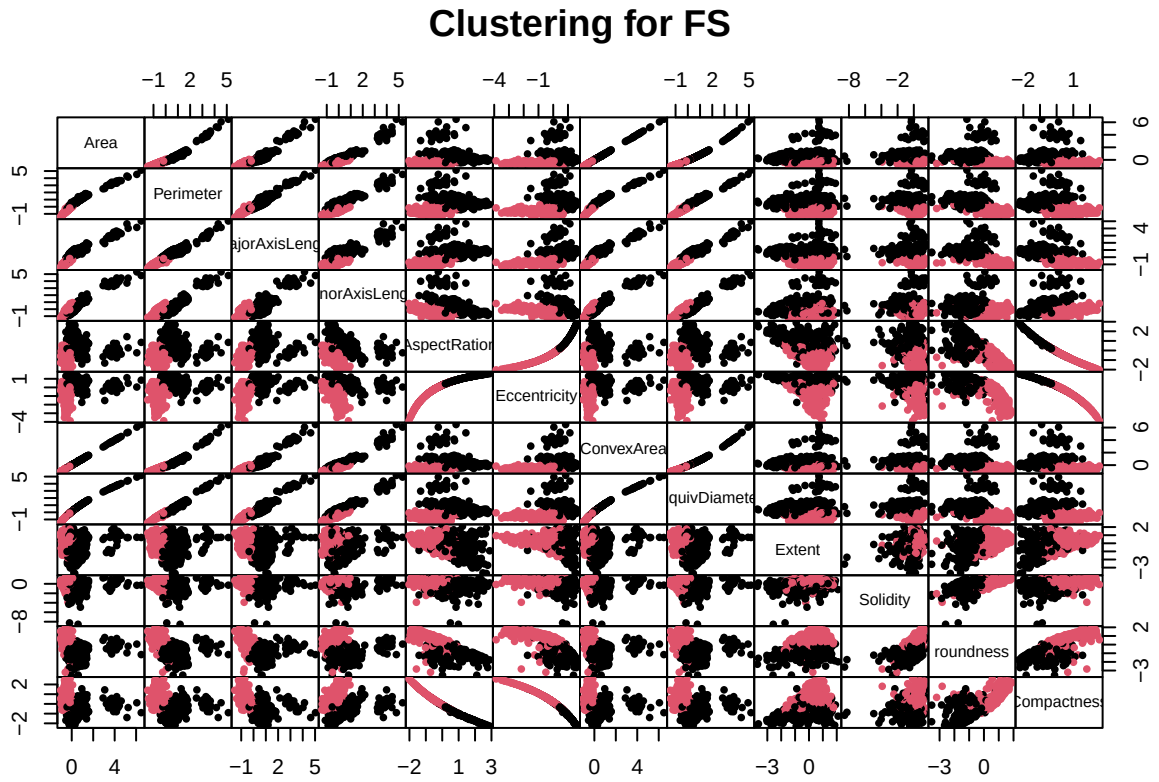
```
##
```

```
##              1   2   3   4   5   6   7
##    BARBUNYA  5  41   0   0   0   1   0
##    BOMBAY    0   0   0   0  25   0   0
##    CALI      2  61   3   0   0   1   0
##    DERMASON 14   0   0   8   0  22 144
##    HOROZ     3   2  89   0   0   3   0
##    SEKER     4   0   0  84   0   5   3
##    SIRA     62   0   4   0   0  70  16
```

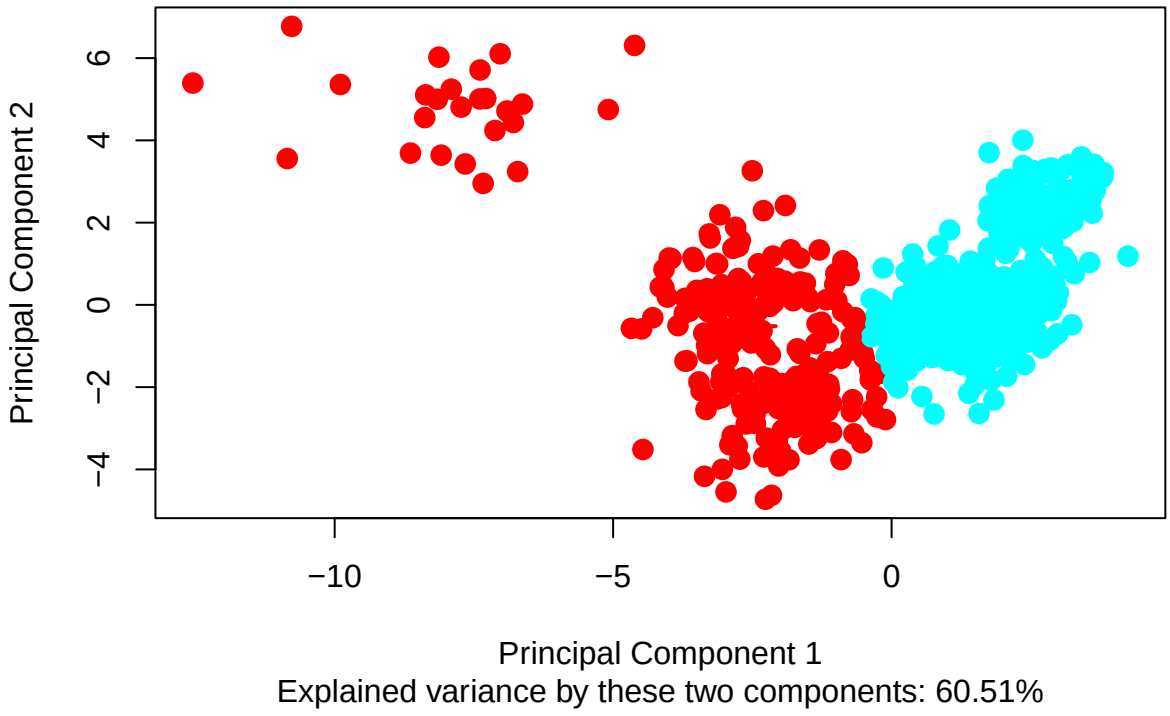
For 2 clusters the 7 classes are divided in two. Instead for 5 or 7 clusters we can see a better identification of each class for each cluster.

7.3 Plots

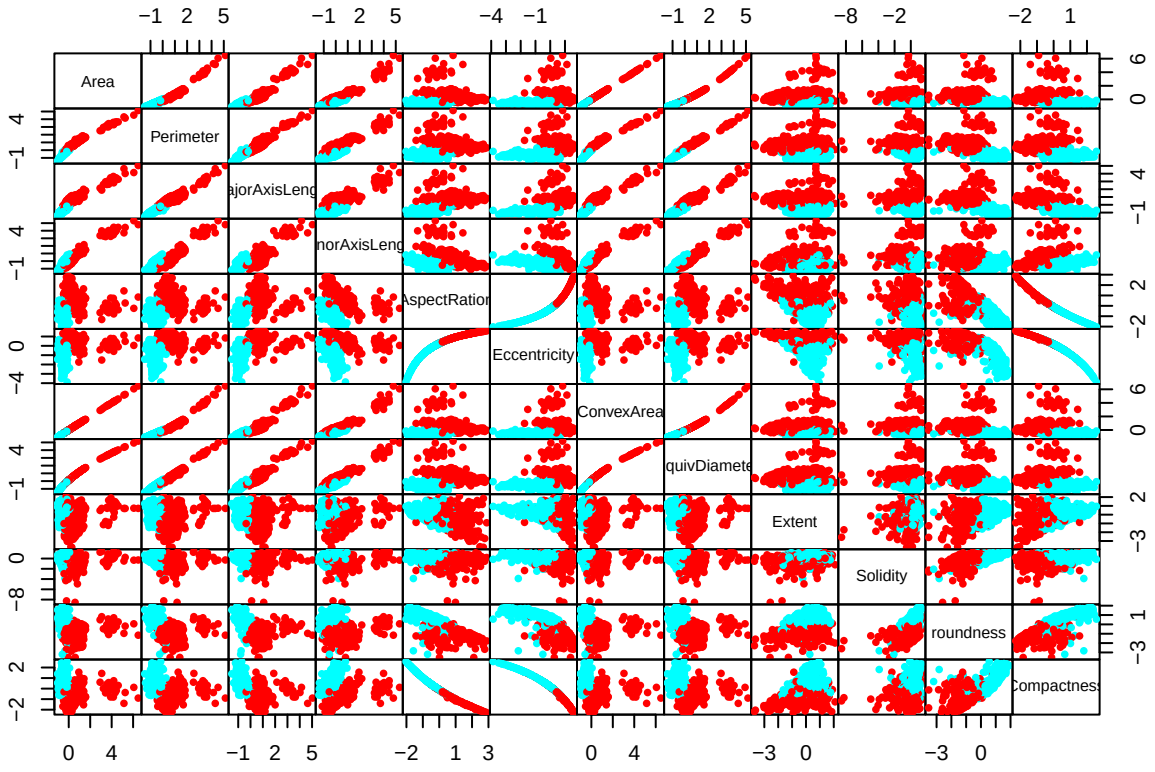
```
pairs(std.Beans, col = Beans.FKM.SIL$clus[,1], pch = 16, cex = 0.8 , gap = 0, main = "Cl
```



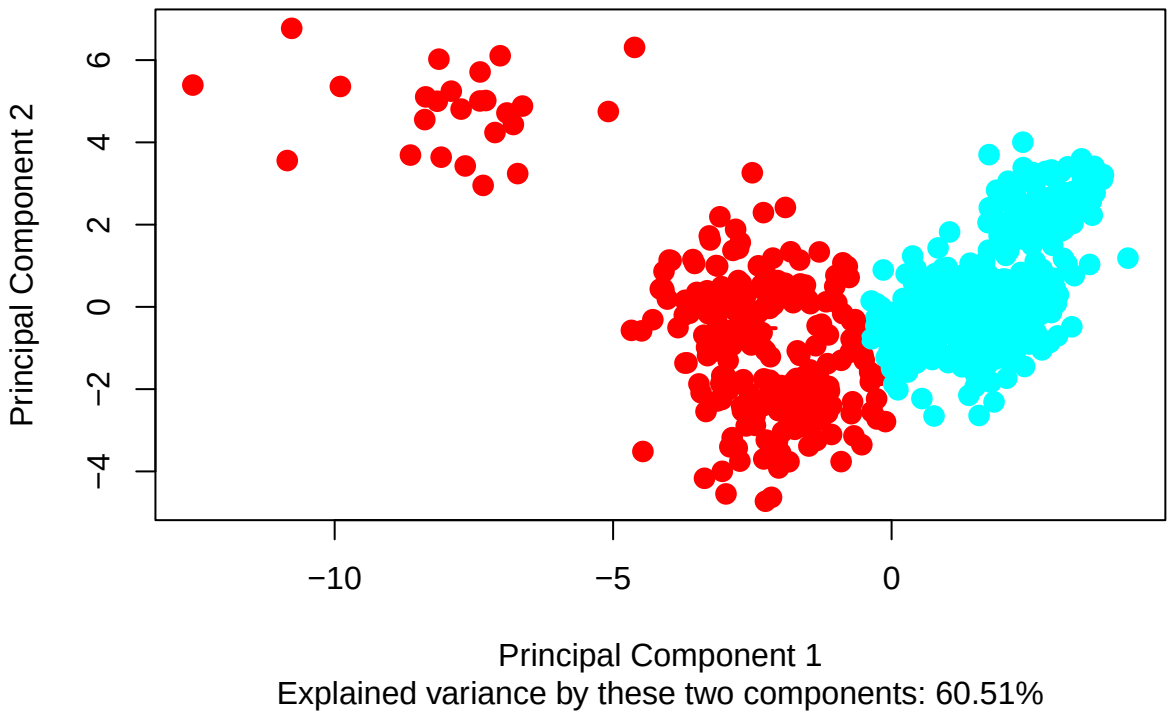

```
plot.fclust(x = Beans.FKM.SIL, pca = TRUE, main = "Clustering for FS")
```



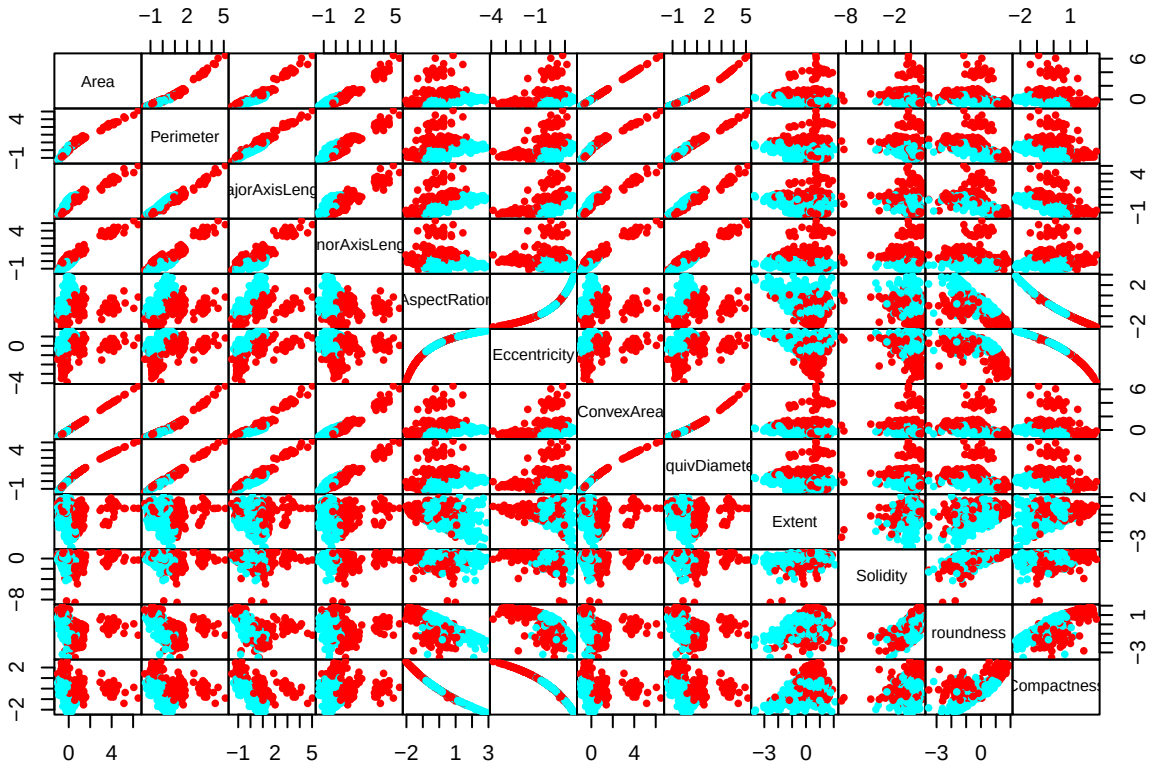

```
pairs(std.Beans, col = Beans.FKM.XB$clus[,1], pch = 16, cex = 0.8 , gap = 0)
```



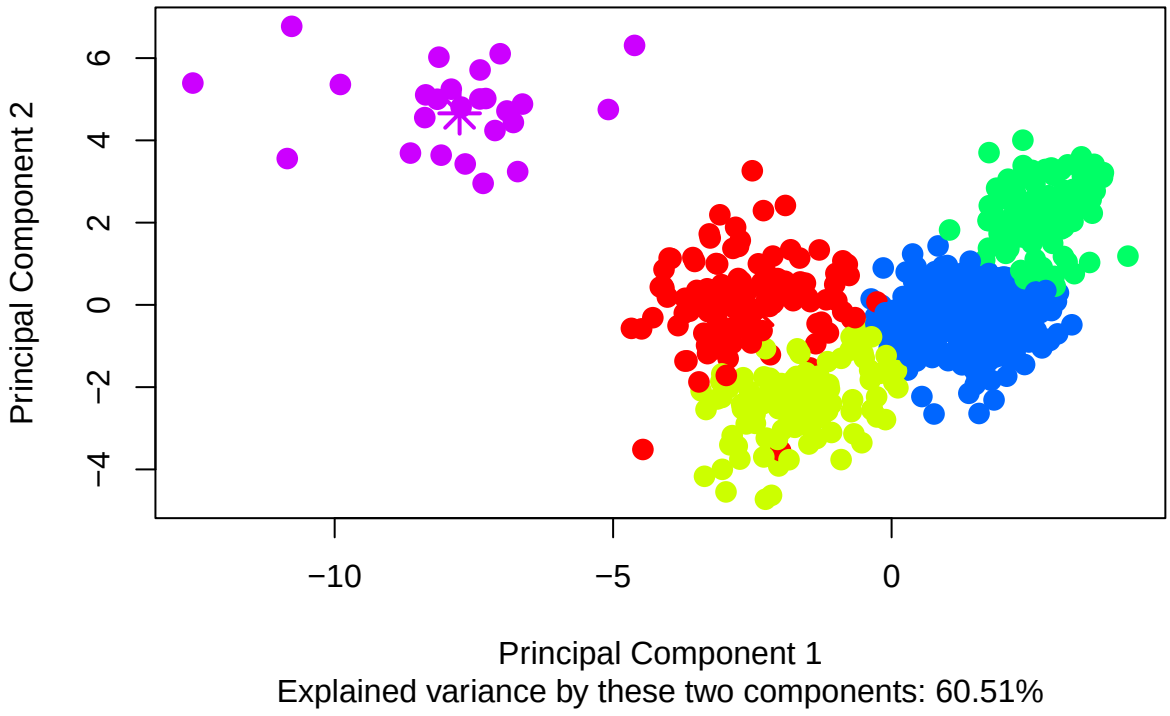

```
plot.fclust(x = Beans.FKM.XB, pca = TRUE)
```



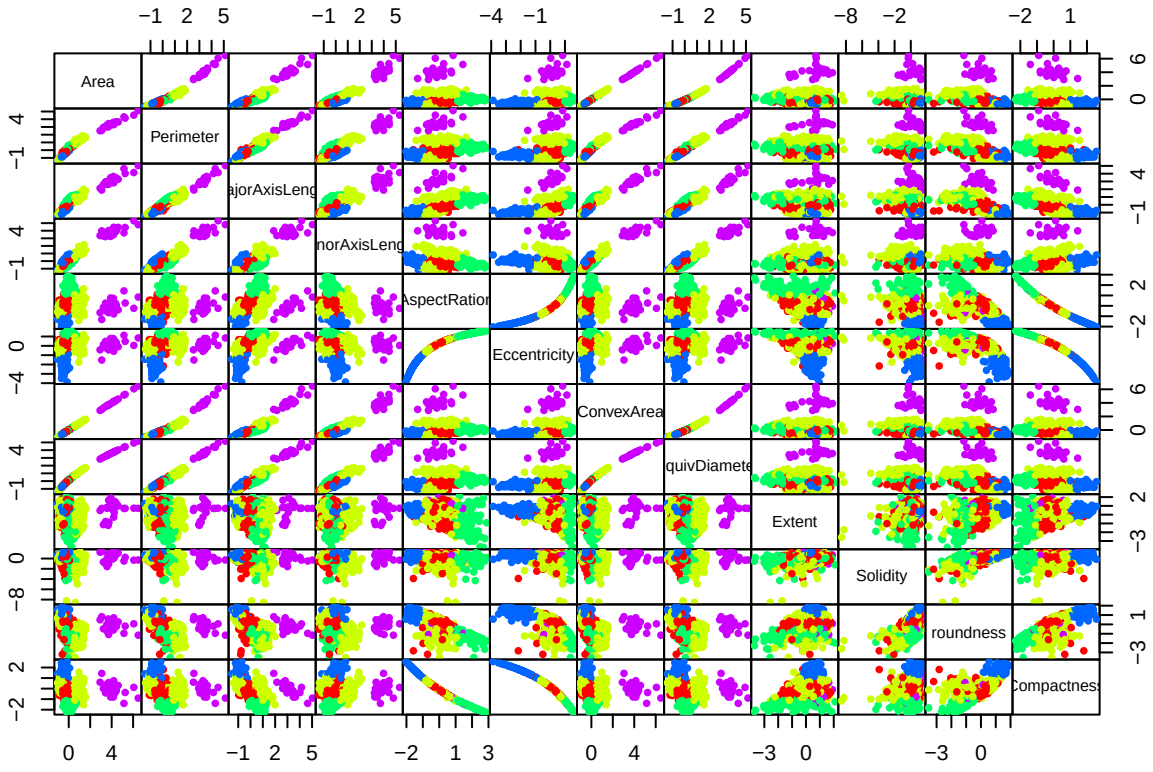

```
pairs(std.Beans, col = Beans.FKM5$clus[,1], pch = 16, cex = 0.8 , gap = 0)
```



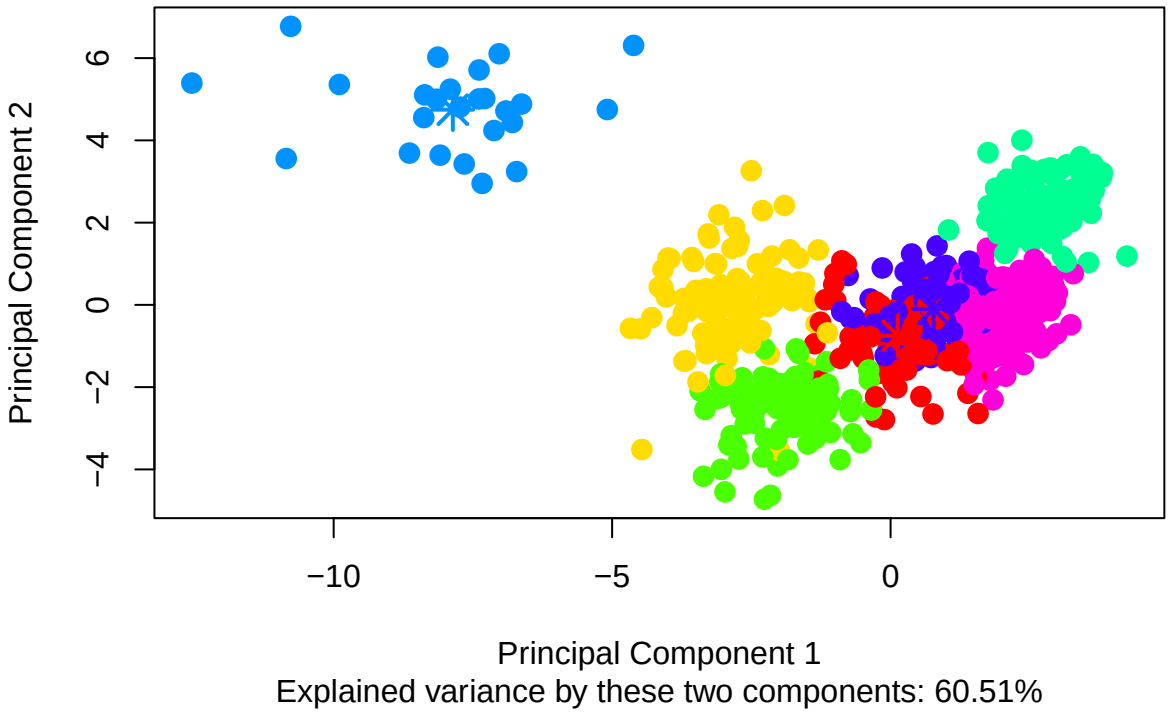

```
plot.fclust(x = Beans.FKM5, pca = TRUE)
```




```
pairs(std.Beans, col = Beans.FKM7$clus[,1], pch = 16, cex = 0.8 , gap = 0)
```




```
plot.fclust(x = Beans.FKM7, pca = TRUE)
```

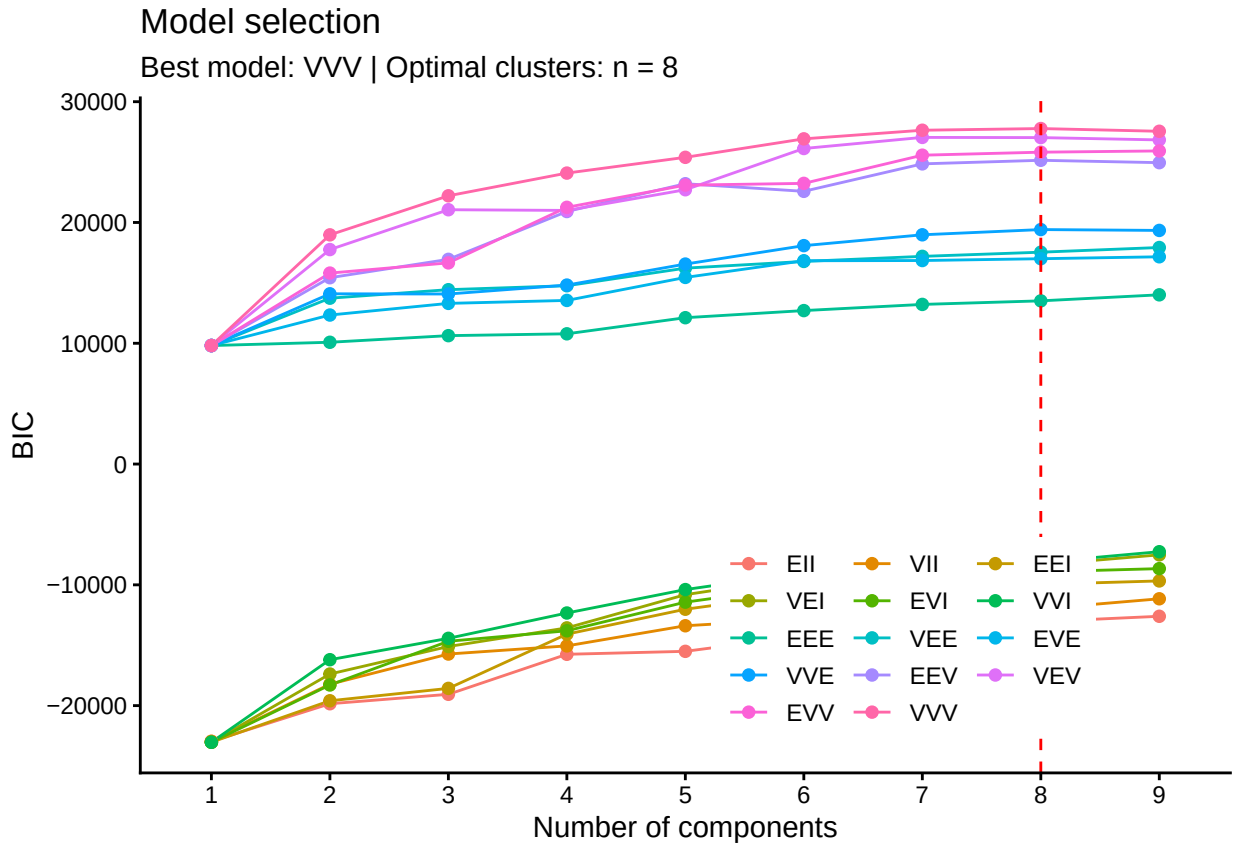


Comparing all the plots we can see that only 2 clusters have too much parsimony. In the case of 5 or 7 is better, but in the case of 7 the different clusters overlap.

8 Model based Clustering

One limitation of traditional clustering methods, such as hierarchical and partitioning algorithms, is that they rely on heuristic procedures and lack a formal statistical framework. As a result, these approaches do not support formal inference. A more principled alternative is model-based clustering, which assumes that the data are generated from a mixture of underlying probability distributions—one for each cluster. Each cluster is described by a specific probability density (or mass) function and is associated with a weight in the mixture. Unlike traditional methods, model-based clustering uses a soft assignment approach, meaning that each data point is assigned a probability of belonging to each cluster, rather than being definitively allocated to just one.

```
Mbc <- Mclust(std.Beans)
fviz_mclust(Mbc, "BIC", palette("jco"))
```




```
summary(Mbc)
```

```
## ----------------------------------------------------
## Gaussian finite mixture model fitted by EM algorithm
## ----------------------------------------------------
##
## Mclust VVV (ellipsoidal, varying volume, shape, and orientation) model with 8
## components:
##
##  log-likelihood   n  df      BIC      ICL
##        16255.18 672 727 27777.41 27774.81
##
## Clustering table:
##   1   2   3   4   5   6   7   8
##  67 162  25 168  69  56  97  28
```

We can see that in this case the best model is a Gaussian mixture model with varying volume, shape and orientation. It identify 8 clusters and the $BIC = 27777.41$. We can compare the three best BIC.

```
summary(Mbc$BIC)
```

```
## Best BIC values:
```

```
##             VVV,8      VVV,7     VVV,9
## BIC      27777.41 27637.9504 27551.678
## BIC diff     0.00  -139.4561  -225.728
```

We can notice that for the VVV with 7 clusters the value of BIC is slightly lower. From now on we choose the configuration with 8 clusters instead of 7. We now compare the 8 clusters with the 7 clusters already present in the data.

```
table(Beans$Class,Mbc$classification)
```

```
##
##               1   2   3   4   5   6   7   8
##    BARBUNYA  11   0   0   1   0  34   0   1
##    BOMBAY     0   0  25   0   0   0   0   0
##    CALI      56   0   0   0   0   6   0   5
##    DERMASON   0 155   0  19   0   3  11   0
##    HOROZ      0   0   0   4  66   5   0  22
##    SEKER      0   0   0   6   0   4  86   0
##    SIRA       0   7   0 138   3   4   0   0
```

We can see that:

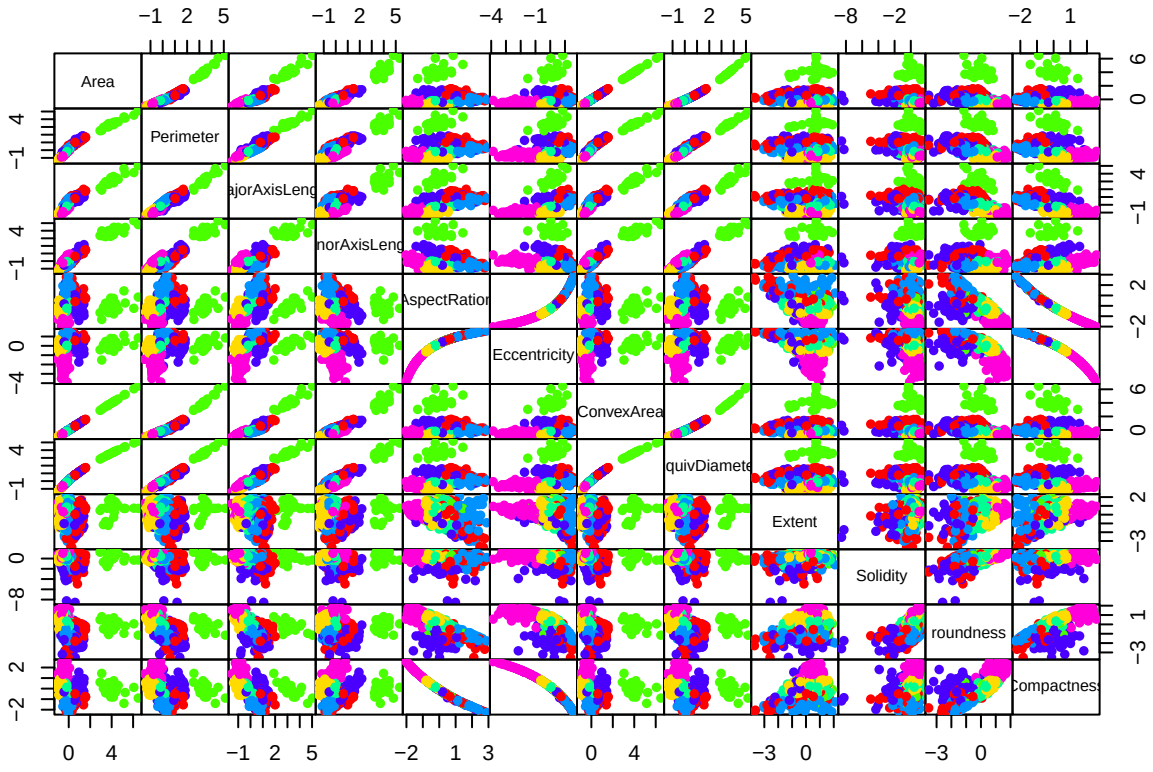
- **Barbunya** in majority belongs to two clusters: the 1st and the 6th.
- **Bombay** is the only one clustered well in the 3rd.
- **Cali** in majority belongs to the 1st cluster, but we have values on 6th and 8th.
- **Dermason** in majority belongs to the 2nd clusters.
- **Horzo** in majority belongs to the 5th cluster, but we have a great number of them inside the 8th cluster.
- **Seker** in majority belongs to the 7th cluster.
- **Sira** in majority belongs to the 4th cluster.

```
adjustedRandIndex(Beans$Class, Mbc$classification)
```

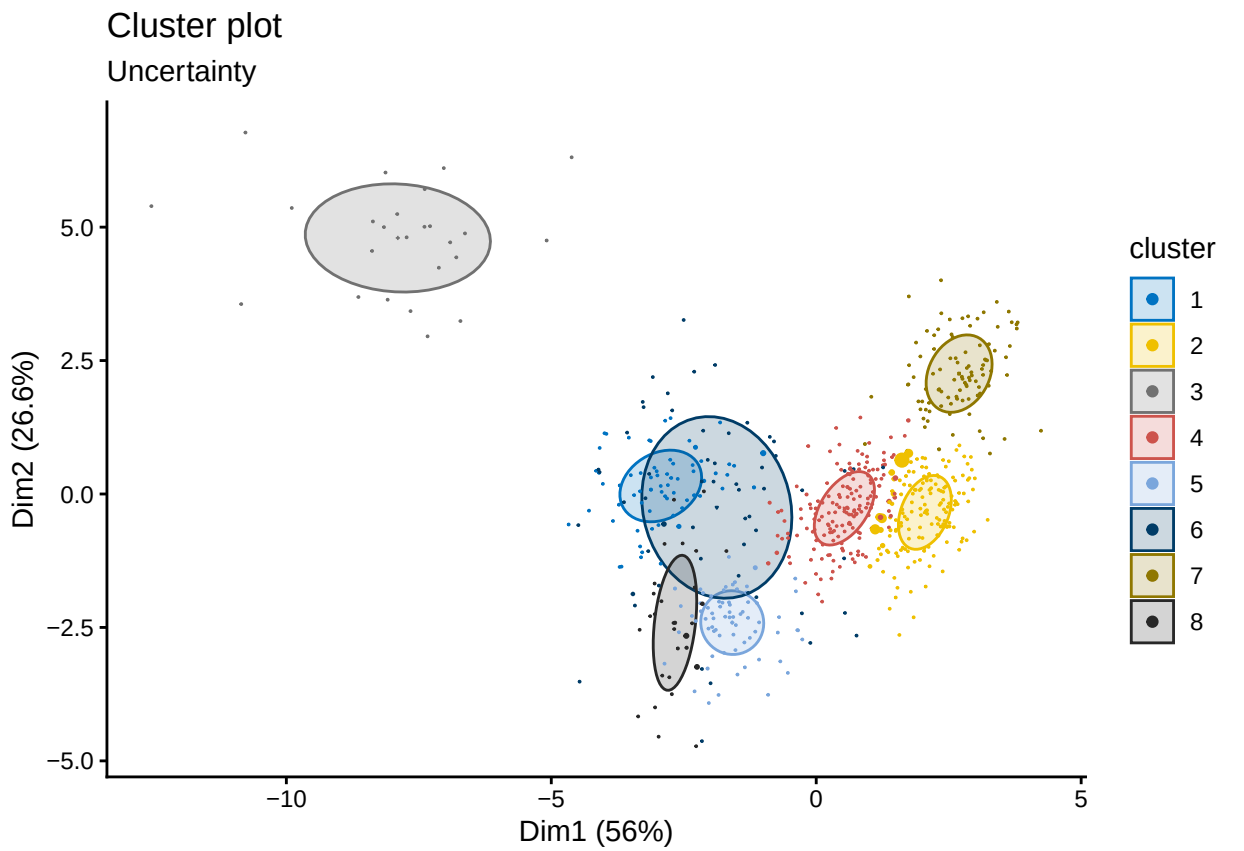
```
## [1] 0.6977213
```

The rand index indicates an high agreement between the clustering proposed by the Model based clustering and the original classes. We can see it from the above table. We can now plot the values on the biplot and the space spanned by the two first PCs.

```
pairs(std.Beans, gap = 0, pch = 16, col = Mbc$classification)
```




```
fviz_mclust(Mbc, "uncertainty", palette="jco")
```



The figure displays the cluster structure estimated via model-based clustering, projected onto the first two principal components. The shape and orientation of each ellipse reflect the underlying Gaussian component. The point size represents the uncertainty of the assignment: smaller points indicate higher classification ambiguity. Eight distinct clusters were identified, as suggested by the highest BIC value.

9 Conclusion

The analysis underling a difficulty for the most heuristic models to predict the right number of clusters. The geometrical factor, with these algorithms, are not sufficient to underling differences between the units. Probably as we can see in the space spanned by the first two PCs the algorithm has difficulty in separating that classes of beans in which the geometrical property are the same, but they are different type of beans. For example Bombay is the only one cluster that frequently is always represented well without misleading assignment to other clusters for their units. The only one method that give a similar number of clusters is the Model based approach. The rand index in this case is grater then all the others.